

ABC80 BASIC-ROM

Kommenterad disassemblering
med skillnaderna i den senare versionen 9913

Z80-kod, \$0000–\$3FFF (16 KB)

Innehåll

1	Inledning	1
2	Systemvariabler och externa adresser	2
3	Listning	3
3.1	RST-vektorer och fasta ingångar	3
3.2	Kallstart	4
3.3	Kommandoloopen	5
3.3.1	Anrop till tilläggs-ROM	6
3.4	Enhetslistan	7
3.5	PRTXT: skriv text på skärmen	7
3.6	INLINE: radinmatning från tangentbordet	10
3.7	GETKEY: vänta på en tangent	10
3.8	Kassetten	12
3.9	OPEN för kassett: leta efter filen	15
3.10	CASRBL: hämta ett block från den avbrottsstyrda läsaren	16
3.11	CASINT: avbrott från kassetten, ett per bit	17
3.12	Felhantering	19
3.13	INSLIN: lägg in, ersätt eller radera en programrad	22
3.14	OPEN: öppna en fil	23
3.15	DVCALL: anropa enhetsrutin nummer 0-8	25
3.16	TXTprt: skriv text till blockad fil (enhetsrutin 4 för CAS)	26
3.17	TXTINP: läs en textrad från blockad fil (enhetsrutin 3 för CAS)	28
3.18	KWSRCH: sök nyckelord i tabell	29
3.19	ED n: redigera rad n	30
3.20	LIST [n][-m] eller LIST fil	31
3.21	LOAD och MERGE	32
3.22	REN [start][,steg]: numrera om programmet	35
3.23	RUN [fil]: starta programmet	37
3.24	EXESTM: utför en sats	38
3.25	SAVE fil: spara programmet som BAC-fil	38
3.26	PRELNK: förlänkning innan programmet körs	40
3.27	SKPELM: hoppa över ett element i tokeniserad text	41
3.28	DETOK: gör om en programrad till text (för LIST, ED med mera)	49
3.29	DETEXP: skriv ett kompilerat uttryck (omvänd polsk notation) som infix	51
3.30	ROTTXT: flytta den senast skrivna texten bakåt	57
3.31	FNAME: tolka ett filnamn [enhet:]namn[.typ]	58
3.32	GETFLT: läs ett flyttal från texten	60
3.33	FLTASC: skriv flyttalet vid HL som text vid DE	64
3.34	TOKLIN: gör om en inskriven rad till intern kod	70
3.35	TOKKW: kompilera en sats	71
3.36	STTAB1/STTAB2: satsernas kompileringsrutiner	73
3.37	CMPDEF: kompilera DEF FNx(p1,p2,...)=uttryck	74
3.38	CMPFN: kompilera ett anrop FNx(a1,a2,...) i ett uttryck	75
3.39	FN vid körning (funktionskod 0 i EXPTAB2)	76
3.40	LET: kompilering	77
3.41	DIM: kompilering	78
3.42	DIMELM: \$CC i uttryckskoden, dimensionera en variabel	79
3.43	FOR: kompilering	81
3.44	FOR vid körning	82
3.45	NEXT vid körning	83
3.46	IF: kompilering	86
3.47	INPUT: kompilering	87
3.48	INPUT vid körning	87

3.49	READ vid körning	88
3.50	RDVAL: läs ett värde ur texten vid HL och tilldela det	89
3.51	ON: kompilering	91
3.52	ON vid körning	92
3.53	OPEN / PREPARE: kompilering	93
3.54	OPEN / PREPARE vid körning	93
3.55	OUT / POKE: kompilering	95
3.56	PRINT: kompilering	96
3.57	PRINT vid körning	97
3.58	RETURN	100
3.59	DOTADR: skärmadress och bit för en grafikpunkt	101
3.60	Uttryckskompilatorn: en post per prioritetsnivå	104
3.61	FNTYPE: typer för funktionernas resultat och argument	106
3.62	TYPCNV: omvandla värdet till typen A (bit 0-1)	106
3.63	PRIMARY: kompilera ett primärt uttryck	107
3.64	CMPVAR: kompilera en variabel, med index	108
3.65	VARNAM: läs ett variabelnamn	111
3.66	CMPEXP: kompilera ett uttryck till omvänd polsk notation	112
3.67	EXEXP: utför kompilerad uttryckskod (RST \$38)	115
3.68	Värdestacken när uttryckskoden körs	115
3.69	EXPTAB1: rutiner för token \$B6-\$FF vid körning	117
3.70	EXPTAB2: funktionerna, index = kod (token i FNKW - \$80)	118
3.71	ASGNF / ASGNI / ASGNS: tilldelning (\$B6-\$B8)	119
3.72	Jämförelse av flyttal, \$E2-\$F3	122
3.73	FCMP: jämför flyttal	122
3.74	ADDF / SUBF / MULF / DIVF	123
3.75	Logiska operatörer, \$DC-\$E1	124
3.76	Jämförelse av heltal, \$E2-\$F3	125
3.77	ADD\$ SUB\$ MUL\$ DIV\$ COMP%: aritmetik på tal skrivna som text	126
3.78	ASCARG: kopiera ett strängargument till bufferten (DE)	128
3.79	VAR0 (\$C0-\$C2): variabel utan index	129
3.80	VAR1x / VAR2x: element i vektor eller matris	130
3.81	Flyttalsfunktioner med ett argument	132
3.82	Potens: POWF (\$FD), POWI (\$FE), POWFI (\$FF)	133
3.83	Strängfunktionerna	136
3.84	CONCAT (\$F6): a\$ + b\$	137
3.85	Strängjämförelser: EQS NES GTS LES LTS GES	139
3.86	EXINST: INSTR(i, a\$, b\$)	140
3.87	EXVAL: VAL(s\$)	143
3.88	EXRND: RND och RND(x)	145
3.89	FNPAR: adressen till en DEF FN-parameter	149
3.90	Flyttalsaritmetik: FADD FSUB FMUL FDIV	150
3.91	IMUL / IDIV: 16-bitars heltal utan tecken	160
3.92	ASCII-aritmetik: AADD ASUB ACOMP ADIV AMUL	160
3.93	ADIV: a = a / b	162
3.94	AMUL: a = a * b	163
3.95	ASPARS: läs ett tal i text	166
3.96	ASALGN: ställ upp a och b så att decimalerna står under varandra	167
3.97	ASFIN: gör resultatet i a läsbart	169
3.98	Flyttalsfunktioner: ATN EXP INT LOG LOG10 SIN COS TAN SQR	171
3.99	FSIN / FCOS	175
3.100	FSQR: kvadratroten, siffra för siffra med udda tal	177
3.101	FTOINT: flyttalet vid HL till heltal i DE	179
3.102	INTTOF: heltalet i HL (med tecken) som flyttal vid DE (exponenten)	180
3.103	FPOLY: P(z) = c0 + c1 z + ... + cn z^n med Horner's metod	181
A	Interpretatorns arkitektur	183
B	Hårdvaran	187
C	Felkoder	191

D	Tokens och tabeller	193
E	Skillnader i version 9913	197
F	Oklara ställen	200
G	Källor	201
H	Symbolförteckning	211

Inledning

Z80-disassemblering av ABC80:s BASIC-ROM (16 KB, \$0000-\$3FFF). Återassemblerad ger listningen exakt samma bytes som originalet (version 11273).

Minneskarta (som ROM-koden använder den)

\$0000-\$3FFF denna ROM (BASIC, skärm, tangentbord, kassett)
\$4000-\$7BFF plats för tilläggs-ROM: \$405C (RST \$08), ingångar på \$60xx och \$70xx (via EXTJP1/EXTJP2), hopptabellerna för IEC: (\$7009) och PR: (\$7800)
\$7C00-\$7FFF bildminne, 24 rader x 40 tecken (se ROWTAB)
\$8000- RAM; programmet börjar där RAM-testet hittar RAM (BOFA)
\$FB00 stackens topp (SP växer nedåt)
\$FB00-\$FCFF två 256-bytes kassettbuffertar
\$FDxx-\$FExx systemvariabler, IY pekar på \$FE16

Radformat i programminnet (tokeniserad BASIC)

+0 radens längd i bytes inklusive denna byte
+1,+2 radnummer (lo, hi)
+3... tokeniserad text
Programmet avslutas med en "rad" vars längdbyte är \$01.

Felkoder

RST \$10 följs av en byte = felnummer + \$80. Felnumret skrivs ut som "ERR n". Texterna i kommentarerna kommer från ABC80:s fellista.

I/O-portar

\$06 ljudgeneratorn (SN76477)
\$07 läses vid reset (funktion framgår inte av koden)
\$38/\$39 PIO A data/styr: tangentbord, bit 7 = tangent nedtryckt, bit 0-6 = teckenkod; ger avbrott via vektor \$34
\$3A/\$3B PIO B data/styr: kassett. Ut: bit 5 = motor, bit 6 = skrivsignal. In: bit 7 = läst signal; avbrott via vektor \$36 vid läsning

Att läsa listningen

Varje rad visar adress, bytes (högst fyra; fler markeras med ...), etikett, instruktion och kommentar. Etiketter, variabelnamn och adresser i ROM är länkar i PDF-filen. Längre kommentarer före en rutin står som löptext; rubricerade block ger avsnitten i innehållsförteckningen. Tolkningar som är osäkra är markerade med (?). Allt är härlett ur koden i ROM:en; uppgifter från andra källor har källan inom parentes, t.ex. "(Finnved 1979)", och källorna står i kapitlet Källor. Symbolförteckningen sist ger adress och sida för varje etikett.

Systemvariabler och externa adresser

Namn	Adress	Beskrivning
V405C	\$405C	
V604B	\$604B	
V6051	\$6051	
ASBBEG	\$FDD0	
ASBEND	\$FDD2	
ASABEG	\$FDD4	
ASAEND	\$FDD6	
SINQ	\$FDE4	
ASARG1	\$FDE7	
ASEND	\$FDE9	
ASDEC	\$FDEB	
ASCODE	\$FDED	
NMICNT	\$FDEF	NMI: skrivs tillbaka oförändrad
CLOCKH	\$FDF1	
CURROW	\$FDF3	markörens rad (0-23)
CURCOL	\$FDF4	markörens kolumn (0-39)
KEYFLG	\$FDF5	bit 7 = tangent väntar (sätts av KEYINT)
RPTDLY	\$FDF7	autorepetition: fördröjning
CASST	\$FDF8	kassettläsning: \$FF = pågår, annars resultat
CASBUF	\$FDF9	kassett: aktuell läsbuffert (0 = ingen)
EDPTR	\$FDFA	INLINE: mallrad för TAB
CASBLK	\$FDFD	kassett: blocknummer
RCVPTR	\$FDFF	kassettläsning: sparat DE
RCVBC	\$FE01	kassettläsning: C = skiftregister, B = bitar kvar
RCVCONT	\$FE03	kassettläsning: korutinens fortsättningsadress
RCVSUM	\$FE05	kassettläsning: checksumma
BRKFLG	\$FE07	<> 0 om BREAK (CTRL-C) har tryckts
DEVHEAD	\$FE0A	första posten i enhetslistan
EXTFLG	\$FE0F	bit 0: tilläggs-ROM skriver felmeddelanden
BOFA	\$FE1C	programmets början
EOFA	\$FE1E	programmets slut (slutmarkeringen \$01)
HEAP	\$FE20	slutet av variablerna = första lediga byte
FLAGS	\$FE25	flaggor: bit 4 = TRACE, bit 5 = skyddat program (IY+\$0F)
STKTOP	\$FE27	stackens startvärde
VARLST	\$FE29	variabellistan
CURLIN	\$FE2C	adress till raden som körs (eller hade fel)
DATPTR	\$FE2E	DATA: pekare till nästa värde
DATLIN	\$FE30	DATA: aktuell rad
FILLST	\$FE32	
VFE33	\$FE33	
ONERR	\$FE34	ON ERROR GOTO: radadress, 0 = inget (IY+\$1E/\$1F)
STMSP	\$FE36	SP när satsen startade, för ON ERROR
FNFRAM	\$FE3E	
BUF1	\$FE40	BUF1: radbuffert, vid PRELNK pekare till FOR-kedjan

Listning

3.1 RST-vektorer och fasta ingångar

\$0000

Mellan RST-adresserna ligger JP-instruktioner som fasta ingångar för maskinkodsprogram:

```
$0002 GETKEY   vänta på tangent, A = tecken
$0005 INLINE   läs en rad från tangentbordet (HL = buffert, C = max)
$000B PRTXT    skriv text på skärmen (HL = text, BC = längd)
$0015 TXTINP   läs rad från blockad textfil (enhetsrutin 3)
$001B TXTPRT   skriv till blockad textfil (enhetsrutin 4)
$0023 TXTEND   avsluta blockad textfil
$003B, $003E OPNSTR, $0041, $0044 KWSRCH: interpretatorns rutiner
```

RST-rutinerna:

```
RST $08 hopp till tilläggs-ROM på $405C
RST $10 fel; efter RST-instruktionen följer felkoden
RST $18 INC HL och hoppa över blanktecken, A = nästa tecken
RST $20 hoppa över blanktecken, A = nästa tecken
RST $28 kastar returadressen och går till $2841 (PUSH HL, sedan som RST $38)
RST $30 kastar returadressen och går till RST $38
RST $38 utför kompillerad uttrycks kod vid DE ($2842, EXPTAB1)
```

0000	18 66	RESET:	JR	INIT	hoppa över ingångarna
0002	C3 EA 02	J0002:	JP	GETKEY	
0005	C3 AA 02	J0005:	JP	INLINE	
0008	C3 5C 40	RST08:	JP	V405C	
000B	C3 A8 01	J000B:	JP	PRTXT	
000E	00 00		DEFB	\$00,\$00	
0010	E1	RST10:	POP	HL	HL = adressen till felkoden
0011	7E		LD	A, (HL)	
0012	C3 2E 06		JP	ERROR	
0015	C3 A0 09	J0015:	JP	TXTINP	
0018	C3 BE 25	RST18:	JP	SKIPSP	från nästa tecken, utan blanka; A = tecknet
001B	C3 12 09	J001B:	JP	TXTPRT	
001E	00 00		DEFB	\$00,\$00	
0020	C3 BF 25	RST20:	JP	SKIPS0	som RST \$18, men från (HL) självt
0023	C3 85 09	J0023:	JP	TXTEND	
0026	00 00		DEFB	\$00,\$00	
0028	33	RST28:	INC	SP	kasta returadressen; HL blir retur
0029	33		INC	SP	
002A	C3 41 28		JP	EXEXPH	
002D	C3 E2 07	J002D:	JP	OPEN	
0030	33	RST30:	INC	SP	
0031	33		INC	SP	
0032	18 04		JR	RST38	

IM 2-vektortabell ($I = 0$). PIO A (tangentbord) har vektor \$34 och PIO B (kassett) har vektor \$36.

```
0034 1E 03      IM2VEC: DEFW KEYINT
0036 94 05      DEFW CASINT
0038 C3 42 28    RST38:  JP  EXEXP
003B C3 6C 23    J003B:  JP  CMPSTX
003E C3 C2 07    J003E:  JP  OPNSTR
```

```
0041 C3 78 14 J0041: JP FNAME
0044 C3 F5 09 J0044: JP KWSRCH
```

NMI: realtidsklocka. NMI kommer 50 gånger per sekund. Räknnaren CLOCK (\$FDF0-\$FDF2, 24 bitar) räknas ned med 1 per NMI. Den läses in som BC = (\$FDEF): B = \$FDF0 räknas med DJNZ, och vid noll räknas \$FDF1-\$FDF2 ned. \$FDEF (C) skrivs tillbaka oförändrad.

```
0047 C5 NMICLK: PUSH BC          B = låga byten, C = $FDEF orörd
0048 ED 4B EF FD LD BC, (NMICNT)
004C 10 10 DJNZ L005E          inte noll än: bara spara
004E ED 43 EF FD LD (NMICNT), BC
0052 ED 4B F1 FD LD BC, (CLOCKH)      var 256:e NMI: de höga bytena
0056 0B DEC BC
0057 ED 43 F1 FD LD (CLOCKH), BC
005B C1 POP BC
005C ED 45 RETN

005E ED 43 EF FD L005E: LD (NMICNT), BC
0062 C1 POP BC
0063 ED 45 RETN

0065 45 DEFB $45
0066 18 DF NMI: JR NMICLK
```

3.2 Kallstart

\$0068

Initierar PIO:erna, sätter IM 2, stacken på \$FB00 och IY = \$FE16. Letar sedan nedåt från \$FA00 efter lägsta sidan (256 bytes) med skrivbart minne, dock inte under \$8000. Där börjar programmet (BOFA). Enhetslistan initieras och en eventuell tilläggs-ROM anropas om det står JP (\$C3) på \$604B.

```
0068 DB 07 INIT: IN A, ($07)      port 7: återställ ABC-bussen (Markesjö 1978, s. 122)
006A ED 5E IM 2
006C 3E CF LD A, $CF          PIO B: bitläge ...
006E D3 3B OUT ($3B), A
0070 3E 87 LD A, $87          ... riktningsmask
0072 D3 3B OUT ($3B), A
0074 3E 34 LD A, $34          PIO A: avbrottsvektor $34
0076 D3 39 OUT ($39), A
0078 3E B7 LD A, $B7          PIO A: avbrott på, mask följer
007A D3 39 OUT ($39), A
007C 3E 7F LD A, $7F          bara bit 7 (strobe) ger avbrott
007E D3 39 OUT ($39), A
0080 21 00 FB LD HL, $FB00      stackens topp
0083 F9 LD SP, HL
0084 22 27 FE LD (STKTOP), HL
0087 FD 21 16 FE LD IY, $FE16    IY = bas för systemvariablerna
008B AF XOR A
008C ED 47 LD I, A            I = 0: vektortabellen ligger på $00xx
008E 32 0F FE LD (EXTFLG), A
0091 32 25 FE LD (FLAGS), A
0094 32 33 FE LD (VFE33), A
0097 6F LD L, A
0098 D3 06 OUT ($06), A        tyst ljudgenerator (SN76477; Markesjö 1978, s. 97)
009A 32 F5 FD LD (KEYFLG), A
009D 3D DEC A
009E D3 39 OUT ($39), A
00A0 D3 39 OUT ($39), A
```

RAM-test: skriv komplementet av innehållet, läs tillbaka och återställ. Nollflagga = minnet gick att skriva.

```
00A2 25 L00A2: DEC H
00A3 7E LD A, (HL)
00A4 47 LD B, A
00A5 2F CPL
00A6 77 LD (HL), A
00A7 AE XOR (HL)
00A8 70 LD (HL), B
```

00A9	20 05	JR	NZ,L00B0	
00AB	7C	LD	A,H	
00AC	FE 80	CP	\$80	
00AE	30 F2	JR	NC,L00A2	
00B0	24	L00B0:	INC H	första sidan med RAM
00B1	22 1C FE	LD	(BOFA),HL	BOFA
00B4	CD 76 02	CALL	CLS	sudda skärmen
00B7	21 8C 01	LD	HL,DEVLIST	enhetslistan börjar i ROM
00BA	22 0A FE	LD	(DEVHEAD),HL	
00BD	3A 4B 60	LD	A,(V604B)	tilläggs-ROM med JP på \$604B?
00C0	FE C3	CP	\$C3	
00C2	CC 4B 60	CALL	Z,V604B	... anropa den
00C5	FB	EI		

3.3 Kommandoloopen

\$00C6

COLD: NEW + CLEAR. **READY:** om bit 5 i \$FE25 är satt (programmet var laddat som skyddat, typ \$83) raderas det först. Skriver "ABC80". **CMDLP** läser en rad till bufferten \$FE40 (högst 120 tecken). Tom rad ignoreras. Börjar raden med ett direktkommando (**CMDKW**) körs det via **CMDTAB** med **READY** som returadress. Annars tokeniseras raden:

- med radnummer: raden läggs in i programmet (**INSLIN**)
- utan radnummer: raden utförs direkt

00C6	CD 70 0A	COLD:	CALL NEW	
00C9	CD 80 0A	ENDPRG:	CALL CLEAR	
00CC	FD CB 0F 6E	READY:	BIT 5,(IY+\$0F)	skyddat program: radera
00D0	20 F4		JR NZ,COLD	
00D2	21 2D 07		LD HL,D072D	CR LF "ABC80"
00D5	01 07 00		LD BC,\$0007	
00D8	CD A8 01		CALL PRTXT	
00DB	CD 42 07	CMDLP:	CALL NEWLIN	
00DE	FD 36 0E 01		LD (IY+\$0E),\$01	\$FE24 = 1: direktläge
00E2	2A 27 FE		LD HL,(STKTOP)	
00E5	F9		LD SP,HL	
00E6	21 40 FE		LD HL,\$FE40	radbuffert
00E9	01 78 00		LD BC,\$0078	max 120 tecken
00EC	E5		PUSH HL	
00ED	CD AA 02		CALL INLINE	
00F0	CD 3E 03		CALL TSTBRK	
00F3	E1		POP HL	
00F4	E7	L00F4:	RST \$20	första tecknet utan blanka
00F5	FE 0D		CP \$0D	
00F7	28 E2		JR Z,CMDLP	
00F9	11 20 0A		LD DE,CMDKW	direktkommando?
00FC	CD F5 09		CALL KWSRCH	
00FF	28 32		JR Z,DOCMD	
0101	CD 90 18		CALL TOKLIN	tokenisera raden
0104	DA 69 06		JP C,ERRSTP	
0107	DD 6E 01		LD L,(IX+\$01)	HL = radnumret
010A	DD 66 02		LD H,(IX+\$02)	
010D	7D		LD A,L	
010E	B4		OR H	
010F	28 08		JR Z,L0119	radnummer 0: kör direkt
0111	CD 50 07		CALL INSLIN	annars in i programmet
0114	30 C5		JR NC,CMDLP	
0116	C3 2E 06		JP ERROR	
0119	DD 7E 00	L0119:	LD A,(IX+\$00)	tom rad?
011C	A7		AND A	
011D	28 BC		JR Z,CMDLP	
011F	CD 42 07		CALL NEWLIN	
0122	21 BB FE		LD HL,\$FEBB	koden i direktläge, sparas för RUNSTM
0125	E5		PUSH HL	
0126	FD 36 2C 00		LD (IY+\$2C),\$00	inga DEF-parametrar
012A	CD AA 0E		CALL LNKELM	länka raden
012D	D1		POP DE	
012E	CD 92 0D		CALL RUNSTM	
0131	18 99		JR READY	

Direktkommando: A = kommandonummer från KWSRCH. Hämta adressen ur CMDTAB och hoppa dit med READY som returadress.

0133	01 CC 00	DOCMD:	LD BC,READY	
0136	C5		PUSH BC	
0137	11 5A 0A		LD DE,CMDTAB	två bytes per kommando
013A	87		ADD A,A	
013B	4F		LD C,A	
013C	06 00		LD B,\$00	
013E	EB		EX DE,HL	
013F	09		ADD HL,BC	
0140	7E		LD A,(HL)	
0141	23		INC HL	
0142	66		LD H,(HL)	
0143	6F		LD L,A	
0144	EB		EX DE,HL	DE = rutinen
0145	D5		PUSH DE	hoppa dit, READY som retur
0146	C9		RET	

3.3.1 Anrop till tilläggs-ROM

\$0147

Efter CALL EXTJP1/EXTJP2 följer adressen till en ingång i en tilläggs-ROM. Om det står JP (\$C3) på den adressen hoppar man dit. Annars gäller:

EXTJP1: återvänder till anroparens anropare med CY = 1, A = \$88

EXTJP2: fel 8 (FINNS EJ I DETTA SYSTEM)

0147	CD 56 01	L0147:	CALL EXTJP1	
014A	00 70		DEFB \$00,\$70	
014C	CD 56 01	L014C:	CALL EXTJP1	
014F	5A 60		DEFB \$5A,\$60	
0151	CD 56 01	L0151:	CALL EXTJP1	
0154	54 60		DEFB \$54,\$60	
0156	E3	EXTJP1:	EX (SP),HL	HL = adressen efter CALL
0157	7E		LD A,(HL)	
0158	23		INC HL	
0159	66		LD H,(HL)	
015A	6F		LD L,A	
015B	7E		LD A,(HL)	
015C	FE C3		CP \$C3	JP-instruktion där?
015E	E3		EX (SP),HL	
015F	C8		RET Z	ja: hoppa dit via RET
0160	33		INC SP	nej: kasta returadressen
0161	33		INC SP	
0162	3E 88		LD A,\$88	A = fel 8, CY = 1
0164	37		SCF	
0165	C9		RET	
0166	CD 7F 01	L0166:	CALL EXTJP2	
0169	03 70		DEFB \$03,\$70	
016B	CD 7F 01	EXIEC:	CALL EXTJP2	
016E	06 70		DEFB \$06,\$70	
0170	CD 7F 01	L0170:	CALL EXTJP2	
0173	5D 60		DEFB \$5D,\$60	
0175	CD 7F 01	L0175:	CALL EXTJP2	
0178	57 60		DEFB \$57,\$60	
017A	CD 7F 01	L017A:	CALL EXTJP2	
017D	4E 60		DEFB \$4E,\$60	
017F	E3	EXTJP2:	EX (SP),HL	HL = adressen efter CALL
0180	7E		LD A,(HL)	
0181	23		INC HL	
0182	66		LD H,(HL)	
0183	6F		LD L,A	
0184	7E	L0184:	LD A,(HL)	
0185	FE C3		CP \$C3	
0187	C2 09 07		JP NZ,L0709	saknas: fel 8
018A	E3		EX (SP),HL	hoppa dit, anroparen som retur

018B C9

RET

3.4 Enhetslistan

\$018C

DEVHEAD (\$FE0A) pekar på första posten. Varje post är 7 bytes:

- +0,+1 pekare till nästa post (0 = sista)
- +2..+4 enhetsnamn, 3 tecken (t.ex. "CAS", "PR ")
- +5,+6 adress till enhetens hopptabell med 9 JP-instruktioner:
 - 0 **OPEN** öppna befintlig fil för läsning
 - 1 **PREPARE** skapa ny fil för skrivning
 - 2 **CLOSE** stäng
 - 3 **INPUT** läs en textrad
 - 4 **PRINT** skriv text
 - 5 **GET** läs nästa block (blockade filer)
 - 6 **PUT** skriv block
 - 7, 8 används inte av CAS (fel 52)

Posten med blankt namn " " är standardenheten när inget namn anges. Den pekar på samma tabell som CAS:. PR: och IEC: har sina hopptabeller i tilläggs-kretsar (\$7800, \$7009). Eftersom listan börjar i RAM-variabeln **DEVHEAD** kan program och tilläggs-ROM:ar länka in nya enheter först i listan.

Uppslagning av enheten sker i **OPEN** (\$07E2) och anrop av en rutin via **DVCALL** (\$08A5).

```

018C 93 01      DEVLST: DEFW DEVDFL
018E 43 41 53      DEFM "CAS"
0191 A4 03      DEFW CASTAB
0193 9A 01      DEVDFL: DEFW DEVPR
0195 20 20 20      DEFM " "
0198 A4 03      DEFW CASTAB
019A A1 01      DEVPR: DEFW DEVIEC
019C 50 52 20      DEFM "PR "
019F 00 78      DEFW $7800
01A1 00 00      DEVIEC: DEFW $0000
01A3 49 45 43      DEFM "IEC"
01A6 09 70      DEFW $7009

```

3.5 PRTXT: skriv text på skärmen

\$01A8

In: HL = text, BC = antal tecken.

Styrtecken:

- \$07 pip (ljudgeneratorn via port 6)
- \$08 markören ett steg vänster
- \$0A ny rad (rullar skärmen på sista raden)
- \$0C sudda skärmen
- \$0D markören till början av raden
- \$1B ESC "=" rad+32 kolumn+32: flytta markören (som ADM-3A)

Övriga tecken skrivs med bit 7 nollställd. Efter kolumn 39 blir det automatiskt ny rad.

```

01A8 DD E5      PRTXT: PUSH IX
01AA CD 93 02   L01AA: CALL CURADR      DE = markören, IX = $FDF3
01AD 79         L01AD: LD    A,C
01AE B0         OR     B
01AF CA E7 02   JP     Z,L02E7      slut: stäng
01B2 7E         LD     A,(HL)
01B3 23         INC    HL
01B4 0B         DEC    BC
01B5 FE 07      CP     $07           BEL
01B7 28 32      JR     Z,L01EB
01B9 FE 08      CP     $08           BS
01BB 28 5D      JR     Z,L021A
01BD FE 0A      CP     $0A           LF
01BF 28 25      JR     Z,L01E6
01C1 FE 0D      CP     $0D           CR
01C3 28 1B      JR     Z,L01E0
01C5 FE 0C      CP     $0C           FF

```

01C7	28 2B	JR	Z, L01F4	
01C9	FE 1B	CP	\$1B	ESC
01CB	28 2C	JR	Z, L01F9	
01CD	E6 7F	AND	\$7F	vanligt tecken
01CF	12	LD	(DE), A	rakt in i bildminnet
01D0	13	INC	DE	
01D1	DD 34 01	INC	(IX+\$01)	kolumn + 1
01D4	3A F4 FD	LD	A, (CURCOL)	
01D7	FE 28	CP	\$28	kolumn 40?
01D9	38 D2	JR	C, L01AD	
01DB	CD 34 02	CALL	CRLF	rad full: ny rad
01DE	18 CA	JR	L01AA	
01E0	DD 36 01 00	L01E0: LD	(IX+\$01), \$00	CR: kolumn 0
01E4	18 C4	JR	L01AA	
01E6	CD 38 02	L01E6: CALL	LINF	
01E9	18 BF	JR	L01AA	
01EB	AF	L01EB: XOR	A	pip
01EC	D3 06	OUT	(\$06), A	
01EE	3E 83	LD	A, \$83	styrord för tonen
01F0	D3 06	OUT	(\$06), A	
01F2	18 B6	JR	L01AA	
01F4	CD 76 02	L01F4: CALL	CLS	
01F7	18 B1	JR	L01AA	
01F9	EB	L01F9: EX	DE, HL	ESC: minst 3 tecken kvar?
01FA	21 FD FF	LD	HL, \$FFFD	
01FD	09	ADD	HL, BC	
01FE	EB	EX	DE, HL	
01FF	30 A9	JR	NC, L01AA	
0201	7E	LD	A, (HL)	
0202	FE 3D	CP	\$3D	ESC "="
0204	20 A4	JR	NZ, L01AA	
0206	23	INC	HL	hoppa över "="
0207	0B	DEC	BC	
0208	1E 18	LD	E, \$18	rad 0..23
020A	CD 27 02	CALL	ESCARG	
020D	DD 77 00	LD	(IX+\$00), A	rad
0210	1E 28	LD	E, \$28	kolumn 0..39
0212	CD 27 02	CALL	ESCARG	
0215	DD 77 01	LD	(IX+\$01), A	kolumn
0218	18 90	JR	L01AA	
021A	1B	L021A: DEC	DE	BS: kolumn - 1 om möjligt
021B	DD 35 01	DEC	(IX+\$01)	
021E	F2 AD 01	JP	P, L01AD	var inte 0: klart
0221	13	INC	DE	kolumn 0: ångra
0222	DD 34 01	INC	(IX+\$01)	
0225	18 86	JR	L01AD	

ESCARG: läs koordinat (tecken - 32), begränsad till 0..E-1.

0227	7E	ESCARG: LD	A, (HL)	
0228	23	INC	HL	
0229	0B	DEC	BC	
022A	D6 20	SUB	\$20	koordinat - 32
022C	30 01	JR	NC, L022F	
022E	AF	XOR	A	negativ: 0
022F	BB	L022F: CP	E	
0230	D8	RET	C	
0231	7B	LD	A, E	för stor: E - 1
0232	3D	DEC	A	
0233	C9	RET		

CRLF: markören till kolumn 0 och ny rad. **LINF:** ny rad. På rad 23 rullas skärmen: rad 1..23 kopieras upp ett steg via ROWTAB och sista raden suddas.

0234	DD 36 01 00	CRLF: LD	(IX+\$01), \$00	
------	-------------	----------	-----------------	--

0238	DD 34 00	LINF:	INC	(IX+\$00)	
023B	DD 7E 00		LD	A, (IX+\$00)	
023E	FE 18		CP	\$18	
0240	D8		RET	C	
0241	DD 36 00 17		LD	(IX+\$00), \$17	stanna på rad 23 och rulla
0245	DD 21 74 03		LD	IX, ROWTAB	rulla skärmen: kopiera rad n+1 till rad n
0249	3E 18		LD	A, \$18	
024B	E5		PUSH	HL	
024C	C5		PUSH	BC	
024D	01 28 00	L024D:	LD	BC, \$0028	40 tecken per rad
0250	DD 5E 00		LD	E, (IX+\$00)	DE = rad n
0253	DD 56 01		LD	D, (IX+\$01)	
0256	3D		DEC	A	
0257	28 10		JR	Z, L0269	sista raden: sudda den
0259	DD 6E 02		LD	L, (IX+\$02)	HL = rad n + 1
025C	DD 66 03		LD	H, (IX+\$03)	
025F	ED B0		LDIR		
0261	DD 23		INC	IX	
0263	DD 23		INC	IX	
0265	18 E6		JR	L024D	

CLRROW: fyll raden på DE (40 tecken) med blanktecken.

0267	E5	CLRROW:	PUSH	HL	
0268	C5		PUSH	BC	
0269	62	L0269:	LD	H, D	blanka i första tecknet ...
026A	6B		LD	L, E	
026B	36 20		LD	(HL), \$20	
026D	13		INC	DE	
026E	01 27 00		LD	BC, \$0027	
0271	ED B0		LDIR		... och kopiera det framåt
0273	C1		POP	BC	
0274	E1		POP	HL	
0275	C9		RET		

CLS: sudda alla 24 rader och sätt markören i övre vänstra hörnet.

0276	3E 18	CLS:	LD	A, \$18	24 rader
0278	DD 21 74 03		LD	IX, ROWTAB	
027C	DD 5E 00	L027C:	LD	E, (IX+\$00)	
027F	DD 56 01		LD	D, (IX+\$01)	
0282	DD 23		INC	IX	
0284	DD 23		INC	IX	
0286	CD 67 02		CALL	CLRROW	
0289	3D		DEC	A	
028A	20 F0		JR	NZ, L027C	
028C	5F		LD	E, A	A = 0: rad 0, kolumn 0
028D	57		LD	D, A	
028E	ED 53 F3 FD		LD	(CURROW), DE	
0292	C9		RET		

CURADR: DE = adressen i bildminnet för markörpositionen.

Ut: IX = \$FDF3, alltså IX+0 rad, IX+1 kolumn, IX+2 tangentflagga.

0293	E5	CURADR:	PUSH	HL	
0294	2A F3 FD		LD	HL, (CURROW)	L = rad, H = kolumn
0297	7C		LD	A, H	
0298	26 00		LD	H, \$00	
029A	29		ADD	HL, HL	två bytes per rad i ROWTAB
029B	11 74 03		LD	DE, ROWTAB	
029E	19		ADD	HL, DE	
029F	86		ADD	A, (HL)	radens adress + kolumnen
02A0	23		INC	HL	
02A1	66		LD	H, (HL)	
02A2	6F		LD	L, A	
02A3	EB		EX	DE, HL	
02A4	E1		POP	HL	
02A5	DD 21 F3 FD		LD	IX, \$FDF3	

02A9 C9

RET

3.6 INLINE: radinmatning från tangentbordet

\$02AA

In: HL = buffert, C = max antal tecken. Raden ekas på skärmen och avslutas med CR i bufferten.

CR avslutar
 \$18 (CTRL-X) suddar hela raden
 \$08 suddar ett tecken
 \$09 (TAB) hämtar nästa tecken ur mallraden som **EDPTR** pekar på
 (används av ED; **INLINE** sätter mallen till en tom rad)

INLIN2 tar mallens adress i DE.

02AA	11 4E 03	INLINE:	LD DE,\$034E	mall = \$0D inne i CP \$0D: tom
02AD	ED 53 FB FD	INLIN2:	LD (EDPTR),DE	
02B1	DD E5		PUSH IX	
02B3	41		LD B,C	B = platser kvar
02B4	CD 5B 03	L02B4:	CALL RUBOUT	radera allt inmatat
02B7	20 FB		JR NZ,L02B4	
02B9	CD 5B 03	L02B9:	CALL RUBOUT	
02BC	CD EA 02	L02BC:	CALL GETKEY	
02BF	FE 09		CP \$09	
02C1	CC 48 03		CALL Z,EDTAB	TAB: tecken från mallen
02C4	FE 0D		CP \$0D	
02C6	28 1D		JR Z,L02E5	
02C8	FE 18		CP \$18	CTRL-X
02CA	28 E8		JR Z,L02B4	
02CC	FE 08		CP \$08	BS
02CE	28 E9		JR Z,L02B9	
02D0	FE 20		CP \$20	övriga styrtecken ignoreras
02D2	38 E8		JR C,L02BC	
02D4	77		LD (HL),A	in i bufferten ...
02D5	23		INC HL	
02D6	12		LD (DE),A	... och på skärmen
02D7	DD 34 01		INC (IX+\$01)	kolumn + 1
02DA	DD 7E 01		LD A,(IX+\$01)	
02DD	FE 28		CP \$28	
02DF	D4 34 02		CALL NC,CRLF	kolumn 40: ny rad
02E2	10 D8		DJNZ L02BC	plats kvar: nästa tangent
02E4	2B		DEC HL	full: sista tecknet blir CR
02E5	36 0D	L02E5:	LD (HL),\$0D	
02E7	DD E1	L02E7:	POP IX	
02E9	C9		RET	

3.7 GETKEY: vänta på en tangent

\$02EA

Sätter bit 7 i tecknet under markören medan den väntar. Tangenten kommer från avbrottsrutinen **KEYINT**, som sätter bit 7 i **KEYFLG**. Autorepetition: så länge tangenten hålls ned (bit 7 i port \$38) räknas **RPTCNT/RPTDLY** ned. Första fördröjningen är 70*256 varv, och därefter upprepas tangenten var 8*256:e varv.

Ut: A = teckenkod (7 bitar).

02EA	FB	GETKEY:	EI	
02EB	CD 93 02		CALL CURADR	DE = markören, IX = \$FDF3
02EE	EB		EX DE,HL	
02EF	CB FE		SET 7,(HL)	markera markören
02F1	DD CB 02 7E	L02F1:	BIT 7,(IX+\$02)	tangent från avbrottet?
02F5	20 1B		JR NZ,L0312	
02F7	DB 38		IN A,(\$38)	hålls tangenten ned?
02F9	87		ADD A,A	bit 7 (nedtryckt) -> CY
02FA	38 06		JR C,L0302	
02FC	DD 36 04 46		LD (IX+\$04),\$46	nej: starta om fördröjningen
0300	18 EF		JR L02F1	
0302	DD 35 03	L0302:	DEC (IX+\$03)	inre räknare, 256 varv
0305	20 EA		JR NZ,L02F1	

0307	DD 35 04	DEC	(IX+\$04)	yttre: 70 första gången, sedan 8
030A	20 E5	JR	NZ, L02F1	
030C	DD 36 04 08	LD	(IX+\$04), \$08	repetitionstakt
0310	18 04	JR	L0316	
0312	DD CB 02 BE	L0312: RES	7, (IX+\$02)	
0316	DB 38	L0316: IN	A, (\$38)	tangentens kod utan strobe
0318	E6 7F	AND	\$7F	
031A	CB BE	RES	7, (HL)	ta bort markören
031C	EB	EX	DE, HL	
031D	C9	RET		

Tangentbordsavbrott (vektor \$34). CTRL-C (\$83 med strobebiten) sätter BRKFLG. Vid varje tangent skrivs \$80 till ljudporten och KEYFLG. Repetitionsfördröjningen startas om.

031E	F5	KEYINT: PUSH	AF	
031F	DB 38	IN	A, (\$38)	
0321	FE 83	CP	\$83	CTRL-C (med strobebiten)
0323	20 03	JR	NZ, L0328	
0325	32 07 FE	LD	(BRKFLG), A	
0328	3E 80	L0328: LD	A, \$80	A = \$80 både ...
032A	D3 06	OUT	(\$06), A	... till ljudet och ...
032C	32 F5 FD	LD	(KEYFLG), A	... som tangentflagga (bit 7)
032F	3E 46	LD	A, \$46	
0331	32 F7 FD	LD	(RPTDLY), A	
0334	F1	POP	AF	
0335	FB	EI		
0336	ED 4D	RETI		

SCRPOS: A = aktuell kolumn, E = radbredd (40), för skärmen.

0338	3A F4 FD	SCRPOS: LD	A, (CURCOL)
033B	1E 28	LD	E, \$28
033D	C9	RET	

TSTBRK: NZ om BREAK (CTRL-C) har tryckts. Flaggan nollställs.

033E	3A 07 FE	TSTBRK: LD	A, (BRKFLG)	
0341	A7	AND	A	
0342	3E 00	LD	A, \$00	A = 0 utan att röra flaggorna
0344	32 07 FE	LD	(BRKFLG), A	
0347	C9	RET		

EDTAB: TAB i INLINE, hämta nästa tecken ur mallraden.

0348	E5	EDTAB: PUSH	HL	mallens nästa tecken
0349	2A FB FD	LD	HL, (EDPTR)	
034C	7E	LD	A, (HL)	
034D	FE 0D	CP	\$0D	
034F	28 06	JR	Z, L0357	
0351	23	INC	HL	
0352	22 FB FD	LD	(EDPTR), HL	
0355	E1	POP	HL	
0356	C9	RET		
0357	3E 09	L0357: LD	A, \$09	mallen slut: ge TAB (ignoreras)
0359	E1	POP	HL	
035A	C9	RET		

RUBOUT: backa ett tecken (inte förbi radens början).

035B	78	RUBOUT: LD	A, B	inget inmatat: Z, inget görs
035C	B9	CP	C	
035D	C8	RET	Z	
035E	DD 35 01	DEC	(IX+\$01)	
0361	F2 6B 03	JP	P, L036B	kolumn >= 0: samma rad
0364	DD 35 00	DEC	(IX+\$00)	annars sista kolumnen på raden ovanför
0367	DD 36 01 27	LD	(IX+\$01), \$27	
036B	2B	L036B: DEC	HL	

036C	CD 93 02	CALL	CURADR	
036F	3E 20	LD	A, \$20	
0371	12	LD	(DE), A	sudda tecknet
0372	04	INC	B	en plats till; NZ
0373	C9	RET		

Bildminnets radadresser. Raderna ligger i tre grupper om 8. Inom en grupp är avståndet \$80, och grupperna börjar på \$7C00, \$7C28 och \$7C50.

0374	00 7C	ROWTAB:	DEFW \$7C00
0376	80 7C		DEFW \$7C80
0378	00 7D		DEFW \$7D00
037A	80 7D		DEFW \$7D80
037C	00 7E		DEFW \$7E00
037E	80 7E		DEFW \$7E80
0380	00 7F		DEFW \$7F00
0382	80 7F		DEFW \$7F80
0384	28 7C		DEFW \$7C28
0386	A8 7C		DEFW \$7CA8
0388	28 7D		DEFW \$7D28
038A	A8 7D		DEFW \$7DA8
038C	28 7E		DEFW \$7E28
038E	A8 7E		DEFW \$7EA8
0390	28 7F		DEFW \$7F28
0392	A8 7F		DEFW \$7FA8
0394	50 7C		DEFW \$7C50
0396	D0 7C		DEFW \$7CD0
0398	50 7D		DEFW \$7D50
039A	D0 7D		DEFW \$7DD0
039C	50 7E		DEFW \$7E50
039E	D0 7E		DEFW \$7ED0
03A0	50 7F		DEFW \$7F50
03A2	D0 7F		DEFW \$7FD0

3.8 Kassetten

\$03A4

Hopptabell för CAS: (och för standardenhetsen " "), se enhetslistan. Kassettfil:

Varje block är 256 bytes. Det skrivs som 256 nollbitar, 3 x SYN (\$16), STX (\$02), 256 databytes, ETX (\$03), en 16-bitars checksumma (summan av databytes + 3) och sist en byte \$00.
 Namnblock: \$FF \$FF \$FF + 11 tecken namn (8+3, blankutfyllt).
 Datablock: \$00, blocknummer (lo, hi), 253 bytes data.

Bufferten är \$FC00, eller vid läsning växelvis \$FC00/\$FB00. FCB-fälten (IX) som används:

+7 radbredd (\$84 = 132)
 +8,+9 buffertens dataområde (= \$FC03)
 +A,+B aktuell position i bufferten
 +D lediga bytes kvar i blocket
 +E bit 7: textdata har skrivits

03A4	C3 91 04	CASTAB:	JP	CASOPN
03A7	C3 BF 03	L03A7:	JP	CASPRE
03AA	C3 28 04	L03AA:	JP	CASCLO
03AD	C3 A0 09	L03AD:	JP	TXTINP
03B0	C3 12 09	L03B0:	JP	TXTPRT
03B3	C3 09 05	L03B3:	JP	CASGET
03B6	C3 0E 04	L03B6:	JP	CASPUT
03B9	C3 27 07	L03B9:	JP	L0727
03BC	C3 27 07	L03BC:	JP	L0727

PREPARE för kassett: bygg namnblocket i \$FC00 (namnet från DE), starta motorn, vänta, skriv namnblocket och nollställ blocknumret.

03BF	21 00 FC	CASPRE:	LD HL, \$FC00	namnblocket byggs i \$FC00
03C2	E5		PUSH HL	
03C3	36 00	L03C3:	LD (HL), \$00	nolla hela sidan
03C5	2C		INC L	
03C6	20 FB		JR NZ, L03C3	
03C8	06 03		LD B, \$03	\$FF \$FF \$FF = namnblock
03CA	36 FF	L03CA:	LD (HL), \$FF	
03CC	23		INC HL	
03CD	10 FB		DJNZ L03CA	
03CF	EB		EX DE, HL	
03D0	01 0B 00		LD BC, \$000B	11 tecken namn
03D3	ED B0		LDIR	
03D5	3E 20		LD A, \$20	motor på
03D7	D3 3A		OUT (\$3A), A	
03D9	06 0C		LD B, \$0C	vänta på att bandet kommer upp i fart
03DB	2B	L03DB:	DEC HL	12 x ca 65536 varv à 26 T, ca 6,8 s
03DC	7C		LD A, H	
03DD	B5		OR L	
03DE	20 FB		JR NZ, L03DB	
03E0	10 F9		DJNZ L03DB	
03E2	E1		POP HL	
03E3	CD 39 04		CALL CASWBL	skriv namnblocket
03E6	21 00 00		LD HL, \$0000	
03E9	22 F9 FD		LD (CASBUF), HL	CASBUF = 0: ingen mottagning
03EC	DD 36 0E 01		LD (IX+\$0E), \$01	+E = 1: filen öppen, ingen text än
03F0	2B		DEC HL	blocknummer = -1
03F1	22 FD FD		LD (CASBLK), HL	

Initiera FCB:ns buffertpekare till \$FC03 och 252 lediga bytes.

03F4	21 03 FC	CASBRS:	LD HL, \$FC03	
03F7	75	L03F7:	LD (HL), L	första databyten \$03 (ETX): tomt
03F8	DD 75 0A		LD (IX+\$0A), L	
03FB	DD 75 08		LD (IX+\$08), L	
03FE	DD 74 0B		LD (IX+\$0B), H	
0401	DD 74 09		LD (IX+\$09), H	
0404	DD 36 0D FC		LD (IX+\$0D), \$FC	252 lediga bytes
0408	DD 36 07 84		LD (IX+\$07), \$84	radbredd 132
040C	EB		EX DE, HL	DE = bufferten
040D	C9		RET	

PUT: öka blocknumret, skriv blockhuvudet (0, nr lo, nr hi) före data och skriv blocket.

040E	2A FD FD	CASPUT:	LD HL, (CASBLK)	
0411	23		INC HL	
0412	22 FD FD		LD (CASBLK), HL	
0415	EB		EX DE, HL	DE = blocknumret
0416	DD 6E 08		LD L, (IX+\$08)	
0419	DD 66 09		LD H, (IX+\$09)	
041C	2B		DEC HL	huvudet före data: 0, lo, hi
041D	72		LD (HL), D	
041E	2B		DEC HL	
041F	73		LD (HL), E	
0420	2B		DEC HL	
0421	36 00		LD (HL), \$00	
0423	CD 39 04		CALL CASWBL	
0426	18 CC		JR CASBRS	ny tom buffert

CLOSE: har textdata skrivits avslutas filen först (TXTEND). Motorn stängs av.

0428	DD 7E 0E	CASCLO:	LD A, (IX+\$0E)	
042B	A7		AND A	
042C	28 04		JR Z, L0432	
042E	87		ADD A, A	
042F	DC 85 09		CALL C, TXTEND	
0432	AF	L0432:	XOR A	
0433	DD 77 0E		LD (IX+\$0E), A	
0436	D3 3A		OUT (\$3A), A	A = 0: motor av
0438	C9		RET	

CASWBL: skriv blocket i sidan HL (256 bytes) med inledning, sync, STX, data, ETX och checksumma. Efter checksumman faller rutinen igenom in i CASWBY med C = 0 och skriver en nollbyte till: blocket blir $256 + 8 \times 264 = 2368$ bitar.

0439	0E 00	CASWBL:	LD	C, \$00	256 nollbitar som inledning
043B	CD 76 04	L043B:	CALL	CASW0	
043E	0D		DEC	C	
043F	20 FA		JR	NZ, L043B	
0441	1E 03		LD	E, \$03	3 x SYN
0443	0E 16	L0443:	LD	C, \$16	
0445	CD 6A 04		CALL	CASWBY	
0448	1D		DEC	E	
0449	20 F8		JR	NZ, L0443	
044B	0E 02		LD	C, \$02	STX
044D	CD 6A 04		CALL	CASWBY	
0450	11 03 00		LD	DE, \$0003	checksumman börjar på 3
0453	4E	L0453:	LD	C, (HL)	
0454	EB		EX	DE, HL	
0455	09		ADD	HL, BC	B = 0 efter CASHLF: bara C adderas
0456	EB		EX	DE, HL	
0457	CD 6A 04		CALL	CASWBY	
045A	2C		INC	L	sidan runt
045B	20 F6		JR	NZ, L0453	
045D	0E 03		LD	C, \$03	ETX
045F	CD 6A 04		CALL	CASWBY	
0462	4B		LD	C, E	checksumma lo ...
0463	CD 6A 04		CALL	CASWBY	
0466	4A		LD	C, D	... och hi
0467	CD 6A 04		CALL	CASWBY	faller igenom: en \$00-byte till

CASWBY: skriv byten C, minst signifikanta biten först. SCF före första RR C lägger en markeringsbit i bit 7. När den har skiftats ut är C = 0, och då har exakt 8 bitar skickats.

046A	37	CASWBY:	SCF	
046B	CB 19		RR	C
046D	CD 77 04	L046D:	CALL	CASWBI
0470	CB 19		RR	C
0472	20 F9		JR	NZ, L046D
0474	A7		AND	A
0475	C9		RET	

CASW0: skriv en nollbit. **CASWBI:** skriv biten i CY.

Skrivsignalen (bit 6 i port \$3A) vänds en gång per bitcell för en nolla. För en etta vänds den två gånger, med halv fördröjning emellan.

0476	AF	CASW0:	XOR	A	CY = 0: nolla
0477	3E 96	CASWBI:	LD	A, \$96	halv bitcell
0479	30 09		JR	NC, CASHLF	
047B	CB 3F		SRL	A	etta: två halva bitceller
047D	CD 84 04		CALL	CASHLF	
0480	3E 96		LD	A, \$96	
0482	CB 3F		SRL	A	
0484	47	CASHLF:	LD	B, A	
0485	10 FE	L0485:	DJNZ	L0485	
0487	47		LD	B, A	
0488	10 FE	L0488:	DJNZ	L0488	
048A	DB 3A		IN	A, (\$3A)	vänd skrivsignalen
048C	EE 40		XOR	\$40	
048E	D3 3A		OUT	(\$3A), A	
0490	C9		RET		

3.9 OPEN för kassett: leta efter filen

\$0491

Läs block tills ett namnblock (\$FF \$FF \$FF) med rätt namn hittas. Datablock hoppas över. Börjar det sökta namnet med blanktecken duger första filen, och dess namn skrivs ut som "FOUND namn.ext". Tidsgräns utan signal: A = \$AA.

0491	21 00 00	CASOPN:	LD HL,\$0000	
0494	22 F9 FD		LD (CASBUF),HL	
0497	D5	L0497:	PUSH DE	DE = namnet
0498	CD 2B 05	L0498:	CALL CASRBL	
049B	D1		POP DE	
049C	30 15		JR NC,L04B3	
049E	FE AA		CP \$AA	tidsgräns?
04A0	4F		LD C,A	
04A1	3E 00		LD A,\$00	
04A3	D3 3A		OUT (\$3A),A	motor av
04A5	79		LD A,C	
04A6	37		SCF	
04A7	C8		RET Z	tidsgräns: tillbaka med CY
04A8	7E		LD A,(HL)	
04A9	A7		AND A	
04AA	28 EB		JR Z,L0497	fel i ett datablock: läs vidare
04AC	AF	L04AC:	XOR A	
04AD	D3 3A		OUT (\$3A),A	
04AF	79		LD A,C	
04B0	C3 2E 06		JP ERROR	A = felkoden
04B3	01 A5 03	L04B3:	LD BC,\$03A5	B = 3, C = \$A5
04B6	7E		LD A,(HL)	
04B7	A7		AND A	
04B8	28 DD		JR Z,L0497	datablock: hoppa över
04BA	7E	L04BA:	LD A,(HL)	namnblock (\$FF \$FF \$FF)?
04BB	23		INC HL	
04BC	3C		INC A	
04BD	20 ED		JR NZ,L04AC	varken \$00 eller \$FF: fel 37
04BF	10 F9		DJNZ L04BA	
04C1	1A		LD A,(DE)	tomt namn: ta första filen
04C2	FE 20		CP \$20	
04C4	28 19		JR Z,L04DF	
04C6	D5		PUSH DE	
04C7	06 0B		LD B,\$0B	jämför 11 tecken
04C9	1A	L04C9:	LD A,(DE)	
04CA	AE		XOR (HL)	
04CB	20 CB		JR NZ,L0498	fel namn: nästa block
04CD	23		INC HL	
04CE	13		INC DE	
04CF	10 F8		DJNZ L04C9	
04D1	D1		POP DE	
04D2	AF	L04D2:	XOR A	
04D3	11 FF FF		LD DE,\$FFFF	
04D6	ED 53 FD FD		LD (CASBLK),DE	nästa block ska vara nr 0
04DA	2E 03		LD L,\$03	L = 3: \$03 först, tom buffert
04DC	C3 F7 03		JP L03F7	

Skriv "FOUND " och namnet. Punkten stoppas in genom att tecknen från position 8 flyttas ett steg framåt i bufferten.

04DF	E5	L04DF:	PUSH HL	
04E0	21 01 05		LD HL,D0501	
04E3	01 08 00		LD BC,\$0008	
04E6	CD A8 01		CALL PRTXT	
04E9	E1		POP HL	
04EA	E5		PUSH HL	
04EB	01 2E 03		LD BC,\$032E	C = ".", B = 3
04EE	2E 0B		LD L,\$0B	
04F0	7E	L04F0:	LD A,(HL)	typdelen ett steg, "." in
04F1	71		LD (HL),C	
04F2	4F		LD C,A	
04F3	23		INC HL	
04F4	10 FA		DJNZ L04F0	

```

04F6 71          LD  (HL),C
04F7 2E 03       LD  L,$03
04F9 0E 0C       LD  C,$0C      12 tecken: namn.typ
04FB CD A8 01    CALL PRTXT
04FE E1          POP  HL
04FF 18 D1       JR   L04D2

```

```

0501 0D          D0501: DEFB $0D
0502 0A 46 4F 55... DEFB $0A,"FOUND "

```

GET för kassett: läs nästa datablock. Blocket måste börja med \$00 och ha blocknummer CASBLK+1, annars fel 37.

```

0509 CD 2B 05    CASGET: CALL CASRBL
050C 4F          LD  C,A
050D 38 9D       JR   C,L04AC    fel: C = koden
050F ED 5B FD FD LD  DE,(CASBLK)    väntat blocknummer
0513 13          INC  DE
0514 ED 53 FD FD LD  (CASBLK),DE
0518 97          SUB  A
0519 01 A5 03    LD  BC,$03A5    B = 3, C = $A5 (fel 37)
051C AE          L051C: XOR  (HL)    0, nr lo, nr hi
051D 20 8D       JR   NZ,L04AC
051F 23          INC  HL
0520 7B          LD  A,E          A = väntad nästa byte
0521 5A          LD  E,D
0522 10 F8       DJNZ L051C
0524 DD 75 08    LD  (IX+$08),L    data efter huvudet
0527 DD 74 09    LD  (IX+$09),H
052A C9          RET

```

3.10 CASRBL: hämta ett block från den avbrottsstyrda läsaren

\$052B

Mottagningen sker under avbrott (CASINT/RCVBLK) medan programmet arbetar. CASST (\$FDF8) är \$FF medan ett block tas emot, och därefter resultatet:

0 = ok, \$A5 = formatfel (ETX saknas), \$A3 = checksummafel.

När ett block är klart startas mottagningen av nästa block i den andra bufferten. Ut: HL = bufferten, CY = 1 och A = felkod vid fel (\$AA = tidsgräns utan signal).

Tidsgränsen: DE räknar 65536 varv à $66 + 24 \times 13 + 8 = 386$ T-perioder, 25,3 miljoner T = ca 8,5 s vid 2,9952 MHz (NMI-klockan tar lite till). Version 9913 har \$3C i st. f. \$19 i den inre slingan: 841 T per varv, ca 18,4 s. I CASOPN blir tidsgränsen fel 21 (HITTAR EJ FILEN) via OPEN; kommer det block med andra namn fortsätter sökningen.

Christensen 1979 anger 10 s väntan på kassettfilen, vilket bör vara version 11273:s 8,5 s.

```

052B 2A F9 FD    CASRBL: LD  HL,(CASBUF)
052E 7C          LD  A,H          bara H provas (OR H)
052F B4          OR   H
0530 CC 56 05    CALL Z,RCVSTA    ingen mottagning igång: starta
0533 11 00 00    LD  DE,$0000
0536 ED 53 F9 FD LD  (CASBUF),DE    0: nästa anrop startar om
053A 1B          L053A: DEC  DE    DE = 0: 65536 varv
053B 06 19       LD  B,$19      tidsgräns
053D 10 FE       L053D: DJNZ L053D
053F 7A          LD  A,D
0540 B3          OR   E
0541 3E AA       LD  A,$AA
0543 37          SCF
0544 C8          RET  Z          tidsgräns: A = $AA, CY = 1
0545 3A F8 FD    LD  A,(CASST)    blocket klart?
0548 3C          INC  A
0549 28 EF       JR   Z,L053A    $FF: mottagning pågår
054B 3D          DEC  A
054C E5          PUSH HL
054D F5          PUSH AF
054E CC 56 05    CALL Z,RCVSTA    starta nästa block
0551 F1          POP  AF
0552 E1          POP  HL

```

0553	C0	RET	NZ	fel: A = koden, CY = 1 (SCF)
0554	AF	XOR	A	
0555	C9	RET		klart: A = 0, CY = 0, HL = blocket

RCVSTA: starta mottagning. Programmerar PIO B med vektor \$36 och avbrott på bit 7. Mottagaren startar i **RCVBLK**, som ligger i **RCVCONT**. Buffertsidan väljs växelvis: \$FC -> \$FB -> \$FC ...

0556	F3	RCVSTA:	DI	
0557	3E FF	LD	A, \$FF	
0559	32 F8 FD	LD	(CASST), A	
055C	3E 36	LD	A, \$36	PIO B: vektor \$36
055E	D3 3B	OUT	(\$3B), A	
0560	3E 97	LD	A, \$97	avbrott på, mask följer
0562	D3 3B	OUT	(\$3B), A	
0564	3E 7F	LD	A, \$7F	bara bit 7
0566	D3 3B	OUT	(\$3B), A	
0568	EB	EX	DE, HL	
0569	21 00 32	LD	HL, \$3200	hoppa över 50 bitar innan synkningen börjar
056C	22 01 FE	LD	(RCVBC), HL	
056F	21 D0 05	LD	HL, \$05D0	korutinen börjar i RCVBLK
0572	22 03 FE	LD	(RCVCONT), HL	
0575	EB	EX	DE, HL	
0576	7C	LD	A, H	förra bufferten; H = 0: ingen
0577	A7	AND	A	
0578	20 05	JR	NZ, L057F	
057A	21 00 FC	LD	HL, \$FC00	
057D	18 03	JR	L0582	
057F	EE 07	L057F:	XOR \$07	\$FC <-> \$FB
0581	67	LD	H, A	
0582	22 F9 FD	L0582:	LD (CASBUF), HL	
0585	22 FF FD	LD	(RCVPTR), HL	
0588	36 AA	LD	(HL), \$AA	markering: ingen data än
058A	3E 20	LD	A, \$20	
058C	D3 3A	OUT	(\$3A), A	
058E	3E 60	LD	A, \$60	motor på, bit 6 = 1
0590	D3 3A	OUT	(\$3A), A	
0592	FB	EI		
0593	C9	RET		

3.11 CASINT: avbrott från kassetten, ett per bit

\$0594

Avbrottet kommer på en flank i början av bitcellen. Rutinen pulserar bit 6 i port \$3A (troligen för att nollställa en flanklatch i kassettgränssnittet), väntar ungefär 3/4 bitcell och läser bit 7.

CY = inverterad bit 7 = den mottagna biten (?). Biten skiftas in i **RCVBC** (C). När B bitar är mottagna hoppar rutinen till adressen i **RCVCONT** med A = byten och DE = **RCVPTR**. **RCVSUM** summerar mottagna bytes.

Mottagarens huvudprogram (**RCVBLK**) fungerar alltså som en korutin: **RCVBYT/RCVBIT** sparar returadressen i **RCVCONT** och avslutar avbrottet. Nästa gång B bitar är klara fortsätter programmet efter anropet.

0594	FB	CASINT:	EI	
0595	F5	PUSH	AF	
0596	C5	PUSH	BC	
0597	DB 3A	IN	A, (\$3A)	bit 6 = 0 ...
0599	E6 BF	AND	\$BF	
059B	D3 3A	OUT	(\$3A), A	
059D	4F	LD	C, A	
059E	3E 78	LD	A, \$78	
05A0	47	LD	B, A	
05A1	10 FE	L05A1:	DJNZ L05A1	
05A3	47	LD	B, A	
05A4	3E 40	LD	A, \$40	
05A6	B1	OR	C	
05A7	D3 3A	OUT	(\$3A), A	... bit 6 = 1
05A9	10 FE	L05A9:	DJNZ L05A9	
05AB	47	LD	B, A	
05AC	DB 3A	IN	A, (\$3A)	

05AE	87	ADD	A, A	CY = inverterad bit 7
05AF	3F	CCF		
05B0	79	LD	A, C	
05B1	D3 3A	OUT	(\$3A), A	
05B3	78	LD	A, B	
05B4	D3 3A	OUT	(\$3A), A	
05B6	ED 4B 01 FE	LD	BC, (RCVBC)	
05BA	CB 19	RR	C	skifta in biten
05BC	10 68	DJNZ	L0626	fler bitar i byten
05BE	79	LD	A, C	
05BF	E5	PUSH	HL	
05C0	D5	PUSH	DE	
05C1	2A 05 FE	LD	HL, (RCVSUM)	summera byten
05C4	09	ADD	HL, BC	B = 0 efter DJNZ: bara C
05C5	22 05 FE	LD	(RCVSUM), HL	
05C8	2A 03 FE	LD	HL, (RCVCONT)	
05CB	ED 5B FF FD	LD	DE, (RCVPTR)	
05CF	E9	JP	(HL)	fortsätt korutinen

RCVBLK: synka bit för bit på SYN (\$16), vänta på STX (\$02), ta emot 256 bytes till (DE), kontrollera ETX och checksumman och lägg resultatet i CASST. Sedan stängs PIO B-avbrottet av.

05D0	06 01	RCVBLK:	LD	B, \$01	sök SYN bit för bit
05D2	CD 1C 06		CALL	RCVBIT	B = 1: en bit i taget
05D5	FE 16		CP	\$16	
05D7	20 F7		JR	NZ, RCVBLK	
05D9	CD 1A 06	L05D9:	CALL	RCVBYT	vänta bort resten av SYN
05DC	FE 16		CP	\$16	
05DE	28 F9		JR	Z, L05D9	
05E0	FE 02		CP	\$02	STX?
05E2	20 EC		JR	NZ, RCVBLK	
05E4	21 00 00		LD	HL, \$0000	
05E7	22 05 FE		LD	(RCVSUM), HL	
05EA	CD 1A 06	L05EA:	CALL	RCVBYT	256 databytes
05ED	12		LD	(DE), A	DE = RCVPTR
05EE	1C		INC	E	
05EF	20 F9		JR	NZ, L05EA	
05F1	CD 1A 06		CALL	RCVBYT	
05F4	FE 03		CP	\$03	ETX?
05F6	3E A5		LD	A, \$A5	fel 37 (formatfel)
05F8	20 12		JR	NZ, L060C	
05FA	ED 5B 05 FE		LD	DE, (RCVSUM)	
05FE	CD 1A 06		CALL	RCVBYT	
0601	AB		XOR	E	jämför checksumman
0602	5F		LD	E, A	
0603	CD 1A 06		CALL	RCVBYT	
0606	AA		XOR	D	
0607	B3		OR	E	
0608	28 02		JR	Z, L060C	
060A	3E A3		LD	A, \$A3	fel 35 (checksummafel)
060C	32 F8 FD	L060C:	LD	(CASST), A	
060F	3E 07		LD	A, \$07	PIO B: avbrott av
0611	D3 3B		OUT	(\$3B), A	
0613	DB 3A		IN	A, (\$3A)	
0615	E6 BF		AND	\$BF	
0617	D3 3A		OUT	(\$3A), A	
0619	C5		PUSH	BC	skenadress; avbrotten är av
061A	06 08	RCVBYT:	LD	B, \$08	
061C	E1	RCVBIT:	POP	HL	korutin: spara fortsättningsadressen
061D	22 03 FE		LD	(RCVCONT), HL	
0620	ED 53 FF FD		LD	(RCVPTR), DE	
0624	D1		POP	DE	DE, HL från \$05BF/\$05C0
0625	E1		POP	HL	
0626	ED 43 01 FE	L0626:	LD	(RCVBC), BC	B = bitar kvar, C = byten
062A	C1		POP	BC	
062B	F1		POP	AF	
062C	ED 4D		RETI		

3.12 Felhantering

\$062E

ERROR: A = felkod (\$80 + nummer). Om ett program körs, felet finns i listan **ERRTRP** och **ON ERROR GOTO** är aktivt (\$FE35 <> 0), sparas koden i \$FE38. Stacken återställs och programmet fortsätter på **ON ERROR**-raden.

Annars **ERRSTP**: stacken återställs, radnumret för den rad som kördes letas fram och "ERR n LINE m" skrivs ut. Sedan går man tillbaka till kommandoloopen. Med A = 0 skrivs "STOP" i stället.

Om bit 0 i **EXTFLG** är satt får tilläggs-ROM:en (\$6051) skriva ut felet (till exempel som text).

062E 5F	ERROR: LD E,A	
062F FD 7E 0E	LD A,(IY+\$0E)	direktläge: inget ON ERROR
0632 A7	AND A	
0633 7B	LD A,E	
0634 20 33	JR NZ, ERRSTP	direktläge: stoppa
0636 21 59 06	LD HL, ERRTRP	kan felet fångas?
0639 01 10 00	LD BC,\$0010	
063C ED B1	CPIR	
063E 20 29	JR NZ, ERRSTP	inte i ERRTRP : stoppa
0640 FD 56 1F	LD D,(IY+\$1F)	ON ERROR GOTO aktivt?
0643 7A	LD A,D	
0644 B7	OR A	
0645 7B	LD A,E	
0646 28 21	JR Z, ERRSTP	
0648 FD 73 22	LD (IY+\$22),E	felkoden sparas för ERRCODE
064B FD 5E 1E	LD E,(IY+\$1E)	
064E FD 36 1F 00	LD (IY+\$1F),\$00	ON ERROR gäller en gång
0652 2A 36 FE	LD HL,(STMSP)	
0655 F9	LD SP,HL	SP som vid satsens början
0656 C3 71 0D	JP RUNLIN	fortsätt på ON ERROR -raden

*Felkoder som kan fångas med **ON ERROR GOTO**.*

0659 87 84 8C 9E...	ERRTRP: DEFB \$87,\$84,\$8C,\$9E,\$A2,\$93,\$95,\$A3	
0661 A4 A5 A6 A7...	DEFB \$A4,\$A5,\$A6,\$A7,\$A8,\$A9,\$AA,\$AB	
0669 2A 27 FE	ERRSTP: LD HL,(STKTOP)	
066C F9	LD SP,HL	
066D F5	PUSH AF	felkoden på stacken
066E AF	XOR A	
066F 6F	LD L,A	
0670 67	LD H,A	
0671 FD B6 0E	OR (IY+\$0E)	HL = 0 i direktläge: inget radnr
0674 20 1E	JR NZ, L0694	
0676 2A 1C FE	LD HL,(BOFA)	leta rätt på raden som kördes
0679 44	L0679: LD B,H	BC = radens början
067A 4D	LD C,L	
067B 5E	LD E,(HL)	
067C 16 00	LD D,\$00	
067E 19	ADD HL,DE	HL = nästa rad
067F 7B	LD A,E	
0680 EB	EX DE,HL	
0681 6C	LD L,H	HL = 0 (H var 0)
0682 FE 01	CP \$01	längd 1: programslut, inget radnr
0684 28 0E	JR Z, L0694	
0686 2A 2C FE	LD HL,(CURLIN)	
0689 ED 52	SBC HL,DE	CURLIN >= nästa rad: vidare
068B EB	EX DE,HL	
068C 30 EB	JR NC, L0679	
068E 03	INC BC	radnumret efter längdbyten
068F 0A	LD A,(BC)	
0690 6F	LD L,A	
0691 03	INC BC	
0692 0A	LD A,(BC)	
0693 67	LD H,A	
0694 F1	L0694: POP AF	
0695 CD 9B 06	CALL ERRMSG	
0698 C3 CC 00	JP READY	

ERRMSG: bygg meddelandet på stacken (80 bytes): CR LF BEL, "STOP" eller "ERR n", "LINE m" om HL <> 0. Skriv sedan ut

<i>det.</i>			
069B	EB	ERRMSG: EX DE,HL	
069C	21 B0 FF	LD HL,\$FFB0	80 bytes buffert på stacken
069F	39	ADD HL,SP	
06A0	F9	LD SP,HL	
06A1	E5	PUSH HL	
06A2	36 0D	LD (HL),\$0D	CR LF BEL först
06A4	23	INC HL	
06A5	36 0A	LD (HL),\$0A	
06A7	23	INC HL	
06A8	36 07	LD (HL),\$07	
06AA	23	INC HL	
06AB	EB	EX DE,HL	
06AC	E5	PUSH HL	radnumret
06AD	A7	AND A	
06AE	20 0A	JR NZ,L06BA	A = 0: STOP
06B0	21 3A 07	LD HL,D073A	
06B3	01 04 00	LD BC,\$0004	
06B6	ED B0	LDIR	
06B8	18 1A	JR L06D4	
06BA	21 0F FE	L06BA: LD HL,\$FE0F	tilläggs-ROM skriver felet?
06BD	CB 46	BIT 0,(HL)	
06BF	37	SCF	
06C0	C4 51 60	CALL NZ,V6051	NC tillbaka: tillägget skrev felet
06C3	30 0F	JR NC,L06D4	
06C5	21 3E 07	LD HL,D073E	
06C8	01 04 00	LD BC,\$0004	
06CB	ED B0	LDIR	
06CD	E6 7F	AND \$7F	felnummer
06CF	6F	LD L,A	
06D0	60	LD H,B	
06D1	CD 62 18	CALL PRNUM	
06D4	E1	L06D4: POP HL	
06D5	7C	LD A,H	
06D6	B5	OR L	
06D7	28 0D	JR Z,L06E6	HL = 0: ingen radtext
06D9	E5	PUSH HL	
06DA	21 34 07	LD HL,D0734	
06DD	01 06 00	LD BC,\$0006	
06E0	ED B0	LDIR	
06E2	E1	POP HL	
06E3	CD 62 18	CALL PRNUM	
06E6	E1	L06E6: POP HL	
06E7	EB	EX DE,HL	
06E8	B7	OR A	
06E9	ED 52	SBC HL,DE	BC = textens längd
06EB	4D	LD C,L	
06EC	44	LD B,H	
06ED	EB	EX DE,HL	
06EE	CD A8 01	CALL PRTXT	
06F1	21 50 00	LD HL,\$0050	bufferten bort
06F4	39	ADD HL,SP	
06F5	F9	LD SP,HL	
06F6	C9	RET	

Felingångar: en RST \$10 för varje felkod, så att man kan hoppa hit med JP.

06F7	D7	L06F7: RST \$10	
06F8	B5	DEFB \$B5	fel 53: FELAKTIG RAD
06F9	D7	L06F9: RST \$10	
06FA	80	DEFB \$80	fel 0: EJ TILLÅTET ÖKA "DIM"
06FB	D7	L06FB: RST \$10	
06FC	81	DEFB \$81	fel 1: FEL ANTAL INDEX
06FD	D7	L06FD: RST \$10	
06FE	82	DEFB \$82	fel 2: OTILLÅTET SOM KOMMANDO

06FF D7	L06FF:	RST \$10	
0700 83		DEFB \$83	fel 3: MINNET FULLT
0701 D7	L0701:	RST \$10	
0702 84		DEFB \$84	fel 4: FÖR STORT FLYTTAL
0703 D7	L0703:	RST \$10	
0704 85		DEFB \$85	fel 5: FÖR STORT INDEX
0705 D7	L0705:	RST \$10	
0706 86		DEFB \$86	fel 6: HITTAR EJ DETTA RADNUMMER
0707 D7	L0707:	RST \$10	
0708 87		DEFB \$87	fel 7: FÖR STORT HELTAL
0709 D7	L0709:	RST \$10	
070A 88		DEFB \$88	fel 8: FINNS EJ I DETTA SYSTEM
070B D7	L070B:	RST \$10	
070C 89		DEFB \$89	fel 9: INDEX UTANFÖR STRÄNGEN
070D D7	L070D:	RST \$10	
070E 8A		DEFB \$8A	fel 10: TEXTEN FÅR EJ PLATS I STRÄNGEN
070F D7	L070F:	RST \$10	
0710 95		DEFB \$95	fel 21: HITTAR EJ FILEN
0711 D7	L0711:	RST \$10	
0712 96		DEFB \$96	fel 22: OTILLÅTEN SATS
0713 D7	L0713:	RST \$10	
0714 8C		DEFB \$8C	fel 12: FELAKTIGT TAL
0715 D7	L0715:	RST \$10	
0716 98		DEFB \$98	fel 24: "NEXT" SAKNAS
0717 D7	L0717:	RST \$10	
0718 9C		DEFB \$9C	fel 28: FEL VARIABEL EFTER "NEXT"
0719 D7	L0719:	RST \$10	
071A 9B		DEFB \$9B	fel 27: "NEXT" UTAN "FOR"
071B D7	L071B:	RST \$10	
071C 9D		DEFB \$9D	fel 29: "RETURN" UTAN "GOSUB"
071D D7	L071D:	RST \$10	
071E 9A		DEFB \$9A	fel 26: FEL I ON-UTTRYCK
071F D7	L071F:	RST \$10	
0720 9E		DEFB \$9E	fel 30: DATA SLUT
0721 D7	L0721:	RST \$10	
0722 9F		DEFB \$9F	fel 31: FEL DATA TILL KOMMANDO
0723 D7	L0723:	RST \$10	
0724 A0		DEFB \$A0	fel 32: FILEN EJ ÖPPNAD
0725 D7	L0725:	RST \$10	
0726 B2		DEFB \$B2	fel 50: KVADRATROT UR NEGATIVT TAL
0727 D7	L0727:	RST \$10	
0728 B4		DEFB \$B4	fel 52: EJ TILL DENNA ENHET
0729 D7	L0729:	RST \$10	
072A B9		DEFB \$B9	fel 57: FUNKTIONEN EJ DEFINIERAD
072B D7	L072B:	RST \$10	
072C BC		DEFB \$BC	fel 60: BIT ADRESS 16 BITAR
072D 0D	D072D:	DEFB \$0D	
072E 0A 41 42 43...		DEFB \$0A, "ABC80"	
0734 20 4C 49 4E...	D0734:	DEFB " LINE "	
073A 53 54 4F 50	D073A:	DEFB "STOP"	
073E 45 52 52 20	D073E:	DEFB "ERR "	

NEWLIN: skriv CR LF.

0742 D5	NEWLIN:	PUSH DE
0743 E5		PUSH HL

0744	21 2D 07	LD	HL, D072D	
0747	01 02 00	LD	BC, \$0002	CR LF, början av D072D
074A	CD A8 01	CALL	PRTXT	
074D	E1	POP	HL	
074E	D1	POP	DE	
074F	C9	RET		

3.13 INSLIN: lägg in, ersätt eller radera en programrad

\$0750

IX pekar på den tokeniserade raden (+0 längd, +1,+2 radnummer).

Raden ska in där **FINDLN** hittar numret eller första högre nummer. Resten av programmet, till och med \$01:an på **EOFA**, flyttas med skillnaden mellan ny och gammal längd (gammal = 0 för en ny rad): **LDDR** uppåt om raden växer, **LDIR** nedåt om den krymper; **EOFA** följer med. Längd 0 (bara radnummer) raderar raden. Före en flyttning uppåt görs **CLEAR**, så att **HEAP** = **EOFA** + 1, och **MEMRES** kontrollerar minnet. Räcker det inte blir det via **CMPEER** tillbaka med **CY** = 1 och **A** = \$83 (fel 3, MINNET FULLT); programmet är då orört. Därefter kopieras raden in, och **CLEAR** görs eftersom variablerna ligger efter programmet.

Ut: **CY** = 0; **Z** = 1 om raden blev sist (**MERGE** fortsätter på **NZ**).

0750	DD 5E 01	INSLIN:	LD	E, (IX+\$01)	
0753	DD 56 02		LD	D, (IX+\$02)	
0756	CD 39 0F		CALL	FINDLN	finns radnummer?
0759	01 00 00		LD	BC, \$0000	
075C	20 01		JR	NZ, L075F	
075E	4E		LD	C, (HL)	C = gamla radens längd
075F	DD 7E 00	L075F:	LD	A, (IX+\$00)	
0762	91		SUB	C	A = ny - gammal längd
0763	28 30		JR	Z, L0795	lika lång: skriv över
0765	38 42		JR	C, L07A9	kortare: flytta ned
0767	E5		PUSH	HL	
0768	47		LD	B, A	
0769	C5		PUSH	BC	
076A	CD 80 0A		CALL	CLEAR	HEAP = EOFA + 1
076D	16 00		LD	D, \$00	
076F	C1		POP	BC	
0770	C5		PUSH	BC	
0771	58		LD	E, B	
0772	CD 5A 33		CALL	MEMRES	plats i minnet?
0775	C1		POP	BC	
0776	3E 83		LD	A, \$83	fel 3, MINNET FULLT
0778	DA F7 1B		JP	C, CMPEER	
077B	78		LD	A, B	
077C	06 00		LD	B, \$00	
077E	EB		EX	DE, HL	
077F	E1		POP	HL	
0780	E5		PUSH	HL	
0781	09		ADD	HL, BC	
0782	D5		PUSH	DE	
0783	EB		EX	DE, HL	
0784	ED 52		SBC	HL, DE	BC = raderna efter, till EOFA
0786	4D		LD	C, L	
0787	44		LD	B, H	
0788	D1		POP	DE	
0789	6F		LD	L, A	
078A	26 00		LD	H, \$00	
078C	1B		DEC	DE	
078D	19		ADD	HL, DE	
078E	22 1E FE		LD	(EOFA), HL	nytt slut = gammalt + skillnaden
0791	EB		EX	DE, HL	
0792	ED B8		LDDR		flytta upp, bakifrån
0794	E1	L0794:	POP	HL	
0795	EB	L0795:	EX	DE, HL	
0796	DD E5		PUSH	IX	
0798	E1		POP	HL	
0799	4E		LD	C, (HL)	
079A	79		LD	A, C	

079B	A7	AND	A	
079C	28 02	JR	Z, L07A0	längd 0: raden raderas
079E	ED B0	LDIR		
07A0	1A	L07A0:	LD A, (DE)	
07A1	FE 01	CP	\$01	Z: raden är sist
07A3	F5	PUSH	AF	
07A4	CD 80 0A	CALL	CLEAR	
07A7	F1	POP	AF	
07A8	C9	RET		
07A9	E5	L07A9:	PUSH HL	
07AA	09	ADD	HL, BC	
07AB	EB	EX	DE, HL	
07AC	2A 1E FE	LD	HL, (EOFA)	
07AF	ED 52	SBC	HL, DE	
07B1	44	LD	B, H	
07B2	4D	LD	C, L	
07B3	03	INC	BC	
07B4	26 FF	LD	H, \$FF	HL = skillnaden (negativ)
07B6	6F	LD	L, A	
07B7	19	ADD	HL, DE	
07B8	EB	EX	DE, HL	
07B9	ED B0	LDIR		flytta ned, framifrån
07BB	1B	DEC	DE	
07BC	ED 53 1E FE	LD	(EOFA), DE	EOFA = den flyttade \$01:an
07C0	18 D2	JR	L0794	

3.14 OPEN: öppna en fil

\$07C2

OPNSTR: filnamnet står i programtexten på HL, inom " eller ', eller fram till CR. Ut till **OPEN:** HL = namnet, C = längd.

OPEN: A = typ av öppning:

- 0 OPEN
- 1 PREPARE
- 2 för LOAD: prova namn.BAC, och finns den inte namn.BAS
- 3 PREPARE med .BAS som standard (text, t.ex. LIST till fil)
- 4+ PREPARE med .BAC som standard (SAVE)

Namnet tolkas (via \$1478) till 8+3 tecken och ett enhetsnamn, som letas upp i enhetslistan (fel 21 om enheten saknas). Det tolkade namnet ligger i 14 bytes på stacken, och enhetens rutin får det i DE. FCB-fältet +3,+4 pekar sedan på posten i listan (på hopptabellspekaren), +9 = 0, och efter öppningen +5 = 1 (öppen), +6 = 0, +E = 0. Svarar enheten CY blir det fel 21, vad den än lagt i A; bara typ 2 prövar .BAS när .BAC gav CY med A = 0.

07C2	F5	OPNSTR:	PUSH AF	
07C3	7E	LD	A, (HL)	
07C4	23	INC	HL	
07C5	FE 22	CP	\$22	
07C7	28 07	JR	Z, L07D0	
07C9	FE 27	CP	\$27	
07CB	28 03	JR	Z, L07D0	
07CD	2B	DEC	HL	utan citattecken: till CR
07CE	3E 0D	LD	A, \$0D	
07D0	47	L07D0:	LD B, A	
07D1	0E FF	LD	C, \$FF	C = längden
07D3	E5	PUSH	HL	
07D4	7E	L07D4:	LD A, (HL)	
07D5	23	INC	HL	
07D6	0C	INC	C	
07D7	B8	CP	B	
07D8	28 04	JR	Z, L07DE	
07DA	FE 0D	CP	\$0D	CR slutar alltid
07DC	20 F6	JR	NZ, L07D4	
07DE	06 00	L07DE:	LD B, \$00	
07E0	E1	POP	HL	
07E1	F1	POP	AF	
07E2	EB	OPEN:	EX DE, HL	
07E3	21 F2 FF	LD	HL, \$FFF2	14 bytes för FNAME på stacken

07E6	39		ADD HL,SP	
07E7	F9		LD SP,HL	
07E8	EB		EX DE,HL	
07E9	DD E5		PUSH IX	
07EB	F5		PUSH AF	
07EC	D5		PUSH DE	
07ED	CD 78 14		CALL FNAME	tolka filnamnet
07F0	DD E1		POP IX	IX = det tolkade namnet
07F2	38 78		JR C,L086C	felaktigt namn: fel 21
07F4	ED 5B 0A FE		LD DE,(DEVHEAD)	gå igenom enhetslistan
07F8	7A	L07F8:	LD A,D	
07F9	B3		OR E	
07FA	CA 0F 07		JP Z,L070F	fel 21, enheten finns inte
07FD	EB		EX DE,HL	
07FE	CD 1D 10		CALL RDDE	
0801	DD 7E 0B		LD A,(IX+\$0B)	jämför enhetsnamnet
0804	BE		CP (HL)	
0805	23		INC HL	
0806	20 F0		JR NZ,L07F8	
0808	DD 7E 0C		LD A,(IX+\$0C)	
080B	BE		CP (HL)	
080C	23		INC HL	
080D	20 E9		JR NZ,L07F8	
080F	DD 7E 0D		LD A,(IX+\$0D)	
0812	BE		CP (HL)	
0813	23		INC HL	
0814	20 E2		JR NZ,L07F8	
0816	DD E5		PUSH IX	
0818	D1		POP DE	DE = namnet, till enheten
0819	F1		POP AF	
081A	DD E1		POP IX	
081C	DD 75 03		LD (IX+\$03),L	FCB+3,+4 = posten
081F	DD 74 04		LD (IX+\$04),H	
0822	DD 36 09 00		LD (IX+\$09),\$00	
0826	A7		AND A	A = typ av öppning
0827	28 40		JR Z,L0869	
0829	3D		DEC A	
082A	28 1A		JR Z,L0846	
082C	3D		DEC A	
082D	28 1C		JR Z,L084B	
082F	3D		DEC A	
0830	21 D1 08		LD HL,D08D1	typ 3: .BAS
0833	28 03		JR Z,L0838	
0835	21 CE 08		LD HL,D08CE	typ 4+: .BAC
0838	E5	L0838:	PUSH HL	
0839	21 08 00		LD HL,\$0008	
083C	19		ADD HL,DE	
083D	7E		LD A,(HL)	
083E	E1		POP HL	
083F	FE 20		CP \$20	
0841	20 03		JR NZ,L0846	typ angiven: behåll
0843	CD BD 08		CALL SETEXT	
0846	CD 8D 08	L0846:	CALL DVPREP	PREPARE
0849	18 24		JR L086F	
084B	21 08 00	L084B:	LD HL,\$0008	typ 2: filtyp angiven?
084E	19		ADD HL,DE	
084F	7E		LD A,(HL)	
0850	FE 20		CP \$20	
0852	20 15		JR NZ,L0869	
0854	21 CE 08		LD HL,D08CE	nej: prova .BAC
0857	CD BD 08		CALL SETEXT	
085A	CD 8A 08		CALL DVOPEN	
085D	30 0D		JR NC,L086C	fanns: klart
085F	A7		AND A	
0860	C2 0F 07		JP NZ,L070F	A <> 0: fel 21 direkt
0863	21 D1 08		LD HL,D08D1	finns inte: prova .BAS
0866	CD BD 08		CALL SETEXT	
0869	CD 8A 08	L0869:	CALL DVOPEN	OPEN

086C	DA 0F 07	L086C:	JP	C, L070F	CY från enheten: alltid fel 21
086F	DD 36 05 01	L086F:	LD	(IX+\$05), \$01	filen är öppen
0873	21 0E 00		LD	HL, \$000E	
0876	DD 74 0E		LD	(IX+\$0E), H	
0879	DD 74 06		LD	(IX+\$06), H	
087C	39		ADD	HL, SP	de 14 bytena bort
087D	F9		LD	SP, HL	
087E	C9		RET		

CLOSE: stäng filen i FCB IX om den är öppen. FCB+5 nollas före enhetens CLOSE, så filen räknas som stängd även om enheten svarar CY.

087F	DD 7E 05	CLOSE:	LD	A, (IX+\$05)	
0882	A7		AND	A	
0883	C8		RET	Z	redan stängd
0884	DD 36 05 00		LD	(IX+\$05), \$00	stängd före enhetens CLOSE
0888	18 06		JR	DVCLOS	

3.15 DVCALL: anropa enhetsrutin nummer 0-8

\$088A

Raderna CALL **DVCALL** ligger efter varandra. Att anropa **DVOPEN**, **DVPREP**, ... ger alltså returadresser som skiljer sig med 3. **DVCALL** räknar ut rutinnumret (x3) som returadress - \$088D och lägger det till hopptabellens adress, som hämtas via FCB+3,+4. Står det inte JP (\$C3) där blir det fel 8. Övriga register bevaras.

088A	CD A5 08	DVOPEN:	CALL	DVCALL	
088D	CD A5 08	DVPREP:	CALL	DVCALL	
0890	CD A5 08	DVCLOS:	CALL	DVCALL	
0893	CD A5 08	DVINP:	CALL	DVCALL	
0896	CD A5 08	DVPRT:	CALL	DVCALL	
0899	CD A5 08	DVGET:	CALL	DVCALL	
089C	CD A5 08	DVPUT:	CALL	DVCALL	
089F	CD A5 08		CALL	DVCALL	
08A2	CD A5 08		CALL	DVCALL	
08A5	E3	DVCALL:	EX	(SP), HL	HL = returadressen
08A6	D5		PUSH	DE	
08A7	EB		EX	DE, HL	
08A8	DD 6E 03		LD	L, (IX+\$03)	
08AB	DD 66 04		LD	H, (IX+\$04)	
08AE	7E		LD	A, (HL)	HL = hopptabellen
08AF	23		INC	HL	
08B0	66		LD	H, (HL)	
08B1	6F		LD	L, A	
08B2	19		ADD	HL, DE	
08B3	11 8D 08		LD	DE, \$088D	rutinnummer x 3
08B6	A7		AND	A	
08B7	ED 52		SBC	HL, DE	
08B9	D1		POP	DE	
08BA	C3 84 01		JP	L0184	JP där? hoppa

SETEXT: kopiera 3 tecken från HL som filtyp (namn+8) i DE.

08BD	D5	SETEXT:	PUSH	DE	
08BE	E5		PUSH	HL	
08BF	21 08 00		LD	HL, \$0008	
08C2	19		ADD	HL, DE	
08C3	EB		EX	DE, HL	
08C4	E1		POP	HL	
08C5	C5		PUSH	BC	
08C6	01 03 00		LD	BC, \$0003	
08C9	ED B0		LDIR		
08CB	C1		POP	BC	
08CC	D1		POP	DE	
08CD	C9		RET		
08CE	42 41 43	D08CE:	DEFB	"BAC"	
08D1	42 41 53	D08D1:	DEFB	"BAS"	

PRINT, **INPUT** och **GETPOS** går till skärmen/tangentbordet, eller till filen om ett filnummer är valt (\$FE23 <> 0, CURFIL).

PRINT HL = text, BC = längd

INPUT läs en rad, fel 34 vid filslut, annars enhetens fel

GETPOS A = kolumn, E = radbredd

08D4	CD FC 08	PRINT:	CALL CURFIL	
08D7	D2 A8 01		JP NC,PRTXT	ingen fil: skärmen
08DA	C3 96 08		JP DVPRT	
08DD	CD FC 08	INPUT:	CALL CURFIL	
08E0	D2 AA 02		JP NC,INLINE	
08E3	CD 93 08		CALL DVINP	
08E6	D0		RET NC	raden läst
08E7	A7		AND A	
08E8	20 02		JR NZ,L08EC	
08EA	3E A2		LD A,\$A2	A = 0: filslut, fel 34
08EC	C3 2E 06	L08EC:	JP ERROR	
08EF	CD FC 08	GETPOS:	CALL CURFIL	
08F2	D2 38 03		JP NC,SCRPOS	
08F5	DD 7E 06		LD A,(IX+\$06)	fil: A = kolumn, E = radbredd
08F8	DD 5E 07		LD E,(IX+\$07)	
08FB	C9		RET	

CURFIL: Z om inget filnummer är valt. Annars IX = FCB för filen, CY = 1. Är filen inte öppen blir det fel 32.

08FC	FD 7E 0D	CURFIL:	LD A,(IX+\$0D)	
08FF	A7		AND A	
0900	C8		RET Z	Z, NC: ingen fil vald
0901	C5		PUSH BC	
0902	CD E1 20		CALL FNDFIL	leta FCB för filnumret
0905	C2 23 07		JP NZ,L0723	fel 32, FILEN EJ ÖPPNAD
0908	DD CB 05 46		BIT 0,(IX+\$05)	bit 0: öppen?
090C	CA 23 07		JP Z,L0723	
090F	C1		POP BC	
0910	37		SCF	NZ, CY: IX = FCB
0911	C9		RET	

3.16 TXTPRT: skriv text till blockad fil (enhetsrutin 4 för CAS)

\$0912

Tecknen läggs i blockbufferten. LF tas bort. En följd av blanktecken lagras som \$09 + antal. Blocket slutar med \$03, och när det är fullt skrivs det med PUT. FCB+6 är aktuell kolumn.

Två eller fler blanktecken före CR stryks helt; ett ensamt står kvar. Efter en följd av blanktecken som följs av ett annat tecken än CR blir kolumnen ett steg för stor (prövat). \$09 n läggs aldrig i blockets två sista bytes: då skrivs blocket först.

0912	DD 5E 0A	TXTPRT:	LD E,(IX+\$0A)	
0915	DD 56 0B		LD D,(IX+\$0B)	
0918	78	L0918:	LD A,B	BC = kvar av texten
0919	B1		OR C	
091A	28 27		JR Z,L0943	
091C	7E		LD A,(HL)	
091D	FE 0A		CP \$0A	LF tas bort
091F	28 1E		JR Z,L093F	
0921	DD 34 06		INC (IX+\$06)	kolumn + 1
0924	FE 0D		CP \$0D	CR: kolumn 0
0926	20 04		JR NZ,L092C	
0928	DD 36 06 00		LD (IX+\$06),\$00	
092C	FE 20	L092C:	CP \$20	flera blanktecken?
092E	20 05		JR NZ,L0935	
0930	23		INC HL	
0931	BE		CP (HL)	
0932	2B		DEC HL	
0933	28 19		JR Z,L094E	
0935	ED A0	L0935:	LDI	tecknet till bufferten
0937	DD 35 0D		DEC (IX+\$0D)	blocket fullt?
093A	CC 95 09		CALL Z,XTBLK	

093D	18 D9	JR	L0918	
093F	23	L093F:	INC HL	
0940	0B		DEC BC	
0941	18 D5	JR	L0918	
0943	DD 73 0A	L0943:	LD (IX+\$0A),E	spara positionen
0946	DD 72 0B		LD (IX+\$0B),D	
0949	DD 36 0E 80		LD (IX+\$0E),\$80	text skriven: CLOSE gör TXTEND
094D	C9		RET	
094E	D5	L094E:	PUSH DE	
094F	1E 00		LD E,\$00	E = antal blanka
0951	1C	L0951:	INC E	
0952	23		INC HL	
0953	0B		DEC BC	
0954	78		LD A,B	
0955	B1		OR C	
0956	28 0C		JR Z,L0964	
0958	DD 34 06		INC (IX+\$06)	även tecknet efter: kolumnen 1 för stor
095B	7E		LD A,(HL)	
095C	FE 0D		CP \$0D	
095E	28 22		JR Z,L0982	CR: blankorna stryks
0960	FE 20		CP \$20	
0962	28 ED		JR Z,L0951	
0964	3E 02	L0964:	LD A,\$02	högst 2 lediga: nytt block först
0966	DD BE 0D		CP (IX+\$0D)	
0969	7B		LD A,E	
096A	D1		POP DE	
096B	F5		PUSH AF	
096C	D4 95 09		CALL NC,TXTBLK	
096F	F1		POP AF	
0970	EB		EX DE,HL	
0971	36 09		LD (HL),\$09	\$09 + antal blanktecken
0973	23		INC HL	
0974	77		LD (HL),A	
0975	23		INC HL	
0976	EB		EX DE,HL	
0977	DD 35 0D		DEC (IX+\$0D)	två bytes mindre
097A	DD 35 0D		DEC (IX+\$0D)	
097D	CC 95 09		CALL Z,TXTBLK	
0980	18 96	JR	L0918	
0982	D1	L0982:	POP DE	
0983	18 93	JR	L0918	

TXTEND: skriv aktuellt block. Lägg sedan 6 nollbytes (slutmarkering) först i ett nytt block och skriv det också.

TXTBLK: avsluta blocket med \$03 och skriv det (PUT).

0985	DD 5E 0A	TXTEND:	LD E,(IX+\$0A)	
0988	DD 56 0B		LD D,(IX+\$0B)	
098B	CD 95 09		CALL TXTBLK	
098E	06 06		LD B,\$06	6 nollbytes = filslut
0990	AF		XOR A	
0991	12	L0991:	LD (DE),A	
0992	13		INC DE	
0993	10 FC		DJNZ L0991	sedan in i TXTBLK: \$03 efter nollorna
0995	C5	TXTBLK:	PUSH BC	
0996	E5		PUSH HL	
0997	EB		EX DE,HL	HL = bufferten
0998	36 03		LD (HL),\$03	blockslut
099A	CD 9C 08		CALL DVPUT	PUT
099D	E1		POP HL	DE = nya bufferten, från enheten
099E	C1		POP BC	
099F	C9		RET	

3.17 TXTINP: läs en textrad från blockad fil (enhetsrutin 3 för CAS)

\$09A0

Kopierar till DE tills CR. \$03 = blockslut: nästa block hämtas med GET. \$09 n expanderas till n blanktecken. \$00 = filslut (A = 0, CY = 1). Tecken >= \$80 ger fel 58, och för lång rad ger fel 20.

In: HL = mottagaren, C = platsen. Raden med CR måste vara kortare än C, annars fel 20 (prövat). Positionen i FCB:t sparas bara efter CR, och ett LF i filen följer med raden.

09A0	EB	TXTINP:	EX	DE,HL	
09A1	DD 36 0D 01		LD	(IX+\$0D), \$01	lediga = 1 (?)
09A5	DD 6E 0A		LD	L, (IX+\$0A)	HL = positionen i blocket
09A8	DD 66 0B		LD	H, (IX+\$0B)	
09AB	7E	L09AB:	LD	A, (HL)	
09AC	FE 03		CP	\$03	blockslut: läs nästa block
09AE	28 1D		JR	Z, L09CD	
09B0	FE 09		CP	\$09	komprimerade blanktecken
09B2	28 22		JR	Z, L09D6	
09B4	A7		AND	A	\$00: filslut
09B5	37		SCF		
09B6	C8		RET	Z	
09B7	3E BA		LD	A, \$BA	fel 58
09B9	F8		RET	M	
09BA	7E		LD	A, (HL)	
09BB	06 00		LD	B, \$00	C = platser kvar
09BD	ED A0		LDI		
09BF	E2 E9 09		JP	P0, L09E9	BC = 0: fel 20, även med CR sist
09C2	FE 0D		CP	\$0D	till och med CR
09C4	20 E5		JR	NZ, L09AB	
09C6	DD 74 0B		LD	(IX+\$0B), H	positionen sparas bara här
09C9	DD 75 0A		LD	(IX+\$0A), L	
09CC	C9		RET		
09CD	C5	L09CD:	PUSH	BC	
09CE	D5		PUSH	DE	
09CF	CD 99 08		CALL	DVGET	HL = nästa block
09D2	D1		POP	DE	
09D3	C1		POP	BC	
09D4	18 D5		JR	L09AB	
09D6	23	L09D6:	INC	HL	
09D7	46		LD	B, (HL)	
09D8	23		INC	HL	
09D9	79		LD	A, C	ryms n blanka?
09DA	90		SUB	B	
09DB	38 0C		JR	C, L09E9	
09DD	28 0A		JR	Z, L09E9	
09DF	4F		LD	C, A	C minskas i förväg
09E0	EB		EX	DE,HL	
09E1	36 20	L09E1:	LD	(HL), \$20	
09E3	23		INC	HL	
09E4	10 FB		DJNZ	L09E1	
09E6	EB		EX	DE,HL	
09E7	18 C2		JR	L09AB	
09E9	3E 94	L09E9:	LD	A, \$94	fel 20, FÖR LÅNG RAD
09EB	37		SCF		
09EC	C9		RET		
09ED	1B	L09ED:	DEC	DE	tillbaka till tecknet som skilde
09EE	13	L09EE:	INC	DE	
09EF	1A		LD	A, (DE)	
09F0	87		ADD	A, A	
09F1	30 FB		JR	NC, L09EE	fram till nästa posts kod (bit 7)
09F3	E1		POP	HL	texten från början igen
09F4	C1		POP	BC	

3.18 KWSRCH: sök nyckelord i tabell

\$09F5

DE = tabell, HL = text. Varje post är en kod med bit 7 satt, följd av ordet. Tabellen slutar med \$FF. Bokstäver jämförs utan hänsyn till skiftläge, och blanktecken i texten hoppas över.

Ut: Z och A = kod om ordet hittades (HL efter ordet), annars NZ.

09F5	1A	KWSRCH:	LD	A, (DE)	
09F6	FE FF		CP	\$FF	tabellens slut?
09F8	20 02		JR	NZ, L09FC	
09FA	A7		AND	A	ja: NZ = inte hittat
09FB	C9		RET		
09FC	C5	L09FC:	PUSH	BC	
09FD	E5		PUSH	HL	spara textens början
09FE	4F		LD	C, A	C = postens kod
09FF	13		INC	DE	
0A00	1A	L0A00:	LD	A, (DE)	
0A01	FE 41		CP	\$41	bokstav? jämför utan skiftläge
0A03	38 09		JR	C, L0A0E	
0A05	FE 5E		CP	\$5E	
0A07	30 05		JR	NC, L0A0E	
0A09	AE		XOR	(HL)	
0A0A	E6 DF		AND	\$DF	bortse från bit 5, versal/gemen
0A0C	18 01		JR	L0A0F	
0A0E	AE	L0A0E:	XOR	(HL)	
0A0F	F5	L0A0F:	PUSH	AF	
0A10	CC BE 25		CALL	Z, SKIPSP	lika: nästa tecken (hoppa över blanktecken)
0A13	F1		POP	AF	
0A14	1A		LD	A, (DE)	nästa tabellbyte, flaggorna står kvar
0A15	13		INC	DE	
0A16	28 E8		JR	Z, L0A00	lika: fortsätt i ordet
0A18	87		ADD	A, A	sista tecknet i ordet (bit 7 i nästa kod)?
0A19	30 D2		JR	NC, L09ED	nej: olika mitt i ordet, nästa post
0A1B	79		LD	A, C	hittat: A = koden
0A1C	C1		POP	BC	släpp textens början, HL efter ordet
0A1D	C1		POP	BC	
0A1E	BF		CP	A	Z = hittat
0A1F	C9		RET		

Direktkommandon. Koden är index i CMDTAB (NEW och SCR = samma).

0A20	80 42 59 45	CMDKW:	DEFB	\$80, "BYE"
0A24	81 52 55 4E		DEFB	\$81, "RUN"
0A28	82 4C 49 53...		DEFB	\$82, "LIST"
0A2D	83 4E 45 57		DEFB	\$83, "NEW"
0A31	83 53 43 52		DEFB	\$83, "SCR"
0A35	84 43 4C 45...		DEFB	\$84, "CLEAR"
0A3B	85 4C 4F 41...		DEFB	\$85, "LOAD"
0A40	86 4D 45 52...		DEFB	\$86, "MERGE"
0A46	87 52 45 4E		DEFB	\$87, "REN"
0A4A	88 45 44		DEFB	\$88, "ED"
0A4D	89 53 41 56...		DEFB	\$89, "SAVE"
0A52	8A 55 4E 53...		DEFB	\$8A, "UNSAVE"
0A59	FF		DEFB	\$FF
0A5A	7A 01	CMDTAB:	DEFW	L017A
0A5C	4C 0D		DEFW	CMDRUN
0A5E	EF 0A		DEFW	CMDLIST
0A60	70 0A		DEFW	NEW
0A62	80 0A		DEFW	CLEAR
0A64	A8 0B		DEFW	CMDLOAD
0A66	A9 0B		DEFW	CMDMERG
0A68	8A 0C		DEFW	CMDREN
0A6A	A1 0A		DEFW	CMDDED
0A6C	C6 0D		DEFW	CMDSAVE
0A6E	4C 01		DEFW	L014C

NEW (och **SCR**): töm programmet (**EOFA** = **BOFA** och slutmarkering \$01), **TRACE** av och inte längre skyddat (**FLAGS** bit 4, 5). Sedan

CLEAR. **CLEAR:** stäng alla filer och töm fillistan (**FILLST:**s höga byte = 0), **VARLST** = 0, **LNKOK** = 0, **ON ERROR** av (**ONERR:**s höga byte = 0), **ERRCOD** = 0, **HEAP** = **EOFA**+1.

0A70	FD CB 0F A6	NEW:	RES 4, (IY+\$0F)	TRACE av
0A74	FD CB 0F AE		RES 5, (IY+\$0F)	inte skyddat
0A78	2A 1C FE		LD HL, (BOFA)	
0A7B	22 1E FE		LD (EOFA), HL	
0A7E	36 01		LD (HL), \$01	slutmarkering
0A80	DD E5	CLEAR:	PUSH IX	
0A82	CD CD 20		CALL CLSALL	stäng alla filer
0A85	DD E1		POP IX	
0A87	21 00 00		LD HL, \$0000	
0A8A	FD 74 1D		LD (IY+\$1D), H	FILLST: s höga byte = 0: tom
0A8D	22 29 FE		LD (VARLST), HL	
0A90	FD 75 10		LD (IY+\$10), L	LNKOK = 0
0A93	FD 75 1F		LD (IY+\$1F), L	ON ERROR av
0A96	FD 75 22		LD (IY+\$22), L	ERRCOD = 0
0A99	2A 1E FE		LD HL, (EOFA)	HEAP = EOFA +1
0A9C	23		INC HL	
0A9D	22 20 FE		LD (HEAP), HL	
0AA0	C9		RET	

3.19 ED n: redigera rad n

\$0AA1

Raden detokeniseras (\$1195) i 126 bytes på stacken och skrivs ut.

Den blir mall för **INLIN2**: varje TAB hämtar mallens nästa tecken, i tur och ordning och oberoende av vad som skrivits ("2" + 11 TAB på raden "10 PRINT 12345" ger "210 PRINT 12"). Den nya raden (i **BUF1**, högst 120 tecken) körs sedan som om den hade skrivits in (\$00F4), så den kan lika gärna vara ett kommando. Inget tal eller något efter talet: fel 31. Raden saknas: fel 6.

0AA1	CD 42 07	CMDED:	CALL NEWLIN	
0AA4	CD 1E 18		CALL GETLN	
0AA7	DA 21 07		JP C, L0721	inget tal: fel 31
0AAA	7E		LD A, (HL)	
0AAB	FE 0D		CP \$0D	
0AAD	C2 21 07		JP NZ, L0721	
0AB0	CD 39 0F		CALL FINDLN	
0AB3	C2 05 07		JP NZ, L0705	raden saknas: fel 6
0AB6	EB		EX DE, HL	
0AB7	21 82 FF		LD HL, \$FF82	126 bytes på stacken
0ABA	39		ADD HL, SP	
0ABB	F9		LD SP, HL	
0ABC	EB		EX DE, HL	
0ABD	D5		PUSH DE	
0ABE	CD 95 11		CALL DETOK	mallen slutar med CR
0AC1	EB		EX DE, HL	
0AC2	D1		POP DE	
0AC3	D5		PUSH DE	
0AC4	36 0A		LD (HL), \$0A	LF efter
0AC6	23		INC HL	
0AC7	ED 52		SBC HL, DE	
0AC9	44		LD B, H	
0ACA	4D		LD C, L	
0ACB	EB		EX DE, HL	
0ACC	CD A8 01		CALL PRTXT	
0ACF	D1		POP DE	
0AD0	21 40 FE		LD HL, \$FE40	BUF1 , högst 120 tecken
0AD3	0E 78		LD C, \$78	
0AD5	E5		PUSH HL	
0AD6	CD AD 02		CALL INLIN2	TAB hämtar ur mallen (DE)
0AD9	21 80 00		LD HL, \$0080	
0ADC	39		ADD HL, SP	
0ADD	D1		POP DE	
0ADE	F9		LD SP, HL	
0ADF	EB		EX DE, HL	
0AE0	C3 F4 00		JP L00F4	kör raden som om den skrivits in

0AE3	DD 36 0F 01	L0AE3:	LD (IX+\$0F), \$01	till fil: hela programmet
0AE7	3E 03		LD A, \$03	
0AE9	CD C2 07		CALL OPNSTR	
0AEC	AF		XOR A	
0AED	18 25		JR L0B14	

3.20 LIST [n][-m] eller LIST fil

\$0AEF

IX = 16 bytes på stacken: IX+0 = radräknare (22 rader per sida), IX+F = 1 om listningen går till en fil (OPEN typ 3, .BAS).

Ett filnamn börjar med " eller ' eller en bokstav. Till fil skrivs alltid hela programmet (ett radintervall efter namnet läses inte), utan sidvisning; varje rad slutar med CR LF.

På skärmen: n, n-m, n,m, n- (till slutet) och -m (från början). Efter 22 rader väntar LIST på en tangent: CR avbryter, andra tangenter visar en rad till. BREAK avbryter också.

CMDLIST:				
0AEF	DD 21 F0 FF		LD IX, \$FFF0	IX = 16 bytes på stacken
0AF3	DD 39		ADD IX, SP	
0AF5	DD F9		LD SP, IX	
0AF7	DD 36 00 16		LD (IX+\$00), \$16	
0AFB	DD 36 0F 00		LD (IX+\$0F), \$00	
0AFF	7E		LD A, (HL)	filnamn efter LIST?
0B00	FE 22		CP \$22	
0B02	28 06		JR Z, L0B0A	
0B04	FE 27		CP \$27	
0B06	28 02		JR Z, L0B0A	
0B08	FE 41		CP \$41	
0B0A	30 D7	L0B0A:	JR NC, L0AE3	citattecken eller bokstav: fil
0B0C	E5		PUSH HL	
0B0D	CD 42 07		CALL NEWLIN	
0B10	E1		POP HL	
0B11	7E		LD A, (HL)	
0B12	FE 0D		CP \$0D	
0B14	11 00 00	L0B14:	LD DE, \$0000	från 0 till 65535
0B17	01 FF FF		LD BC, \$FFFF	
0B1A	28 1E		JR Z, L0B3A	
0B1C	CD 1E 18		CALL GETLN	DE = första raden
0B1F	D5		PUSH DE	
0B20	7E		LD A, (HL)	
0B21	FE 0D		CP \$0D	
0B23	28 12		JR Z, L0B37	
0B25	FE 2D		CP \$2D	"_"
0B27	28 05		JR Z, L0B2E	
0B29	FE 2C		CP \$2C	","
0B2B	C2 21 07		JP NZ, L0721	
0B2E	DF	L0B2E:	RST \$18	
0B2F	CD 1E 18		CALL GETLN	
0B32	30 03		JR NC, L0B37	
0B34	11 FF FF		LD DE, \$FFFF	inget tal efter -: till slutet
0B37	42	L0B37:	LD B, D	BC = sista raden
0B38	4B		LD C, E	
0B39	D1		POP DE	
0B3A	C5	L0B3A:	PUSH BC	
0B3B	CD 39 0F		CALL FINDLN	HL = första raden >= DE
0B3E	C1	L0B3E:	POP BC	
0B3F	7E		LD A, (HL)	programmets slut?
0B40	EE 01		XOR \$01	
0B42	28 57		JR Z, L0B9B	
0B44	CD 3E 03		CALL TSTBRK	BREAK?
0B47	20 52		JR NZ, L0B9B	
0B49	54		LD D, H	
0B4A	5D		LD E, L	
0B4B	23		INC HL	
0B4C	7E		LD A, (HL)	
0B4D	23		INC HL	

0B4E	66		LD	H, (HL)	
0B4F	6F		LD	L, A	
0B50	ED 42		SBC	HL, BC	radnummer > BC: slut
0B52	38 02		JR	C, L0B56	
0B54	20 45		JR	NZ, L0B9B	
0B56	EB	L0B56:	EX	DE, HL	
0B57	C5		PUSH	BC	
0B58	11 40 FE		LD	DE, \$FE40	detokenisera i BUF1
0B5B	D5		PUSH	DE	
0B5C	DD E5		PUSH	IX	
0B5E	CD 95 11		CALL	DETOK	detokenisera raden
0B61	DD E1		POP	IX	
0B63	E3		EX	(SP), HL	
0B64	EB		EX	DE, HL	
0B65	36 0A		LD	(HL), \$0A	LF efter CR
0B67	23		INC	HL	
0B68	A7		AND	A	
0B69	ED 52		SBC	HL, DE	
0B6B	EB		EX	DE, HL	
0B6C	4B		LD	C, E	
0B6D	42		LD	B, D	
0B6E	DD E5		PUSH	IX	
0B70	DD CB 0F 46		BIT	0, (IX+\$0F)	fil eller skärm
0B74	28 05		JR	Z, L0B7B	
0B76	CD 96 08		CALL	DVPRT	
0B79	18 03		JR	L0B7E	
0B7B	CD A8 01	L0B7B:	CALL	PRTXT	
0B7E	DD E1	L0B7E:	POP	IX	
0B80	E1		POP	HL	
0B81	DD CB 0F 46		BIT	0, (IX+\$0F)	
0B85	20 B7		JR	NZ, L0B3E	till fil: ingen sidvisning
0B87	DD 35 00		DEC	(IX+\$00)	22 rader, sedan vänta
0B8A	20 B2		JR	NZ, L0B3E	
0B8C	DD 34 00		INC	(IX+\$00)	räkaren 1: en rad i taget
0B8F	DD E5		PUSH	IX	
0B91	CD EA 02		CALL	GETKEY	CR avbryter, annan tangent: en rad till
0B94	DD E1		POP	IX	
0B96	FE 0D		CP	\$0D	
0B98	20 A4		JR	NZ, L0B3E	annan tangent: en rad till
0B9A	C1		POP	BC	
0B9B	DD CB 0F 46	L0B9B:	BIT	0, (IX+\$0F)	
0B9F	C4 7F 08		CALL	NZ, CLOSE	stäng filen
0BA2	21 10 00		LD	HL, \$0010	
0BA5	39		ADD	HL, SP	
0BA6	F9		LD	SP, HL	
0BA7	C9		RET		

3.21 LOAD och MERGE

\$0BA8

CMDLOAD börjar på \$0BA8, där bytes \$3E \$AF blir LD A,\$AF (A <> 0 -> NEW först). CMDMERG börjar på \$0BA9 med XOR A (A = 0, inget NEW).

Filen öppnas med OPEN typ 2. Filen läses först som text (INPUT), rad för rad. Varje rad tokeniseras och läggs in som om den hade skrivits in.

En rad som inte går att tokenisera får felutskrift och sparas ändå, som token \$95 + hela textraden (med numret; LIST visar "20 ??? 20 A="). En rad utan radnummer ger fel 16; raderna före står kvar.

Ger INPUT fel 58 (tecken >= \$80) är det en BAC-fil med tokeniserade rader. Då ska buffertens första byte vara \$82 eller \$83 (fel 59 annars). \$83 = skyddat program och sätter bit 5 i \$FE25.

0BA8	3E		DEFB	\$3E	LOAD: A = \$AF, MERGE: A = 0; LD A,\$AF (överlappar nästa)
0BA9	AF		XOR	A	
0BAA	DD 21 F1 FF		LD	IX, \$FFF1	
0BAE	DD 39		ADD	IX, SP	

0BB0	DD F9		LD SP,IX	
0BB2	08		EX AF,AF'	
0BB3	3E 02		LD A,\$02	OPEN typ 2 (BAC eller BAS)
0BB5	CD C2 07		CALL OPNSTR	
0BB8	08		EX AF,AF'	
0BB9	A7		AND A	
0BBA	C4 70 0A		CALL NZ,NEW	LOAD: NEW först
0BBD	21 40 FE	L0BBD:	LD HL,\$FE40	BUF1, högst 120 tecken
0BC0	0E 78		LD C,\$78	
0BC2	E5		PUSH HL	
0BC3	CD 93 08		CALL DVINP	läs en textrad
0BC6	E1		POP HL	
0BC7	38 44		JR C,L0C0D	
0BC9	E7		RST \$20	tom rad hoppas över
0BCA	FE 0D		CP \$0D	
0BCC	28 EF		JR Z,L0BBD	
0BCE	DD E5		PUSH IX	
0BD0	E5		PUSH HL	
0BD1	CD 90 18		CALL TOKLIN	tokenisera
0BD4	D1		POP DE	
0BD5	DD 6E 01		LD L,(IX+\$01)	
0BD8	DD 66 02		LD H,(IX+\$02)	
0BDB	30 23		JR NC,L0C00	NC: tokeniserad
0BDD	E5		PUSH HL	
0BDE	D5		PUSH DE	
0BDF	DD E5		PUSH IX	
0BE1	CD 9B 06		CALL ERRMSG	felet skrivs ut, raden sparas ändå
0BE4	D1		POP DE	
0BE5	21 03 00		LD HL,\$0003	
0BE8	19		ADD HL,DE	
0BE9	EB		EX DE,HL	
0BEA	E3		EX (SP),HL	
0BEB	3E 95		LD A,\$95	\$95 + hela textraden
0BED	12		LD (DE),A	
0BEE	13		INC DE	
0BEF	7E	L0BEF:	LD A,(HL)	
0BF0	ED A0		LDI	
0BF2	FE 0D		CP \$0D	
0BF4	20 F9		JR NZ,L0BEF	
0BF6	E1		POP HL	
0BF7	E5		PUSH HL	
0BF8	DD E1		POP IX	
0BFA	7B		LD A,E	ny längd
0BFB	95		SUB L	
0BFC	DD 77 00		LD (IX+\$00),A	
0BFF	E1		POP HL	
0C00	7C	L0C00:	LD A,H	radnummer 0: saknas
0C01	B5		OR L	
0C02	3E 90		LD A,\$90	fel 16, RADNUMMER SAKNAS
0C04	28 17		JR Z,L0C1D	
0C06	CD 50 07		CALL INSLIN	
0C09	DD E1		POP IX	
0C0B	30 B0		JR NC,L0BBD	CY: INSLIN:s fel
0C0D	FE BA	L0C0D:	CP \$BA	\$BA = binärdata: det är en BAC-fil
0C0F	28 16		JR Z,L0C27	
0C11	A7		AND A	A = 0: textfilens slut
0C12	20 0B		JR NZ,L0C1F	
0C14	CD 7F 08		CALL CLOSE	klart
0C17	21 0F 00		LD HL,\$000F	
0C1A	39		ADD HL,SP	
0C1B	F9		LD SP,HL	
0C1C	C9		RET	
0C1D	DD E1	L0C1D:	POP IX	
0C1F	F5	L0C1F:	PUSH AF	
0C20	CD 7F 08		CALL CLOSE	
0C23	F1		POP AF	
0C24	C3 69 06		JP ERRSTP	stäng och fel

BAC-fil: bufferten ska börja med \$82 (vanligt program) eller \$83 (skyddat program, bit 5 i **FLAGS** sätts; i direktläget raderas det sedan vid **READY**), annars fel 59. En buffertadress 0 i **FCB+8/9** ger fel 52. Sedan följer rader i samma format som i programminnet. Längdbyte \$00 = slut på blocket (nästa block läses med **GET**), \$01 = slut på filen.

Raderna läggs först in med **INSLIN**, som sorterar in dem. När **INSLIN** säger att raden hamnade sist (Z) går det över till en snabbare slinga på \$0C5D, som bara lägger raderna efter varandra i slutet. Resten av filen sorteras alltså inte: MERGE av raderna 10, 30, 25 till ett program med 5 och 20 ger 5, 10, 20, 30, 25.

0C27	DD 66 09	L0C27:	LD	H, (IX+\$09)	
0C2A	7C		LD	A, H	
0C2B	DD 6E 08		LD	L, (IX+\$08)	
0C2E	A7		AND	A	
0C2F	3E B4		LD	A, \$B4	fel 52: ingen buffert
0C31	28 DA		JR	Z, L0C0D	
0C33	7E		LD	A, (HL)	
0C34	FE 82		CP	\$82	BAC, typ \$82
0C36	28 0A		JR	Z, L0C42	
0C38	FE 83		CP	\$83	typ \$83: skyddat program
0C3A	3E BB		LD	A, \$BB	fel 59: okänd typ
0C3C	20 CF		JR	NZ, L0C0D	
0C3E	FD CB 0F EE		SET	5, (IY+\$0F)	skyddat program
0C42	23	L0C42:	INC	HL	
0C43	7E	L0C43:	LD	A, (HL)	
0C44	A7		AND	A	
0C45	28 39		JR	Z, L0C80	
0C47	EE 01		XOR	\$01	\$01 = filslut
0C49	28 C2		JR	Z, L0C0D	
0C4B	E5		PUSH	HL	
0C4C	DD E3		EX	(SP), IX	IX = raden
0C4E	4E		LD	C, (HL)	
0C4F	06 00		LD	B, \$00	
0C51	09		ADD	HL, BC	
0C52	E5		PUSH	HL	
0C53	CD 50 07		CALL	INSLIN	sortera in raden
0C56	E1		POP	HL	
0C57	DD E1		POP	IX	
0C59	38 B2		JR	C, L0C0D	fel 3 från INSLIN
0C5B	20 E6		JR	NZ, L0C43	NZ: fler rader finns efter, fortsatt sortera
0C5D	7E	L0C5D:	LD	A, (HL)	resten läggs sist, osorterat
0C5E	A7		AND	A	
0C5F	28 24		JR	Z, L0C85	
0C61	5F		LD	E, A	
0C62	16 00		LD	D, \$00	
0C64	EE 01		XOR	\$01	
0C66	28 A5		JR	Z, L0C0D	
0C68	E5		PUSH	HL	
0C69	CD 5A 33		CALL	MEMRES	plats för raden?
0C6C	2B		DEC	HL	
0C6D	3E 83		LD	A, \$83	
0C6F	38 9C		JR	C, L0C0D	fel 3, MINNET FULLT
0C71	42		LD	B, D	
0C72	4B		LD	C, E	
0C73	EB		EX	DE, HL	
0C74	E1		POP	HL	
0C75	ED B0		LDIR		lägg raden sist
0C77	EB		EX	DE, HL	
0C78	22 1E FE		LD	(EOFA), HL	ny EOFA och slutmarkering
0C7B	36 01		LD	(HL), \$01	
0C7D	EB		EX	DE, HL	
0C7E	18 DD		JR	L0C5D	
0C80	CD 99 08	L0C80:	CALL	DVGET	nästa block
0C83	18 BE		JR	L0C43	
0C85	CD 99 08	L0C85:	CALL	DVGET	
0C88	18 D3		JR	L0C5D	

3.22 REN [start][,steg]: numrera om programmet

\$0C8A

Både start och steg är 10 om de utelämnas. 0 ger fel 31.

1. **LNKREF**: alla radreferenser (\$BF/\$BE) i programmet får adressen till raden de pekar på (fel 6 om raden saknas).
2. Alla rader får nya nummer: start, start+steg, ... Går nästa nummer över 65535 börjar det om med 10, 20, ... – även när raden som fick det sista numret var programmets sista (REN 65000,500 med två rader ger 10, 20).
3. Varje radreferens får det nya numret från raden som adressen pekar på.

Eftersom referenserna följer adresserna och inte numren, blir de rätt även om numren byts helt. Vid fel 6 i steg 1 är radnumren orörda (bara referensernas adressfält har fyllts i fram till felet).

0C8A	7E	CMDREN:	LD	A, (HL)	
0C8B	11 0A 00		LD	DE, \$000A	förvalt startnummer 10
0C8E	FE 0D		CP	\$0D	
0C90	28 09		JR	Z, L0C9B	
0C92	CD 1E 18		CALL	GETLN	startnummer
0C95	7A		LD	A, D	
0C96	B3		OR	E	
0C97	28 11		JR	Z, L0CAA	0: fel 31
0C99	7E		LD	A, (HL)	
0C9A	23		INC	HL	
0C9B	D5	L0C9B:	PUSH	DE	
0C9C	FE 0D		CP	\$0D	
0C9E	28 0D		JR	Z, L0CAD	
0CA0	FE 2C		CP	\$2C	
0CA2	C2 21 07		JP	NZ, L0721	annat än komma: fel 31
0CA5	CD 1E 18		CALL	GETLN	steg
0CA8	7A		LD	A, D	
0CA9	B3		OR	E	
0CAA	CA 21 07	L0CAA:	JP	Z, L0721	
0CAD	D5	L0CAD:	PUSH	DE	
0CAE	CD 01 0D		CALL	LNKREF	steg 1: alla referenser får adresser
0CB1	C1		POP	BC	BC = steg
0CB2	D1	L0CB2:	POP	DE	DE = startnummer
0CB3	2A 1C FE		LD	HL, (BOFA)	steg 2: nya radnummer
0CB6	7E	L0CB6:	LD	A, (HL)	
0CB7	3D		DEC	A	
0CB8	28 15		JR	Z, L0CCF	
0CBA	23		INC	HL	
0CBB	73		LD	(HL), E	
0CBC	23		INC	HL	
0CBD	72		LD	(HL), D	
0CBE	3D		DEC	A	nytt radnummer
0CBF	85		ADD	A, L	
0CC0	6F		LD	L, A	
0CC1	30 01		JR	NC, L0CC4	
0CC3	24		INC	H	
0CC4	EB	L0CC4:	EX	DE, HL	
0CC5	09		ADD	HL, BC	nästa nummer, spill ger CY
0CC6	EB		EX	DE, HL	
0CC7	30 ED		JR	NC, L0CB6	NC: inget spill
0CC9	01 0A 00		LD	BC, \$000A	spill: börja om med 10, steg 10
0CCC	C5		PUSH	BC	
0CCD	18 E3		JR	L0CB2	

Steg 3: skriv in de nya radnumren i alla referenser.

0CCF	2A 1C FE	L0CCF:	LD	HL, (BOFA)	steg 3: referenserna
0CD2	7E	L0CD2:	LD	A, (HL)	
0CD3	3D		DEC	A	
0CD4	C8		RET	Z	
0CD5	23		INC	HL	
0CD6	23		INC	HL	
0CD7	23		INC	HL	
0CD8	CD FB 0E	L0CD8:	CALL	SKPELM	CY: radens slut
0CDB	38 F5		JR	C, L0CD2	
0CDD	7E		LD	A, (HL)	

00CDE	06 01		LD	B, \$01	
00CE0	FE BF		CP	\$BF	\$BF = en referens
00CE2	28 06		JR	Z, L0CEA	
00CE4	FE BE		CP	\$BE	\$BE = lista, nästa byte = antal
00CE6	20 F0		JR	NZ, L0CD8	
00CE8	23		INC	HL	
00CE9	46		LD	B, (HL)	
00CEA	23	L0CEA:	INC	HL	
00CEB	E5	L0CEB:	PUSH	HL	
00CEC	23		INC	HL	
00CED	CD 1C 10		CALL	RDDE1	DE = målradens adress
00CF0	7B		LD	A, E	
00CF1	B2		OR	D	
00CF2	28 08		JR	Z, L0CFC	adress 0: hoppas över
00CF4	E3		EX	(SP), HL	hämta nytt radnummer från målraden
00CF5	13		INC	DE	
00CF6	1A		LD	A, (DE)	
00CF7	77		LD	(HL), A	
00CF8	23		INC	HL	
00CF9	13		INC	DE	
00CFA	1A		LD	A, (DE)	
00CFB	77		LD	(HL), A	
00CFC	E1	L0CFC:	POP	HL	
00CFD	10 EC		DJNZ	L0CEB	B referenser
00CFF	18 D7		JR	L0CD8	

LNKREF: gå igenom alla satser och ge varje radreferens (\$BF, eller \$BE + antal) adressen till målraden. Saknas raden blir det fel 6, och **CURLIN** pekar då på raden med referensen.

00D01	FD 36 0E 00	LNKREF:	LD	(IY+\$0E), \$00	DIRECT = 0: felet visar raden
00D05	2A 1C FE		LD	HL, (BOFA)	
00D08	7E	L0D08:	LD	A, (HL)	
00D09	3D		DEC	A	
00D0A	C8		RET	Z	
00D0B	23		INC	HL	
00D0C	23		INC	HL	
00D0D	23		INC	HL	
00D0E	22 2C FE		LD	(CURLIN), HL	för felutskriften
00D11	CD FB 0E	L0D11:	CALL	SKPELM	
00D14	38 F2		JR	C, L0D08	
00D16	7E		LD	A, (HL)	
00D17	06 01		LD	B, \$01	
00D19	FE BF		CP	\$BF	
00D1B	28 06		JR	Z, L0D23	
00D1D	FE BE		CP	\$BE	
00D1F	20 F0		JR	NZ, L0D11	
00D21	23		INC	HL	
00D22	46		LD	B, (HL)	
00D23	23	L0D23:	INC	HL	
00D24	CD 1D 10	L0D24:	CALL	RDDE	DE = radnummer
00D27	7A		LD	A, D	
00D28	B3		OR	E	
00D29	C4 33 0D		CALL	NZ, REFADR	0 = ingen referens
00D2C	CD 23 10		CALL	WRDE	adressfältet
00D2F	10 F3		DJNZ	L0D24	
00D31	18 DE		JR	L0D11	
00D33	E5	REFADR:	PUSH	HL	
00D34	C5		PUSH	BC	
00D35	CD 39 0F		CALL	FINDLN	hitta raden
00D38	C2 05 07		JP	NZ, L0705	fel 6, HITTA EJ DETTA RADNUMMER
00D3B	EB		EX	DE, HL	
00D3C	C1		POP	BC	
00D3D	E1		POP	HL	
00D3E	C9		RET		

CHAIN "fil": beräkna strängen (RST \$38), gör **NEW** och sätt **CR** efter namnet. Sedan fortsätter koden i **RUN**, som laddar och startar **filen**. **CHAIN** går också i direktläge. Variablerna nollas (**NEW**, **CLEAR**), och namnet får vara ett uttryck, t.ex. **LEFT\$(A\$,5)**. **CHAIN ""**

gör **NEW** och kör det tomma programmet: det gamla programmet är alltså borta.

0D3F	FF	CHAIN:	RST	\$38	
0D40	CD 70 0A		CALL	NEW	NEW
0D43	E1		POP	HL	
0D44	E1		POP	HL	
0D45	C1		POP	BC	HL = strängen, BC = längd
0D46	54		LD	D,H	
0D47	5D		LD	E,L	
0D48	09		ADD	HL,BC	
0D49	36 0D		LD	(HL), \$0D	CR i strängens eget minne
0D4B	EB		EX	DE,HL	HL = namnet, vidare som RUN

3.23 RUN [fil]: starta programmet

\$0D4C

Med ett filnamn laddas filen först (**CMDLOAD**). Sedan: **CLEAR**, förlänkning (**PRELNK**), **DATA**-pekaren nollställs (**RSTDAT**) och slumpvalsfröet nollställs (**RNDINI**). **DIRECT** = 0.

Stacken nollställs till **STKTOP**, och adressen **\$071B** (fel 29, **RETURN** utan **GOSUB**) läggs överst. En **RETURN** som inte har någon **GOSUB** hamnar alltså där.

RUN radnr finns inte: allt efter **RUN** tas som ett filnamn, så **RUN 100** försöker ladda filen "100". **GOTO** och **GOSUB** ger fel 2 i direktläge (**NODIR**), och bland direktkommandona (**CMDKW**) finns ingen **CON** eller **CONT**. Ett program kan alltså bara startas från början, och efter **STOP** eller **BREAK** kan det inte fortsätta. Detsamma gäller version 9913 (Christensen 1979: **CON** och **RUN** radnr saknas).

0D4C	E7	CMDRUN:	RST	\$20	
0D4D	FE 0D		CP	\$0D	
0D4F	C4 A8 0B		CALL	NZ,CMDLOAD	RUN fil: ladda först
0D52	CD 42 07		CALL	NEWLIN	
0D55	CD 80 0A		CALL	CLEAR	
0D58	CD 6E 0E		CALL	PRELNK	förlänkning
0D5B	CD 93 22		CALL	RSTDAT	första DATA
0D5E	CD 5D 22		CALL	RNDINI	slumpvalsfrö = 0
0D61	FD 36 0E 00		LD	(IY+\$0E), \$00	programläge
0D65	ED 5B 1C FE		LD	DE, (BOFA)	DE = första raden
0D69	2A 27 FE		LD	HL, (STKTOP)	
0D6C	F9		LD	SP,HL	stacken från början
0D6D	21 1B 07		LD	HL, \$071B	"RETURN utan GOSUB" i botten av stacken
0D70	E5		PUSH	HL	

RUNLIN: kör programmet från raden vid **DE**. Längdbyte **\$01** = slut, och då går det till **ENDPRG** (**CLEAR**, sedan **READY**). För varje rad sparas radadressen i **CURLIN**, **TRACE** skriver radnumret (bit 4 i **FLAGS**), och **SP** sparas i **STMSP** så att **ON ERROR** kan återställa stacken.

0D71	1A	RUNLIN:	LD	A, (DE)	
0D72	FE 01		CP	\$01	
0D74	CA C9 00		JP	Z,ENDPRG	programmets slut
0D77	ED 53 2C FE		LD	(CURLIN), DE	för fel och TRACE
0D7B	FD CB 0F 66		BIT	4, (IY+\$0F)	TRACE?
0D7F	C4 2F 23		CALL	NZ,TRCPRT	
0D82	21 00 00		LD	HL, \$0000	
0D85	39		ADD	HL, SP	
0D86	22 36 FE		LD	(STMSP), HL	SP för ON ERROR
0D89	13		INC	DE	förbi längd och radnummer
0D8A	13		INC	DE	
0D8B	13		INC	DE	
0D8C	CD 92 0D		CALL	RUNSTM	utför raden
0D8F	13		INC	DE	förbi CR
0D90	18 DF		JR	RUNLIN	

RUNSTM: utför satserna på en rad, till **CR**. Före varje sats testas **BREAK**, som ger **STOP**.

0D92	EB	RUNSTM:	EX	DE,HL	
0D93	CD 3E 03		CALL	TSTBRK	
0D96	C2 23 23		JP	NZ,BRKSTP	BREAK: STOP
0D99	EB		EX	DE,HL	
0D9A	1A		LD	A, (DE)	

0D9B	FE 0D	CP	\$0D	
0D9D	C8	RET	Z	CR: raden slut
0D9E	CD A3 0D	CALL	EXESTM	
0DA1	18 EF	JR	RUNSTM	

3.24 EXESTM: utför en sats

\$0DA3

Bytes under \$80 (":") mellan satser) hoppas över. Satstoken \$80-\$96 är index i STEXE1. Token \$86 betyder att nästa byte är index i STEXE2 (DIM, POKE, OPEN ...). Hoppet sker via \$284D med BC = tabell. Varje sats har två rutiner: STTAB1/STTAB2 (\$1A08) används när raden matas in och kompileras, och STEXE1/STEXE2 används när den körs.

0DA3	01 14 0E	EXESTM:	LD	BC, STEXE1	
0DA6	1A	L0DA6:	LD	A, (DE)	
0DA7	13		INC	DE	
0DA8	FE 80		CP	\$80	satstoken?
0DAA	38 FA		JR	C, L0DA6	
0DAC	87		ADD	A, A	HL = 2*(token-\$80)
0DAD	6F		LD	L, A	
0DAE	26 00		LD	H, \$00	
0DB0	C3 4D 28		JP	L284D	JP via (BC + HL)
0DB3	01 42 0E	EXEST2:	LD	BC, STEXE2	
0DB6	18 EE		JR	L0DA6	

Token \$87 + längd: hoppa framåt. IF lägger \$87 före THEN-delen och ELSE-delen. När villkoret är falskt hoppar IF över THEN-delen. Kommer THEN-delen till \$87 så hoppar den över ELSE-delen.

0DB8	EB	EXESKP:	EX	DE, HL	
0DB9	5E		LD	E, (HL)	
0DBA	16 00		LD	D, \$00	
0DBC	19		ADD	HL, DE	framåt från längdbyten
0DBD	EB		EX	DE, HL	
0DBE	C9		RET		

NODIR: fel 2 om satsen körs i direktläge (DIRECT <> 0). Gäller GOTO, GOSUB och STOP.

0DBF	FD 7E 0E	NODIR:	LD	A, (IY+\$0E)	
0DC2	A7		AND	A	
0DC3	C8		RET	Z	programläge: tillåtet
0DC4	D7		RST	\$10	
0DC5	82		DEFB	\$82	fel 2: OTILLÅTET SOM KOMMANDO

3.25 SAVE fil: spara programmet som BAC-fil

\$0DC6

Filen öppnas med OPEN typ 4. Bufferten fylls med \$82 och sedan hela rader. När nästa rad inte får plats avslutas blocket med \$00 och skrivs med PUT. Efter sista raden kommer \$01 och en sista PUT.

Slutet (CLOSE) är gemensamt med LOAD på \$0C0D.

FCB+\$0D = antal lediga bytes (CAS: 252). \$82 står bara i första blocket; nästa block börjar där enhetens PUT lämnar DE. Enheten måste ge en buffert (FCB+8,+9), annars blir det fel 52. SAVE prövar inte CY från PUT, så ett fel märks bara om enheten själv ger det. En rad som inte får plats i ett tomt block ger tomma block utan slut.

CMDSAVE:					
0DC6	DD 21 F1 FF	LD	IX, \$FFF1	FCB om 15 bytes på stacken	
0DCA	DD 39	ADD	IX, SP		
0DCC	DD F9	LD	SP, IX		
0DCE	3E 04	LD	A, \$04	OPEN typ 4 (skriv .BAC)	
0DD0	CD C2 07	CALL	OPNSTR		
0DD3	DD 6E 08	LD	L, (IX+\$08)		
0DD6	DD 66 09	LD	H, (IX+\$09)		
0DD9	7C	LD	A, H	H = 0: ingen buffert	
0DDA	A7	AND	A		
0ddb	3E B4	LD	A, \$B4	fel 52 om enheten saknar buffert	
0DDD	28 32	JR	Z, L0E11		
0DDF	36 82	LD	(HL), \$82	BAC-huvud	

0DE1	23		INC	HL	
0DE2	DD 4E 0D		LD	C, (IX+\$0D)	C = plats i bufferten
0DE5	0D		DEC	C	\$82 tar en plats
0DE6	EB		EX	DE, HL	
0DE7	2A 1C FE		LD	HL, (BOFA)	från första raden
0DEA	7E	L0DEA:	LD	A, (HL)	
0DEB	47		LD	B, A	
0DEC	EE 01		XOR	\$01	
0DEE	28 1A		JR	Z, L0E0A	slutmarkering?
0DF0	79		LD	A, C	
0DF1	90		SUB	B	
0DF2	4F		LD	C, A	
0DF3	38 08		JR	C, L0DFD	får inte raden plats?
0DF5	7E	L0DF5:	LD	A, (HL)	kopiera raden, B bytes
0DF6	23		INC	HL	
0DF7	12		LD	(DE), A	
0DF8	13		INC	DE	
0DF9	10 FA		DJNZ	L0DF5	
0DFB	18 ED		JR	L0DEA	
0DFD	E5	L0DFD:	PUSH	HL	
0DFE	EB		EX	DE, HL	
0DFF	36 00		LD	(HL), \$00	\$00 = blocket slut
0E01	CD 9C 08		CALL	DVPUT	CY från PUT prövas inte
0E04	E1		POP	HL	
0E05	DD 4E 0D		LD	C, (IX+\$0D)	nytt block, DE från PUT
0E08	18 E0		JR	L0DEA	
0E0A	EB	L0E0A:	EX	DE, HL	
0E0B	36 01		LD	(HL), \$01	\$01 = filslut
0E0D	CD 9C 08		CALL	DVPUT	
0E10	AF		XOR	A	A = 0: inget fel
0E11	C3 0D 0C	L0E11:	JP	L0C0D	

Satsernas körrutiner, index = token - \$80 (se STKW1). \$86 = byt till STEXE2 och \$87 = hoppa framåt. \$95 är en rad som inte gick att tokenisera och ger fel 53 om den körs.

0E14	DB 1D	STEXE1:	DEFW	GOTO
0E16	DB 1D		DEFW	GOTO
0E18	FD 1B		DEFW	LET
0E1A	FD 1B		DEFW	LET
0E1C	9F 21		DEFW	PRINTS
0E1E	9F 21		DEFW	PRINTS
0E20	B3 0D		DEFW	EXEST2
0E22	B8 0D		DEFW	EXESKP
0E24	4B 1E		DEFW	IF
0E26	78 1E		DEFW	INPT
0E28	82 1F		DEFW	INPLIN
0E2A	C2 1C		DEFW	FOR
0E2C	21 1D		DEFW	NEXT
0E2E	09 07		DEFW	L0709
0E30	C5 1E		DEFW	READ
0E32	86 22		DEFW	RESTOR
0E34	E9 1D		DEFW	GOSUB
0E36	B0 22		DEFW	RETURN
0E38	09 20		DEFW	ON
0E3A	73 22		DEFW	DATA
0E3C	41 20		DEFW	ONERRG
0E3E	F7 06		DEFW	L06F7
0E40	AA 1A		DEFW	DEF

Körrutiner för tabell 2, index = kod i STKW2.

0E42	FA 1B	STEXE2:	DEFW	DIM
0E44	44 21		DEFW	POKE
0E46	2A 21		DEFW	OUTST
0E48	09 07		DEFW	L0709
0E4A	76 22		DEFW	REM
0E4C	74 20		DEFW	OPENST

```

0E4E 77 20      DEFW PREPST
0E50 BF 20      DEFW CLOSST
0E52 58 22      DEFW RANDOM
0E54 20 23      DEFW STOP
0E56 C9 00      DEFW ENDPRG
0E58 66 01      DEFW L0166
0E5A 09 07      DEFW L0709
0E5C 09 07      DEFW L0709
0E5E C7 22      DEFW SETDOT
0E60 CF 22      DEFW CLRDOT
0E62 B1 1D      DEFW GET
0E64 3F 0D      DEFW CHAIN
0E66 70 01      DEFW L0170
0E68 75 01      DEFW L0175
0E6A 25 23      DEFW TRACE
0E6C 2A 23      DEFW NOTRAC

```

3.26 PRELNK: förlänkning innan programmet körs

\$0E6E

RUN går igenom hela programmet en gång innan det startar.

- Radreferenser (\$BF/\$BE) får adressen till målraden (fel 6).
- Variabler i uttryckskoden (token >= \$C0) får plats på heapen och får adressen till sitt värde (LNKEXP).
- FOR och NEXT paras ihop redan här. BUF1 används som pekare till innersta öppna FOR (LNKFOR/LNKNXT). En FOR utan NEXT ger fel 24, och CURLIN pekar in i FOR-raden (ERRMSG letar upp raden).
- DEF FN: antalet parametrar i \$FE42 och namnen i \$FE43- så att de känns igen i funktionskroppen (LNKDEF).
- ON ERROR GOTO 0 får adressen 0.
- Token \$8D ger fel 8. Den finns inte i STKW1 och kan bara komma från en fil eller POKE.

DIRECT sätts till 0 först, så att felen visar raden. Inget körs förrän hela programmet är länkat: ett fel i sista raden stoppar programmet innan första raden har körts.

LNKOK = 1 när allt har gått bra.

```

0E6E 2A 1C FE      PRELNK: LD HL, (BOFA)
0E71 AF           XOR A
0E72 FD 77 0E      LD (IY+$0E), A      programläge: felen visar raden
0E75 FD 77 2B      LD (IY+$2B), A      FOR-kedjan tom
0E78 FD 77 2C      LD (IY+$2C), A      inga DEF FN-parametrar
0E7B 7E           LD A, (HL)
0E7C D6 01        SUB $01      $01: programmets slut
0E7E 28 0B        JR Z, L0E8B
0E80 22 2C FE      LD (CURLIN), HL    för felen
0E83 23           INC HL
0E84 23           INC HL
0E85 23           INC HL
0E86 CD AA 0E      CALL LNKELM    hela raden, till CR
0E89 18 F0        JR L0E7B
0E8B FD B6 2B      L0E8B: OR (IY+$2B)    FOR utan NEXT?
0E8E 20 05        JR NZ, L0E95
0E90 FD 36 10 01   LD (IY+$10), $01    förlänkat
0E94 C9           RET
0E95 67           L0E95: LD H, A      HL = innersta FOR-blocket
0E96 FD 6E 2A      LD L, (IY+$2A)
0E99 23           INC HL
0E9A 23           INC HL
0E9B 23           INC HL
0E9C CD 1C 10      CALL RDDE1    DE = +4, slingans början
0E9F ED 53 2C FE   LD (CURLIN), DE    CURLIN = en adress i FOR-raden
0EA3 C3 15 07      JP L0715    fel 24, "NEXT" SAKNAS
0EA6 CD FB 0E      L0EA6: CALL SKPELM
0EA9 D8           RET C      CR: raden slut

```

LNKELM: länka ett element i satsen vid HL. Returadressen \$0EAA läggs på stacken, så rutinerna för \$C0-, \$8B och \$8C kommer tillbaka hit och tar nästa element.

0EAA	7E	LNKELM:	LD	A, (HL)	
0EAB	E5		PUSH	HL	
0EAC	21 AA 0E		LD	HL, \$0EAA	LNKELM som returadress
0EAF	E3		EX	(SP), HL	
0EB0	FE C0		CP	\$C0	uttryckskod
0EB2	D2 32 10		JP	NC, LNKEXP	
0EB5	FE 8B		CP	\$8B	FOR
0EB7	CA 7C 0F		JP	Z, LNKFOR	
0EBA	FE 8C		CP	\$8C	NEXT
0EBC	CA DB 0F		JP	Z, LKNXT	
0EBF	FE 8D		CP	\$8D	\$8D: fel 8
0EC1	CA 09 07		JP	Z, L0709	
0EC4	E3		EX	(SP), HL	ta bort returadressen
0EC5	E1		POP	HL	
0EC6	06 01		LD	B, \$01	en radreferens
0EC8	FE 94		CP	\$94	ON ERROR GOTO
0ECA	28 25		JR	Z, L0EF1	
0ECC	FE 96		CP	\$96	DEF FN
0ECE	CA 59 0F		JP	Z, LNKDEF	
0ED1	FE BF		CP	\$BF	\$BF: radreferens
0ED3	28 06		JR	Z, L0EDB	
0ED5	FE BE		CP	\$BE	\$BE: B = antal
0ED7	20 CD		JR	NZ, L0EA6	
0ED9	23		INC	HL	
0EDA	46		LD	B, (HL)	
0EDB	23	L0EDB:	INC	HL	
0EDC	CD 1D 10	L0EDC:	CALL	RDDE	DE = radnumret
0EDF	C5	L0EDF:	PUSH	BC	
0EE0	E5		PUSH	HL	
0EE1	CD 39 0F		CALL	FINDLN	
0EE4	C2 05 07		JP	NZ, L0705	fel 6
0EE7	EB		EX	DE, HL	DE = radens adress
0EE8	E1		POP	HL	
0EE9	C1		POP	BC	
0EEA	CD 23 10	L0EEA:	CALL	WRDE	efter radnumret
0EED	10 ED		DJNZ	L0EDC	
0EEF	18 B9		JR	LNKELM	
0EF1	23	L0EF1:	INC	HL	
0EF2	CD 1C 10		CALL	RDDE1	förbi \$BF, DE = radnumret
0EF5	7B		LD	A, E	
0EF6	B2		OR	D	
0EF7	20 E6		JR	NZ, L0EDF	
0EF9	18 EF		JR	L0EEA	GOTO 0: adress 0

3.27 SKPELM: hoppa över ett element i tokeniserad text

\$0EFB

Visar hur satserna lagras:

\$0D	slut på raden (CY = 1)
\$86 k	sats k i tabell 2
\$87 n	hopp framåt n bytes (IF/ELSE)
\$B9 a b	två bytes data
\$BF n1 nh a1 ah	radreferens: radnummer, sedan adress
\$BE k + k*4 bytes	lista med k radreferenser (ON ... GOTO)
\$96 ..	DEF FN: 2 bytes, antal, 2 bytes/parameter
\$C0-\$FF	uttryckskod, hoppas över med SKPEXP

Ut: CY = 0 och HL efter elementet.

0EFB	7E	SKPELM:	LD	A, (HL)	
0EFC	23		INC	HL	
0EFD	FE 0D		CP	\$0D	CR: slut, CY = 1
0EFF	37		SCF		
0F00	C8		RET	Z	
0F01	FE 96		CP	\$96	

0F03	28 23	JR	Z, L0F28	DEF FN
0F05	FE C0	CP	\$C0	
0F07	30 28	JR	NC, L0F31	uttryckskod
0F09	06 01	LD	B, \$01	
0F0B	FE 87	CP	\$87	\$87 och \$86: en byte
0F0D	28 14	JR	Z, L0F23	
0F0F	FE 86	CP	\$86	
0F11	28 10	JR	Z, L0F23	
0F13	FE B9	CP	\$B9	\$B9: två bytes
0F15	28 0B	JR	Z, L0F22	
0F17	FE BF	CP	\$BF	\$BF: fyra bytes
0F19	28 05	JR	Z, L0F20	
0F1B	EE BE	XOR	\$BE	\$BE: B gånger fyra
0F1D	C0	RET	NZ	annat: bara token, CY = 0
0F1E	46	LD	B, (HL)	
0F1F	23	INC	HL	
0F20	23	L0F20: INC	HL	
0F21	23	INC	HL	
0F22	23	L0F22: INC	HL	
0F23	23	L0F23: INC	HL	
0F24	10 FA	DJNZ	L0F20	
0F26	A7	AND	A	
0F27	C9	RET		
0F28	23	L0F28: INC	HL	
0F29	23	INC	HL	
0F2A	7E	LD	A, (HL)	
0F2B	87	ADD	A, A	två bytes per parameter
0F2C	23	L0F2C: INC	HL	
0F2D	3D	DEC	A	
0F2E	F8	RET	M	HL på "="
0F2F	18 FB	JR	L0F2C	
0F31	2B	L0F31: DEC	HL	tillbaka till koden
0F32	CD C3 10	L0F32: CALL	SKPEXP	
0F35	30 FB	JR	NC, L0F32	
0F37	A7	AND	A	
0F38	C9	RET		

FINDLN: leta efter raden med nummer DE. Ut: Z = hittad, HL = raden. NZ = saknas, och HL pekar då på första raden med högre nummer (eller på slutmarkeringen). Det är där en ny rad ska in.

0F39	2A 1C FE	FINDLN: LD	HL, (BOFA)	
0F3C	42	LD	B, D	BC = sökt nummer
0F3D	4B	LD	C, E	
0F3E	7E	L0F3E: LD	A, (HL)	
0F3F	FE 01	CP	\$01	\$01: slut, NZ
0F41	28 14	JR	Z, L0F57	
0F43	23	INC	HL	
0F44	5E	LD	E, (HL)	
0F45	23	INC	HL	
0F46	56	LD	D, (HL)	
0F47	2B	DEC	HL	
0F48	2B	DEC	HL	
0F49	EB	EX	DE, HL	
0F4A	A7	AND	A	
0F4B	ED 42	SBC	HL, BC	
0F4D	EB	EX	DE, HL	
0F4E	C8	RET	Z	Z: hittad
0F4F	30 06	JR	NC, L0F57	större: NZ
0F51	5F	LD	E, A	nästa rad
0F52	16 00	LD	D, \$00	
0F54	19	ADD	HL, DE	
0F55	18 E7	JR	L0F3E	
0F57	B7	L0F57: OR	A	
0F58	C9	RET		

LNKDEF: DEF FN. Antalet parametrar sparas i IY+\$2C och deras namn (2 bytes var) i IY+\$2D-. Sedan länkas funktionskroppen,

och IY+\$2C nollställs.

0F59	23	LNKDEF:	INC	HL	
0F5A	23		INC	HL	
0F5B	23		INC	HL	
0F5C	46		LD	B, (HL)	B = antal parametrar
0F5D	23		INC	HL	
0F5E	FD 70 2C		LD	(IY+\$2C), B	
0F61	78		LD	A, B	
0F62	A7		AND	A	
0F63	28 0F		JR	Z, L0F74	
0F65	FD E5		PUSH	IY	
0F67	CB 10		RL	B	två bytes per parameter
0F69	7E	L0F69:	LD	A, (HL)	
0F6A	23		INC	HL	
0F6B	FD 77 2D		LD	(IY+\$2D), A	namnen till \$FE43-
0F6E	FD 23		INC	IY	
0F70	10 F7		DJNZ	L0F69	
0F72	FD E1		POP	IY	
0F74	CD AA 0E	L0F74:	CALL	LNKELM	resten av raden
0F77	FD 36 2C 00		LD	(IY+\$2C), \$00	parametrarna gäller bara här
0F7B	C9		RET		

LNKFOR: FOR $v = a$ TO b [STEP c]. Fel 2 i direktläge. Ett block reserveras på heapen: 16 bytes för flyttalsvariabel, 10 bytes för heltal (token \$C0 = flyttal). Blocket länkas in först i FOR-kedjan (BUF1), och dess adress skrivs efter \$B9 i FOR-satsen. Blocket:

- +0 länken i FOR-kedjan, sedan adressen efter NEXT (LNKNXT)
- +2 variabelns adress
- +4 slingans början = efter FOR-satsens \$B9-fält
- +6 gränsen (TO) och steget: heltal 2 + 2 bytes, flyttal 5 + 5

Allt utom gränsen och steget skrivs här; steget får värdet 1 (1,0 = \$10 ... \$81), och FOR skriver resten vid körning.

0F7C	CD BF 0D	LNKFOR:	CALL	NODIR	fel 2 i direktläge
0F7F	23		INC	HL	
0F80	E5		PUSH	HL	
0F81	CD 32 10		CALL	LNKEXP	$v = a$, variabeln länkas
0F84	E3		EX	(SP), HL	
0F85	7E		LD	A, (HL)	variabelns typ
0F86	23		INC	HL	
0F87	23		INC	HL	
0F88	23		INC	HL	
0F89	4E		LD	C, (HL)	BC = variabelns adress
0F8A	23		INC	HL	
0F8B	46		LD	B, (HL)	
0F8C	11 10 00		LD	DE, \$0010	
0F8F	FE C0		CP	\$C0	flyttal: 16 bytes
0F91	28 0F		JR	Z, L0FA2	
0F93	11 0A 00		LD	DE, \$000A	heltal: 10 bytes
0F96	CD 74 33		CALL	ALLOC	
0F99	EB		EX	DE, HL	
0F9A	19		ADD	HL, DE	
0F9B	2B		DEC	HL	
0F9C	2B		DEC	HL	
0F9D	36 01		LD	(HL), \$01	steget 1
0F9F	EB		EX	DE, HL	
0FA0	18 0F		JR	L0FB1	
0FA2	CD 74 33	L0FA2:	CALL	ALLOC	
0FA5	EB		EX	DE, HL	
0FA6	19		ADD	HL, DE	
0FA7	2B		DEC	HL	
0FA8	36 81		LD	(HL), \$81	steget 1,0: exponent
0FAA	21 0B 00		LD	HL, \$000B	
0FAD	19		ADD	HL, DE	
0FAE	36 10		LD	(HL), \$10	och mantissa
0FB0	EB		EX	DE, HL	
0FB1	ED 5B 40 FE	L0FB1:	LD	DE, (BUF1)	länka in i FOR-kedjan
0FB5	22 40 FE		LD	(BUF1), HL	
0FB8	E5		PUSH	HL	
0FB9	CD 23 10		CALL	WRDE	+0 länken, +2 variabeln

0FBC	CD 29 10	CALL	WRBC	
0FBF	C1	POP	BC	
0FC0	E3	EX	(SP),HL	
0FC1	C5	PUSH	BC	
0FC2	23	INC	HL	
0FC3	CD 32 10	CALL	LNKEXP	gränsen
0FC6	7E	LD	A,(HL)	
0FC7	FE BB	CP	\$BB	STEP?
0FC9	20 04	JR	NZ,L0FCF	
0FCB	23	INC	HL	
0FCC	CD 32 10	CALL	LNKEXP	steget
0FCF	C1	L0FCF: POP	BC	
0FD0	D1	POP	DE	
0FD1	23	INC	HL	
0FD2	CD 29 10	CALL	WRBC	blockets adress i FOR
0FD5	EB	EX	DE,HL	
0FD6	73	LD	(HL),E	+4 = slingans början
0FD7	23	INC	HL	
0FD8	72	LD	(HL),D	
0FD9	EB	EX	DE,HL	
0FDA	C9	RET		

LNKNXT: NEXT [v]. Plockar innersta **FOR** från kedjan (fel 27 om den är tom). Hör **NEXT** till en variabel ska den vara samma som i **FOR** (fel 28). **NEXT** och **FOR** får varandras adresser. **NEXT** utan variabel jämförs med 0 och ger därför alltid fel 28 (samma i 9913). I direktläge är **BUF1** den inskrivna raden, så **NEXT** tar två tecken ur texten som blockets adress och skriver dit; utan minne där blir det fel 28.

0FDB	23	LNKNXT: INC	HL	
0FDC	7E	LD	A,(HL)	
0FDD	11 00 00	LD	DE,\$0000	utan variabel: 0
0FE0	FE C0	CP	\$C0	
0FE2	38 0B	JR	C,L0FEF	
0FE4	E5	PUSH	HL	
0FE5	CD 32 10	CALL	LNKEXP	variabelns adress
0FE8	E3	EX	(SP),HL	
0FE9	23	INC	HL	
0FEA	23	INC	HL	
0FEB	CD 1C 10	CALL	RDDE1	
0FEE	E1	POP	HL	
0FEF	D5	L0FEF: PUSH	DE	
0FF0	ED 4B 40 FE	LD	BC,(BUF1)	BC = innersta FOR
0FF4	78	LD	A,B	
0FF5	A7	AND	A	
0FF6	CA 19 07	JP	Z,L0719	fel 27, "NEXT" UTAN "FOR"
0FF9	CD 28 10	CALL	WRBC1	blockets adress i NEXT
0FFC	0A	LD	A,(BC)	
0FFD	FD 77 2A	LD	(IY+\$2A),A	kedjan ett steg bakåt
1000	7D	LD	A,L	
1001	02	LD	(BC),A	+0 = efter NEXT
1002	03	INC	BC	
1003	0A	LD	A,(BC)	
1004	FD 77 2B	LD	(IY+\$2B),A	
1007	7C	LD	A,H	
1008	02	LD	(BC),A	
1009	03	INC	BC	
100A	D1	POP	DE	
100B	0A	LD	A,(BC)	
100C	03	INC	BC	
100D	AB	XOR	E	samma variabel? 0 aldrig
100E	5F	LD	E,A	
100F	0A	LD	A,(BC)	
1010	AA	XOR	D	
1011	B3	OR	E	
1012	C2 17 07	JP	NZ,L0717	fel 28, FEL VARIABEL EFTER "NEXT"
1015	C9	RET		

Små hjälprutiner: RDDE/RDDE läser ett ord vid HL, och WRBC/WRDE skriver ett. Varianterna med 1 hoppar först över en byte.

```

1016 23      RDBC1: INC HL
1017 4E      RDBC: LD C,(HL)
1018 23              INC HL
1019 46              LD B,(HL)
101A 23              INC HL
101B C9              RET

101C 23      RDDE1: INC HL
101D 5E      RDDE: LD E,(HL)
101E 23              INC HL
101F 56              LD D,(HL)
1020 23              INC HL
1021 C9              RET

1022 23      WRDE1: INC HL
1023 73      WRDE: LD (HL),E
1024 23              INC HL
1025 72              LD (HL),D
1026 23              INC HL
1027 C9              RET

1028 23      WRBC1: INC HL
1029 71      WRBC: LD (HL),C
102A 23              INC HL
102B 70              LD (HL),B
102C 23              INC HL
102D C9              RET

```

LNKSKP: inte en variabel; hoppa över elementet. CY: uttrycket är slut.

```

102E CD C3 10      LNKSKP: CALL SKPEXP
1031 D8              RET C

```

LNKEXP: länka uttryckskod. Funktionsanrop (\$CD-\$D1) går till **LNKFN**. Variabeltoken (de nio första paren i tabellen på \$1105) jämförs först med **DEF** FN-parametrarna i IY+\$2D- (IY+\$2C stycken, bara inne i en **DEF**). Träff ger adressen \$00nn, där nn = antalet parametrar efter den här (den sista får 0); **FNP**AR (\$33AB) tolkar den vid körning. Annars slås variabeln upp i variabellistan, som börjar vid \$FE29+\$12. Finns variabeln inte reserveras plats på heapen (storlek enligt tabellen på \$10B8). Adressen skrivs in i koden.

```

1032 7E      LNKEXP: LD A,(HL)      token
1033 FE CD      CP $CD              funktionsanrop?
1035 38 05      JR C,LNKVAR
1037 FE D2      CP $D2
1039 DA 1D 11   JP C,LNKFN
103C EB      LNKVAR: EX DE,HL
103D 21 05 11   LD HL,TOKLEN      variabeltoken?
1040 01 12 00   LD BC,$0012      9 par
1043 ED B1      CPIR
1045 EB      EX DE,HL
1046 20 E6      JR NZ,LNKSHP      nej
1048 CD 16 10   CALL RDBC1        BC = namnet, HL = adressfältet
104B FD 5E 2C   LD E,(IY+$2C)     antal DEF-parametrar
104E FD E5      PUSH IY
1050 1D      LNKPAR: DEC E        fler parametrar?
1051 FA 6B 10   JP M,LNKLST
1054 FD 7E 2D   LD A,(IY+$2D)     jämför parametrarnas namn
1057 A9      XOR C
1058 57      LD D,A
1059 FD 23      INC IY
105B FD 7E 2D   LD A,(IY+$2D)
105E FD 23      INC IY
1060 A8      XOR B
1061 B2      OR D
1062 20 EC      JR NZ,LNKPAR      nästa parameter
1064 FD E1      POP IY
1066 CD 23 10   CALL WRDE          adressen $00nn
1069 18 C7      JR LNKEXP

```

LNKLST: sök i variabellistan. Varje post:

+0 namnbyte +1 bokstav +2/3 länk till nästa post (0 = sist)
+4 värdet (flyttal 5 bytes, heltal 2, sträng- eller vektorpost)

Namnbyten (bit 4-7 siffran, bit 2-3 antal index, bit 0-1 typen) skiljer A, A%, A\$, A(..) och A(....) åt. Samma namn med ett annat antal index ger fel 1 (bara bit 2-3 skiljer: XOR = \$0C).

Adressen som skrivs in i koden är +4, för flyttal utan index +8 (exponenten).

106B	FD E1	LNKLST:	POP IY	
106D	FD E5		PUSH IY	
106F	E3		EX (SP), HL	HL = IY, (SP) = koden
1070	11 12 00		LD DE, \$0012	\$FE28
1073	19		ADD HL, DE	
1074	23	LNKSRC:	INC HL	länken, första gången VARLST
1075	5E		LD E, (HL)	
1076	23		INC HL	
1077	56		LD D, (HL)	
1078	7B		LD A, E	0: sist
1079	B2		OR D	
107A	28 12		JR Z, LNKNEW	
107C	EB		EX DE, HL	HL = posten
107D	5E		LD E, (HL)	namnbyten
107E	23		INC HL	
107F	7E		LD A, (HL)	bokstaven
1080	A8		XOR B	
1081	20 F1		JR NZ, LNKSRC	annan bokstav
1083	7B		LD A, E	
1084	A9		XOR C	samma namnbyte?
1085	28 1D		JR Z, LNKADR	
1087	FE 0C		CP \$0C	bara antal index skiljer?
1089	20 E9		JR NZ, LNKSRC	
108B	C3 FB 06		JP L06FB	fel 1

LNKNEW: ny variabel. Storleken ur **VARsiz** (index = namnbyte AND \$0F), nollställd plats på heapen (**ALLOC**). Posten länkas in sist i listan.

108E	E5	LNKNEW:	PUSH HL	den sista länkens plats
108F	79		LD A, C	namnbyte AND \$0F
1090	E6 0F		AND \$0F	
1092	5F		LD E, A	
1093	21 B8 10		LD HL, VARsiz	
1096	19		ADD HL, DE	
1097	5E		LD E, (HL)	storleken
1098	CD 74 33		CALL ALLOC	nollställd plats
109B	EB		EX DE, HL	
109C	E1		POP HL	
109D	72		LD (HL), D	länka in
109E	2B		DEC HL	
109F	73		LD (HL), E	
10A0	EB		EX DE, HL	
10A1	71		LD (HL), C	namnbyte
10A2	23		INC HL	
10A3	70		LD (HL), B	bokstav
10A4	23	LNKADR:	INC HL	förbi bokstav och länk
10A5	23		INC HL	
10A6	23		INC HL	
10A7	79		LD A, C	flyttal utan index?
10A8	E6 0F		AND \$0F	
10AA	20 04		JR NZ, LNKWR	
10AC	11 04 00		LD DE, \$0004	exponenten, +4
10AF	19		ADD HL, DE	
10B0	EB	LNKWR:	EX DE, HL	DE = värdets adress
10B1	E1		POP HL	koden
10B2	CD 23 10		CALL WRDE	skriv in adressen
10B5	C3 32 10		JP LNKEXP	

VARsiz: postens storlek för namnbyte AND \$0F (4 bytes huvud + värdet): flyttal 9, heltal 6, sträng 10 (strängpost 6 bytes), med ett index 10 (vektorpost 6 bytes), med två index 14. Typ 3 används inte.

10B8 09 06 0A 00... **VARsiz:** DEFB \$09, \$06, \$0A, \$00, \$0A, \$0A, \$0A, \$0A storlek per variabeltyp

10C0 0E 0E 0E DEFB \$0E,\$0E,\$0E

SKPEXP: hoppa över ett element i uttryckskoden. Tabellen på \$10FB listar token som inte följs av data (CY = 1: slut på uttrycket). Tabellen på \$1105 ger längden för token med data.

10C3	7E	SKPEXP:	LD A, (HL)	token
10C4	23		INC HL	
10C5	01 0A 00		LD BC, \$000A	10 token
10C8	EB		EX DE, HL	
10C9	21 FB 10		LD HL, NODATA	
10CC	ED B1		CPIR	
10CE	37		SCF	CY = 1
10CF	28 17		JR Z, SKPRET	utan data: slut
10D1	FE CB		CP \$CB	strängkonstant
10D3	28 1F		JR Z, SKPSTR	
10D5	FE CD		CP \$CD	funktionsanrop?
10D7	38 04		JR C, SKPTOK	
10D9	FE D2		CP \$D2	
10DB	38 0D		JR C, SKPFN	
10DD	21 05 11	SKPTOK:	LD HL, TOKLEN	token med data?
10E0	0E 18		LD C, \$18	12 par
10E2	ED B1		CPIR	
10E4	4E		LD C, (HL)	C = antal bytes
10E5	28 11		JR Z, SKPLEN	
10E7	A7		AND A	CY = 0
10E8	EB	SKPRET:	EX DE, HL	
10E9	C9		RET	

SKPFN: funktionsanrop \$CD-\$D1, kod. Inbyggd funktion (kod <> 0): klart (XOR ger CY = 0). FN (kod 0): 5 bytes till (namn, adress, avstånd).

10EA	EB	SKPFN:	EX DE, HL	
10EB	7E		LD A, (HL)	funktionens kod
10EC	23		INC HL	
10ED	EE 00		XOR \$00	CY = 0
10EF	C0		RET NZ	inbyggd
10F0	0E 05		LD C, \$05	FN: 5 bytes
10F2	18 05		JR SKPADD	

SKPSTR: \$CB, citattecken, längd, text.

10F4	13	SKPSTR:	INC DE	hoppa över citattecknet
10F5	1A		LD A, (DE)	längden
10F6	4F		LD C, A	
10F7	13		INC DE	
10F8	EB	SKPLEN:	EX DE, HL	
10F9	09	SKPADD:	ADD HL, BC	hoppa över datat
10FA	C9		RET	

NODATA: token utan data, \$B6-\$BF. De avslutar uttrycket (CY = 1).

10FB	B6 B7 B8 B9...	NODATA:	DEFB \$B6, \$B7, \$B8, \$B9, \$BA, \$BB, \$BC, \$BD	token utan data
1103	BE BF		DEFB \$BE, \$BF	

TOKLEN: token och antal bytes data efter det. De nio första paren är variablerna (\$C0 + 4*index + typ, 4 bytes); **LNKEXP** söker bara dem. Sedan \$DB (FN-argumentpost), \$C3 (flyttalskonstant, 5 bytes) och \$C7 (heltalskonstant, 2 bytes).

1105	C0 04 C4 04...	TOKLEN:	DEFB \$C0, \$04, \$C4, \$04, \$C8, \$04, \$C1, \$04	token + längd
110D	C5 04 C9 04...		DEFB \$C5, \$04, \$C9, \$04, \$C2, \$04, \$C6, \$04	
1115	CA 04 DB 04...		DEFB \$CA, \$04, \$DB, \$04, \$C3, \$05, \$C7, \$02	

LNKFN: länka ett funktionsanrop \$CD+k, kod. Inbyggda funktioner (kod <> 0) har inget att länka. För FN (kod 0) söks programmet från **BOFA** efter en rad med \$96 och samma namn (fel 57, FUNKTIONEN EJ DEFINIERAD, om den saknas). Adressen till DEF-radens "=" (radens början + 7 + 2 * antal parametrar) skrivs in i anropet.

Sedan gås argumentposterna igenom bakåt från anropet, och samtidigt parameternamnerna bakåt från "=" (DE). I varje post skrivs parameterns typ (bit 0-1 i namnbyten), och **TYPCNV** skriver över \$D3 med \$D9 eller \$DA om argumentet måste omvandlas. Fel om typerna inte går ihop. Olika antal argument och parametrar ger fel 13.

111D	23	LNKFN:	INC	HL	hoppa över token
111E	7E		LD	A, (HL)	
111F	23		INC	HL	
1120	FE 00		CP	\$00	kod 0 = FN?
1122	C2 32 10		JP	NZ, LNKEXP	
1125	CD 17 10		CALL	RDBC	funktionens namn
1128	DD 2A 1C FE		LD	IX, (BOFA)	sök från programmets början
112C	DD 7E 00	L112C:	LD	A, (IX+\$00)	
112F	FE 01		CP	\$01	programmets slut: fel 57
1131	CA 29 07		JP	Z, L0729	
1134	5F		LD	E, A	radens längd
1135	DD 7E 03		LD	A, (IX+\$03)	DEF?
1138	EE 96		XOR	\$96	
113A	57		LD	D, A	
113B	DD 7E 04		LD	A, (IX+\$04)	samma namn?
113E	A9		XOR	C	
113F	B2		OR	D	
1140	57		LD	D, A	
1141	DD 7E 05		LD	A, (IX+\$05)	
1144	A8		XOR	B	
1145	B2		OR	D	
1146	28 06		JR	Z, L114E	
1148	16 00		LD	D, \$00	
114A	DD 19		ADD	IX, DE	nästa rad
114C	18 DE		JR	L112C	
114E	DD 4E 06	L114E:	LD	C, (IX+\$06)	antal parametrar
1151	3E 07		LD	A, \$07	7 + 2 * antal
1153	81		ADD	A, C	
1154	81		ADD	A, C	
1155	4F		LD	C, A	
1156	42		LD	B, D	
1157	EB		EX	DE, HL	
1158	DD E5		PUSH	IX	
115A	E1		POP	HL	
115B	09		ADD	HL, BC	adressen till "="
115C	EB		EX	DE, HL	
115D	CD 23 10		CALL	WRDE	skriv in DEF-adressen
1160	DD 46 06		LD	B, (IX+\$06)	parametrar kvar
1163	23		INC	HL	
1164	E5		PUSH	HL	efter anropet
1165	2B		DEC	HL	
1166	7E	L1166:	LD	A, (HL)	avstånd till posten
1167	A7		AND	A	
1168	28 20		JR	Z, L118A	0: inga fler
116A	05		DEC	B	
116B	1B		DEC	DE	förra parametern
116C	1B		DEC	DE	
116D	85		ADD	A, L	HL = postens avståndsbyte
116E	6F		LD	L, A	
116F	38 01		JR	C, L1172	
1171	25		DEC	H	
1172	E5	L1172:	PUSH	HL	
1173	C5		PUSH	BC	
1174	23		INC	HL	
1175	46		LD	B, (HL)	argumentets typ
1176	23		INC	HL	
1177	1A		LD	A, (DE)	parametrarnas typ
1178	E6 03		AND	\$03	
117A	77		LD	(HL), A	i posten
117B	23		INC	HL	
117C	36 D3		LD	(HL), \$D3	gör inget, om ingen omvandling
117E	EB		EX	DE, HL	
117F	CD 51 25		CALL	TYPENV	omvandla argumentet
1182	DA 2E 06		JP	C, ERROR	
1185	EB		EX	DE, HL	
1186	C1		POP	BC	
1187	E1		POP	HL	
1188	18 DC		JR	L1166	

118A	78	L118A:	LD	A,B	alla parametrar använda?
118B	A7		AND	A	
118C	3E 8D		LD	A,\$8D	fel 13
118E	C2 2E 06		JP	NZ,ERROR	
1191	E1		POP	HL	
1192	C3 32 10		JP	LNKEXP	

3.28 DETOK: gör om en programrad till text (för LIST, ED med mera)

\$1195

In: HL = radens längdbyte, DE = buffert för texten.

Ut: texten "nnn sats..." slutar med \$0D; DE pekar efter \$0D. Radnumret skrivs med PRNUM. Sedan går slingan igenom raden:

```

': '      blir " : " (mellanslag före och efter)
$87 n      hopp i IF/ELSE; skrivs inte ut
$B9 x x    FOR-post (se FOR); skrivs inte ut
$BF n n a a   en radreferens: numret skrivs
$BE k ...    k radreferenser, skrivs med komma emellan
$96        DEF FN: "DEFFN", namnet och (param,param,...)
$80-$BF     nyckelord: texten hämtas ur STKW1 (eller ur STKW2 om
            token är $86, då nästa byte är token i STKW2);
            mellanslag före och efter
$C0-$FF     uttryckskod: DETEXP skriver om den till infix
$00-$7F     vanligt tecken, kopieras
$0D         radslut, kopieras, klart

```

Mellanslagen skrivs av SPACE, som inte skriver ett mellanslag till om det senaste tecknet redan är ett. Mellanslag mellan uttryckets delar sparas inte, så "PRINT A; B" listas "PRINT A;B" och "A=1:B=2" listas "A=1 : B=2". Vanliga tecken (\$00-\$7F) kopieras däremot som de står: REM-texten (utan de inledande blanka, som inte sparas) och strängarna behåller sina mellanslag. "DEF FN" listas "DEFFNA(P,Q)=..." utan mellanslag.

1195	23	DETOK:	INC	HL	HL på radnumret
1196	D5		PUSH	DE	
1197	CD 1D 10		CALL	RDDE	radnummer
119A	E3		EX	(SP),HL	
119B	EB		EX	DE,HL	HL = numret, DE = bufferten
119C	CD 62 18		CALL	PRNUM	
119F	E1	L119F:	POP	HL	
11A0	CD EA 11	L11A0:	CALL	SPACE	efter numret, nyckelord och ":"
11A3	7E	L11A3:	LD	A,(HL)	
11A4	FE 3A		CP	\$3A	": "
11A6	28 4D		JR	Z,L11F5	
11A8	FE 87		CP	\$87	IF/ELSE-hopp
11AA	28 59		JR	Z,L1205	
11AC	FE B9		CP	\$B9	FOR-post
11AE	28 54		JR	Z,L1204	
11B0	FE BF		CP	\$BF	radreferens
11B2	CA 3D 12		JP	Z,L123D	
11B5	FE BE		CP	\$BE	lista med radreferenser
11B7	28 7C		JR	Z,L1235	
11B9	FE 0D		CP	\$0D	radslut
11BB	28 3F		JR	Z,L11FC	
11BD	FE C0		CP	\$C0	uttryckskod
11BF	30 3E		JR	NC,L11FF	
11C1	FE 96		CP	\$96	DEF FN
11C3	28 44		JR	Z,L1209	
11C5	FE 80		CP	\$80	vanligt tecken
11C7	38 70		JR	C,L1239	
11C9	CD EA 11		CALL	SPACE	före nyckelordet
11CC	7E		LD	A,(HL)	
11CD	23		INC	HL	
11CE	01 11 19		LD	BC,STKW1	
11D1	FE 86		CP	\$86	\$86: token i STKW2
11D3	20 05		JR	NZ,L11DA	
11D5	01 9C 19		LD	BC,STKW2	
11D8	7E		LD	A,(HL)	
11D9	23		INC	HL	

Nyckelordstabellerna har formen [token][text]..[token][text]. CPIR letar upp token-byten, och texten kopieras tills nästa byte med bit 7 satt. Därför har \$81 och \$83 (underförstådd GOTO/LET) ingen text.

11DA	E5	L11DA:	PUSH HL	
11DB	60		LD H,B	
11DC	69		LD L,C	
11DD	01 00 00		LD BC,\$0000	
11E0	ED B1		CPIR	hitta token
11E2	7E	L11E2:	LD A,(HL)	
11E3	87		ADD A,A	bit 7: nästa token, texten slut
11E4	38 B9		JR C,L119F	
11E6	ED A0		LDI	texten till bufferten
11E8	18 F8		JR L11E2	

SPACE: skriv ett mellanslag vid DE, om inte tecknet före DE är ett.

11EA	3E 20	SPACE:	LD A,\$20	
11EC	EB		EX DE,HL	
11ED	2B		DEC HL	
11EE	BE		CP (HL)	förra tecknet ett mellanslag?
11EF	23		INC HL	
11F0	EB		EX DE,HL	
11F1	C8		RET Z	
11F2	12		LD (DE),A	
11F3	13		INC DE	
11F4	C9		RET	
11F5	CD EA 11	L11F5:	CALL SPACE	
11F8	ED A0		LDI	": " (mellanslaget efter vid \$11A0)
11FA	18 A4		JR L11A0	
11FC	ED A0	L11FC:	LDI	CR med, DE efter
11FE	C9		RET	
11FF	CD 58 12	L11FF:	CALL DETEXP	HL efter uttrycket
1202	18 9F		JR L11A3	
1204	23	L1204:	INC HL	\$B9: 3 bytes
1205	23	L1205:	INC HL	\$87 n: 2 bytes
1206	23		INC HL	
1207	18 9A	L1207:	JR L11A3	

DEF FN: "DEFFN" kopieras ur STKW1-tabellen (\$196D, efter token-byten), sedan funktionens namn (DETVAR). Efter namnet kommer antalet parametrar och 2 bytes per parameter (namnbyte, bokstav). DETVAR går alltid fram 4 bytes, så HL backas 2 efter varje anrop.

1209	23	L1209:	INC HL	förbi \$96
120A	E5		PUSH HL	
120B	21 6D 19		LD HL,KWDEFN	"DEFFN"
120E	ED A0	L120E:	LDI	
1210	CB 7E		BIT 7,(HL)	
1212	28 FA		JR Z,L120E	
1214	E1		POP HL	
1215	CD 22 14		CALL DETVAR	funktionens namn
1218	2B		DEC HL	
1219	2B		DEC HL	
121A	7E		LD A,(HL)	antal parametrar
121B	23		INC HL	
121C	A7		AND A	
121D	28 84		JR Z,L11A3	inga parametrar: inga parenteser
121F	47		LD B,A	
1220	3E 28		LD A,\$28	"("
1222	12	L1222:	LD (DE),A	
1223	13		INC DE	
1224	C5		PUSH BC	
1225	CD 22 14		CALL DETVAR	parameterns namn
1228	2B		DEC HL	
1229	2B		DEC HL	
122A	C1		POP BC	
122B	3E 2C		LD A,\$2C	","
122D	10 F3		DJNZ L1222	

122F	3E 29	LD	A, \$29	)")"
1231	12	LD	(DE), A	
1232	13	INC	DE	
1233	18 D2	JR	L1207	

Radreferenser: B = antal, var och en är 4 bytes (nummer, adress).

1235	23	L1235:	INC	HL	
1236	46		LD	B, (HL)	B = antal referenser
1237	18 06		JR	L123F	
1239	ED A0	L1239:	LDI		vanligt tecken, som det står
123B	18 CA		JR	L1207	
123D	06 01	L123D:	LD	B, \$01	
123F	23	L123F:	INC	HL	
1240	D5	L1240:	PUSH	DE	
1241	CD 1D 10		CALL	RDDE	radnumret
1244	23		INC	HL	förbi adressen
1245	23		INC	HL	
1246	E3		EX	(SP), HL	
1247	EB		EX	DE, HL	
1248	C5		PUSH	BC	
1249	CD 62 18		CALL	PRNUM	
124C	C1		POP	BC	
124D	E1		POP	HL	
124E	10 02		DJNZ	L1252	fler: ", " emellan
1250	18 B5		JR	L1207	
1252	3E 2C	L1252:	LD	A, \$2C	
1254	12		LD	(DE), A	
1255	13		INC	DE	
1256	18 E8		JR	L1240	

3.29 DETEXP: skriv ett kompilerat uttryck (omvänd polsk notation) som infix

\$1258

Uttryckskoden har operanderna först och operatoren efter (se [CMPEXP](#)). Texten byggs i samma ordning som koden läses:

- * En operand skrivs direkt vid DE, och adressen där den börjar läggs på stacken.
- * En operator skriver sin text sist i bufferten och flyttar den sedan bakåt till rätt plats med [ROTTXT](#). För "A+B" står i koden A, B, +. Texten blir först "A" "B" "+" och efter rotationen "A" "+" "B". Startadressen för B tas bort från stacken, så att "A+B" nu räknas som en operand som börjar vid A.
- * Parenteserna i originaltexten finns kvar som token \$D3. Den sätter "(" framför den senaste operanden och ")" efter.

Så blir uttrycket skrivet med exakt de parenteser som användaren skrev, trots att koden i sig inte behöver några.

Slut: \$B6-\$B8 skrivs som "=" framför den senaste operanden (tilldelning i [LET/FOR](#), typ 0-2). Andra token < \$C0 (t.ex. \$B9+typ, som kompilatorn lägger sist i ett uttryck) avslutar uttrycket och skrivs inte ut. Stackens översta adress tas bort vid slutet.

1258	7E	DETEXP:	LD	A, (HL)	nästa token
1259	23		INC	HL	
125A	FE B6		CP	\$B6	tilldelning, typ 0-2
125C	28 3C	L125C:	JR	Z, L129A	
125E	FE B7		CP	\$B7	
1260	28 FA		JR	Z, L125C	
1262	FE B8		CP	\$B8	
1264	28 F6		JR	Z, L125C	
1266	FE C0		CP	\$C0	annan token < \$C0: slut
1268	38 35		JR	C, L129F	
126A	FE E1		CP	\$E1	NOT
126C	CA 88 13		JP	Z, L1388	
126F	FE E2		CP	\$E2	jämförelser och aritmetik

1271	D2 94 13	JP	NC,L1394	
1274	FE DC	CP	\$DC	logiska operatörer
1276	D2 90 13	JP	NC,L1390	
1279	FE D7	CP	\$D7	unärt minus
127B	CA 1B 14	L127B:	JP Z,L141B	
127E	FE D8	CP	\$D8	
1280	28 F9	JR	Z,L127B	
1282	FE DB	CP	\$DB	FN-argumentpost, 4 bytes, skrivs inte
1284	28 1B	JR	Z,L12A1	
1286	FE D3	CP	\$D3	parentes
1288	28 1D	JR	Z,L12A7	
128A	30 CC	JR	NC,DETEXP	\$D4-\$D6 (hämta värde) skrivs inte

Token \$C0-\$D2 går via hopptabellen DETTAB (2 bytes per token):

\$C0-\$C2	variabel utan index	-> DETVAR
\$C3	flyttalskonstant	-> \$12FD
\$C4-\$C6	variabel med ett index	-> namn + "(" index ")"
\$C7	heltalskonstant	-> tal + "%"
\$C8-\$CA	variabel med två index	-> index sätts ihop med ","
\$CB	strängkonstant	-> \$131E
\$CC	variabel i DIM	-> \$1337
\$D2	DIM av sträng utan längd	-> \$137E
\$CD-\$D1	funktion med 0-4 argument och namnet sätts framför	-> argumenten sätts ihop med ","

128C	D6 C0	SUB	\$C0	
128E	87	ADD	A,A	
128F	4F	LD	C,A	
1290	06 00	LD	B,\$00	
1292	DD 21 B6 12	LD	IX,DETTAB	
1296	DD 09	ADD	IX,BC	
1298	DD E9	JP	(IX)	
129A	3E 3D	L129A:	LD A,\$3D	"="
129C	CD 0C 14		CALL JOINA	
129F	C1	L129F:	POP BC	
12A0	C9		RET	
12A1	23	L12A1:	INC HL	förbi de 4 bytena
12A2	23		INC HL	
12A3	23		INC HL	
12A4	23		INC HL	
12A5	18 B1	JR	DETEXP	
12A7	D5	L12A7:	PUSH DE	p2 = där "(" skrivs
12A8	3E 28	L12A8:	LD A,\$28	"(" framför operanden
12AA	12		LD (DE),A	
12AB	13		INC DE	
12AC	CD 49 14		CALL ROTTXT	
12AF	3E 29		LD A,\$29	")"
12B1	12		LD (DE),A	
12B2	13		INC DE	
12B3	C1		POP BC	p2 bort igen
12B4	18 A2	L12B4:	JR DETEXP	
12B6	18 28	DETTAB:	JR L12E0	
12B8	18 26	L12B8:	JR L12E0	
12BA	18 24	L12BA:	JR L12E0	
12BC	18 3F	L12BC:	JR L12FD	
12BE	18 20	L12BE:	JR L12E0	
12C0	18 1E	L12C0:	JR L12E0	
12C2	18 1C	L12C2:	JR L12E0	
12C4	18 46	L12C4:	JR L130C	
12C6	18 15	L12C6:	JR L12DD	
12C8	18 13	L12C8:	JR L12DD	

12CA	18 11	L12CA:	JR	L12DD	
12CC	18 50	L12CC:	JR	L131E	
12CE	18 67	L12CE:	JR	L1337	
12D0	18 25	L12D0:	JR	L12F7	
12D2	18 1D	L12D2:	JR	L12F1	
12D4	18 18	L12D4:	JR	L12EE	
12D6	18 13	L12D6:	JR	L12EB	
12D8	18 0E	L12D8:	JR	L12E8	
12DA	C3 7E 13	L12DA:	JP	L137E	
12DD	CD 0A 14	L12DD:	CALL	JOINC	två index: ihop med ",,"
12E0	D5	L12E0:	PUSH	DE	operandens start
12E1	CD 22 14		CALL	DETVAR	
12E4	28 CE		JR	Z, L12B4	utan index: klar
12E6	18 C0		JR	L12A8	namn och "(" framför indexet
12E8	CD 0A 14	L12E8:	CALL	JOINC	4 argument: tre komman
12EB	CD 0A 14	L12EB:	CALL	JOINC	
12EE	CD 0A 14	L12EE:	CALL	JOINC	2 argument
12F1	D5	L12F1:	PUSH	DE	funktion med argument
12F2	CD E9 13		CALL	FNTEXT	
12F5	18 B1		JR	L12A8	
12F7	D5	L12F7:	PUSH	DE	funktion utan argument (RND)
12F8	CD E9 13		CALL	FNTEXT	
12FB	18 B7	L12FB:	JR	L12B4	
12FD	D5	L12FD:	PUSH	DE	
12FE	01 04 00		LD	BC, \$0004	HL på exponenten
1301	09		ADD	HL, BC	
1302	E5		PUSH	HL	
1303	3E 2C		LD	A, \$2C	
1305	CD 79 16		CALL	FLTASC	skriv flyttalet
1308	E1		POP	HL	
1309	23		INC	HL	förbi exponenten
130A	18 EF	L130A:	JR	L12FB	
130C	D5	L130C:	PUSH	DE	
130D	D5		PUSH	DE	
130E	5E		LD	E, (HL)	
130F	23		INC	HL	
1310	56		LD	D, (HL)	
1311	23		INC	HL	
1312	E3		EX	(SP), HL	
1313	EB		EX	DE, HL	HL = talet, DE = bufferten
1314	CD 62 18		CALL	PRNUM	
1317	3E 25		LD	A, \$25	"%"
1319	12		LD	(DE), A	
131A	13		INC	DE	
131B	E1		POP	HL	
131C	18 EC	L131C:	JR	L130A	

Strängkonstant: \$CB, citattecken (" eller '), längd, text. Ett citattecken inne i texten skrivs dubbelt, så att texten kan läsas in igen.

131E	D5	L131E:	PUSH	DE	
131F	CD 17 10		CALL	RDBC	C = citattecken, B = längd
1322	79	L1322:	LD	A, C	citattecknet
1323	12		LD	(DE), A	
1324	13		INC	DE	
1325	05	L1325:	DEC	B	
1326	FA 32 13		JP	M, L1332	
1329	7E		LD	A, (HL)	
132A	23		INC	HL	
132B	12		LD	(DE), A	
132C	13		INC	DE	
132D	B9		CP	C	citattecken i texten skrivs dubbelt

132E	28 F2		JR	Z, L1322	
1330	18 F3		JR	L1325	
1332	79	L1332:	LD	A, C	och det sista
1333	12		LD	(DE), A	
1334	13		INC	DE	
1335	18 E5	L1335:	JR	L131C	

DIM-post. I koden: [index ...] [längd] \$CC [variabeltoken] [namnbyte, bokstav, 2 bytes]. En sträng utan längd har \$D2 i stället för längden; \$D2 kommer då före \$CC, och \$137E hoppar över \$CC och variabeltoken. Utskrift: tal "C(i,j)", sträng med längd "A\$=n" eller "A\$(i,j)=n", sträng utan längd "A\$(i,j)". Efter posten kommer ")," eller \$BF, som avslutar DIM-listan.

DIM A\$ utan både index och längd (går att skriva in i 11273, 9913 ger fel 15) blir \$D2 \$CC ... utan någon operand före. Då får **ROTTXT** en adress på stacken som inte hör till texten, och **LIST** skriver sönder minnet. Körningen går bra.

1337	23	L1337:	INC	HL	förbi variabeltoken
1338	3E 02		LD	A, \$02	
133A	A6		AND	(HL)	strängvariabel?
133B	20 1D		JR	NZ, L135A	
133D	AF	L133D:	XOR	A	CY = 0: ")" sist
133E	08	L133E:	EX	AF, AF'	
133F	CB 5E		BIT	3, (HL)	två index?
1341	C4 0A 14		CALL	NZ, JOINC	
1344	D5		PUSH	DE	p2 = namnets start
1345	3E 28		LD	A, \$28	
1347	F5	L1347:	PUSH	AF	
1348	CD 22 14		CALL	DETVAR	
134B	F1		POP	AF	
134C	12		LD	(DE), A	
134D	13		INC	DE	
134E	CD 49 14		CALL	ROTTXT	namn och tecknet framför operanden
1351	08		EX	AF, AF'	
1352	38 1C		JR	C, L1370	
1354	3E 29		LD	A, \$29	
1356	12		LD	(DE), A	
1357	13		INC	DE	
1358	18 16		JR	L1370	
135A	D5	L135A:	PUSH	DE	sträng: p2 = där ")"=" skrivs
135B	3E 0C		LD	A, \$0C	
135D	A6		AND	(HL)	
135E	28 22		JR	Z, L1382	utan index: "A\$=n"
1360	3E 29		LD	A, \$29	
1362	12		LD	(DE), A	
1363	13		INC	DE	
1364	3E 3D		LD	A, \$3D	
1366	12		LD	(DE), A	
1367	13		INC	DE	
1368	CD 49 14		CALL	ROTTXT	)=" framför längden
136B	C1		POP	BC	båda adresserna bort
136C	C1		POP	BC	
136D	37		SCF		CY: inget ")" sist
136E	18 CE		JR	L133E	
1370	C1	L1370:	POP	BC	operandens adresser bort
1371	C1		POP	BC	
1372	7E		LD	A, (HL)	
1373	23		INC	HL	
1374	FE BF		CP	\$BF	\$BF: DIM-listan slut
1376	C8		RET	Z	
1377	2B		DEC	HL	
1378	3E 2C		LD	A, \$2C	","
137A	12		LD	(DE), A	
137B	13		INC	DE	
137C	18 B7	L137C:	JR	L1335	
137E	23	L137E:	INC	HL	\$D2: förbi \$CC och variabeltoken
137F	23		INC	HL	
1380	18 BB		JR	L133D	
1382	37	L1382:	SCF		CY: inget ")"

```

1383 08          EX  AF,AF'
1384 3E 3D      LD  A,$3D
1386 18 BF      JR   L1347

```

Operatorer: CALL DETOP följs av en flaggbyte.

bit 0 = mellanslag runt operatoren (EQV, IMP, OR, XOR, AND, NOT)

bit 1 = unär operator (bara en operand)

\$E1 NOT: 3, \$DC-\$E0: 1, \$E2-\$FF (jämförelser, aritmetik): 0.

Ingången \$138C (flagga 2, unär utan mellanslag) används inte i denna ROM.

```

1388 CD 98 13    L1388: CALL DETOP
138B 03          DEFB $03

138C CD 98 13    L138C: CALL DETOP
138F 02          DEFB $02

1390 CD 98 13    L1390: CALL DETOP
1393 01          DEFB $01

1394 CD 98 13    L1394: CALL DETOP
1397 00          DEFB $00

```

DETOP: skriv operatortexten sist och rotera den på plats. På stacken ligger startadresserna för operanderna. Vid binär operator flyttas texten in framför högra operanden, vid unär framför den enda operanden.

```

1398 DD E1      DETOP: POP  IX      IX = flaggbyten
139A C1         POP  BC      BC = högra operandens start
139B C5         PUSH BC
139C D5         PUSH DE      p = där operatoren skrivs
139D DD CB 00 46 BIT  0,(IX+$00)
13A1 28 0B      JR   Z,L13AE
13A3 C5         PUSH BC
13A4 0B         DEC  BC      tecknet före operatoren
13A5 0A         LD   A,(BC)
13A6 FE 20      CP   $20
13A8 28 04      JR   Z,L13AE
13AA 3E 20      LD   A,$20
13AC 12         LD   (DE),A
13AD 13         INC  DE
13AE 2B         L13AE: DEC  HL      token
13AF 7E         LD   A,(HL)
13B0 23         INC  HL
13B1 CD D6 13    CALL OPTEXT      operatorns text
13B4 DD CB 00 46 BIT  0,(IX+$00)
13B8 28 0A      JR   Z,L13C4
13BA C1         POP  BC
13BB 0A         LD   A,(BC)      tecknet efter
13BC FE 20      CP   $20
13BE 28 04      JR   Z,L13C4
13C0 3E 20      LD   A,$20
13C2 12         LD   (DE),A
13C3 13         INC  DE
13C4 DD CB 00 4E L13C4: BIT  1,(IX+$00) unär?
13C8 20 06      JR   NZ,L13D0
13CA CD 49 14    CALL ROTTXT      binär: framför högra operanden
13CD C1         POP  BC      p bort
13CE 18 03      JR   L13D3

13D0 CD 49 14    L13D0: CALL ROTTXT unär: framför operanden
13D3 C1         L13D3: POP  BC      binär: högra operandens adress bort
13D4 18 A6      JR   L137C

```

OPTEXT: skriv texten för operatortoken A. Jämförelser och aritmetik har upp till tre token (en per typ), men bara den första står i tabellen. Därför provas A, A-1, A-2 ... tills token hittas i KWEQV... \$FF (flyttal^heltal) börjar på \$FE.

```

13D6 E5      OPTEXT: PUSH HL
13D7 FE FF      CP   $FF      $FF: börja på $FE
13D9 28 01      JR   Z,L13DC
13DB 3C         INC  A
13DC 3D         L13DC: DEC  A

```

13DD	21 F5 23	LD	HL, KWEQV	operatorernas text
13E0	01 3C 00	LD	BC, \$003C	
13E3	ED B1	CPIR		
13E5	20 F5	JR	NZ, L13DC	inte funnen: token - 1
13E7	18 11	JR	L13FA	

FNTEXT: skriv funktionsnamnet. Byten i koden är 2*kod; SCF + RRA ger token \$80+kod i **FNKW**. Kod 0 är användarfunktion: "FN" + namnet.

13E9	7E	FNTEXT: LD	A, (HL)	
13EA	37		SCF	
13EB	1F		RRA	
13EC	23		INC HL	
13ED	FE 80	CP	\$80	kod 0: FN
13EF	28 11	JR	Z, L1402	
13F1	E5	FNKWTX: PUSH	HL	
13F2	21 31 24	LD	HL, FNKW	funktionsnamnen
13F5	01 00 00	LD	BC, \$0000	
13F8	ED B1	CPIR		
13FA	ED A0	L13FA: LDI		texten till DE
13FC	7E	LD	A, (HL)	
13FD	87	ADD	A, A	bit 7: slut på texten
13FE	30 FA	JR	NC, L13FA	
1400	E1	POP	HL	
1401	C9	RET		
1402	CD F1 13	L1402: CALL	FNKWTX	"FN" och namnet
1405	CD 22 14	CALL	DETVAR	
1408	23	INC	HL	
1409	C9	RET		

JOINC/JOINA: skriv tecknet A (**JOINC:** ",") framför den senaste operanden. De två senaste operanderna blir då en.

140A	3E 2C	JOINC: LD	A, \$2C	
140C	DD E1	JOINA: POP	IX	
140E	C1	POP	BC	BC = senaste operandens start
140F	DD E5	PUSH	IX	
1411	C5	PUSH	BC	
1412	D5	PUSH	DE	
1413	12	LD	(DE), A	
1414	13	INC	DE	
1415	CD 49 14	CALL	ROTTXT	
1418	C1	POP	BC	båda bort: nu en operand
1419	C1	POP	BC	
141A	C9	RET		

Unärt minus (\$D7/\$D8): "-" framför operanden.

141B	D5	L141B: PUSH	DE	p = där "-" skrivs
141C	3E 2D	LD	A, \$2D	"_"
141E	12	LD	(DE), A	
141F	13	INC	DE	
1420	18 AE	JR	L13D0	

DETVAR: skriv variabelnamnet. HL pekar på namnbyten (siffra i bit 4-7, \$F = ingen siffra; typ i bit 0-1) följt av bokstaven och 2 bytes adress. Skrivs: bokstav, siffra, "%" (typ 1) eller "\$" (typ 2). Ut: HL += 4, A = antal index * 4 (bit 2-3), Z om inga index.

1422	7E	DETVAR: LD	A, (HL)	namnbyten
1423	23	INC	HL	
1424	ED A0	LDI		bokstaven
1426	23	INC	HL	förbi adressen
1427	23	INC	HL	
1428	4F	LD	C, A	
1429	E6 F0	AND	\$F0	siffra i namnet?
142B	FE F0	CP	\$F0	
142D	28 08	JR	Z, L1437	
142F	0F	RRCA		
1430	0F	RRCA		
1431	0F	RRCA		

1432	0F		RRCA	
1433	F6 30		OR \$30	som ASCII
1435	12		LD (DE),A	
1436	13		INC DE	
1437	3E 03	L1437:	LD A,\$03	typen
1439	A1		AND C	
143A	28 09		JR Z,L1445	
143C	3D		DEC A	
143D	3E 25		LD A,\$25	"%"
143F	28 02		JR Z,L1443	
1441	3E 24		LD A,\$24	"\$"
1443	12	L1443:	LD (DE),A	
1444	13		INC DE	
1445	3E 0C	L1445:	LD A,\$0C	Z: inga index
1447	A1		AND C	
1448	C9		RET	

3.30 ROTTXT: flytta den senast skrivna texten bakåt

\$1449

På stacken (under returadressen) ligger två adresser p2 och p1 (p2 lades senast). Texten mellan p1 och DE ser ut så här:

```

p1      p2      DE
[ text 1 ][ text 2 ]

```

Efter **ROTTXT**:

```

[ text 2 ][ text 1 ]

```

Först flyttas hela p1..DE uppåt med längden av text 2 (LDDR), och sedan flyttas text 2, som nu ligger efter DE, ner till p1. DE och stacken är oförändrade.

1449	E5	ROTTXT:	PUSH HL	
144A	D5		PUSH DE	
144B	DD 21 00 00		LD IX,\$0000	IX = stacken
144F	DD 39		ADD IX,SP	
1451	DD 4E 06		LD C,(IX+\$06)	BC = p2
1454	DD 46 07		LD B,(IX+\$07)	
1457	62		LD H,D	
1458	6B		LD L,E	
1459	ED 42		SBC HL,BC	längden av text 2
145B	E5		PUSH HL	
145C	19		ADD HL,DE	
145D	E5		PUSH HL	
145E	62		LD H,D	
145F	6B		LD L,E	
1460	DD 4E 08		LD C,(IX+\$08)	BC = p1
1463	DD 46 09		LD B,(IX+\$09)	
1466	ED 42		SBC HL,BC	längden av text 1 + text 2
1468	44		LD B,H	
1469	4D		LD C,L	
146A	E1		POP HL	HL = nya slutet
146B	2B		DEC HL	
146C	1B		DEC DE	
146D	EB		EX DE,HL	HL = gamla slutet - 1, DE = nya - 1
146E	D5		PUSH DE	
146F	ED B8		LDDR	flytta allt uppåt
1471	E1		POP HL	HL = nya slutet - 1
1472	C1		POP BC	BC = längden av text 2
1473	ED B8		LDDR	text 2 ner till p1
1475	D1		POP DE	DE och HL tillbaka
1476	E1		POP HL	
1477	C9		RET	

3.31 FNAME: tolka ett filnamn [enhet:]namn[.typ]

\$1478

In: HL = text, BC = längd (1-127), DE = 14 bytes för resultatet.

Ut:

```
+0 namn, 8 tecken
+8 typ (filändelse), 3 tecken
+11 enhet, 3 tecken
```

Det som inte anges blir mellanslag. CY = felaktigt namn (**OPEN** ger då fel 21); resultatet kan då vara delvis ifyllt.

Texten måste börja med en bokstav (kod $\geq \$41$). Om det finns ett ":" någonstans i texten är det som står före enhetens namn (1-3 tecken). Namnet får vara tomt ("PR:", "PR:BAC"), och efter enheten får det börja med en siffra. Mellanslag hoppas över var de än står ("AB C" = "ABC"). Tillåtna tecken är siffror och A-Z samt [\] (ÄÖÅ i den svenska 7-bitarskoden). Små bokstäver görs om till stora (AND \$5F); samma AND stryker bit 7, så \$C1 blir "A". Varje del har ett högsta antal tecken; fler tecken ger CY, liksom varje annat tecken som blir kvar ("AB\$", "A.B:C").
 Provat: "cas:ab c.bac" -> ABC, BAC, CAS.

1478	78	FNAME:	LD	A,B	
1479	A7		AND	A	
147A	37		SCF		
147B	C0		RET	NZ	längden måste vara 1-127
147C	B1		OR	C	
147D	37		SCF		
147E	C8		RET	Z	
147F	F8		RET	M	C $\geq$ \$80: CY
1480	7E		LD	A, (HL)	
1481	FE 41		CP	\$41	måste börja med en bokstav
1483	D8		RET	C	
1484	C5		PUSH	BC	
1485	E5		PUSH	HL	
1486	D5		PUSH	DE	
1487	0E 0D		LD	C, \$0D	füll 14 bytes med mellanslag
1489	6B		LD	L, E	
148A	62		LD	H, D	
148B	13		INC	DE	
148C	36 20		LD	(HL), \$20	
148E	ED B0		LDIR		
1490	D1		POP	DE	
1491	E1		POP	HL	
1492	C1		POP	BC	
1493	C5		PUSH	BC	
1494	E5		PUSH	HL	
1495	3E 3A		LD	A, \$3A	finns ett ":"?
1497	ED B1		CPIR		i hela texten
1499	E1		POP	HL	
149A	C1		POP	BC	
149B	20 18		JR	NZ, L14B5	
149D	D5		PUSH	DE	
149E	3E 0B		LD	A, \$0B	enhet på +11
14A0	83		ADD	A, E	
14A1	5F		LD	E, A	
14A2	30 01		JR	NC, L14A5	
14A4	14		INC	D	
14A5	06 03	L14A5:	LD	B, \$03	högst 3 tecken
14A7	CD DA 14		CALL	FNPART	
14AA	D1		POP	DE	
14AB	D8		RET	C	fel i enheten
14AC	7E		LD	A, (HL)	
14AD	FE 3A		CP	\$3A	":" måste följa
14AF	37		SCF		
14B0	C0		RET	NZ	
14B1	23		INC	HL	
14B2	A7		AND	A	
14B3	0D		DEC	C	
14B4	C8		RET	Z	bara enhet: klart
14B5	06 08	L14B5:	LD	B, \$08	namn, högst 8 tecken
14B7	CD DA 14		CALL	FNPART	
14BA	D8		RET	C	

14BB	79		LD	A,C	
14BC	A7		AND	A	
14BD	C8		RET	Z	slut efter namnet
14BE	7E		LD	A,(HL)	
14BF	FE 2E		CP	\$2E	""
14C1	37		SCF		
14C2	C0		RET	NZ	
14C3	78		LD	A,B	hoppa till +8
14C4	83		ADD	A,E	
14C5	5F		LD	E,A	
14C6	30 01		JR	NC,L14C9	
14C8	14		INC	D	
14C9	23	L14C9:	INC	HL	
14CA	A7		AND	A	
14CB	0D		DEC	C	
14CC	C8		RET	Z	
14CD	06 03		LD	B,\$03	typ, högst 3 tecken
14CF	CD DA 14		CALL	FNPART	
14D2	79		LD	A,C	något kvar efter typen: CY
14D3	A7		AND	A	
14D4	C8		RET	Z	
14D5	37		SCF		
14D6	C9		RET		
14D7	23	L14D7:	INC	HL	
14D8	0D		DEC	C	
14D9	C8		RET	Z	

FNPART: kopiera högst B tillåtna tecken från HL till DE. Slutar vid första otillåtna tecken (NC) eller när texten är slut (C = 0, Z).
CY om delen blir för lång.

14DA	7E	FNPART:	LD	A,(HL)	
14DB	FE 20		CP	\$20	mellanslag hoppas över
14DD	28 F8		JR	Z,L14D7	
14DF	FE 30		CP	\$30	
14E1	3F		CCF		under "0": slut (NC)
14E2	D0		RET	NC	
14E3	FE 3A		CP	\$3A	
14E5	38 09		JR	C,L14F0	
14E7	E6 5F		AND	\$5F	små bokstäver -> stora
14E9	FE 41		CP	\$41	
14EB	3F		CCF		under "A": slut
14EC	D0		RET	NC	
14ED	FE 5E		CP	\$5E	A-Z och [\] (ÄÖÅ)
14EF	D0		RET	NC	
14F0	05	L14F0:	DEC	B	för många tecken?
14F1	37		SCF		
14F2	F8		RET	M	
14F3	23		INC	HL	
14F4	12		LD	(DE),A	
14F5	13		INC	DE	
14F6	0D		DEC	C	
14F7	20 E1		JR	NZ,FNPART	
14F9	A7		AND	A	texten slut: NC
14FA	C9		RET		

GETSFL: som GETFLT, men blanka och ett "+" eller "-" före talet är tillåtna. Vid "-" sätts teckenbyten (HL-1) till 1 efteråt, även vid CY (DE pekar då efter tecknet) och även för talet 0: "-0" blir 00 00 00 01 00, som räknas som 0 (provat: <0 falskt, =0 sant, SGN 0).

Ut vid "-": HL = teckenbyten, annars som GETFLT.

14FB	EB	GETSFL:	EX	DE,HL	
14FC	E7		RST	\$20	blanka före tecknet
14FD	EB		EX	DE,HL	
14FE	1A		LD	A,(DE)	
14FF	13		INC	DE	
1500	FE 2B		CP	\$2B	""
1502	28 0F		JR	Z,GETFLT	

```

1504 1B          DEC DE
1505 FE 2D      CP  $2D          "-"
1507 20 0A      JR  NZ,GETFLT
1509 13          INC DE
150A E5         PUSH HL
150B CD 13 15   CALL GETFLT
150E E1         POP  HL          även vid CY
150F 2B         DEC HL
1510 36 01      LD  (HL), $01    teckenbyte = negativt
1512 C9         RET

```

3.32 GETFLT: läs ett flyttal från texten

\$1513

In: DE = texten, HL = talets sista byte (exponenten).

Ut: NC: talet är lagrat, HL = mantissans början och DE = ett steg efter första tecknet efter talet (anroparna gör DEC DE).

CY: inget giltigt tal (DE oförändrad).

Flyttalsformatet (5 bytes, BCD):

```

+0 +1 +2  mantissa, 6 BCD-siffror, första siffran i bit 4-7 av +0
+3      tecken: 0 = positivt, 1 = negativt
+4      exponent + $80; $00 betyder att talet är 0
Värdet är 0.d1d2d3d4d5d6 * 10^(exponent-$80).
Exempel ur ROM:en: 1 = 10 00 00 00 81, PI/2 = 15 70 80 00 81,
2/PI = 63 66 20 00 80.

```

Rutinerna pekar alltid på den sista byten (exponenten).

Inläsningen använder 8 bytes på stacken, IX pekar på dem:

```

IX+0,1  textpekaren
IX+2,3  var nästa siffra ska skrivas i mantissan
IX+4    bit 4/5: minst en siffra har lästs
IX+6    E: bit 7 = decimalpunkt läst, bit 6 = punkten kom före
         första siffran som inte är 0; bit 0-5 = antal siffror
         före punkten, eller antal nollor efter punkten
IX+7    1 = negativ exponent efter "E"

```

D räknar de gällande siffrorna (bit 7 = halv byte). Bara de 6 första sparas; resten räknas bara in i exponenten.

Inledande nollor sparas inte. Siffrorna 7, 8, ... avrundas inte, de huggs av (9999999 -> 9.999999E+6). Tecknen läses med RST \$18, så blanka inne i talet hoppas över ("1 2 . 5" = 12.5). En andra decimalpunkt ändrar inget ("1.2.3" = 1.23). Högst 63 siffror före punkten.

Efter "E" (eller "e") följer ett heltal med tecken, 0-127, som läses med GETLN. Följer ingen siffra direkt efter "E" räknas "E" inte till talet (talet slutar före "E", VAL("1E") ger fel 12). Men efter "E+" och "E-" utan siffror slutar talet efter tecknet: "1E+" = 1.

Omfånget (provat): största tal .999999E127 (9.999999E126), minsta .1E-127 (".1E-127" går, "1E-128" ger CY).

Rutinen på \$1544 läggs som returadress på stacken, så varje tecken-rutin som gör RET går tillbaka till slingan.

```

1513 06 05      GETFLT: LD  B,$05          nollställ de 5 bytena
1515 36 00      L1515: LD  (HL), $00
1517 2B         DEC  HL
1518 10 FB      DJNZ L1515
151A 23         INC  HL
151B E5         PUSH HL
151C D5         PUSH DE
151D DD 21 F8 FF LD  IX,$FFF8          8 bytes arbetsyta
1521 DD 39      ADD  IX,SP
1523 DD F9      LD   SP,IX
1525 1B         DEC  DE          RST $18 börjar på nästa
1526 EB        EX   DE,HL
1527 CD 70 16   CALL FLTSAV
152A 21 04 00   LD   HL,$0004
152D 06 04      LD   B,$04
152F 39         ADD  HL,SP
1530 36 00      L1530: LD  (HL), $00
1532 23         INC  HL

```

1533	10 FB		DJNZ	L1530	
1535	11 00 00		LD	DE,\$0000	
1538	DD 6E 0A		LD	L,(IX+\$0A)	IX+2,3 = mantissans början
153B	DD 66 0B		LD	H,(IX+\$0B)	
153E	DD 75 02	L153E:	LD	(IX+\$02),L	
1541	DD 74 03		LD	(IX+\$03),H	
1544	21 44 15		LD	HL,\$1544	returadress = slingan
1547	E5		PUSH	HL	
1548	DD 36 07 00		LD	(IX+\$07),\$00	E-tecknet nollas i varje varv
154C	CD C4 15		CALL	FLTCH	
154F	38 25		JR	C,L1576	
1551	FE 2E		CP	\$2E	”.”
1553	28 18		JR	Z,L156D	
1555	E6 DF		AND	\$DF	”E” eller ”e”
1557	FE 45		CP	\$45	
1559	CA 00 16		JP	Z,L1600	
155C	47	L155C:	LD	B,A	
155D	DD 7E 04		LD	A,(IX+\$04)	
1560	E6 10		AND	\$10	har någon siffra lästs?
1562	20 71		JR	NZ,L15D5	
1564	21 0A 00	L1564:	LD	HL,\$000A	inget tal: CY
1567	39		ADD	HL,SP	
1568	F9		LD	SP,HL	
1569	D1		POP	DE	
156A	E1		POP	HL	
156B	37		SCF		
156C	C9		RET		
156D	AF	L156D:	XOR	A	decimalpunkt
156E	B2		OR	D	
156F	20 02		JR	NZ,L1573	
1571	CB F3		SET	6,E	punkt före första gällande siffran
1573	CB FB	L1573:	SET	7,E	en andra punkt ändrar inget
1575	C9		RET		
1576	E6 0F	L1576:	AND	\$0F	siffra
1578	47		LD	B,A	
1579	3E 30		LD	A,\$30	
157B	DD B6 04		OR	(IX+\$04)	
157E	DD 77 04		LD	(IX+\$04),A	
1581	AF		XOR	A	
1582	B0		OR	B	inledande nolla?
1583	20 0D		JR	NZ,L1592	
1585	B2		OR	D	
1586	20 0A		JR	NZ,L1592	
1588	B3		OR	E	
1589	5F		LD	E,A	ingen punkt än: nollan hoppas över
158A	C8		RET	Z	
158B	1C		INC	E	nolla efter punkten: räkna
158C	3E 3F		LD	A,\$3F	
158E	A3		AND	E	
158F	C0		RET	NZ	
1590	18 D2		JR	L1564	
1592	7B	L1592:	LD	A,E	
1593	17		RLA		
1594	38 06		JR	C,L159C	
1596	1C		INC	E	siffra före punkten: räkna
1597	3E 3F		LD	A,\$3F	
1599	A3		AND	E	
159A	28 C8		JR	Z,L1564	
159C	DD 73 06	L159C:	LD	(IX+\$06),E	
159F	14		INC	D	
15A0	7A		LD	A,D	
15A1	E6 7F		AND	\$7F	
15A3	28 BF		JR	Z,L1564	fler än 127 siffror
15A5	FE 07		CP	\$07	mer än 6 siffror: sparas inte
15A7	D0		RET	NC	
15A8	CB 7A		BIT	7,D	första eller andra halvan av byten?
15AA	78		LD	A,B	

15AB	DD 6E 02		LD	L, (IX+\$02)	
15AE	DD 66 03		LD	H, (IX+\$03)	
15B1	20 08		JR	NZ, L15BB	
15B3	CB FA		SET	7, D	
15B5	07		RLCA		till bit 4-7
15B6	07		RLCA		
15B7	07		RLCA		
15B8	07		RLCA		
15B9	77		LD	(HL), A	
15BA	C9		RET		
15BB	CB BA	L15BB:	RES	7, D	
15BD	B6		OR	(HL)	bit 0-3
15BE	77		LD	(HL), A	
15BF	23		INC	HL	
15C0	C1		POP	BC	släng returen, spara HL
15C1	C3 3E 15		JP	L153E	
15C4	DD 6E 00	FLTCH:	LD	L, (IX+\$00)	
15C7	DD 66 01		LD	H, (IX+\$01)	
15CA	DF		RST	\$18	nästa tecken
15CB	CD 70 16		CALL	FLTSAV	
15CE	FE 3A		CP	\$3A	
15D0	D0		RET	NC	NC: inte siffra
15D1	FE 30		CP	\$30	
15D3	3F		CCF		
15D4	C9		RET		

Talet är färdigt. Om mantissan är 0 blir exponenten 0 (talet 0), annars räknar FLTEXP ut exponenten + \$80.

15D5	11 00 00	L15D5:	LD	DE, \$0000	
15D8	C5	L15D8:	PUSH	BC	
15D9	EB		EX	DE, HL	
15DA	DD 4E 0A		LD	C, (IX+\$0A)	
15DD	DD 46 0B		LD	B, (IX+\$0B)	
15E0	0A		LD	A, (BC)	
15E1	C5		PUSH	BC	
15E2	B7		OR	A	mantissan 0?
15E3	28 05		JR	Z, L15EA	
15E5	CD 42 16		CALL	FLTEXP	
15E8	C6 80		ADD	A, \$80	exponent + \$80
15EA	DD E3	L15EA:	EX	(SP), IX	IX = mantissan: skriv exponenten
15EC	DD 77 04		LD	(IX+\$04), A	
15EF	DD E1		POP	IX	
15F1	C1		POP	BC	
15F2	D1		POP	DE	
15F3	E1		POP	HL	HL = IX+0, det sista tecknet
15F4	11 0A 00		LD	DE, \$000A	
15F7	DD 19		ADD	IX, DE	
15F9	DD F9		LD	SP, IX	
15FB	D1		POP	DE	
15FC	EB		EX	DE, HL	
15FD	13		INC	DE	DE = ett efter det
15FE	B7		OR	A	
15FF	C9		RET		
1600	DD 7E 04	L1600:	LD	A, (IX+\$04)	
1603	E6 10		AND	\$10	"E" utan siffror före
1605	CA 64 15		JP	Z, L1564	
1608	DD 6E 00		LD	L, (IX+\$00)	
160B	DD 66 01		LD	H, (IX+\$01)	
160E	E5		PUSH	HL	
160F	CD C4 15		CALL	FLTCH	
1612	38 1B		JR	C, L162F	siffror direkt efter "E"
1614	E1		POP	HL	
1615	FE 2B		CP	\$2B	
1617	28 10		JR	Z, L1629	
1619	FE 2D		CP	\$2D	
161B	28 08		JR	Z, L1625	
161D	CD 70 16	L161D:	CALL	FLTSAV	utan tal: slutar före "E" (efter "E+")

1620	3E 45		LD	A, \$45	
1622	C3 5C 15		JP	L155C	
1625	DD 36 07 01	L1625:	LD	(IX+\$07), \$01	negativ exponent
1629	CD C4 15	L1629:	CALL	FLTCH	"E+" eller "E-"
162C	30 EF		JR	NC, L161D	
162E	D5		PUSH	DE	
162F	D1	L162F:	POP	DE	
1630	CD 1E 18		CALL	GETLN	exponenten
1633	46		LD	B, (HL)	slutade vid HL
1634	CD 70 16		CALL	FLTSAV	
1637	7A		LD	A, D	
1638	B7		OR	A	
1639	20 04		JR	NZ, L163F	högst 127
163B	B3		OR	E	
163C	F2 D8 15		JP	P, L15D8	
163F	C3 64 15	L163F:	JP	L1564	

FLTEXP: A = exponenten ur antalet siffror (IX+6) och talet efter "E" (L). Spill (utanför -128..127) går till fel (CY från GETFLT).

1642	DD 7E 06	FLTEXP:	LD	A, (IX+\$06)	
1645	5F		LD	E, A	
1646	E6 3F		AND	\$3F	
1648	47		LD	B, A	
1649	DD 7E 07		LD	A, (IX+\$07)	
164C	B7		OR	A	
164D	28 10		JR	Z, L165F	
164F	CB 73		BIT	6, E	punkt före första siffran?
1651	28 08		JR	Z, L165B	
1653	7D		LD	A, L	
1654	80		ADD	A, B	
1655	FA 6B 16		JP	M, L166B	för litet: fel
1658	2F		CPL		
1659	3C		INC	A	
165A	C9		RET		
165B	AF	L165B:	XOR	A	siffror före punkten - exponent
165C	95		SUB	L	
165D	80		ADD	A, B	
165E	C9		RET		
165F	3E 40	L165F:	LD	A, \$40	
1661	A3		AND	E	
1662	28 04		JR	Z, L1668	
1664	AF		XOR	A	exponent - nollor
1665	90		SUB	B	
1666	85		ADD	A, L	
1667	C9		RET		
1668	78	L1668:	LD	A, B	siffror före punkten + exponent
1669	85		ADD	A, L	
166A	F0		RET	P	
166B	E1	L166B:	POP	HL	för stort: fel
166C	E1		POP	HL	
166D	E1		POP	HL	
166E	18 CF		JR	L163F	
1670	DD 75 00	FLTSAV:	LD	(IX+\$00), L	
1673	DD 74 01		LD	(IX+\$01), H	
1676	C9		RET		

FLTAS6: FLTASC med format \$0C (6 siffror, mellanslag före positivt tal).

1677	3E 0C	FLTAS6:	LD	A, \$0C
------	-------	----------------	----	---------

3.33 FLTASC: skriv flyttalet vid HL som text vid DE

\$1679

In: HL = talets exponentbyte, DE = buffert, A = format:

- bit 0 alltid E-format (d.dddddE+nn)
- bit 1-4 antal gällande siffror (ROM:en använder 6)
- bit 5 inget mellanslag före ett positivt tal
- bit 6 E-format om antalet heltalssiffror (eller nollor efter punkten) + antal siffror ≥ 7 . Med 6 siffror blir det E-format för allt utom $.1 \leq x < 1$ (provat)
- bit 7 för små tal (fler nollor än siffror) skrivs som "." och nollor i stället för "0" (provat: .0005 med 3 siffror ger ".000")

ROM:en använder bara \$0C (FLTAS6: 6 siffror, PRINT m.m.) och \$2C (6 siffror utan mellanslag, när konstanter listas).

Ut: DE efter texten.

Mantissan packas upp till en siffra per byte på stacken, med en nolla före (för spill vid avrundning) och en efter. Sedan avrundas talet till det antal siffror som ska skrivas (siffra ≥ 5 ger +1 på siffran före, med minnessiffra), och nollor i slutet tas bort. Skrivsätt:

- "-" för negativa tal, annars " " (om inte bit 5 är satt)
- exponent 1..6: siffrorna, med "." där heltalsdelen slutar, t.ex. 12.5
- exponent ≤ 0 : "." och nollor först, t.ex. .05 (ingen nolla före punkten)
- annars E-format: första siffran, ".", resten, "E", tecken och exponenten (med PRNUM)
- talet 0 skrivs som "0"

Stackutrymme: 14 bytes. IX+0 = format, +1 = teckenbyten, +2 = antal siffror före punkten (eller nollor efter), +\$0D = 1 om exponenten är ≤ 0 , +5.. = siffrorna.

Med 6 siffror (provat):

- $x \geq 100000$ E-format: 123456 -> 1.23456E+5
- $.1 \leq x < 100000$ alla siffror: 12345.6, .123456
- $1E-6 \leq x < .1$ högst 6 decimaler, avrundat: .0123456 -> .012346, .0000015 -> .000002
- $x < 1E-6$ E-format: 5E-7, 1.2E-7

Fel: när avrundningen går upp till nästa tiopotens (x i [.0999995, .1), [.00999995, .01) ...) blir det en nolla för mycket: .0999999 skrivs ".001" och .00999999 ".0001". Se [\\$1787](#).

Rättat i den nya ROM:en: där står LD E,A i stället för INC E (där blir .0999999 ".1").

```

1679 DD 21 F2 FF FLTASC: LD IX,$FFF2
167D DD 39 ADD IX,SP
167F DD F9 LD SP,IX
1681 DD 77 00 LD (IX+$00),A
1684 D5 PUSH DE
1685 01 FC FF LD BC,$FFFC HL till mantissans början
1688 09 ADD HL,BC
1689 DD E5 PUSH IX
168B 06 03 LD B,$03
168D DD 36 05 00 LD (IX+$05),$00
1691 DD 23 L1691: INC IX packa upp 6 siffror
1693 7E LD A,(HL)
1694 57 LD D,A
1695 1F RRA
1696 1F RRA
1697 1F RRA
1698 1F RRA
1699 E6 0F AND $0F
169B DD 77 05 LD (IX+$05),A
169E DD 23 INC IX
16A0 7A LD A,D
16A1 E6 0F AND $0F
16A3 DD 77 05 LD (IX+$05),A
16A6 23 INC HL
16A7 10 E8 DJNZ L1691
16A9 DD 70 06 LD (IX+$06),B

```

16AC	DD E1	POP	IX	
16AE	7E	LD	A, (HL)	teckenbyten
16AF	DD 77 01	LD	(IX+\$01), A	
16B2	DD 70 0D	LD	(IX+\$0D), B	
16B5	23	INC	HL	

Exponenten ger $IX+2$ och $IX+$0D$:

$e > \$80$ $IX+2 = e - \$80$ (heltalssiffror), $IX+$0D = 0$
 $e \leq \$80$ $IX+2 = \$80 - e$ (nollor efter punkten), $IX+$0D = 1$
 $x = 0$ $IX+2 = 1$, $IX+$0D = 0$

$E = IX+2$. 6 eller fler (heltalssiffror eller nollor) ger E-format.

16B6	7E	LD	A, (HL)	exponenten
16B7	B7	OR	A	
16B8	28 0E	JR	Z, FAZERS	talet 0
16BA	D6 80	SUB	\$80	- \$80
16BC	20 03	JR	NZ, FAEXP	$e = \$80: .1 \leq x < 1$
16BE	DD 34 0D	INC	(IX+\$0D)	
16C1	F2 C9 16	FAEXP:	JP P, FAPOS	$x \geq .1$
16C4	2F	CPL		nollor - 1
16C5	DD 34 0D	INC	(IX+\$0D)	$x < 1$
16C8	3C	FAZERS:	INC A	nollor ($x = 0: 1$)
16C9	DD 77 02	FAPOS:	LD (IX+\$02), A	$IX+2 =$ heltalssiffror/nollor
16CC	5F	LD	E, A	
16CD	FE 06	CP	\$06	6 eller fler siffror före punkten: E-format
16CF	38 04	JR	C, FAFMT	
16D1	DD CB 00 C6	FAEFMT:	SET 0, (IX+\$00)	E-format

FAFMT: C = antal siffror som behålls, räknat från d1. Siffran efter dem avrundar (FAROUN). E-format: $n+1$, högst 6 (?: med $n < 6$ behålls en siffra för mycket). Annars $n + E$ för $x \geq 1$ (högst 7, så med 6 siffror avrundas inget), och $n - E$ (nollorna) för $x < 1$. Med 6 siffror avrundas alltså bara tal i $[1E-6, .1)$.

16D5	DD 7E 00	FAFMT:	LD A, (IX+\$00)	
16D8	1F	RRA		CY = bit 0
16D9	30 0C	JR	NC, FAFIX	
16DB	E6 0F	AND	\$0F	n = antal siffror
16DD	FE 06	CP	\$06	
16DF	38 02	JR	C, FAEDIG	
16E1	3E 05	LD	A, \$05	högst 5
16E3	57	FAEDIG:	LD D, A	D = n
16E4	3C	INC	A	C = D + 1
16E5	18 32	JR	FAROUN	
16E7	E6 0F	FAFIX:	AND \$0F	D = n
16E9	57	LD	D, A	
16EA	83	ADD	A, E	+ E
16EB	FE 07	CP	\$07	$\geq 7?$
16ED	38 06	JR	C, FAFIX1	
16EF	DD CB 00 76	BIT	6, (IX+\$00)	bit 6: E-format
16F3	20 DC	JR	NZ, FAEFMT	
16F5	DD CB 0D 46	FAFIX1:	BIT 0, (IX+\$0D)	$x < 1?$
16F9	20 02	JR	NZ, FASMAL	
16FB	18 1C	JR	FAROUN	$x \geq 1: C = n + E$
16FD	7A	FASMAL:	LD A, D	n - nollor
16FE	93	SUB	E	
16FF	38 02	JR	C, FATINY	
1701	20 16	JR	NZ, FAROUN	$> 0: C = n - nollor$

FATINY: $x < 1$ och inte fler siffror än nollor (bara med $n < 6$). Utan bit 7: " 0". Med bit 7: "." och nollor (?).

1703	DD 7E 00	FATINY:	LD A, (IX+\$00)	
1706	B7	OR	A	
1707	F2 51 17	JP	P, FAZERO	bit 7 = 0: " 0"
170A	E6 0E	AND	\$0E	n
170C	28 43	JR	Z, FAZERO	
170E	0F	RRCA		
170F	5F	LD	E, A	
1710	1D	DEC	E	

1711	0E 01	LD	C, \$01
1713	21 06 00	LD	HL, \$0006
1716	39	ADD	HL, SP
1717	18 71	JR	FAOUT

FAROUN: HL = siffran efter de C som behålls (IX+3/4 sparar den). ≥ 5 : +1 på siffran före, med minnessiffra åt vänster. Minnet kan gå ända in i platsen före d1 (IX+5).

1719	E6 1F	FAROUN: AND	\$1F	C = högst 7
171B	FE 07	CP	\$07	
171D	38 02	JR	C, FAROU1	
171F	3E 07	LD	A, \$07	
1721	4F	FAROU1: LD	C, A	
1722	06 00	LD	B, \$00	
1724	21 08 00	LD	HL, \$0008	IX+6 + C (SP = IX-2)
1727	39	ADD	HL, SP	
1728	09	ADD	HL, BC	
1729	DD 75 03	LD	(IX+\$03), L	spara platsen
172C	DD 74 04	LD	(IX+\$04), H	
172F	7E	LD	A, (HL)	
1730	FE 05	CP	\$05	avrunda
1732	38 13	JR	C, FASTRP	< 5: ingen avrundning
1734	2B	FACARY: DEC	HL	
1735	34	INC	(HL)	+1 med minnessiffra
1736	7E	LD	A, (HL)	
1737	B7	OR	A	kan inte bli 0 (?)
1738	28 0E	JR	Z, FASTR1	
173A	D6 0A	SUB	\$0A	10?
173C	20 03	JR	NZ, FARND1	
173E	77	LD	(HL), A	0 och minne
173F	18 F3	JR	FACARY	
1741	DD 6E 03	FARND1: LD	L, (IX+\$03)	HL = avrundningsplatsen
1744	DD 66 04	LD	H, (IX+\$04)	

FASTRP: gå bakåt över nollor i slutet. C blir antalet siffror som ska skrivas. Är alla noll ($C < 0$) skrivs " 0".

1747	2B	FASTRP: DEC	HL	
1748	7E	FASTR1: LD	A, (HL)	ta bort nollor i slutet
1749	B7	OR	A	
174A	20 1A	JR	NZ, FAOVFL	
174C	2B	DEC	HL	förra siffran
174D	0D	DEC	C	C = antal siffror
174E	F2 48 17	JP	P, FASTR1	
1751	D1	FAZERO: POP	DE	
1752	DD CB 00 6E	BIT	5, (IX+\$00)	bit 5: inget mellanslag
1756	20 04	JR	NZ, FAZER1	
1758	3E 20	LD	A, \$20	" 0"
175A	12	LD	(DE), A	
175B	13	INC	DE	
175C	CD 61 17	FAZER1: CALL	WRZERO	
175F	18 6D	JR	FAEND	
1761	3E 30	WRZERO: LD	A, \$30	
1763	12	LD	(DE), A	
1764	13	INC	DE	
1765	C9	RET		

FAOVFL: har avrundningen gett en etta i platsen före d1 (IX+5) börjar siffrorna där. För $x \geq 1$ blir det en heltalssiffra till ($B = 1$), för $x < 1$ en nolla mindre ($B = -1$). Var nollorna 0 (.999999...) blir talet ≥ 1 . IX+2 ändras rätt med B, men E (som styr utskriften) ökas alltid med 1, även för $x < 1$. Därför skrivs .0999999 som ".001" (provat). Den nya ROM:en har LD E,A ($E = IX+2$) här.

1766	21 07 00	FAOVFL: LD	HL, \$0007	platsen före d1
1769	39	ADD	HL, SP	
176A	7E	LD	A, (HL)	
176B	B7	OR	A	minne dit?
176C	28 1C	JR	Z, FAOUT	
176E	06 01	LD	B, \$01	$x \geq 1$: en heltalssiffra till

1770	DD 7E 0D	LD	A, (IX+\$0D)	
1773	B7	OR	A	
1774	28 02	JR	Z,FAOVF1	
1776	06 FF	LD	B,\$FF	x < 1: en nolla mindre
1778	DD 7E 02	FAOVF1: LD	A, (IX+\$02)	nollor = 0?
177B	B7	OR	A	
177C	20 05	JR	NZ,FAOVF2	
177E	DD 77 0D	LD	(IX+\$0D),A	nu x >= 1
1781	06 01	LD	B,\$01	
1783	80	FAOVF2: ADD	A,B	IX+2 + B
1784	DD 77 02	LD	(IX+\$02),A	
1787	1C	INC	E	fel: E + 1 även när B = -1 (ny ROM: LD E,A)
1788	0C	INC	C	en siffra till
1789	2B	DEC	HL	börja vid IX+5
178A	23	FAOUT: INC	HL	första siffran
178B	79	LD	A,C	högst 6 siffror
178C	FE 07	CP	\$07	
178E	20 01	JR	NZ,FASIGN	
1790	0D	DEC	C	

FAOVF1: B = E (heltalssiffror eller nollor), DE = bufferten. "-"," " eller inget (bit 5).

1791	DD 7E 01	FASIGN: LD	A, (IX+\$01)	tecknet
1794	43	LD	B,E	B = heltalssiffror/nollor
1795	D1	POP	DE	bufferten
1796	1F	RRA		CY = negativt
1797	3E 2D	LD	A,\$2D	"_"
1799	38 09	JR	C,FASGN1	
179B	DD 7E 00	LD	A, (IX+\$00)	bit 5?
179E	E6 20	AND	\$20	
17A0	20 04	JR	NZ,FAFORM	
17A2	3E 20	LD	A,\$20	" "
17A4	12	FASGN1: LD	(DE),A	
17A5	13	INC	DE	
17A6	DD CB 00 46	FAFORM: BIT	0, (IX+\$00)	E-format?
17AA	20 37	JR	NZ,FAEOUT	
17AC	DD CB 0D 46	BIT	0, (IX+\$0D)	x < 1?
17B0	28 22	JR	Z,FAINT	
17B2	79	LD	A,C	inga siffror: "0"
17B3	B7	OR	A	
17B4	20 05	JR	NZ,FADOT	
17B6	CD 61 17	CALL	WRZERO	
17B9	18 13	JR	FAEND	
17BB	3E 2E	FADOT: LD	A,\$2E	"."
17BD	12	LD	(DE),A	
17BE	13	INC	DE	
17BF	AF	FALEAD: XOR	A	B nollor
17C0	B0	OR	B	
17C1	28 06	JR	Z,FAFRAC	
17C3	CD 61 17	CALL	WRZERO	nollor efter punkten
17C6	05	DEC	B	
17C7	20 F6	JR	NZ,FALEAD	
17C9	CD 15 18	FAFRAC: CALL	WRDIG	C siffror
17CC	20 FB	JR	NZ,FAFRAC	
17CE	21 0E 00	FAEND: LD	HL,\$000E	ta bort arbetsytan
17D1	39	ADD	HL,SP	
17D2	F9	LD	SP,HL	
17D3	C9	RET		

FAINT: x >= 1. B heltalssiffror (nollor om siffrorna tar slut), sedan "." och resten om det finns fler (C > 0).

17D4	CD 15 18	FAINT: CALL	WRDIG	heltalsdelen
17D7	05	DEC	B	
17D8	20 FA	JR	NZ,FAINT	
17DA	79	LD	A,C	fler siffror?
17DB	B7	OR	A	
17DC	28 F0	JR	Z,FAEND	
17DE	FA CE 17	JP	M,FAEND	

17E1 18 D8 JR FADOT

FAEOUT: E-format. d1, "." och resten om det finns fler, "E" och exponenten med tecken: $x \geq 1$ ger $+(IX+2-1)$, $x < 1$ ger $-(IX+2+1)$.

17E3	CD 15 18	FAEOUT: CALL	WRDIG	d1
17E6	28 09	JR	Z,FAEXP	bara en siffra
17E8	3E 2E	LD	A,\$2E	"."
17EA	12	LD	(DE),A	
17EB	13	INC	DE	
17EC	CD 15 18	FAEDGT: CALL	WRDIG	resten
17EF	20 FB	JR	NZ,FAEDGT	
17F1	3E 45	FAEXP: LD	A,\$45	"E"
17F3	12	LD	(DE),A	
17F4	13	INC	DE	
17F5	DD 7E 0D	LD	A,(IX+\$0D)	$x < 1?$
17F8	B7	OR	A	
17F9	28 0A	JR	Z,FAEPOS	
17FB	3E 2D	LD	A,\$2D	"-"
17FD	12	LD	(DE),A	
17FE	13	INC	DE	
17FF	DD 7E 02	LD	A,(IX+\$02)	nollor + 1
1802	3C	INC	A	
1803	18 08	JR	FAEPRN	
1805	3E 2B	FAEPOS: LD	A,\$2B	"+"
1807	12	LD	(DE),A	
1808	13	INC	DE	
1809	DD 7E 02	LD	A,(IX+\$02)	heltalssiffror - 1
180C	3D	DEC	A	
180D	6F	FAEPRN: LD	L,A	
180E	26 00	LD	H,\$00	
1810	CD 62 18	CALL	PRNUM	exponenten
1813	18 B9	JR	FAEND	

WRDIG: skriv siffran vid HL som ASCII, HL+1, C-1 (Z när C = 0).

1815	7E	WRDIG: LD	A,(HL)
1816	C6 30	ADD	A,\$30
1818	12	LD	(DE),A
1819	13	INC	DE
181A	23	INC	HL
181B	0D	DEC	C
181C	C9	RET	

GETLN1: HL+1 och hoppa över mellanslag, sedan GETLN.

181D DF **GETLN1:** RST \$18

GETLN: läs ett decimalt heltal (radnummer) vid HL till DE.

Ut: CY = 1 om det inte börjar med en siffra eller om talet går över 65535. HL står då kvar. Annars CY = 0 och HL efter talet.

Mellan siffrorna hoppas mellanslag över (RST \$18), så "1 0" = 10, och efter talet står HL på nästa tecken som inte är blankt. Vid spill är DE odefinierat.

181E	7E	GETLN: LD	A,(HL)	första tecknet
181F	11 00 00	LD	DE,\$0000	DE = 0
1822	FE 30	CP	\$30	
1824	D8	RET	C	inte en siffra: CY
1825	FE 3A	CP	\$3A	
1827	3F	CCF		
1828	D8	RET	C	över "9": CY
1829	E5	PUSH	HL	spara HL för spill
182A	7E	GETLP: LD	A,(HL)	
182B	D6 30	SUB	\$30	
182D	38 04	JR	C,GETEND	inte en siffra: klart
182F	FE 0A	CP	\$0A	0-9?
1831	38 04	JR	C,GETDIG	
1833	33	GETEND: INC	SP	släng sparad HL

1834	33	INC	SP	
1835	A7	AND	A	CY = 0
1836	C9	RET		
1837	CD 00 3F	GETDIG:	CALL MUL10A	DE = DE*10 + siffra, CY vid spill
183A	38 03	JR	C,GETOVF	spill
183C	DF	RST	\$18	nästa, mellanslag hoppas över
183D	18 EB	JR	GETLP	
183F	E1	GETOVF:	POP HL	HL tillbaka, CY
1840	C9	RET		

GETSIN: heltal med tecken: mellanslag hoppas över, sedan "+" eller "-" och **GETLN**. Ut: DE, CY = 1 om det inte är ett tal. "-" ger $DE = -DE$, men intervallet kontrolleras bara som 0..65535, så "40000" blir -25536 och "-40000" blir 25536. Blanka får stå före och efter tecknet. Kommer det ingen siffra efter tecknet står HL efter det.

1841	E7	GETSIN:	RST \$20	hoppa över mellanslag
1842	FE 2B	CP	\$2B	"+"
1844	28 D7	JR	Z,GETLN1	"+": som utan tecken
1846	FE 2D	CP	\$2D	"-"
1848	20 D4	JR	NZ,GETLN	
184A	CD 1D 18	CALL	GETLN1	talet
184D	D8	RET	C	ingen siffra: HL efter tecknet
184E	EB	EX	DE,HL	
184F	CD EE 2A	CALL	NEGHL	DE = -DE
1852	EB	EX	DE,HL	
1853	A7	AND	A	CY = 0
1854	C9	RET		

PRNSIN: skriv heltalet HL med tecken vid DE: mellanslag eller "-", sedan siffrorna (**PRNUM**).

1855	CB 7C	PRNSIN:	BIT 7,H	
1857	3E 20	LD	A,\$20	mellanslag
1859	28 05	JR	Z,PRSGN	
185B	CD EE 2A	CALL	NEGHL	HL = -HL
185E	3E 2D	LD	A,\$2D	"_"
1860	12	PRSGN:	LD (DE),A	tecknet först
1861	13	INC	DE	

PRNUM: skriv HL decimalt som ASCII vid DE, utan inledande nollor. Dividerar med 10000, 1000, 100 och 10 genom upprepad subtraktion.

1862	AF	PRNUM:	XOR A	inga siffror ännu
1863	01 F0 D8	LD	BC,\$D8F0	-10000
1866	CD 7D 18	CALL	PRDIG	
1869	01 18 FC	LD	BC,\$FC18	-1000
186C	CD 7D 18	CALL	PRDIG	
186F	01 9C FF	LD	BC,\$FF9C	-100
1872	CD 7D 18	CALL	PRDIG	
1875	0E F6	LD	C,\$F6	-10
1877	CD 7D 18	CALL	PRDIG	
187A	7D	LD	A,L	entalssiffran skrivs alltid
187B	18 0C	JR	PRDIG1	

PRDIG: $E = HL / -BC$ genom upprepad addition av BC (negativt), och HL = resten. A är 0 så länge ingen siffra har skrivits, så inledande nollor hoppas över (RET Z). Efter en skriven siffra är $A = \$30$, och OR E ger siffran även när $E = 0$. Ut: $A = \$30$.

187D	D5	PRDIG:	PUSH DE	
187E	1E FF	LD	E,\$FF	$E = -1$
1880	1C	L1880:	INC E	räkna
1881	09	ADD	HL,BC	HL - divisorn
1882	38 FC	JR	C,L1880	tills HL blir negativt
1884	ED 42	SBC	HL,BC	en gång för mycket: lägg tillbaka
1886	B3	OR	E	inledande nolla?
1887	D1	POP	DE	
1888	C8	RET	Z	hoppa över
1889	F6 30	PRDIG1:	OR \$30	ASCII
188B	12	LD	(DE),A	
188C	13	INC	DE	

188D	3E 30	LD	A, \$30	nu skrivs även nollor
188F	C9	RET		

3.34 TOKLIN: gör om en inskriven rad till intern kod

\$1890

In: HL = texten (slutar med \$0D).

Ut: IX = \$FEB8, där raden ligger i samma format som i programmet: [längd][nr lo][nr hi][kod ...][\$0D]. HL = längden.

CY + A = felkod vid fel.

Utan radnummer blir numret 0 (direktkommando). Om det bara finns ett radnummer är längden 0 (raden ska tas bort).

Numret läses direkt vid HL, utan blanka före (kommandoslingan har redan hoppat över dem). Numret 0 är detsamma som inget nummer, så "0 PRINT 7" körs direkt. Ett nummer över 65535 räknas inte som nummer: texten kompileras som en sats och ger fel 11.

Satserna, åtskilda av ":", kompileras en i taget av TOKSTM. ":" följer med i koden. Efter den sista måste \$0D komma, annars fel 14.

1890	CD 1E 18	TOKLIN:	CALL GETLN	radnummer
1893	30 03		JR NC, L1898	
1895	11 00 00		LD DE, \$0000	inget nummer: 0
1898	E5	L1898:	PUSH HL	
1899	21 B8 FE		LD HL, \$FEB8	bygg raden vid \$FEB8
189C	E5		PUSH HL	
189D	36 00		LD (HL), \$00	längden 0 tills vidare
189F	CD 22 10		CALL WRDE1	radnumret efter längden
18A2	EB		EX DE, HL	
18A3	E1		POP HL	
18A4	E3		EX (SP), HL	HL = texten, \$FEB8 på stacken
18A5	7E		LD A, (HL)	
18A6	FE 0D		CP \$0D	bara ett radnummer?
18A8	20 03		JR NZ, L18AD	
18AA	DD E1		POP IX	IX = \$FEB8, längd 0
18AC	C9		RET	
18AD	CD C7 18	L18AD:	CALL TOKSTM	
18B0	DD E1		POP IX	
18B2	D8		RET C	
18B3	7E		LD A, (HL)	
18B4	FE 0D		CP \$0D	måste sluta med \$0D
18B6	3E 8E		LD A, \$8E	fel 14
18B8	37		SCF	
18B9	C0		RET NZ	
18BA	DD E5		PUSH IX	
18BC	ED A0		LDI	\$0D följer med
18BE	EB		EX DE, HL	
18BF	A7		AND A	
18C0	C1		POP BC	
18C1	ED 42		SBC HL, BC	längden
18C3	DD 75 00		LD (IX+\$00), L	HL ut = längden
18C6	C9		RET	

TOKSTM: kompilera satser tills det inte kommer något ":".

18C7	CD D4 18	TOKSTM:	CALL TOKKW	
18CA	D8		RET C	
18CB	7E		LD A, (HL)	
18CC	EE 3A		XOR \$3A	":"?
18CE	C0		RET NZ	
18CF	ED A0		LDI	":" följer med i koden
18D1	E7		RST \$20	blanka efter ":"
18D2	18 F3		JR TOKSTM	

3.35 TOKKW: kompilera en sats

\$18D4

Nyckelordet letas upp med **KWSRCH**, först i **STKW1** och sedan i **STKW2**.

STKW1: token \$80-\$96 skrivs, rutinen tas ur **STTAB1**

STKW2: \$86 och sedan k (plats i **STKW2**), rutinen tas ur **STTAB2**

inget: \$83 (underförstådd **LET**) och **LET**-rutinen. Om det inte heller är en tilldelning blir felet 11, **FÖRSTÅR EJ**, i stället för 15.

Rutinen hämtas ur tabellen med $2 * \text{token}$; för token \$80-\$96 blir det $2 * (\text{token} - \$80)$, eftersom biten som skiftas ut försvinner. Rutinen anropas med **HL** = texten efter nyckelordet och **DE** = var koden ska skrivas. Den ger **CY** och felkod om satsen är fel.

18D4	D5	TOKKW:	PUSH DE	
18D5	11 11 19		LD DE,STKW1	STKW1
18D8	CD F5 09		CALL KWSRCH	
18DB	01 08 1A		LD BC,STTAB1	
18DE	28 22		JR Z,L1902	funnen i STKW1
18E0	11 9C 19		LD DE,STKW2	STKW2
18E3	CD F5 09		CALL KWSRCH	
18E6	01 36 1A		LD BC,STTAB2	
18E9	D1		POP DE	
18EA	EB		EX DE,HL	
18EB	28 0F		JR Z,L18FC	
18ED	36 83		LD (HL),\$83	\$83: underförstådd LET
18EF	EB		EX DE,HL	
18F0	13		INC DE	
18F1	CD 89 1B		CALL CMPLT	en tilldelning?
18F4	D0		RET NC	
18F5	FE 8F		CP \$8F	fel 15 blir fel 11
18F7	37		SCF	
18F8	C0		RET NZ	
18F9	3E 8B		LD A,\$8B	
18FB	C9		RET	
18FC	36 86	L18FC:	LD (HL),\$86	\$86 + token i STKW2
18FE	23		INC HL	
18FF	EB		EX DE,HL	
1900	18 01		JR L1903	
1902	D1	L1902:	POP DE	
1903	12	L1903:	LD (DE),A	token i koden
1904	13		INC DE	
1905	E5		PUSH HL	
1906	87		ADD A,A	2*token
1907	6F		LD L,A	
1908	26 00		LD H,\$00	
190A	09		ADD HL,BC	
190B	4E		LD C,(HL)	BC = rutinen
190C	23		INC HL	
190D	46		LD B,(HL)	
190E	E1		POP HL	HL = texten efter nyckelordet
190F	C5		PUSH BC	hoppa till rutinen
1910	C9		RET	

STKW1: satsernas nyckelord, tabell 1.

Varje post är token (bit 7 satt) följt av ordet, och tabellen slutar med \$FF. Tabellen används på två sätt:

- * **KWSRCH** (inmatning) jämför texten med orden i tur och ordning och tar det första som stämmer. Därför står **INPUTLINE** före **INPUT** och **ONERRORGOTO** före **ON**. **KWSRCH** hoppar över blanktecken, så "**ON ERROR GOTO**" och "**INPUT LINE**" stämmer. Ordet behöver bara vara början av texten: "**LETTER=1**" blir **LET TER=1**.
- * **DETOK** (listning) letar upp token med **CPIR** från **STKW1** och framåt, och texten slutar vid nästa byte med bit 7. Därför kan tokens efter \$FF (**KWLIST**, **THEN**, **ELSE** ...) också listas.

"," ger samma token som **PRINT**, men listas som ";" (\$85).

1911 82 4C 45 54 **STKW1:** DEFB \$82,"LET"

```

1915 80 47 4F 54... DEFB $80,"GOTO" GOTO
191A 84 50 52 49... DEFB $84,"PRINT"
1920 85 3B DEFB $85,";" ; = PRINT
1922 88 49 46 DEFB $88,"IF"
1925 8A 49 4E 50... DEFB $8A,"INPUTLINE" före INPUT
192F 89 49 4E 50... DEFB $89,"INPUT"
1935 8B 46 4F 52 DEFB $8B,"FOR"
1939 8C 4E 45 58... DEFB $8C,"NEXT"
193E 8E 52 45 41... DEFB $8E,"READ"
1943 8F 52 45 53... DEFB $8F,"RESTORE"
194B 90 47 4F 53... DEFB $90,"GOSUB"
1951 91 52 45 54... DEFB $91,"RETURN"
1958 94 4F 4E 45... DEFB $94,"ONERRORGOTO" före ON
1964 92 4F 4E DEFB $92,"ON"
1967 93 44 41 54... DEFB $93,"DATA"
196C 96 DEFB $96 DEF FN
196D 44 45 46 46... KWDEFN: DEFB "DEFFN" listas via LD HL,KWDEFN ($120B)
1972 FF DEFB $FF slut på sökningen

```

KWLIST: tokens som bara listas. \$81 (**GOTO** efter THEN/ELSE) och \$83 (**LET** utan ordet) har ingen text. \$95 är en rad som inte gick att tokenisera (**\$0BEB**): den sparas som \$95 och den råa texten, och listas som "???" följt av texten.

```

1973 81 83 95 3F... KWLIST: DEFB $81,$83,$95,"???" " $81,$83: ingen text; $95: felaktig rad

```

Tabeller med ett ord var, som **KWSRCH** söker i under kompileringen: THEN (**\$1E00**), ELSE (**\$1E22**, **\$2196**), STEP (**\$1C9F**), TO (**\$1C89**) och **ASFILE** (**\$2062**, **OPEN/PREPARE**). Tokens \$BA-\$BD och \$B8 står sedan i koden och listas genom **STKW1**-tabellen.

```

197A BD 54 48 45... KWTHEN: DEFB $BD,"THEN"
197F FF DEFB $FF
1980 BC 45 4C 53... KWELSE: DEFB $BC,"ELSE"
1985 FF DEFB $FF
1986 BB 53 54 45... KWSTEP: DEFB $BB,"STEP"
198B FF DEFB $FF
198C BA 54 4F KWTO: DEFB $BA,"TO"
198F FF DEFB $FF
1990 B8 41 53 46... KWASF: DEFB $B8,"ASFILE"
1997 FF DEFB $FF

```

AS (\$B7): ingen söker i den här tabellen, och \$B7 i uttryckskoden betyder tilldelning (typ 1), inte AS. Posten verkar oanvänd (?).

```

1998 B7 41 53 KWAS: DEFB $B7,"AS"
199B FF DEFB $FF

```

STKW2: satsernas nyckelord, tabell 2. Koden k skrivs efter token \$86 och är index i **STTAB2** och **STEXE2**. Koderna 3, C och D saknar ord.

```

199C 80 44 49 4D STKW2: DEFB $80,"DIM"
19A0 81 50 4F 4B... DEFB $81,"POKE"
19A5 82 4F 55 54 DEFB $82,"OUT"
19A9 84 52 45 4D DEFB $84,"REM"
19AD 85 4F 50 45... DEFB $85,"OPEN"
19B2 86 50 52 45... DEFB $86,"PREPARE"
19BA 87 43 4C 4F... DEFB $87,"CLOSE"
19C0 88 52 41 4E... DEFB $88,"RANDOMIZE"
19CA 89 53 54 4F... DEFB $89,"STOP"
19CF 8A 45 4E 44 DEFB $8A,"END"
19D3 8B 43 4D 44 DEFB $8B,"CMD"
19D7 8E 53 45 54... DEFB $8E,"SETDOT"
19DE 8F 43 4C 52... DEFB $8F,"CLRDOT"
19E5 90 47 45 54 DEFB $90,"GET"
19E9 91 43 48 41... DEFB $91,"CHAIN"
19EF 92 4B 49 4C... DEFB $92,"KILL"
19F4 93 4E 41 4D... DEFB $93,"NAME"
19F9 94 54 52 41... DEFB $94,"TRACE"
19FF 95 4E 4F 54... DEFB $95,"NOTRACE"
1A07 FF DEFB $FF slut

```

3.36 STTAB1/STTAB2: satsernas kompileringsrutiner

\$1A08

Index = token - \$80 (STKW1) eller kod (STKW2). Körrutinerna står i STEXE1/STEXE2 (\$0E14) i samma ordning.

token	sats	kompilering	körning
\$80	GOTO	CMPREF	GOTO
\$81	(GOTO efter THEN/ELSE)	CMPREF	GOTO
\$82	LET	CMPLET	LET
\$83	(LET utan ordet)	CMPLET	LET
\$84	PRINT	CMPprt	PRINTS
\$85	; (= PRINT)	CMPprt	PRINTS
\$86	tabell 2	-	EXEST2
\$87	hopp (IF)	-	EXESKP
\$88	IF	CMPIF	IF
\$89	INPUT	CMPINP	INPT
\$8A	INPUT LINE	CMPINL	INPLIN
\$8B	FOR	CMPFOR	FOR
\$8C	NEXT	CMPNXT	NEXT
\$8D	(inget ord)	fel 8	fel 8
\$8E	READ	CMPRD	READ
\$8F	RESTORE	CMPRES	RESTOR
\$90	GOSUB	CMPREF	GOSUB
\$91	RETURN	NOCODE	RETURN
\$92	ON	CMPON	ON
\$93	DATA	CMPREM	DATA
\$94	ON ERROR GOTO	CMPREF	ONERRG
\$95	(felaktig rad) -	-	fel 53
\$96	DEF FN	CMPDEF	DEF

Tabell 2 (\$86 k):

0 DIM	CMPDIM	DIM	1 POKE	CMPPOK	POKE
2 OUT	CMPOUT	OUTST	3 (inget ord)	CMPINX	fel 8
4 REM	CMPREM	REM	5 OPEN	CMPOPn	OPENST
6 PREPARE	CMPOPn	PREPST	7 CLOSE	CMPCLS	CLOSST
8 RANDOMIZE	CMPNOP	RANDOM	9 STOP	NOCODE	STOP
A END	NOCODE	ENDPRG	B CMD	tillägg	tillägg
C, D (inget ord)	fel 8		E SETDOT	CMPDOT	SETDOT
F CLRDOT	CMPDOT	CLRDOT	10 GET	CMPGET	GET
11 CHAIN	CMPSTX	CHAIN	12 KILL	CMPSTX	tillägg
13 NAME	tillägg	tillägg	14 TRACE	NOCODE	TRACE
15 NOTRACE	NOCODE	NOTRAC			

”tillägg” = ingång till tilläggs-ROM:en (\$0147-), fel 8 om den saknas. Kod 3, C och D har inga nyckelord och kan inte skrivas in; de är nog platser för satser som inte finns i denna version (?).

1A08	C7 1D	STTAB1:	DEFW CMPREF
1A0A	C7 1D		DEFW CMPREF
1A0C	89 1B		DEFW CMPLET
1A0E	89 1B		DEFW CMPLET
1A10	54 21		DEFW CMPprt
1A12	54 21		DEFW CMPprt
1A14	00 00		DEFW RESET
1A16	00 00		DEFW RESET
1A18	FB 1D		DEFW CMPIF
1A1A	58 1E		DEFW CMPINP
1A1C	6B 1F		DEFW CMPINL
1A1E	83 1C		DEFW CMPFOR
1A20	B7 1C		DEFW CMPNXT
1A22	09 07		DEFW L0709
1A24	6E 1E		DEFW CMPRD
1A26	81 22		DEFW CMPRES
1A28	C7 1D		DEFW CMPREF
1A2A	B5 22		DEFW NOCODE
1A2C	BF 1F		DEFW CMPON
1A2E	6B 22		DEFW CMPREM
1A30	C7 1D		DEFW CMPREF
1A32	00 00		DEFW RESET
1A34	62 1A		DEFW CMPDEF
1A36	AA 1B	STTAB2:	DEFW CMPDIM
1A38	04 21		DEFW CMPPOK
1A3A	FF 20		DEFW CMPOUT

1A3C	70 23	DEFW	CMPINX
1A3E	6B 22	DEFW	CMPREM
1A40	5D 20	DEFW	CMPOPN
1A42	5D 20	DEFW	CMPOPN
1A44	71 20	DEFW	CMPCLS
1A46	6A 22	DEFW	CMPNOP
1A48	B5 22	DEFW	NOCODE
1A4A	B5 22	DEFW	NOCODE
1A4C	47 01	DEFW	L0147
1A4E	09 07	DEFW	L0709
1A50	09 07	DEFW	L0709
1A52	B6 22	DEFW	CMPDOT
1A54	B6 22	DEFW	CMPDOT
1A56	A8 23	DEFW	CMPGET
1A58	6C 23	DEFW	CMPSTX
1A5A	6C 23	DEFW	CMPSTX
1A5C	51 01	DEFW	L0151
1A5E	B5 22	DEFW	NOCODE
1A60	B5 22	DEFW	NOCODE

3.37 CMPDEF: kompilera DEF FNx(p1,p2,...)=uttryck

\$1A62

Koden blir:

\$96 [namnbyte][bokstav] [antal param] [2 bytes per param] "=" uttryck \$B9+typ

Högst 4 parametrar (annars fel 13). Uttrycket omvandlas till funktionens typ (efter namnet: A = flyttal, A% = heltal). Parametrarnas namn kopplas vid förlänkeningen (LNKDEF) till funktionens argument. En parameter döljer variabeln med samma namn bara i uttrycket; andra variabler är de vanliga. Finns två DEF för samma namn gäller den första. Ett heltalsargument avrundas nedåt som vid tilldelning (2.7 -> 2, -2.5 -> -3). En strängfunktion (FNA\$) går att skriva in men ger fel 8 vid anropet (FN).

1A62	CD CB 26	CMPDEF:	CALL	VARNAM	funktionens namn
1A65	D8		RET	C	
1A66	EB		EX	DE,HL	
1A67	CD 29 10		CALL	WRBC	namnet efter \$96
1A6A	36 00		LD	(HL), \$00	antal parametrar = 0
1A6C	E5		PUSH	HL	
1A6D	DD E1		POP	IX	IX = antalet
1A6F	23		INC	HL	
1A70	EB		EX	DE,HL	
1A71	7E		LD	A, (HL)	
1A72	FE 28		CP	\$28	"("?
1A74	20 22		JR	NZ, L1A98	
1A76	DF	L1A76:	RST	\$18	
1A77	CD CB 26		CALL	VARNAM	parameterns namn
1A7A	D8		RET	C	
1A7B	EB		EX	DE,HL	
1A7C	CD 29 10		CALL	WRBC	parameterns namn
1A7F	EB		EX	DE,HL	
1A80	DD 7E 00		LD	A, (IX+\$00)	
1A83	FE 04		CP	\$04	högst 4
1A85	3E 8D		LD	A, \$8D	fel 13
1A87	37		SCF		
1A88	C8		RET	Z	
1A89	DD 34 00		INC	(IX+\$00)	en parameter till
1A8C	7E		LD	A, (HL)	
1A8D	FE 2C		CP	\$2C	","
1A8F	28 E5		JR	Z, L1A76	
1A91	FE 29		CP	\$29	)"
1A93	3E 92		LD	A, \$92	fel 18
1A95	37		SCF		
1A96	C0		RET	NZ	
1A97	DF		RST	\$18	
1A98	FE 3D	L1A98:	CP	\$3D	"="
1A9A	3E 8F		LD	A, \$8F	fel 15
1A9C	37		SCF		
1A9D	C0		RET	NZ	

1A9E	ED A0	LDI	"=" följer med
1AA0	E7	RST \$20	
1AA1	DD 7E FE	LD A, (IX-\$02)	funktionens typ
1AA4	E6 03	AND \$03	
1AA6	47	LD B,A	
1AA7	C3 72 23	JP CMPTYX	uttrycket, omvandlat till typen

DEF vid körning: hoppa över resten av raden. DE-4 är radens längd-byte om DEF står först på raden, och DE sätts på radens \$0D. Resten av raden körs alltså inte ("10 DEF FNB(X)=X:PRINT 5" skriver inget). DEF FN får bara stå först på en rad: annars hittar förlänkningen den inte (anropet ger fel 57), och DEF hoppar till en felaktig adress (t.ex. fel 4). I direktläge fel 2 (NODIR).

1AAA	CD BF 0D	DEF: CALL NODIR	
1AAD	21 FC FF	LD HL, \$FFFC	DE-4 = radens längdbyte
1AB0	19	ADD HL, DE	
1AB1	5E	LD E, (HL)	E = längden
1AB2	16 00	LD D, \$00	
1AB4	19	ADD HL, DE	
1AB5	EB	EX DE, HL	
1AB6	1B	DEC DE	DE på \$0D
1AB7	C9	RET	

3.38 CMPFN: kompilera ett anrop FNx(a1,a2,...) i ett uttryck

\$1AB8

Varje argument kompileras som ett uttryck och följs av en 5 bytes lång post:

\$DB [avstånd bakåt till förra \$DB-posten, 0 för den första]
[argumentets typ] [parameterns typ] [omvandling]

De två sista fylls i vid förlänkningen (LNKFN, \$111D): parameterns typ, och \$D3 (gör inget) eller \$D9/\$DA som omvandlar argumentet. Efter argumenten:

\$CD+antal \$00 (funktionskod 0 = användarfunktion)
[namnbyte][bokstav] [2 bytes: adress till DEF, fylls i vid förlänkningen] [avstånd bakåt till sista \$DB-posten]

Högst 4 argument (fel 13). Ut: B = funktionens typ.

Avstånden gör att argumentposterna kan hittas bakåt från anropet.

1AB8	CD CB 26	CMPFN: CALL VARNA	
1ABB	D8	RET C	
1ABC	DD 21 00 00	LD IX, \$0000	
1AC0	DD E5	PUSH IX	IX+0 = antal arg, IX+1 = senaste \$DB-post
1AC2	DD 39	ADD IX, SP	
1AC4	C5	PUSH BC	BC = funktionens namn
1AC5	7E	LD A, (HL)	
1AC6	FE 28	CP \$28	"(?"
1AC8	20 37	JR NZ, L1B01	
1ACA	DF	L1ACA: RST \$18	
1ACB	DD E5	PUSH IX	
1ACD	CD 01 27	CALL CMPEXP	argumentet
1AD0	DD E1	POP IX	
1AD2	38 52	JR C, L1B26	
1AD4	EB	EX DE, HL	
1AD5	36 DB	LD (HL), \$DB	argumentpost \$DB
1AD7	23	INC HL	
1AD8	DD 7E 00	LD A, (IX+\$00)	
1ADB	A7	AND A	
1ADC	28 0A	JR Z, L1AE8	
1ADE	FE 04	CP \$04	
1AE0	3E 8D	LD A, \$8D	fel 13
1AE2	28 42	JR Z, L1B26	
1AE4	DD 7E 01	LD A, (IX+\$01)	avstånd till förra posten
1AE7	95	SUB L	
1AE8	77	L1AE8: LD (HL), A	avståndet, 0 först
1AE9	DD 75 01	LD (IX+\$01), L	
1AEC	DD 34 00	INC (IX+\$00)	
1AEF	23	INC HL	
1AF0	70	LD (HL), B	typ
1AF1	23	INC HL	

1AF2	23		INC	HL	
1AF3	23		INC	HL	
1AF4	EB		EX	DE,HL	DE efter posten
1AF5	7E		LD	A,(HL)	
1AF6	FE 2C		CP	\$2C	
1AF8	28 D0		JR	Z,L1ACA	
1AFA	FE 29		CP	\$29	
1AFC	3E 92		LD	A,\$92	fel 18
1AFE	20 26		JR	NZ,L1B26	
1B00	DF		RST	\$18	
1B01	EB	L1B01:	EX	DE,HL	
1B02	3E CD		LD	A,\$CD	\$CD + antal argument
1B04	DD 86 00		ADD	A,(IX+\$00)	
1B07	77		LD	(HL),A	
1B08	23		INC	HL	
1B09	36 00		LD	(HL),\$00	funktionskod 0
1B0B	23		INC	HL	
1B0C	C1		POP	BC	
1B0D	CD 29 10		CALL	WRBC	namnet
1B10	23		INC	HL	plats för DEF-adressen
1B11	23		INC	HL	
1B12	DD 7E 00		LD	A,(IX+\$00)	
1B15	A7		AND	A	
1B16	28 04		JR	Z,L1B1C	
1B18	DD 7E 01		LD	A,(IX+\$01)	
1B1B	95		SUB	L	
1B1C	77	L1B1C:	LD	(HL),A	
1B1D	23		INC	HL	
1B1E	EB		EX	DE,HL	
1B1F	DD E1		POP	IX	släng ramen
1B21	3E 03		LD	A,\$03	B = typ
1B23	A1		AND	C	
1B24	47		LD	B,A	
1B25	C9		RET		
1B26	C1	L1B26:	POP	BC	släng namn och ram
1B27	C1		POP	BC	
1B28	37		SCF		
1B29	C9		RET		

3.39 FN vid körning (funktionskod 0 i EXPTAB2)

\$1B2A

1. **STKCHK**, sedan läses adressen till **DEF**-raden ur anropet.
2. Den gamla ramen (FE3E) sparas på stacken och FE3E pekar på den nya. Genom FE3E hittar parametrarna i **DEF**-uttrycket argumenten.
3. **RST \$38** räknar ut **DEF**-uttrycket. Därefter återställs FE3E.
4. Resultatet läggs på stacken som uttryckets värde: flyttal (5 bytes) eller heltal. En strängfunktion ger fel 8, FINNS EJ I DETTA SYSTEM, alltså finns det inga strängfunktioner med **DEF FN**.

1B2A	CD 89 33	FN:	CALL	STKCHK	fel 3 vid djup rekursion
1B2D	EB		EX	DE,HL	
1B2E	23		INC	HL	
1B2F	CD 1C 10		CALL	RDDE1	DEF-adressen
1B32	E5		PUSH	HL	
1B33	2A 3E FE		LD	HL,(FNFRAM)	spara ramen
1B36	E5		PUSH	HL	
1B37	21 04 00		LD	HL,\$0004	
1B3A	39		ADD	HL,SP	
1B3B	22 3E FE		LD	(FNFRAM),HL	
1B3E	13		INC	DE	
1B3F	FF		RST	\$38	räkna ut DEF-uttrycket
1B40	78		LD	A,B	
1B41	ED 5B 3E FE		LD	DE,(FNFRAM)	
1B45	D5		PUSH	DE	IX = ramen
1B46	DD E1		POP	IX	

1B48	DD 5E FC	LD	E, (IX-\$04)	
1B4B	DD 56 FD	LD	D, (IX-\$03)	
1B4E	ED 53 3E FE	LD	(FNFRAM), DE	återställ ramen
1B52	DD 5E FE	LD	E, (IX-\$02)	
1B55	DD 56 FF	LD	D, (IX-\$01)	
1B58	A7	AND	A	flyttal?
1B59	28 0C	JR	Z, L1B67	
1B5B	3D	DEC	A	
1B5C	C2 09 07	JP	NZ, L0709	sträng: fel 8
1B5F	E5	PUSH	HL	
1B60	CD 7B 1B	CALL	FNRES	
1B63	C1	POP	BC	
1B64	F9	LD	SP, HL	
1B65	C5	PUSH	BC	
1B66	F7	RST	\$30	
1B67	CD 7B 1B	L1B67: CALL	FNRES	
1B6A	D5	PUSH	DE	
1B6B	EB	EX	DE, HL	
1B6C	1B	DEC	DE	
1B6D	21 06 00	LD	HL, \$0006	
1B70	39	ADD	HL, SP	
1B71	01 05 00	LD	BC, \$0005	
1B74	ED B8	LDDR		flyttalet till stacken
1B76	EB	EX	DE, HL	
1B77	23	INC	HL	
1B78	D1	POP	DE	
1B79	F9	LD	SP, HL	resultatet överst, argumenten borta
1B7A	F7	RST	\$30	

FNRES: hitta slutet på argumenten. HL = IX = ramen (FNFRAM), E = \$FF: ARGWLK går igenom alla argumentposter och ger HL = första byten ovanför det första argumentet, där resultatet ska ligga.

Ut: DE = koden + 1 (efter avståndsbyten).

1B7B	13	FNRES: INC	DE	efter avståndsbyten
1B7C	D5	PUSH	DE	
1B7D	1E FF	LD	E, \$FF	gå igenom alla
1B7F	DD E5	PUSH	IX	
1B81	E1	POP	HL	
1B82	C3 B0 33	JP	ARGWLK	
1B85	13	SKPARG: INC	DE	
1B86	13	INC	DE	
1B87	13	INC	DE	
1B88	F7	RST	\$30	

3.40 LET: kompilering

\$1B89

[LET] v = uttryck (LET kan utelämnas, se TOKKW)

Koden blir ett enda uttryck som slutar med tilldelningen:

[index] [variabel, 5 bytes] [uttryck] [\$B6 + typ]

Uttrycket omvandlas till variabelns typ (TPCNPV): flyttal till heltal avrundas nedåt (FTOINT), A\$=5 ger fel 13 och A%="Q" fel 17. Utan "=" blir det fel 15.

Tilldelningen (\$B6-\$B8) avslutar också körningen av uttrycket: den gör RET i stället för att hämta nästa token.

1B89	CD EE 25	CMPLT: CALL	CMPVAR	
1B8C	D8	L1B8C: RET	C	
1B8D	7E	LD	A, (HL)	
1B8E	FE 3D	CP	\$3D	"="?
1B90	3E 8F	LD	A, \$8F	fel 15
1B92	37	SCF		
1B93	C0	RET	NZ	
1B94	C5	PUSH	BC	variabelns typ
1B95	DF	RST	\$18	
1B96	CD 01 27	CALL	CMPEXP	uttrycket
1B99	38 5C	JR	C, CMPERR	

1B9B	F1	POP	AF	
1B9C	CD 51 25	CALL	TYPCNV	till variabelns typ
1B9F	D8	RET	C	
1BA0	3E B6	LD	A,\$B6	\$B6 + typ: tilldelning
1BA2	C3 68 23	JP	PUTTOK	

CMPFLV: som **CMPLET**, men variabeln får inte vara en sträng (fel 17) och inte ha index (då följer "(" i stället för "=": fel 15).
Används av **FOR**.

1BA5	CD 7F 23	CMPFLV:	CALL	CMPSV
1BA8	18 E2		JR	L1B8C

3.41 DIM: kompilering

\$1BAA

DIM post [, post ...]

Varje post blir:

[index] [längd eller \$D2] \$CC [variabel, 5 bytes]

Hela listan slutar med \$BF. Tal ska ha 1 eller 2 index. En sträng kan ha 0-2 index och "=längd" efter (**DIM** A\$(5)=20, **DIM** B\$=40); utan längd skrivs \$D2, som ger längden 0 vid körning. Indexen och längden är heltal. Ett tal utan index eller med tre index ger fel 4, en längd efter ett tal eller något annat än ",", mellan posterna fel 14.

1BAA	CD BD 1B	CMPDIM:	CALL	CMPDMI
1BAD	D8		RET	C
1BAE	7E		LD	A,(HL) ",,"?
1BAF	EE 2C		XOR	\$2C
1BB1	06 00		LD	B,\$00
1BB3	20 03		JR	NZ,CMPBF
1BB5	DF		RST	\$18
1BB6	18 F2		JR	CMPDIM

Skriv \$BF, slut på **DIM**-listan.

1BB8	3E BF	CMPBF:	LD	A,\$BF
1BBA	12		LD	(DE),A
1BBB	13		INC	DE
1BBC	C9		RET	

CMPDMI: en post i **DIM**-listan.

1BBD	CD CB 26	CMPDMI:	CALL	VARNAM
1BC0	D8		RET	C
1BC1	3E 03		LD	A,\$03
1BC3	A1		AND	C
1BC4	EE 02		XOR	\$02 Z = sträng
1BC6	C5		PUSH	BC
1BC7	DD 21 48 25		LD	IX,\$2548 sträng: 0-2 index
1BCB	28 04		JR	Z,L1BD1
1BCD	DD 21 4D 25		LD	IX,\$254D tal: 1-2 index
1BD1	F5	L1BD1:	PUSH	AF
1BD2	CD 4E 26		CALL	CMPARG
1BD5	38 20		JR	C,CMPERR
1BD7	F1		POP	AF
1BD8	20 16		JR	NZ,L1BF0
1BDA	7E		LD	A,(HL)
1BDB	FE 3D		CP	\$3D "=" längd?
1BDD	28 06		JR	Z,L1BE5
1BDF	3E D2		LD	A,\$D2 \$D2: längd 0
1BE1	12		LD	(DE),A
1BE2	13		INC	DE
1BE3	18 0B		JR	L1BF0
1BE5	DF	L1BE5:	RST	\$18
1BE6	C5		PUSH	BC
1BE7	CD 01 27		CALL	CMPEXP
1BEA	D4 61 25		CALL	NC,TOINT längden är ett heltal
1BED	C1		POP	BC

```

1BEE 38 07          JR  C,CMPERR
1BF0 3E CC          L1BF0: LD  A,$CC          $CC: DIM-post
1BF2 12             LD  (DE),A
1BF3 13             INC  DE
1BF4 C3 FC 25       JP   L25FC          sedan variabeln

```

CMPERR: fel under kompileringen; ta bort sparad BC och sätt CY.

```

1BF7 C1          CMPERR: POP  BC
1BF8 37          SCF
1BF9 C9          RET

```

DIM vid körning: fel 2 i direktläge. Sedan körs koden som ett uttryck: indexen och längden hamnar på stacken, och \$CC (**DIMELM**) dimensionerar variabeln. \$BF avslutar.

```

1BFA CD BF 0D      DIM:   CALL NODIR

```

LET vid körning: bara RST \$30, dvs. kör koden vid DE. Tilldelningen sist gör RET direkt till den som anropade satsen.

```

1BFD F7          LET:   RST  $30

```

3.42 DIMELM: \$CC i uttryckskoden, dimensionera en variabel

\$1BFE

Efter \$CC kommer variabeln (5 bytes); IX = dess post. Värdena på stacken, överst först: [längd (strängar)] [index 2] [index 1].

Bit 2-3 i namnbyten (antal index) väljer fall:

- 0 index sträng: **DIMSTR** med längden
- 1 index **DIM1**, sedan varje element (6 bytes) som en sträng
- 2 index **DIM2**, sedan likadant

Tal behöver inte någon längd. En strängvektor med längden 0 får bara sina element, och texten reserveras när något tilldelas: 80 bytes, eller textens längd om den är längre (**ASGNS**). Högsta index 0 lämnar $+2/3 = 0$, dvs. odimensionerad, och första användningen ger **DIM** (10) (**VAR1**). Högsta index -1 ger 0 bytes, och då nollar **ALLOC** hela minnet ($BC = 0 - 1$): återhoppet går till \$0000, kallstart.

En post för en vektor:

- +0/1 adress till elementen +2/3 antal element - 1
- +4 elementets storlek +5 offset till värdet
- +6/7 första indexets max +8/9 andra indexets max (**DIM2**)

En post för en sträng (även varje element i en strängvektor):

- +0/1 reserverad längd +2/3 adress till texten +4/5 längd

```

1BFE EB          DIMELM: EX  DE,HL
1BFF 23          INC  HL
1C00 4E          LD   C,(HL)          namnbyte
1C01 23          INC  HL
1C02 CD 1C 10     CALL RDDE1          variabelns adress
1C05 D5          PUSH DE
1C06 DD E1       POP  IX
1C08 E5          PUSH HL
1C09 3E 0C       LD   A,$0C          antal index
1C0B A1          AND  C
1C0C 28 56       JR   Z,L1C64
1C0E FE 04       CP   $04
1C10 28 3C       JR   Z,L1C4E
1C12 CD 6C 1C     CALL ELMSIZ          två index
1C15 C1          POP  BC
1C16 E1          POP  HL
1C17 D1          POP  DE          första index
1C18 20 06       JR   NZ,L1C20       sträng?
1C1A C5          PUSH BC
1C1B CD 11 33     CALL DIM2          tal: DIM2
1C1E D1          POP  DE
1C1F F7          RST  $30
1C20 E3          L1C20: EX  (SP),HL          sträng, två index

```

```

1C21 C5          PUSH BC
1C22 EB          EX   DE,HL
1C23 CD 11 33    CALL DIM2

```

Strängvektor: gå igenom alla (+2/3 plus 1) element, 6 bytes var, och reservera längden DE för varje (DIMSTR).

```

1C26 C1          L1C26: POP  BC
1C27 D1          POP  DE
1C28 7A          LD   A,D
1C29 B3          OR   E
1C2A 28 3D       JR   Z,L1C69
1C2C C5          PUSH BC
1C2D DD 4E 02    LD   C,(IX+$02)    antal element - 1
1C30 DD 46 03    LD   B,(IX+$03)
1C33 03          INC  BC
1C34 DD 6E 00    LD   L,(IX+$00)    elementen
1C37 DD 66 01    LD   H,(IX+$01)
1C3A E5          PUSH HL
1C3B DD E1       POP  IX
1C3D CD 38 33    L1C3D: CALL DIMSTR    reservera strängen
1C40 0B          DEC  BC
1C41 EB          EX   DE,HL
1C42 11 06 00    LD   DE,$0006    nästa element
1C45 DD 19       ADD  IX,DE
1C47 EB          EX   DE,HL
1C48 78          LD   A,B
1C49 B1          OR   C
1C4A 20 F1       JR   NZ,L1C3D
1C4C D1          POP  DE
1C4D F7          RST  $30

1C4E CD 6C 1C    L1C4E: CALL ELMSIZ    ett index
1C51 C1          POP  BC
1C52 20 07       JR   NZ,L1C5B
1C54 D1          POP  DE
1C55 C5          PUSH BC
1C56 CD E6 32    CALL DIM1    tal: DIM1
1C59 D1          POP  DE
1C5A F7          RST  $30

1C5B E1          L1C5B: POP  HL    sträng, ett index
1C5C D1          POP  DE
1C5D E5          PUSH HL
1C5E C5          PUSH BC
1C5F CD E6 32    CALL DIM1
1C62 18 C2       JR   L1C26

1C64 C1          L1C64: POP  BC    inget index: sträng
1C65 D1          POP  DE
1C66 CD 38 33    CALL DIMSTR    DE = längden
1C69 50          L1C69: LD   D,B    DE = koden
1C6A 59          LD   E,C
1C6B F7          RST  $30

```

ELMSIZ: sätt elementets storlek i IX+4: flyttal 5, heltal 2, sträng 6. Ut: Z för tal, NZ för sträng.

```

1C6C DD 36 04 05 ELMSIZ: LD   (IX+$04),$05    flyttal: 5 bytes
1C70 3E 03       LD   A,$03
1C72 A1          AND  C
1C73 C8          RET  Z
1C74 DD 36 04 02 LD   (IX+$04),$02    heltal: 2 bytes
1C78 3D          DEC  A
1C79 C8          RET  Z
1C7A DD 36 04 06 LD   (IX+$04),$06    sträng: 6 bytes
1C7E C9          ENDDIM: RET

```

PUSH0: \$D2, värdet 0 (DIM av sträng utan längd).

```

1C7F 21 00 00    PUSH0: LD   HL,$0000
1C82 EF          RST  $28

```

3.43 FOR: kompilering

\$1C83

FOR v = a TO b [STEP c]

v a \$B6+t \$BA b \$B9+t [\$BB c \$B9+t] \$B9 [2 bytes]

v får inte ha index och inte vara en sträng. b och c omvandlas till v:s typ. \$B9+t avslutar ett uttryck och lämnar värdet av typ t (se [EXPTAB1](#)). De två sista byten får adressen till [FOR](#)-blocket vid förlänkningen ([LNKFOR](#)). Fel 23 om TO saknas.

1C83	CD A5 1B	CMPFOR:	CALL	CMPFLV	v = a
1C86	D8		RET	C	
1C87	C5		PUSH	BC	
1C88	D5		PUSH	DE	
1C89	11 8C 19		LD	DE, KWTO	tabellen med TO
1C8C	CD F5 09		CALL	KWSRCH	
1C8F	D1		POP	DE	
1C90	C1		POP	BC	
1C91	4F		LD	C, A	
1C92	3E 97		LD	A, \$97	fel 23
1C94	37		SCF		
1C95	C0		RET	NZ	
1C96	79		LD	A, C	\$BA
1C97	12		LD	(DE), A	
1C98	13		INC	DE	
1C99	CD 72 23		CALL	CMPTYX	gränsen
1C9C	D8		RET	C	
1C9D	D5		PUSH	DE	
1C9E	C5		PUSH	BC	
1C9F	11 86 19		LD	DE, KWSTEP	tabellen med STEP
1CA2	CD F5 09		CALL	KWSRCH	
1CA5	C1		POP	BC	
1CA6	D1		POP	DE	
1CA7	20 06		JR	NZ, CMPB9	
1CA9	12		LD	(DE), A	\$BB
1CAA	13		INC	DE	
1CAB	CD 72 23		CALL	CMPTYX	steget
1CAE	D8		RET	C	

CMPB9: skriv \$B9 och lämna plats för två bytes (adressen till [FOR](#)-blocket, som fylls i vid förlänkningen). Ut: CY = 0.

1CAF	3E B9	CMPB9:	LD	A, \$B9
1CB1	12		LD	(DE), A
1CB2	13		INC	DE
1CB3	13		INC	DE
1CB4	13		INC	DE
1CB5	A7		AND	A
1CB6	C9		RET	

NEXT: kompilering. NEXT [v]

[v \$BC+t] \$B9 [2 bytes]

Utan variabel slutar satsen vid ":", CR eller ELSE (S218D).

Obs: [LNKNXT](#) jämför variabeln med den i [FOR](#), och utan variabel blir det alltid fel 28 vid RUN. [NEXT](#) ska alltså ha en variabel.

1CB7	CD 8D 21	CMPNXT:	CALL	STMEND	slut på satsen?
1CBA	28 F3		JR	Z, CMPB9	
1CBC	CD 92 23		CALL	CMPSVR	variabeln + \$BC + typ
1CBF	D8		RET	C	
1CC0	18 ED		JR	CMPB9	

3.44 FOR vid körning

\$1CC2

1. Första RST \$38: $v = a$ (tilldelningen avslutar).
2. Andra: gränsen b . Ett heltal läggs på stacken, ett flyttal ligger redan där.
3. Om STEP: steget c .
4. $BC = \text{FOR-blocket}$ (från \$B9-byten). Gränsen och steget skrivs in i blocket. Utan STEP gäller steget som LNKFOR lade in (1).
5. Testa direkt om slingan ska köras (FORTSI/FORTSF). Är a redan förbi b körs den noll gånger och körningen fortsätter efter NEXT ($v = a$).

Gränsen och steget räknas bara här. STEP 0 räknas som positivt, och slingan tar aldrig slut. Ett heltal slår runt: FOR I%=1 TO 32767 tar inte heller slut (v blir -32768, som är mindre än gränsen). FOR-blocket (reserverat på heapen av LNKFOR):

```
+0/1  adress efter NEXT (under förlänkningen länk till yttre FOR)
+2/3  variabelns adress
+4/5  adress efter FOR-satsen (slingans början)
heltal: +6/7 gränsen    +8/9 steget
flyttal: +6..+10 gränsen +11..+15 steget
```

Ut: DE = var körningen ska fortsätta.

1CC2	FF	FOR:	RST \$38	$v = a$
1CC3	13		INC DE	hoppa över \$BA
1CC4	FF		RST \$38	gränsen
1CC5	78		LD A,B	
1CC6	FE 00		CP \$00	heltal?
1CC8	28 01		JR Z,L1CCB	
1CCA	E5		PUSH HL	heltalet till stacken
1CCB	1A	L1CCB:	LD A,(DE)	
1CCC	FE BB		CP \$BB	STEP?
1CCE	13		INC DE	
1CCF	20 25		JR NZ,L1CF6	
1CD1	FF		RST \$38	steget
1CD2	78		LD A,B	
1CD3	EB		EX DE,HL	
1CD4	CD 16 10		CALL RDBC1	$BC = \text{FOR-blocket}$
1CD7	C5		PUSH BC	
1CD8	DD E1		POP IX	
1CDA	FE 00		CP \$00	flyttal?
1CDC	28 3D		JR Z,L1D1B	
1CDE	DD 73 08		LD (IX+\$08),E	heltal: steget
1CE1	DD 72 09		LD (IX+\$09),D	
1CE4	D1	L1CE4:	POP DE	gränsen
1CE5	DD 73 06		LD (IX+\$06),E	
1CE8	DD 72 07		LD (IX+\$07),D	
1CEB	DD 6E 02		LD L,(IX+\$02)	variabelns adress
1CEE	DD 66 03		LD H,(IX+\$03)	
1CF1	5E		LD E,(HL)	$DE = v$
1CF2	23		INC HL	
1CF3	56		LD D,(HL)	
1CF4	18 63		JR FORTSI	

Ingen STEP. \$B9 är redan överhoppad; $BC = \text{FOR-blocket}$.

1CF6	78	L1CF6:	LD A,B	
1CF7	EB		EX DE,HL	
1CF8	4E		LD C,(HL)	$BC = \text{FOR-blocket}$
1CF9	23		INC HL	
1CFA	46		LD B,(HL)	
1CFB	C5		PUSH BC	
1CFC	DD E1		POP IX	
1CFE	FE 01		CP \$01	heltal?
1D00	28 E2		JR Z,L1CE4	

Flytta 5 bytes (ett flyttal) från stacken till blocket: +11 för steget ($CY = 1$, sedan en gång till för gränsen), +6 för gränsen.

1D02	21 06 00	L1D02:	LD	HL,\$0006	
1D05	AF		XOR	A	
1D06	08	L1D06:	EX	AF,AF'	
1D07	09		ADD	HL,BC	
1D08	EB		EX	DE,HL	
1D09	21 00 00		LD	HL,\$0000	
1D0C	39		ADD	HL,SP	
1D0D	01 05 00		LD	BC,\$0005	
1D10	ED B0		LDIR		flyttalet från stacken
1D12	F9		LD	SP,HL	ta bort det
1D13	08		EX	AF,AF'	
1D14	30 72		JR	NC,FORTSF	
1D16	DD E5		PUSH	IX	
1D18	C1		POP	BC	
1D19	18 E7		JR	L1D02	
1D1B	21 0B 00	L1D1B:	LD	HL,\$000B	steget till +11
1D1E	37		SCF		
1D1F	18 E5		JR	L1D06	

3.45 NEXT vid körning

\$1D21

IX = **FOR**-blocket, vars adress står 7 bytes in (efter v och \$BC+t). Heltal: $v = v + \text{steg}$ (ingen kontroll av spill).
 Flyttal: flyttals-addition med steget på blocket+15 (\$33F7).

Sedan samma test som i **FOR**:

positivt steg: fortsatt slingan om $v \leq \text{gränsen}$
 negativt steg: fortsatt slingan om $v \geq \text{gränsen}$

Ut: DE = blocket+4 (slingans början) eller blocket+0 (efter **NEXT**). Hoppet sker bara genom DE, och **CURLIN** sätts inte om: står **FOR** mitt i en rad får ett fel i resten av den raden **NEXT**-radens nummer, tills nästa rad börjar.

1D21	1A	NEXT:	LD	A,(DE)	
1D22	FE B9		CP	\$B9	ingen variabel?
1D24	20 10		JR	NZ,L1D36	

*Utan variabel (\$B9 först): blocket+4 används som uttrycks kod och körs; ger det 0 fortsätter körningen vid ordet som blocket pekar på. Grenen passar inte med **FOR**-blocket, och **LNKNXT** tillåter inte **NEXT** utan variabel. Den används nog aldrig (?).*

1D26	13		INC	DE	
1D27	EB		EX	DE,HL	
1D28	5E		LD	E,(HL)	
1D29	23		INC	HL	
1D2A	56		LD	D,(HL)	
1D2B	EB		EX	DE,HL	
1D2C	E5		PUSH	HL	
1D2D	23		INC	HL	
1D2E	23		INC	HL	
1D2F	23		INC	HL	
1D30	23		INC	HL	
1D31	5E		LD	E,(HL)	DE = (blocket+4)
1D32	23		INC	HL	
1D33	56		LD	D,(HL)	
1D34	18 72		JR	L1DA8	
1D36	21 07 00	L1D36:	LD	HL,\$0007	adressen till FOR -blocket
1D39	19		ADD	HL,DE	
1D3A	5E		LD	E,(HL)	
1D3B	23		INC	HL	
1D3C	56		LD	D,(HL)	
1D3D	D5		PUSH	DE	
1D3E	DD E1		POP	IX	
1D40	DD 6E 02		LD	L,(IX+\$02)	variabelns adress
1D43	DD 66 03		LD	H,(IX+\$03)	
1D46	FE C0		CP	\$C0	flyttal?
1D48	28 2F		JR	Z,L1D79	
1D4A	DD 4E 08		LD	C,(IX+\$08)	steget
1D4D	DD 46 09		LD	B,(IX+\$09)	

1D50	5E	LD	E, (HL)	
1D51	23	INC	HL	
1D52	56	LD	D, (HL)	
1D53	EB	EX	DE, HL	v + steget
1D54	09	ADD	HL, BC	
1D55	EB	EX	DE, HL	
1D56	72	LD	(HL), D	
1D57	2B	DEC	HL	
1D58	73	LD	(HL), E	

FORTSI: heltal. HL = gränsen, DE = v. Med negativt steg byts de. Om gränsen - v är negativ (med hänsyn till spill, P/V) är slingan klar.

1D59	DD 6E 06	FORTSI:	LD	L, (IX+\$06)	
1D5C	DD 66 07		LD	H, (IX+\$07)	
1D5F	DD 7E 09		LD	A, (IX+\$09)	stegets tecken
1D62	A7		AND	A	
1D63	F2 67 1D		JP	P, L1D67	positivt?
1D66	EB		EX	DE, HL	negativt: byt
1D67	ED 52	L1D67:	SBC	HL, DE	gränsen - v
1D69	E2 6F 1D		JP	PO, L1D6F	inget spill?
1D6C	3E 80		LD	A, \$80	spill: vänd tecknet
1D6E	AC		XOR	H	
1D6F	FA A1 1D	L1D6F:	JP	M, L1DA1	negativt: klar
1D72	DD 5E 04	L1D72:	LD	E, (IX+\$04)	slingans början
1D75	DD 56 05		LD	D, (IX+\$05)	
1D78	C9		RET		
1D79	44	L1D79:	LD	B, H	
1D7A	4D		LD	C, L	
1D7B	21 0F 00		LD	HL, \$000F	steget, blocket+15
1D7E	19		ADD	HL, DE	
1D7F	50		LD	D, B	
1D80	59		LD	E, C	
1D81	DD E5		PUSH	IX	
1D83	CD F7 33		CALL	FADD	v = v + steget
1D86	DD E1		POP	IX	

FORTSF: flyttal. Jämför v med gränsen (blocket+10) med \$2A93, och byt ordning om steget är negativt (teckenbyten blocket+14).

1D88	DD E5	FORTSF:	PUSH	IX	
1D8A	C1		POP	BC	
1D8B	21 0A 00		LD	HL, \$000A	gränsen, blocket+10
1D8E	09		ADD	HL, BC	
1D8F	DD 5E 02		LD	E, (IX+\$02)	
1D92	DD 56 03		LD	D, (IX+\$03)	
1D95	DD 7E 0E		LD	A, (IX+\$0E)	stegets tecken
1D98	A7		AND	A	
1D99	28 01		JR	Z, L1D9C	
1D9B	EB		EX	DE, HL	
1D9C	CD 93 2A	L1D9C:	CALL	FCMP	jämför
1D9F	30 D1		JR	NC, L1D72	
1DA1	DD 5E 00	L1DA1:	LD	E, (IX+\$00)	efter NEXT
1DA4	DD 56 01		LD	D, (IX+\$01)	
1DA7	C9		RET		
1DA8	FF	L1DA8:	RST	\$38	
1DA9	7C		LD	A, H	
1DAA	B5		OR	L	
1DAB	E1		POP	HL	
1DAC	C0		RET	NZ	
1DAD	5E		LD	E, (HL)	
1DAE	23		INC	HL	
1DAF	56		LD	D, (HL)	
1DB0	C9		RET		

GET v\$: vänta på en tangent (GETKEY) och tilldela v\$ en sträng med det tecknet. Går även i direktläge; GET till en talvariabel ger fel 17 vid kompileringen. Tecknet läggs på stacken som en tillfällig sträng:

[antal tillfälliga bytes = 1] [adress] [längd = 1] [tecknet]

och sedan hoppar det till **ASGNS**.

1DB1	FF	GET:	RST	\$38	
1DB2	E5		PUSH	HL	
1DB3	D5		PUSH	DE	
1DB4	CD EA 02		CALL	GETKEY	vänta på en tangent
1DB7	D1		POP	DE	
1DB8	21 FF FF		LD	HL, \$FFFF	
1DBB	39		ADD	HL, SP	
1DBC	F9		LD	SP, HL	en byte på stacken
1DBD	01 01 00		LD	BC, \$0001	
1DC0	77		LD	(HL), A	tecknet på stacken
1DC1	C5		PUSH	BC	längd 1
1DC2	E5		PUSH	HL	
1DC3	C5		PUSH	BC	
1DC4	C3 C3 29		JP	ASGNS	tilldela

CMPREF: kompilera en radreferens. Radnumret läses från texten och \$BF nl nh 0 0 skrivs ut. Adressen fylls i vid RUN. Inget tal eller ett tal över 65535 ger CY och A = \$90 (fel 16). Blanka inne i numret hoppas över ("GOTO 1 0" = GOTO 10).

1DC7	D5	CMPREF:	PUSH	DE	
1DC8	CD 1E 18		CALL	GETLN	
1DCB	42		LD	B, D	
1DCC	4B		LD	C, E	
1DCD	D1		POP	DE	
1DCE	3E 90		LD	A, \$90	fel 16, RADNUMMER SAKNAS
1DD0	D8		RET	C	inget nummer
1DD1	EB		EX	DE, HL	
1DD2	36 BF		LD	(HL), \$BF	radreferens
1DD4	CD 28 10		CALL	WRBC1	
1DD7	23		INC	HL	
1DD8	23		INC	HL	
1DD9	EB		EX	DE, HL	
1DDA	C9		RET		

GOTO: fel 2 i direktläge. Adressen hämtas från \$BF-elementet och **RUNLIN** fortsätter där. De två POP tar bort returadresserna till **RUNSTM** och **RUNLIN**. En rad som saknas ger fel 6 redan vid länkningen.

1DDB	CD BF 0D	GOTO:	CALL	NODIR	
1DDE	13		INC	DE	
1DDF	13		INC	DE	
1DE0	EB		EX	DE, HL	
1DE1	CD 1C 10		CALL	RDDE1	DE = målradens adress
1DE4	E1		POP	HL	bort med RUNSTM :s och
1DE5	E1		POP	HL	RUNLIN :s returadresser
1DE6	C3 71 0D		JP	RUNLIN	

GOSUB: som **GOTO**, men **RUNLIN** anropas med CALL. **RETURN** går tillbaka hit, och körningen fortsätter efter **GOSUB**. Varje **GOSUB**-nivå kostar alltså stack, och **STKCHK** kontrollerar att den räcker: med 16 KB RAM och ett kort program blir det fel 3 efter omkring 1870 nivåer.

1DE9	CD BF 0D	GOSUB:	CALL	NODIR	
1DEC	13		INC	DE	
1DED	13		INC	DE	
1DEE	EB		EX	DE, HL	
1DEF	CD 1C 10		CALL	RDDE1	
1DF2	CD 89 33		CALL	STKCHK	
1DF5	E5		PUSH	HL	fortsättningen i raden
1DF6	CD 71 0D		CALL	RUNLIN	kör från målraden till RETURN
1DF9	D1		POP	DE	
1DFA	C9		RET		

3.46 IF: kompilering

\$1DFB

IF villkor [THEN] satser [ELSE satser]

villkoret (kod) [\$BD] \$87 n THEN-del [\$87 m \$BC ELSE-del]

Ett radnummer direkt efter THEN eller ELSE blir en **GOTO**: token \$81 och en radreferens. n och m är längderna att hoppa, och de fylls i när delen är kompilerad.

En sats efter THEN eller ELSE tar resten av raden (**TOKSTM**), utom att THEN-delen slutar vid ELSE. Ett radnummer däremot är hela delen: det som följer efter ":" är en vanlig sats efter IF. I "IF v THEN 100 : S" körs S alltså när v är falskt, och i "IF v THEN T ELSE 100 : S" när v är sant. ELSE hör till det närmaste IF:et. Villkoret är ett heltalsuttryck (**CMPINX**): ett strängvillkor ger fel 17, och utan THEN-del eller med tom THEN- eller ELSE-del blir det fel 11. **REM** i THEN-delen tar även ELSE-delen.

1DFB	CD 70 23	CMPIF:	CALL CMPINX	
1DFE	D8		RET C	villkoret: heltal
1DFF	D5		PUSH DE	
1E00	11 7A 19		LD DE, KWTHEN	"THEN"
1E03	CD F5 09		CALL KWSRCH	
1E06	D1		POP DE	
1E07	20 02		JR NZ, L1E0B	
1E09	12		LD (DE), A	\$BD
1E0A	13		INC DE	
1E0B	3E 87	L1E0B:	LD A, \$87	hopp förbi THEN-delen
1E0D	12		LD (DE), A	
1E0E	13		INC DE	
1E0F	D5		PUSH DE	platsen för n
1E10	13		INC DE	
1E11	3E 81		LD A, \$81	THEN nnn = GOTO nnn
1E13	12		LD (DE), A	
1E14	13		INC DE	
1E15	CD C7 1D		CALL CMPREF	
1E18	30 04		JR NC, L1E1E	radnummer: klart
1E1A	1B		DEC DE	
1E1B	CD C7 18		CALL TOKSTM	annars satser
1E1E	C1	L1E1E:	POP BC	
1E1F	D8		RET C	fel i THEN-delen
1E20	C5		PUSH BC	
1E21	D5		PUSH DE	
1E22	11 80 19		LD DE, KWELSE	"ELSE"
1E25	CD F5 09		CALL KWSRCH	
1E28	D1		POP DE	
1E29	C1		POP BC	
1E2A	20 1A		JR NZ, L1E46	
1E2C	EB		EX DE, HL	
1E2D	36 87		LD (HL), \$87	
1E2F	23		INC HL	
1E30	E5		PUSH HL	
1E31	23		INC HL	
1E32	77		LD (HL), A	
1E33	23		INC HL	
1E34	7D		LD A, L	
1E35	91		SUB C	
1E36	02		LD (BC), A	n = THEN-delens längd
1E37	36 81		LD (HL), \$81	
1E39	23		INC HL	
1E3A	EB		EX DE, HL	
1E3B	CD C7 1D		CALL CMPREF	
1E3E	30 04		JR NC, L1E44	
1E40	1B		DEC DE	
1E41	CD C7 18		CALL TOKSTM	
1E44	C1	L1E44:	POP BC	
1E45	D8		RET C	
1E46	7B	L1E46:	LD A, E	
1E47	91		SUB C	
1E48	02		LD (BC), A	n eller m
1E49	A7		AND A	
1E4A	C9		RET	

IF: beräkna villkoret (RST \$38). HL = 0 betyder falskt. Då pekar DE på \$87, och hoppet tar körningen till ELSE-delen. Annars hoppar DE över \$87 n och THEN-delen körs. Ett flyttal avrundas nedåt till heltal: IF .5 är falskt, IF -.5 sant, och från 65536 blir det fel 7.

1E4B	FF	IF:	RST	\$38	
1E4C	1A		LD	A, (DE)	
1E4D	FE BD		CP	\$BD	THEN?
1E4F	20 01		JR	NZ, L1E52	
1E51	13		INC	DE	
1E52	7D	L1E52:	LD	A, L	falskt?
1E53	B4		OR	H	
1E54	C8		RET	Z	
1E55	13		INC	DE	över \$87 n: THEN-delen
1E56	13		INC	DE	
1E57	C9		RET		

3.47 INPUT: kompilering

\$1E58

INPUT [#n,] v [, v ...]
 ["#" n \$BA ",,"] v \$BC+t [",," v \$BC+t ...]

Tecknen "#" och "," kopieras direkt till koden. Filnumret är ett heltal. Fel 13 om "," saknas efter filnumret. Variablerna kan ha index och vara av alla typer.

CMPRD (READ) är samma sak utan filnumret.

1E58	7E	CMPINP:	LD	A, (HL)	
1E59	FE 23		CP	\$23	"#"?
1E5B	20 11		JR	NZ, CMPRD	
1E5D	ED A0		LDI		kopiera "#"
1E5F	E7		RST	\$20	
1E60	CD 70 23		CALL	CMPINX	filnumret
1E63	D8		RET	C	
1E64	7E		LD	A, (HL)	
1E65	FE 2C		CP	\$2C	","?
1E67	3E 8D		LD	A, \$8D	annars fel 13
1E69	37		SCF		
1E6A	C0		RET	NZ	
1E6B	ED A0	L1E6B:	LDI		kopiera ","
1E6D	E7		RST	\$20	
1E6E	CD 97 23	CMPRD:	CALL	CMPVR	variabel + \$BC + typ
1E71	D8		RET	C	
1E72	7E		LD	A, (HL)	
1E73	EE 2C		XOR	\$2C	","?
1E75	C0		RET	NZ	
1E76	18 F3		JR	L1E6B	

3.48 INPUT vid körning

\$1E78

Raden läses in i **BUF1** (\$FE40, högst 120 tecken). Filnumret (0 = tangentbordet) sparas i IY+\$0D. För varje variabel:

- står det "," i bufferten tas nästa värde efter det,
- annars läses en ny rad. Från tangentbordet skrivs först "?", och efter raden en ny rad (**NEWLIN**).

Värdet läses och tilldelas av **RDVAL**. Kommer det för få värden frågar datorn alltså igen med "?". Värden som blir över glöms. Står det något annat än "," efter ett värde ("5X,6") glöms också resten, och nästa variabel får en ny rad. **INPUT LINE** går till **INPLIN** (\$1F82).

1E78	21 40 FE	INPT:	LD	HL, \$FE40	
1E7B	E5		PUSH	HL	
1E7C	36 0D		LD	(HL), \$0D	tom buffert
1E7E	FD 36 0D 00		LD	(IY+\$0D), \$00	fil 0: tangentbordet
1E82	1A		LD	A, (DE)	
1E83	FE 23		CP	\$23	"#"?
1E85	20 06		JR	NZ, L1E8D	
1E87	13		INC	DE	
1E88	FF		RST	\$38	filnumret

1E89	13		INC	DE	
1E8A	FD 75 0D		LD	(IY+\$0D),L	IY+\$0D = filnumret
1E8D	E1	L1E8D:	POP	HL	
1E8E	E5		PUSH	HL	
1E8F	7E	L1E8F:	LD	A,(HL)	
1E90	23		INC	HL	
1E91	FE 2C		CP	\$2C	"," : nästa värde på raden
1E93	28 24		JR	Z,L1EB9	
1E95	FD 7E 0D		LD	A,(IY+\$0D)	
1E98	A7		AND	A	tangentbordet?
1E99	20 0B		JR	NZ,L1EA6	
1E9B	D5		PUSH	DE	
1E9C	21 6A 1F		LD	HL,D1F6A	"?"
1E9F	01 01 00		LD	BC,\$0001	
1EA2	CD A8 01		CALL	PRTXT	
1EA5	D1		POP	DE	
1EA6	E1	L1EA6:	POP	HL	
1EA7	E5		PUSH	HL	
1EA8	D5		PUSH	DE	
1EA9	01 78 00		LD	BC,\$0078	högst 120 tecken
1EAC	CD DD 08		CALL	INPUT	läs en rad
1EAF	FD 7E 0D		LD	A,(IY+\$0D)	
1EB2	A7		AND	A	
1EB3	CC 42 07		CALL	Z,NEWLIN	tangentbordet: ny rad
1EB6	D1		POP	DE	
1EB7	E1		POP	HL	
1EB8	E5		PUSH	HL	HL = läspekaren
1EB9	CD FE 1E	L1EB9:	CALL	RDVAL	läs värdet och tilldela
1EBC	1A		LD	A,(DE)	
1EBD	13		INC	DE	
1EBE	FE 2C		CP	\$2C	fler variabler?
1EC0	28 CD		JR	Z,L1E8F	
1EC2	1B		DEC	DE	
1EC3	E1		POP	HL	BUF1
1EC4	C9		RET		

3.49 READ vid körning

\$1EC5

DATPTR pekar på nästa värde i en **DATA**-rad, och **DATLIN** på raden. Står **DATPTR** på CR letas nästa rad upp som börjar med **DATA** (\$93 direkt efter radnumret); finns ingen blir det fel 30, **DATA SLUT**. **DATA** räknas alltså bara när det står först på raden. Värdena läses med **RDVAL**, precis som för **INPUT**, och "," mellan dem hoppas över. Till skillnad från **INPUT** krävs inget ",": det som står efter ett värde blir nästa värde. I "DATA "AB"C" är C ett eget värde, och i "DATA 2 : REM" blir ":" REM" nästa värde (fel 12 för ett tal).

1EC5	2A 2E FE	READ:	LD	HL,(DATPTR)	
1EC8	7E	L1EC8:	LD	A,(HL)	
1EC9	FE 0D		CP	\$0D	slut på DATA-raden?
1ECB	20 1D		JR	NZ,L1EEA	
1ECD	2A 30 FE		LD	HL,(DATLIN)	
1ED0	4E	L1ED0:	LD	C,(HL)	
1ED1	79		LD	A,C	
1ED2	FE 01		CP	\$01	slut på programmet?
1ED4	CA 1F 07		JP	Z,L071F	fel 30, DATA SLUT
1ED7	06 00		LD	B,\$00	
1ED9	09		ADD	HL,BC	nästa rad
1EDA	44		LD	B,H	
1EDB	4D		LD	C,L	
1EDC	03		INC	BC	
1EDD	03		INC	BC	
1EDE	03		INC	BC	
1EDF	0A		LD	A,(BC)	första token
1EE0	FE 93		CP	\$93	DATA?
1EE2	20 EC		JR	NZ,L1ED0	
1EE4	22 30 FE		LD	(DATLIN),HL	
1EE7	60		LD	H,B	
1EE8	69		LD	L,C	

1EE9 23	INC HL	efter DATA-token
1EEA CD FE 1E	L1EEA: CALL RDVAL	
1EED 7E	LD A,(HL)	
1EEE FE 2C	CP \$2C	","?
1EF0 CC BE 25	CALL Z,SKIPSP	över kommat
1EF3 1A	LD A,(DE)	
1EF4 13	INC DE	
1EF5 FE 2C	CP \$2C	fler variabler?
1EF7 28 CF	JR Z,L1EC8	
1EF9 1B	DEC DE	
1EFA 22 2E FE	LD (DATPTR),HL	spara pekaren
1EFD C9	RET	

3.50 RDVAL: läs ett värde ur texten vid HL och tilldela det

\$1EFE

DE = koden "v \$BC+t". RST \$38 ger variabelns adress och typ.

flyttal GETSFL (tecken tillåtet), fel 12 om det inte är ett tal (tom rad, "." eller "-"). Text efter talet stoppar bara läsningen: "5X" = 5, och "1E" = 1 med HL på "E".

heltal GETSIN (tecken tillåtet), fel 12, även över 65535; 65535 = -1 och -40000 = 25536 (modulo 65536)

sträng inom " " eller ' ': till samma citattecken eller CR, två citattecken efter varandra blir ett. Annars till "," eller CR, och alla mellanslag hoppas över (RST \$20 före värdet, RST \$18 i slingan), så att "A B" blir "AB". Saknas det avslutande citattecknet tas texten till CR. Texten kopieras till BUF1 (\$FE40) och tilldelas med ASGNS.

Samma sak gäller READ av DATA-strängar utan citattecken. INPUT LINE tar raden som den är. (Christensen 1979 klagar på att INPUT tar bort blanktecken; det stämmer, i båda versionerna.)

INPUT läser också in raden till BUF1, men kopian hamnar aldrig efter originalet, så det går bra.

Ut: HL efter värdet (och mellanslag), DE efter koden.

1EFE E7	RDVAL: RST \$20	
1EFF E5	PUSH HL	
1F00 FF	RST \$38	HL = variabeln, B = typ
1F01 78	LD A,B	
1F02 EB	EX DE,HL	
1F03 E3	EX (SP),HL	(SP) = variabeln, HL = texten
1F04 FE 02	CP \$02	sträng?
1F06 28 1D	JR Z,L1F25	
1F08 FE 01	CP \$01	heltal?
1F0A 28 0B	JR Z,L1F17	
1F0C EB	EX DE,HL	
1F0D CD FB 14	CALL GETSFL	flyttal
1F10 38 09	JR C,L1F1B	
1F12 1B	DEC DE	HL på tecknet efter talet
1F13 EB	EX DE,HL	
1F14 D1	L1F14: POP DE	
1F15 E7	RST \$20	
1F16 C9	RET	
1F17 D5	L1F17: PUSH DE	
1F18 CD 41 18	CALL GETSIN	heltal
1F1B DA 13 07	L1F1B: JP C,L0713	fel 12
1F1E E3	EX (SP),HL	
1F1F 73	LD (HL),E	skriv heltalet
1F20 23	INC HL	
1F21 72	LD (HL),D	
1F22 E1	POP HL	HL = texten
1F23 18 EF	JR L1F14	

Sträng. \$1F13 läggs som returadress, så att ASGNS kommer tillbaka dit. Längden räknas i B (högst 255).

1F25 01 13 1F	L1F25: LD BC,\$1F13	retur till \$1F13
1F28 C5	PUSH BC	
1F29 D5	PUSH DE	

1F2A	11 40 FE	LD	DE,\$FE40	kopian i BUF1
1F2D	06 00	LD	B,\$00	
1F2F	D5	PUSH	DE	
1F30	7E	LD	A,(HL)	
1F31	FE 22	CP	\$22	citattecken?
1F33	28 12	JR	Z,L1F47	
1F35	FE 27	CP	\$27	
1F37	28 0E	JR	Z,L1F47	
1F39	FE 2C	L1F39: CP	\$2C	","?
1F3B	28 20	JR	Z,L1F5D	
1F3D	FE 0D	CP	\$0D	CR?
1F3F	28 1C	JR	Z,L1F5D	
1F41	04	INC	B	
1F42	12	LD	(DE),A	
1F43	13	INC	DE	
1F44	DF	RST	\$18	nästa, utan mellanslag
1F45	18 F2	JR	L1F39	
1F47	4F	L1F47: LD	C,A	
1F48	23	INC	HL	
1F49	7E	L1F49: LD	A,(HL)	
1F4A	FE 0D	CP	\$0D	
1F4C	28 0F	JR	Z,L1F5D	
1F4E	23	INC	HL	
1F4F	B9	CP	C	samma citattecken?
1F50	28 05	JR	Z,L1F57	
1F52	12	L1F52: LD	(DE),A	
1F53	13	INC	DE	
1F54	04	INC	B	
1F55	18 F2	JR	L1F49	
1F57	BE	L1F57: CP	(HL)	två citattecken?
1F58	20 03	JR	NZ,L1F5D	
1F5A	23	INC	HL	
1F5B	18 F5	JR	L1F52	
1F5D	EB	L1F5D: EX	DE,HL	
1F5E	26 00	LD	H,\$00	HL = längden
1F60	68	LD	L,B	
1F61	E3	EX	(SP),HL	(SP) = längden, HL = BUF1
1F62	E5	PUSH	HL	
1F63	21 00 00	LD	HL,\$0000	inget tillfälligt område
1F66	E5	PUSH	HL	
1F67	C3 C3 29	JP	ASGNS	tilldela
1F6A	3F	D1F6A: DEFB	"?"	

INPUT LINE: kompilering. INPUT LINE [#n[,]] v\$
["#" n \$BA [","]] v\$ \$BE

Kommat efter filnumret kan utelämnas.

1F6B	7E	CMPINL: LD	A,(HL)	
1F6C	FE 23	CP	\$23	""?
1F6E	20 0F	JR	NZ,L1F7F	
1F70	ED A0	LDI		
1F72	E7	RST	\$20	
1F73	CD 70 23	CALL	CMPINX	filnumret
1F76	D8	RET	C	
1F77	7E	LD	A,(HL)	
1F78	FE 2C	CP	\$2C	","?
1F7A	20 03	JR	NZ,L1F7F	
1F7C	ED A0	LDI		
1F7E	E7	RST	\$20	
1F7F	C3 A8 23	L1F7F: JP	CMPGET	strängvariabel + \$BE

INPUT LINE vid körning: en hel rad till strängvariabeln, utan "?" och utan ny rad efteråt på skärmen (inget NEWLIN som i INPT).

120 bytes reserveras på stacken och **INPUT** läser högst 119 tecken dit. Efter CR läggs LF, och strängen blir hela raden med CR LF sist. Sedan **ASGNS** med de 120 byten som tillfälligt område.

1F82	FD 36 0D 00	INPLIN: LD	(IY+\$0D),\$00
------	-------------	-------------------	----------------

1F86	1A	LD	A, (DE)	
1F87	FE 23	CP	\$23	
1F89	20 0B	JR	NZ, L1F96	
1F8B	13	INC	DE	
1F8C	FF	RST	\$38	
1F8D	FD 75 0D	LD	(IY+\$0D), L	IY+\$0D = filnumret
1F90	1A	LD	A, (DE)	
1F91	FE 2C	CP	\$2C	
1F93	20 01	JR	NZ, L1F96	
1F95	13	INC	DE	
1F96	FF	L1F96: RST	\$38	variabelns adress
1F97	E5	PUSH	HL	
1F98	21 88 FF	LD	HL, \$FF88	120 bytes på stacken
1F9B	39	ADD	HL, SP	
1F9C	F9	LD	SP, HL	
1F9D	01 77 00	LD	BC, \$0077	högst 119 tecken
1FA0	D5	PUSH	DE	
1FA1	E5	PUSH	HL	
1FA2	CD DD 08	CALL	INPUT	
1FA5	E1	POP	HL	
1FA6	D1	POP	DE	
1FA7	E5	PUSH	HL	
1FA8	01 00 00	LD	BC, \$0000	
1FAB	3E 0D	LD	A, \$0D	
1FAD	ED B1	CPIR		leta upp CR
1FAF	36 0A	LD	(HL), \$0A	LF efter CR
1FB1	AF	XOR	A	
1FB2	67	LD	H, A	
1FB3	91	SUB	C	längden, med CR LF
1FB4	3C	INC	A	
1FB5	6F	LD	L, A	
1FB6	E3	EX	(SP), HL	
1FB7	E5	PUSH	HL	
1FB8	21 78 00	LD	HL, \$0078	120 bytes tillfälligt
1FBB	E5	PUSH	HL	
1FBC	C3 C3 29	JP	ASGNS	

3.51 ON: kompilering

\$1FBF

ON uttryck **GOTO**|**GOSUB**|**RESTORE** nr [, nr ...]
 n \$BA token \$BE k k * [nr lo][nr hi][adress lo][adress hi]

Token är \$80 (**GOTO**), \$90 (**GOSUB**) eller \$8F (**RESTORE**), annars fel 25. Uttrycket är ett heltal. Adresserna fylls i vid förlänkningen, och **REN** ändrar numren. Saknas ett radnummer blir det fel 16.

1FBF	CD 70 23	CMPON:	CALL	CMPINX	
1FC2	D8		RET	C	
1FC3	D5		PUSH	DE	
1FC4	11 11 19		LD	DE, STKW1	GOTO, GOSUB ...
1FC7	CD F5 09		CALL	KWSRCH	
1FCA	D1		POP	DE	
1FCB	20 0C		JR	NZ, L1FD9	
1FCD	12		LD	(DE), A	
1FCE	13		INC	DE	
1FCF	FE 80		CP	\$80	GOTO?
1FD1	28 06		JR	Z, L1FD9	
1FD3	FE 90		CP	\$90	GOSUB?
1FD5	28 02		JR	Z, L1FD9	
1FD7	FE 8F		CP	\$8F	RESTORE?
1FD9	3E 99	L1FD9:	LD	A, \$99	fel 25
1FDB	37		SCF		
1FDC	C0		RET	NZ	
1FDD	3E BE		LD	A, \$BE	\$BE: lista med radreferenser
1FDF	12		LD	(DE), A	
1FE0	13		INC	DE	
1FE1	D5		PUSH	DE	plats för antalet
1FE2	13		INC	DE	
1FE3	06 00		LD	B, \$00	B = antalet

1FE5	C5	L1FE5:	PUSH BC	
1FE6	D5		PUSH DE	
1FE7	CD 1E 18		CALL GETLN	radnumret
1FEA	38 12		JR C, L1FFE	
1FEC	E3		EX (SP),HL	HL = kodpekaren
1FED	CD 23 10		CALL WRDE	
1FF0	23		INC HL	plats för adressen
1FF1	23		INC HL	
1FF2	D1		POP DE	
1FF3	EB		EX DE,HL	
1FF4	C1		POP BC	
1FF5	04		INC B	räkna
1FF6	7E		LD A,(HL)	
1FF7	EE 2C		XOR \$2C	","?
1FF9	20 0A		JR NZ, L2005	
1FFB	DF		RST \$18	över kommat
1FFC	18 E7		JR L1FE5	
1FFE	C1	L1FFE:	POP BC	
1FFF	C1		POP BC	
2000	C1		POP BC	
2001	3E 90		LD A,\$90	fel 16
2003	37		SCF	
2004	C9		RET	
2005	78	L2005:	LD A,B	
2006	C1		POP BC	
2007	02		LD (BC),A	antalet
2008	C9		RET	

3.52 ON vid körning

\$2009

Värdet n (1..k) väljer referens nr n; annars fel 26, även för $n < 0$ och $n > 255$. Ett flyttal avrundas nedåt (ON 2.9 = ON 2), och från 65536 blir det fel 7. Efter ON **GOSUB** fortsätter **RETURN** efter hela listan. DE = målradens adress, HL = efter listan. Hoppet görs genom att RET går till en del av **GOTO** (**\$1DE4**), **GOSUB** (**\$1DF2**) eller **RESTORE** (**\$2297**), som har lagts på stacken.

2009	CD BF 0D	ON:	CALL NODIR	
200C	FF		RST \$38	n
200D	1A		LD A,(DE)	
200E	13		INC DE	
200F	13		INC DE	hoppa över \$BE
2010	01 E4 1D		LD BC,\$1DE4	GOTO
2013	FE 80		CP \$80	
2015	28 0A		JR Z, L2021	
2017	01 F2 1D		LD BC,\$1DF2	GOSUB
201A	FE 90		CP \$90	
201C	28 03		JR Z, L2021	
201E	01 97 22		LD BC,\$2297	RESTORE
2021	C5	L2021:	PUSH BC	
2022	AF		XOR A	
2023	B4		OR H	
2024	C2 1D 07		JP NZ, L071D	n > 255: fel 26
2027	B5		OR L	
2028	CA 1D 07		JP Z, L071D	n = 0: fel 26
202B	1A		LD A,(DE)	
202C	13		INC DE	
202D	95		SUB L	antal - n
202E	DA 1D 07		JP C, L071D	n > antal: fel 26
2031	4F		LD C,A	C = referenser efter nr n
2032	2D		DEC L	
2033	29		ADD HL,HL	4*(n-1)
2034	29		ADD HL,HL	
2035	19		ADD HL,DE	
2036	23		INC HL	
2037	CD 1C 10		CALL RDDE1	DE = målradens adress
203A	06 00		LD B,\$00	

203C	09	ADD	HL,BC	hoppa över resten
203D	09	ADD	HL,BC	
203E	09	ADD	HL,BC	
203F	09	ADD	HL,BC	
2040	C9	RET		

ON **ERROR GOTO** nr: spara målradens adress i **ONERR**. Nummer 0 ger adressen 0 (se **PRELNK**), och då är felhanteringen avstängd.

2041	CD BF 0D	ONERRG:	CALL	NODIR	
2044	EB		EX	DE,HL	
2045	23		INC	HL	hoppa över \$BF och numret
2046	23		INC	HL	
2047	CD 1C 10		CALL	RDDE1	
204A	EB		EX	DE,HL	
204B	22 34 FE		LD	(ONERR),HL	radens adress
204E	C9		RET		

ERRCODE: koden för det senaste felet som ON **ERROR** fångade ($IY + \$22$, sparas bara på **\$0648**) utan bit 7. Fel i direktläge och fel som inte fångas ändrar den inte. **CLEAR** (och därmed **RUN**) nollar den, och 0 ger värdet -1; fel 0 finns inte i **ERRTRP**.

204F	FD 7E 22	EXERR:	LD	A, (IY+\$22)	
2052	21 FF FF		LD	HL, \$FFFF	
2055	A7		AND	A	
2056	28 04		JR	Z, L205C	0: inget fångat fel
2058	E6 7F		AND	\$7F	
205A	6F		LD	L, A	
205B	24		INC	H	H = 0
205C	EF	L205C:	RST	\$28	

3.53 OPEN / PREPARE: kompilering

\$205D

OPEN namn\$ AS FILE n (PREPARE likadant)
 namn\$ \$BB \$B8 n \$BA

Namnet är ett stränguttryck, n ett heltal. Fel 33 om AS FILE saknas. **CLOSE** n: bara n \$BA.

205D	CD 6C 23	CMPOPn:	CALL	CMPSTX	namnet, sträng + \$BB
2060	D8		RET	C	
2061	D5		PUSH	DE	
2062	11 90 19		LD	DE, KWASF	tabellen med AS FILE
2065	CD F5 09		CALL	KWSRCH	
2068	47		LD	B, A	
2069	D1		POP	DE	
206A	3E A1		LD	A, \$A1	fel 33
206C	37		SCF		
206D	C0		RET	NZ	
206E	78		LD	A, B	\$B8
206F	12		LD	(DE), A	AS FILE
2070	13		INC	DE	
2071	C3 70 23	CMPCls:	JP	CMPINX	filnumret, heltal + \$BA

3.54 OPEN / PREPARE vid körning

\$2074

A = 0 (**OPEN**) eller 1 (**PREPARE**) läggs på stacken, sedan namnet (en sträng på stacken) och filnumret. Fil 0 gör ingenting.

Filerna har var sitt FCB på 15 bytes, i en kedja från **FILLST**:

+0/1 nästa FCB +2 filnummer +3- se **OPEN** (**\$07E2**)

Finns filnumret redan stängs den filen först och FCB:t används igen. Annars reserveras ett nytt FCB och läggs sist i kedjan.

Sedan **OPEN** med HL = namnet, BC = längden och A = typen. A hämtas från stacken ovanför strängens tillfälliga område.

Till sist tas strängen och A bort från stacken.

Vid fil 0 tolkas inte ens namnet: **OPEN** "XYZ:" AS FILE 0 ger inget fel. Filen står öppen efter **STOP**, men programslutet (**ENDPRG** -> **CLEAR** -> **CLSALL**) stänger den, och en senare **INPUT #n** ger fel 32.

```

2074 97          OPENST: SUB  A           A = 0: OPEN
2075 18 02              JR  L2079

2077 3E 01          PREPST: LD  A,$01      A = 1: PREPARE
2079 F5          L2079: PUSH AF
207A FF              RST  $38      namnet till stacken
207B 13              INC  DE      hoppa över $B8
207C FF              RST  $38      filnumret
207D D5              PUSH DE
207E 7D              LD   A,L
207F A7              AND  A
2080 28 35          JR   Z,L20B7      fil 0: ingenting
2082 CD E1 20      CALL FNDFIL      finns filen?
2085 28 16          JR   Z,L209D      Z: filen finns
2087 F5              PUSH AF
2088 11 0F 00      LD   DE,$000F      nytt FCB, 15 bytes
208B CD 74 33      CALL ALLOC
208E DD 75 00      LD   (IX+$00),L      sist i kedjan
2091 DD 74 01      LD   (IX+$01),H
2094 E5              PUSH HL
2095 DD E1          POP  IX
2097 F1              POP  AF
2098 DD 77 02      LD   (IX+$02),A      filnumret
209B 18 03          JR   L20A0

209D CD 7F 08      L209D: CALL CLOSE      stäng den gamla
20A0 21 02 00      L20A0: LD   HL,$0002      strängen på stacken
20A3 39              ADD  HL,SP
20A4 CD 1D 10      CALL RDDE
20A7 D5              PUSH DE
20A8 CD 1D 10      CALL RDDE
20AB CD 17 10      CALL RDBC
20AE EB              EX   DE,HL
20AF E3              EX   (SP),HL
20B0 19              ADD  HL,DE
20B1 23              INC  HL
20B2 7E              LD   A,(HL)      A (OPEN/PREPARE)
20B3 E1              POP  HL
20B4 CD E2 07      CALL OPEN      öppna
20B7 D1          L20B7: POP  DE
20B8 E1              POP  HL
20B9 C1              POP  BC
20BA C1              POP  BC
20BB 39              ADD  HL,SP      ta bort strängen
20BC F9              LD   SP,HL
20BD E1              POP  HL      och A
20BE C9              RET

```

CLOSE n: stäng filen om den finns. **CLOSE 0** och **CLOSE** av en fil som inte är öppen gör ingenting.

```

20BF FF          CLOSST: RST  $38
20C0 7D              LD   A,L
20C1 A7              AND  A
20C2 C8              RET  Z      fil 0
20C3 CD E1 20      CALL FNDFIL      finns filen?
20C6 C0              RET  NZ      finns inte
20C7 D5              PUSH DE
20C8 CD 7F 08      CALL CLOSE
20CB D1              POP  DE
20CC C9              RET

```

CLSALL: stäng alla filer i **FILLST**-kedjan (från **CLEAR**).

```

20CD 2A 32 FE      CLSALL: LD   HL,(FILLST)
20D0 7C          L20D0: LD   A,H
20D1 A7              AND  A      slut på kedjan?
20D2 C8              RET  Z

```

```

20D3 E5          PUSH HL
20D4 DD E1       POP  IX
20D6 CD 7F 08    CALL CLOSE
20D9 DD 6E 00    LD   L, (IX+$00)
20DC DD 66 01    LD   H, (IX+$01)      nästa FCB
20DF 18 EF       JR   L20D0

```

FNDFIL: leta efter FCB för fil A. Ut: Z och IX = FCB om den finns, annars NZ och IX = sista FCB i kedjan (eller **FILLST** om kedjan är tom, så att IX+0/1 alltid är länken som ska ändras).

```

20E1 FD E5       FNDFIL: PUSH IY
20E3 DD E1       POP  IX
20E5 01 1C 00    LD   BC, $001C      IX = FILLST ($FE32)
20E8 DD 09       ADD  IX, BC
20EA DD 4E 00    L20EA: LD   C, (IX+$00)
20ED DD 46 01    LD   B, (IX+$01)
20F0 04          INC  B              länk 0: slut
20F1 05          DEC  B
20F2 20 02       JR   NZ, L20F6
20F4 04          INC  B              NZ: hittades inte
20F5 C9          RET
20F6 C5          L20F6: PUSH BC
20F7 DD E1       POP  IX
20F9 DD BE 02    CP   (IX+$02)      rätt filnummer?
20FC 20 EC       JR   NZ, L20EA
20FE C9          RET

```

3.55 OUT / POKE: kompilering

\$20FF

OUT p, v [, p, v ...] **POKE** a, v [, v ...]

Alla är heltal, och kommatecknen kopieras till koden:

```

p $BA ", " v $BA [", " p $BA ", " v $BA ...]
a $BA ", " v $BA [", " v $BA ...]

```

Slingan görs med en returadress i BC: \$2107 för OUT (nästa par), \$2119 för POKE (nästa värde). Fel 13 om ", " saknas efter p eller a (och därmed för ett udda antal tal i OUT); ett ", " sist ger fel 11.

```

20FF 01 07 21    CMPOUT: LD   BC, $2107      OUT: nästa par
2102 18 03       JR   L2107
2104 01 19 21    CMPPOK: LD   BC, $2119      POKE: nästa värde
2107 C5          L2107: PUSH BC
2108 CD 70 23    CALL CMPINX
210B C1          POP  BC
210C D8          RET  C
210D 7E          LD   A, (HL)
210E EE 2C       XOR  $2C
2110 3E 8D       LD   A, $8D      fel 13
2112 37          SCF
2113 C0          RET  NZ
2114 C5          PUSH BC
2115 ED A0       LDI              kopiera ", "
2117 E7          RST  $20
2118 C1          POP  BC
2119 C5          PUSH BC
211A CD 70 23    CALL CMPINX      v
211D C1          POP  BC
211E D8          RET  C
211F 7E          LD   A, (HL)
2120 EE 2C       XOR  $2C
2122 C0          RET  NZ      inget ", ": klart
2123 C5          PUSH BC
2124 C5          PUSH BC
2125 ED A0       LDI
2127 E7          RST  $20
2128 C1          POP  BC
2129 C9          RET      till $2107 eller $2119

```

OUT vid körning: för varje par skrivs först det sparade värdet för port 1 (IY+\$15) till port 1 och sedan v till port p. Ett värde som skrivs till port 1 sparas där. Port 1 väljer enhet på ABC-bussen (abc80sim), och valet gäller för alla senare OUT och INP.

Både p och v tas modulo 256, så även OUT 257, v räknas som port 1. Det sparade värdet är \$FF från start; CLEAR och RUN ändrar det inte.

212A	FF	OUTST:	RST	\$38	p
212B	13		INC	DE	hoppa över ",,"
212C	E5		PUSH	HL	
212D	FF		RST	\$38	v
212E	C1		POP	BC	
212F	79		LD	A,C	
2130	3D		DEC	A	port 1 = val av enhet?
2131	20 03		JR	NZ,L2136	
2133	FD 75 15		LD	(IY+\$15),L	spara värdet
2136	FD 7E 15	L2136:	LD	A,(IY+\$15)	
2139	D3 01		OUT	(\$01),A	port 1 först
213B	ED 69		OUT	(C),L	sedan v till port p
213D	1A		LD	A,(DE)	
213E	FE 2C		CP	\$2C	
2140	C0		RET	NZ	
2141	13		INC	DE	
2142	18 E6		JR	OUTST	

POKE vid körning: värdena skrivs till a, a+1, a+2 ... med avbrotten avstängda, modulo 256 (POKE a,258 skriver 2).

2144	FF	POKE:	RST	\$38	a
2145	13	L2145:	INC	DE	
2146	E5		PUSH	HL	
2147	FF		RST	\$38	v
2148	7D		LD	A,L	
2149	E1		POP	HL	
214A	F3		DI		
214B	77		LD	(HL),A	skriv
214C	FB		EI		
214D	23		INC	HL	
214E	1A		LD	A,(DE)	
214F	FE 2C		CP	\$2C	
2151	28 F2		JR	Z,L2145	
2153	C9		RET		

3.56 PRINT: kompilering

\$2154

PRINT [#n,] [uttryck] [, eller ;] [uttryck] ... (";" = PRINT)
 ["#" n \$BA] ... uttryck \$B9+t ... ",," ";" ...

Tecknen "#", "," och ";" kopieras till koden, och varje uttryck följs av \$B9+typ. Två uttryck får stå direkt efter varandra (PRINT A\$B\$), utom när det första är ett flyttal och det andra börjar med en bokstav (även gemen och ÄÖÅ): då blir det fel 11 (:? så att t.ex. PRINT A B inte godtas). PRINT A%B och PRINT 1(2) går, och PRINT 1 2 skriver 12, eftersom talet läses förbi blanka (GETFLT).

Satsen slutar vid ":", CR eller ELSE (STMEND).

2154	7E	CMPRT:	LD	A,(HL)	
2155	FE 23		CP	\$23	"#"?
2157	20 07		JR	NZ,L2160	
2159	ED A0		LDI		kopiera "#"
215B	E7		RST	\$20	
215C	CD 70 23		CALL	CMPINX	filnumret
215F	D8		RET	C	
2160	7E	L2160:	LD	A,(HL)	
2161	FE 2C		CP	\$2C	"","?
2163	28 23		JR	Z,L2188	
2165	FE 3B		CP	\$3B	"","?
2167	28 1F		JR	Z,L2188	
2169	CD 8D 21		CALL	STMEND	slut på satsen?
216C	C8		RET	Z	
216D	CD 62 23		CALL	CMPEXB	uttryck + \$B9 + typ
2170	D8		RET	C	

2171	3E 00	LD	A,\$00	
2173	B8	CP	B	inte flyttal: vidare
2174	20 EA	JR	NZ,L2160	
2176	CD 8D 21	CALL	STMEND	
2179	C8	RET	Z	
217A	7E	LD	A,(HL)	
217B	E6 5F	AND	\$5F	stora bokstäver
217D	FE 41	CP	\$41	bokstav, A-Z ÄÖÅ?
217F	38 DF	JR	C,L2160	
2181	FE 5E	CP	\$5E	
2183	30 DB	JR	NC,L2160	
2185	3E 8B	LD	A,\$8B	fel 11
2187	C9	RET		
2188	ED A0	L2188: LDI		kopiera ",," eller ",,"
218A	E7	RST	\$20	
218B	18 D3	JR	L2160	

STMEND: Z om HL står på slutet av en sats: ":", CR eller ELSE.

218D	7E	STMEND: LD	A,(HL)	
218E	FE 3A	CP	\$3A	":,"?
2190	C8	RET	Z	
2191	FE 0D	CP	\$0D	CR?
2193	C8	RET	Z	
2194	D5	PUSH	DE	
2195	E5	PUSH	HL	
2196	11 80 19	LD	DE,KWELSE	ELSE?
2199	CD F5 09	CALL	KWSRCH	
219C	E1	POP	HL	
219D	D1	POP	DE	
219E	C9	RET		

3.57 PRINT vid körning

\$219F

Filnumret väljs med IY+\$0D (0 = skärmen). Sedan, i ordning:

",," nästa tabulatorstopp (var 14:e kolumn), eller ny rad om
det inte får plats
";," ingenting
uttryck skrivs ut (**PRTEXP**)
slut CR LF, utom när satsen slutar med ",," eller ";,"

Ett ",," direkt efter "#n" räknas som ";,".

219F	FD 36 0D 00	PRINTS: LD	(IY+\$0D),\$00	
21A3	1A	LD	A,(DE)	
21A4	FE 23	CP	\$23	"#"?
21A6	20 0A	JR	NZ,L21B2	
21A8	13	INC	DE	
21A9	FF	RST	\$38	filnumret
21AA	FD 75 0D	LD	(IY+\$0D),L	välj filen
21AD	1A	LD	A,(DE)	
21AE	FE 2C	CP	\$2C	":," efter #n: som ";,"
21B0	28 07	JR	Z,L21B9	
21B2	1A	L21B2: LD	A,(DE)	
21B3	FE 2C	CP	\$2C	":,"?
21B5	28 65	JR	Z,PRTCOM	
21B7	FE 3B	CP	\$3B	":,"?
21B9	CA 3A 22	L21B9: JP	Z,PRTSEP	
21BC	FE C0	CP	\$C0	uttryck?
21BE	38 36	JR	C,PRCRLF	nej: CR LF och klart

PRTEXP: räkna ut uttrycket och skriv det.

flyttal **FLTAS6** i ett område på 16 bytes på stacken
heltal **S1855** i bufferten **\$FDD5**: mellanslag eller "-", sedan
siffrorna
sträng texten skrivs direkt från stacken

Tal som inte får plats på raden (PRTFIT) börjar på en ny rad. Strängar kontrolleras inte. Efteråt tas stackområdet bort.

21C0	FF	PRTEXP:	RST	\$38	
21C1	78		LD	A,B	
21C2	FE 01		CP	\$01	heltal?
21C4	28 40		JR	Z,L2206	
21C6	FE 02		CP	\$02	sträng?
21C8	28 49		JR	Z,L2213	
21CA	21 F0 FF		LD	HL,\$FFF0	flyttal: 16 bytes på stacken
21CD	39		ADD	HL,SP	
21CE	F9		LD	SP,HL	
21CF	D5		PUSH	DE	
21D0	EB		EX	DE,HL	
21D1	21 14 00		LD	HL,\$0014	flyttalet (sista byten)
21D4	19		ADD	HL,DE	
21D5	D5		PUSH	DE	
21D6	CD 77 16		CALL	FLTAS6	till text
21D9	21 15 00		LD	HL,\$0015	16 + 5 bytes ska bort
21DC	E3	L21DC:	EX	(SP),HL	
21DD	EB		EX	DE,HL	
21DE	A7		AND	A	
21DF	ED 52		SBC	HL,DE	BC = längden
21E1	44		LD	B,H	
21E2	4D		LD	C,L	
21E3	EB		EX	DE,HL	
21E4	CD F0 21		CALL	PRTFIT	ny rad om det inte får plats
21E7	CD D4 08	L21E7:	CALL	PRINT	skriv
21EA	E1		POP	HL	
21EB	D1		POP	DE	
21EC	39		ADD	HL,SP	ta bort stackområdet
21ED	F9		LD	SP,HL	
21EE	18 C2		JR	L21B2	

PRTFIT: BC = längden. Om kolumn + längd inte är mindre än radbredden, skriv CR LF först (GETPOS: A = kolumn, E = bredd).
 Ett tal som skulle sluta i sista kolumnen börjar alltså på nästa rad.

21F0	CD EF 08	PRTFIT:	CALL	GETPOS	
21F3	81		ADD	A,C	kolumn + längd
21F4	93		SUB	E	- bredd
21F5	D8		RET	C	

PRCRLF: skriv CR LF (\$072D).

21F6	D5	PRCRLF:	PUSH	DE	
21F7	C5		PUSH	BC	
21F8	E5		PUSH	HL	
21F9	21 2D 07		LD	HL,\$072D	CR LF
21FC	01 02 00		LD	BC,\$0002	
21FF	CD D4 08		CALL	PRINT	
2202	E1		POP	HL	
2203	C1		POP	BC	
2204	D1		POP	DE	
2205	C9		RET		
2206	D5	L2206:	PUSH	DE	
2207	11 D5 FD		LD	DE,\$FDD5	buffert
220A	D5		PUSH	DE	
220B	CD 55 18		CALL	PR SIN	tecken och siffror
220E	21 00 00		LD	HL,\$0000	inget stackområde
2211	18 C9		JR	L21DC	
2213	DD E1	L2213:	POP	IX	antal tillfälliga bytes
2215	E1		POP	HL	adress
2216	C1		POP	BC	längd
2217	D5		PUSH	DE	
2218	DD E5		PUSH	IX	
221A	18 CB		JR	L21E7	

PRTCOM: ", ". C = 14 - (kolumn mod 14) mellanslag ur SPACES. Går det över radbredden blir det ny rad i stället. På 40 tecken

finns alltså stoppen 14 och 28, och från kolumn 28 blir det ny rad.

221C	D5	PRTCOM:	PUSH DE	
221D	CD EF 08		CALL GETPOS	A = kolumn, E = bredd
2220	47		LD B,A	
2221	D6 0E	L2221:	SUB \$0E	kolumn mod 14
2223	30 FC		JR NC,L2221	
2225	3D		DEC A	
2226	2F		CPL	
2227	4F		LD C,A	C = mellanslag till nästa stopp
2228	80		ADD A,B	
2229	BB		CP E	får plats?
222A	38 05		JR C,L2231	
222C	CD F6 21		CALL PRCRLF	nej: ny rad
222F	18 08		JR L2239	
2231	06 00	L2231:	LD B,\$00	
2233	21 4A 22		LD HL,SPACES	mellanslag
2236	CD D4 08		CALL PRINT	
2239	D1	L2239:	POP DE	

PRTSEP: efter ",," eller ";". Kommer det inget uttryck efter slutar PRINT utan CR LF.

223A	13	PRTSEP:	INC DE	
223B	1A		LD A,(DE)	
223C	FE 2C		CP \$2C	","?"
223E	28 DC		JR Z,PRTCOM	
2240	FE 3B		CP \$3B	","?"
2242	28 F6		JR Z,PRTSEP	
2244	FE C0		CP \$C0	uttryck?
2246	D2 C0 21		JP NC,PRTSEP	
2249	C9		RET	slut, utan CR LF
224A	20 20 20 20...	SPACES:	DEFB "	"

RANDOMIZE: slumpatsfröet \$FE39-\$FE3D fylls med R-registret.

RNDINI (från RUN) använder masken C = 0, så fröet blir noll och RND ger samma följd varje gång programmet körs.

R ökar med 6 per varv i slingan (sex M1-cykler), så de fem byten blir R, R+6, R+12 ... ROM:en skriver aldrig till R, och bit 7 är 0 efter RESET. RANDOMIZE kan därför bara ge 128 olika frön.

2258	01 FF 05	RANDOM:	LD BC,\$05FF	
225B	18 03		JR L2260	
225D	01 00 05	RNDINI:	LD BC,\$0500	
2260	21 39 FE	L2260:	LD HL,\$FE39	
2263	ED 5F	L2263:	LD A,R	R AND mask
2265	A1		AND C	
2266	77		LD (HL),A	R, R+6, R+12 ...
2267	23		INC HL	
2268	10 F9		DJNZ L2263	

CMPNOP: RANDOMIZE m.fl. har ingen kod efter token.

226A	C9	CMPNOP:	RET
------	----	----------------	-----

CMPREM: DATA och REM. Resten av raden kopieras oförändrad till koden. (Därför måste DATA stå sist, och READ läser värdena direkt ur texten med RDVAL.)

226B	3E 0D	CMPREM:	LD A,\$0D	
226D	BE	L226D:	CP (HL)	till CR
226E	C8		RET Z	
226F	ED A0		LDI	
2271	18 FA		JR L226D	

DATA vid körning: fel 2 i direktläge, annars som REM: hoppa till CR (DE pekar på CR efteråt).

2273	CD BF 0D	DATA:	CALL NODIR
2276	3E 0D	REM:	LD A,\$0D
2278	EB		EX DE,HL
2279	01 00 00		LD BC,\$0000

227C	ED B1	CPIR	leta upp CR
227E	EB	EX DE,HL	
227F	1B	DEC DE	DE på CR
2280	C9	RET	

RESTORE [nr]: kompilering. Med radnummer en radreferens (\$BF), annars ingenting efter token (CMPREF:s fel ignoreras).

2281	CD C7 1D	CMPRES: CALL CMPREF	
2284	A7	AND A	CY = 0 även utan radnummer
2285	C9	RET	

RESTORE vid körning: med radnummer börjar READ på den raden om den börjar med DATA, annars på nästa DATA-rad efter den. Utan radnummer från programmets början (RSTDAT). En rad som saknas ger fel 6 vid länknigen.

2286	1A	RESTOR: LD A, (DE)	
2287	FE BF	CP \$BF	radnummer?
2289	20 08	JR NZ, RSTDAT	
228B	EB	EX DE,HL	
228C	23	INC HL	
228D	23	INC HL	
228E	CD 1C 10	CALL RDDE1	radens adress
2291	18 04	JR L2297	

RSTDAT: DATA-pekaren. DATLIN får första raden, och om den börjar med DATA (\$93) pekar DATPTR på dess data. Annars pekar DATPTR på \$1ECA, en byte \$0D inuti ROM-koden, så att nästa READ letar vidare.

2293	2A 1C FE	RSTDAT: LD HL, (BOFA)	
2296	EB	EX DE,HL	
2297	EB	L2297: EX DE,HL	
2298	22 30 FE	LD (DATLIN), HL	
229B	7E	LD A, (HL)	
229C	FE 01	CP \$01	
229E	28 09	JR Z, L22A9	
22A0	23	INC HL	
22A1	23	INC HL	
22A2	23	INC HL	
22A3	7E	LD A, (HL)	
22A4	23	INC HL	
22A5	FE 93	CP \$93	DATA?
22A7	28 03	JR Z, L22AC	
22A9	21 CA 1E	L22A9: LD HL, \$1ECA	\$0D i READ: nästa DATA-rad
22AC	22 2E FE	L22AC: LD (DATPTR), HL	
22AF	C9	RET	

3.58 RETURN

\$22B0

Stacken när en sats körs: [till RUNSTM] [till RUNLIN] [RUNLIN:s anropare]. De två POP tar bort de två första, och RET går då till den som anropade RUNLIN:

- GOSUB (\$1DF9): DE = koden efter GOSUB, och den gamla raden fortsätter.
- RUN: \$071B, fel 29 ("RETURN" UTAN "GOSUB").

Resten av RETURN-raden körs aldrig. CURLIN pekar på RETURN-raden tills nästa rad börjar: ett fel eller STOP i resten av GOSUB-raden får RETURN-radens nummer. I direktläge fel 2 först när satsen körs.

22B0	CD BF 0D	RETURN: CALL NODIR	fel 2 i direktläge
22B3	E1	POP HL	ta bort RUNSTM och RUNLIN
22B4	E1	POP HL	

NOCODE: satser utan argument (RETURN, STOP, END, TRACE, NOTRACE) kompileras till bara token.

22B5	C9	NOCODE: RET
------	----	--------------------

CPMDOT: SETDOT r,k / CLRDOT r,k: r \$BA ", " k \$BA.

Kontrollen av kommat fungerar inte: SCF före RET NC gör att den aldrig slår till, och tecknet efter r kopieras vad det än är: "SETDOT 5;6" och "SETDOT 5X6" sätter punkten 5,6. Fel 61 (KOMMA SAKNAS) kommer alltså inte härifrån. Utan k beror

utfallet på det som står efter raden: fel 11 eller 14, eller i direktläge en punkt.

22B6	CD 70 23	CMPDOT:	CALL	CMPINX	r
22B9	D8		RET	C	
22BA	7E		LD	A, (HL)	
22BB	EE 2C		XOR	\$2C	","?"
22BD	3E BD		LD	A, \$BD	
22BF	37		SCF		(CY = 1: tas aldrig)
22C0	D0		RET	NC	
22C1	ED A0		LDI		kopiera tecknet
22C3	E7		RST	\$20	
22C4	C3 70 23		JP	CMPINX	k

SETDOT / CLRDOT: sätt eller släck en punkt i skärmens grafik.

Bit 5 sätts alltid, så att tecknet blir ett grafiktecken. Står det text där ORas biten in i tecknet ("A" blir "a"). **CLRDOT** lämnar bit 5, så ett tomt grafiktecken blir \$20. r och k avrundas nedåt.

22C7	CD E3 22	SETDOT:	CALL	DOTARG	
22CA	B6		OR	(HL)	sätt biten
22CB	F6 20	L22CB:	OR	\$20	grafiktecken
22CD	77		LD	(HL), A	
22CE	C9		RET		
22CF	CD E3 22	CLRDOT:	CALL	DOTARG	
22D2	2F		CPL		släck biten
22D3	A6		AND	(HL)	
22D4	18 F5		JR	L22CB	

DOT(r,k): -1 om punkten är satt, annars 0. CY från **DOTADR** provas inte: utanför skärmen blir det inget fel 62, utan A AND en ROM-byte (på adressen k, eller \$0372 när r >= 72). Även texttecken ger -1 när biten råkar vara satt.

22D6	E1	EXDOT:	POP	HL	k
22D7	C1		POP	BC	r
22D8	CD EE 22		CALL	DOTADR	
22DB	A6		AND	(HL)	satt?
22DC	21 00 00		LD	HL, \$0000	
22DF	28 01		JR	Z, L22E2	
22E1	2B		DEC	HL	ja: -1
22E2	EF	L22E2:	RST	\$28	

DOTARG: räkna ut r och k (koden "r \$BA, k \$BA") och ta fram adress och bit med **DOTADR**. Fel 62 om punkten är utanför skärmen.

22E3	FF	DOTARG:	RST	\$38	r
22E4	E5		PUSH	HL	
22E5	13		INC	DE	hoppa över ","
22E6	FF		RST	\$38	k
22E7	C1		POP	BC	
22E8	CD EE 22		CALL	DOTADR	
22EB	D0		RET	NC	
22EC	D7		RST	\$10	
22ED	BE		DEFB	\$BE	fel 62: DOT-ADRESS UTANFÖR SKÄRMEN

3.59 DOTADR: skärmadress och bit för en grafikpunkt

\$22EE

In: BC = r (rad, 0-71), HL = k (kolumn, 0-79).

Ut: HL = tecknets adress på skärmen, A = biten. CY = utanför. Varje tecken är 2 x 3 punkter:

k jämn	k udda	
\$01	\$02	r mod 3 = 0
\$04	\$08	r mod 3 = 1
\$10	\$40	r mod 3 = 2

Teckenraden = r/3 (radtabellen på \$0374), teckenkolumnen = k/2. Biten räknas fram genom att rotera \$40 eller \$80 två steg per rad; bit 5 hoppas över med ADD \$20 / AND \$DF, så \$20 blir \$40.

22EE	7C	DOTADR:	LD	A, H	
22EF	B0		OR	B	över 255?
22F0	37		SCF		

22F1	C0	RET	NZ	
22F2	B5	OR	L	
22F3	FE 50	CP	\$50	k < 80?
22F5	3F	CCF		
22F6	D8	RET	C	
22F7	06 40	LD	B, \$40	jämn kolumn: \$40
22F9	1F	RRA		k/2
22FA	30 02	JR	NC, L22FE	
22FC	06 80	LD	B, \$80	udda: \$80
22FE	6F	L22FE:	LD	L, A
22FF	79	LD	A, C	
2300	4D	LD	C, L	
2301	21 72 03	LD	HL, \$0372	radtabellen - 2
2304	FE 48	CP	\$48	r < 72?
2306	3F	CCF		
2307	D8	RET	C	
2308	23	L2308:	INC	HL
2309	23	INC	HL	
230A	D6 03	SUB	\$03	r/3
230C	30 FA	JR	NC, L2308	
230E	CB 08	L230E:	RRC	B
2310	CB 08	RRC	B	två steg per rad
2312	3C	INC	A	
2313	20 F9	JR	NZ, L230E	
2315	7E	LD	A, (HL)	radens början + k/2
2316	81	ADD	A, C	
2317	23	INC	HL	
2318	66	LD	H, (HL)	
2319	6F	LD	L, A	
231A	78	LD	A, B	
231B	C6 20	ADD	A, \$20	hoppa över bit 5
231D	E6 DF	AND	\$DF	
231F	C9	RET		

STOP: fel 2 i direktläge. Annars "fel" 0, som skrivs som STOP LINE nn. Resten av raden körs inte, och ON ERROR fångar inte STOP (fel 0 finns inte i ERRTRP).

2320 CD BF 0D **STOP:** CALL NODIR

BREAK under körning: "fel" med kod 0, som ERRMSG skriver som STOP.

2323	D7	BRKSTP:	RST	\$10	fångas inte av ON ERROR
2324	00		DEFB	\$00	kod 0: skrivs som STOP
2325	FD CB 0F E6	TRACE:	SET	4, (IY+\$0F)	
2329	C9		RET		
232A	FD CB 0F A6	NOTRAC:	RES	4, (IY+\$0F)	
232E	C9		RET		

TRACE: skriv radnumret för raden som ska köras (från RUNLIN, en gång per rad, inte per sats). DE = raden. Numret skrivs med PRNUM i 10 bytes på stacken, följt av \$1F, som i bildminnet syns som blankt. Ny rad när kolumnen + 7 >= 40: 11 tvåsiffriga nummer per rad, 6 femsiffriga. Direktrader skrivs inte. TRACE mitt på en rad gäller från nästa rad. CLEAR och RUN behåller TRACE, NEW slår av det.

232F	D5	TRCPRT:	PUSH	DE	
2330	CD 38 03		CALL	SCRPOS	A = kolumnen, E = 40
2333	C6 07		ADD	A, \$07	
2335	BB		CP	E	
2336	D4 42 07		CALL	NC, NEWLIN	ryms inte: ny rad
2339	DD E1		POP	IX	IX = raden
233B	21 F6 FF		LD	HL, \$FFF6	10 bytes på stacken
233E	39		ADD	HL, SP	
233F	F9		LD	SP, HL	
2340	EB		EX	DE, HL	
2341	DD 6E 01		LD	L, (IX+\$01)	radnumret
2344	DD 66 02		LD	H, (IX+\$02)	
2347	DD E5		PUSH	IX	
2349	D5		PUSH	DE	

234A	CD 62 18	CALL	PRNUM	
234D	EB	EX	DE,HL	
234E	36 1F	LD	(HL), \$1F	syns som blankt
2350	23	INC	HL	
2351	D1	POP	DE	
2352	A7	AND	A	
2353	ED 52	SBC	HL,DE	HL = längden
2355	EB	EX	DE,HL	
2356	42	LD	B,D	
2357	4B	LD	C,E	
2358	CD A8 01	CALL	PRTXT	direkt i bildminnet
235B	D1	POP	DE	
235C	21 0A 00	LD	HL, \$000A	
235F	39	ADD	HL,SP	
2360	F9	LD	SP,HL	
2361	C9	RET		

CMPEXB: kompilera ett uttryck och skriv \$B9 + typ.

2362	CD 01 27	CMPEXB:	CALL	CMPEXP	
2365	D8	L2365:	RET	C	
2366	3E B9		LD	A,\$B9	\$B9 + typ

PUTTOK: skriv A + B (token + typ).

2368	80	PUTTOK:	ADD	A,B	
2369	12		LD	(DE),A	
236A	13		INC	DE	
236B	C9		RET		

CM PSTX / CMPINX / CMPTYX: kompilera ett uttryck, omvandla det till sträng / heltal / typen B, och skriv \$B9 + typ. **CM PSTX** anropas också från tilläggs-ROM:en via \$003B.

236C	06 02	CM PSTX:	LD	B,\$02	sträng
236E	18 02		JR	CMPTYX	
2370	06 01	CM PINX:	LD	B,\$01	heltal
2372	C5	CM PTYX:	PUSH	BC	
2373	CD 01 27		CALL	CMPEXP	
2376	DA F7 1B		JP	C,CMPEXP	fel: avbryt satsen
2379	F1		POP	AF	
237A	CD 51 25		CALL	TYPCNV	omvandla till typen B
237D	18 E6		JR	L2365	

CMPSV: enkel variabel (utan index) som inte är sträng. Fel 17 för sträng. Koden blir variabelns 5 bytes.

237F	CD CB 26	CMPSV:	CALL	VARNAM	
2382	D8		RET	C	
2383	79		LD	A,C	typen
2384	E6 03		AND	\$03	
2386	FE 02		CP	\$02	sträng?
2388	3E 91		LD	A,\$91	fel 17
238A	37		SCF		
238B	C8		RET	Z	
238C	C5		PUSH	BC	
238D	0E 00		LD	C,\$00	inga index
238F	C3 FC 25		JP	L25FC	

CMPSVR / CMPVR: variabel följd av \$BC + typ. Vid körning avslutar \$BC-\$BE uttrycket med variabelns adress i HL och typen i B (se EXPTAB1). **CMPSVR:** enkel talvariabel, **CMPVR:** vilken variabel som helst, med index. Används där satsen ska skrivas till variabeln.

2392	CD 7F 23	CMPSVR:	CALL	CMPSV	
2395	18 03		JR	L239A	
2397	CD EE 25	CMPVR:	CALL	CMPVAR	
239A	D8	L239A:	RET	C	
239B	3E BC	PUTBC:	LD	A,\$BC	\$BC + typ: variabelns adress
239D	18 C9		JR	PUTTOK	

CMPIVR / CMPGET: variabel med index + \$BC + typ. **CMPIVR** kräver heltal (anropas inte i denna ROM), **CMPGET** kräver sträng (GET, INPUT LINE). Annars fel 17.

```

239F CD EE 25      CMPIVR: CALL CMPVAR
23A2 D8           RET C
23A3 78           LD A,B
23A4 FE 01        CP $01          heltal?
23A6 18 07        JR L23AF

23A8 CD EE 25      CMPGET: CALL CMPVAR
23AB D8           RET C
23AC 78           LD A,B
23AD FE 02        CP $02          sträng?
23AF 3E 91        L23AF: LD A,$91    annars fel 17
23B1 37           SCF
23B2 C0           RET NZ
23B3 18 E6        JR PUTBC

```

3.60 Uttryckskompilatorn: en post per prioritetsnivå

\$23B5

Varje post är [rutin][nästa nivå][operatortabell][klass]. Nivåerna, från lägst till högst prioritet:

```

PREC1  EQV          klass 3 (logisk)
PREC2  IMP          klass 3
PREC3  OR XOR       klass 3
PREC4  AND          klass 3
PREC5  NOT          egen rutin (CMPNOT), unär
PREC6  = < > <= > >= klass 2 (jämförelse)
PREC7  + -          klass 1, egen rutin (CMPADD) för unärt -/+
PREC8  * /          klass 1 (aritmetik)
PREC9  ^ **         klass 4 (upphöjt till)
PREC10 primärt uttryck (PRIMARY)

```

PREC5 har ingen klassbyte; den första byten i **PREC6** hamnar där, men **CMPNOT** läser den inte.

Alla binära operatörer är vänsterassociativa: A-B-C = (A-B)-C.

```

23B5 05 27 BC 23... PREC1: DEFW PRECLV,PREC2,KWEQV
23BB 03            DEFB $03
23BC 05 27 C3 23... PREC2: DEFW PRECLV,PREC3,KWIMP
23C2 03            DEFB $03
23C3 05 27 CA 23... PREC3: DEFW PRECLV,PREC4,KWOR
23C9 03            DEFB $03
23CA 05 27 D1 23... PREC4: DEFW PRECLV,PREC5,KWAND
23D0 03            DEFB $03
23D1 26 28 D7 23... PREC5: DEFW CMPNOT,PREC6,KWNOT
23D7 05 27 DE 23... PREC6: DEFW PRECLV,PREC7,KWCMP
23DD 02            DEFB $02
23DE FB 27 E5 23... PREC7: DEFW CMPADD,PREC8,KWADD
23E4 01            DEFB $01
23E5 05 27 EC 23... PREC8: DEFW PRECLV,PREC9,KWMUL
23EB 01            DEFB $01
23EC 05 27 F3 23... PREC9: DEFW PRECLV,PREC10,KWPOW
23F2 04            DEFB $04
23F3 7F 25        PREC10: DEFW PRIMARY

```

Operatortabellerna har samma form som nyckelordstabellerna: [token][text] ... \$FF. Jämförelser och aritmetik har en token per typ: bas + 0 flyttal, + 1 heltal, + 2 sträng. I tabellen står bara bastoken. "<=" och ">=" står före "<" och ">", annars skulle **KWSRCH** hitta "<" först. "***" är en annan stavning av "^".

```

23F5 DC 45 51 56 KWEQV: DEFB $DC,"EQV"
23F9 FF          DEFB $FF
23FA DD 49 4D 50 KWIMP: DEFB $DD,"IMP"
23FE FF          DEFB $FF
23FF DE 4F 52    KWOR: DEFB $DE,"OR"
2402 DF 58 4F 52 DEFB $DF,"XOR"
2406 FF          DEFB $FF
2407 E0 41 4E 44 KWAND: DEFB $E0,"AND"
240B FF          DEFB $FF
240C E1 4E 4F 54 KWNOT: DEFB $E1,"NOT"

```

2410	FF		DEFB \$FF
2411	F1 3C 3D	KWCMP:	DEFB \$F1, "<="
2414	E5 3C 3E		DEFB \$E5, "<>"
2417	E8 3C		DEFB \$E8, "<"
2419	EB 3E 3D		DEFB \$EB, ">="
241C	EE 3E		DEFB \$EE, ">"
241E	E2 3D		DEFB \$E2, "="
2420	FF		DEFB \$FF
2421	F4 2B	KWADD:	DEFB \$F4, "+"
2423	F7 2D		DEFB \$F7, "-"
2425	FF		DEFB \$FF
2426	F9 2A	KWMUL:	DEFB \$F9, "*"
2428	FB 2F		DEFB \$FB, "/"
242A	FF		DEFB \$FF
242B	FD 5E	KWPOW:	DEFB \$FD, "^"
242D	FD 2A 2A		DEFB \$FD, "**"
2430	FF		DEFB \$FF

*Funktionernas namn. Token \$80+kod; koden (0-\$34) blir 2*kod i den kompillerade koden och används som index i EXPTAB2. LOG10 står före LOG av samma skäl som "<=" före "<".*

2431	80 46 4E	FNKW:	DEFB \$80, "FN"
2434	81 41 42 53		DEFB \$81, "ABS"
2438	82 41 54 4E		DEFB \$82, "ATN"
243C	83 43 4F 53		DEFB \$83, "COS"
2440	84 45 58 50		DEFB \$84, "EXP"
2444	85 46 49 58		DEFB \$85, "FIX"
2448	86 49 4E 54		DEFB \$86, "INT"
244C	88 4C 4F 47...		DEFB \$88, "LOG10"
2452	87 4C 4F 47		DEFB \$87, "LOG"
2456	89 50 49		DEFB \$89, "PI"
2459	8A 52 4E 44		DEFB \$8A, "RND"
245D	8B 53 47 4E		DEFB \$8B, "SGN"
2461	8C 53 49 4E		DEFB \$8C, "SIN"
2465	8D 53 51 52		DEFB \$8D, "SQR"
2469	8E 54 41 4E		DEFB \$8E, "TAN"
246D	8F 41 53 43		DEFB \$8F, "ASC"
2471	90 43 48 52...		DEFB \$90, "CHR\$"
2476	91 4C 45 46...		DEFB \$91, "LEFT\$"
247C	92 52 49 47...		DEFB \$92, "RIGHT\$"
2483	93 4D 49 44...		DEFB \$93, "MID\$"
2488	94 4C 45 4E		DEFB \$94, "LEN"
248C	95 49 4E 53...		DEFB \$95, "INSTR"
2492	96 53 50 41...		DEFB \$96, "SPACE\$"
2499	97 53 54 52...		DEFB \$97, "STRING\$"
24A1	98 4E 55 4D...		DEFB \$98, "NUM\$"
24A6	99 56 41 4C		DEFB \$99, "VAL"
24AA	9A 53 57 41...		DEFB \$9A, "SWAP\$"
24B0	9B 50 45 45...		DEFB \$9B, "PEEK"
24B5	9C 49 4E 50		DEFB \$9C, "INP"
24B9	9D 43 41 4C...		DEFB \$9D, "CALL"
24BE	9E 45 52 52...		DEFB \$9E, "ERRCODE"
24C6	9F 49 45 43...		DEFB \$9F, "IEC\$"
24CB	A0 54 41 42		DEFB \$A0, "TAB"
24CF	A1 43 55 52		DEFB \$A1, "CUR"
24D3	A3 44 4F 54		DEFB \$A3, "DOT"
24D7	B0 41 44 44...		DEFB \$B0, "ADD\$"
24DC	B1 53 55 42...		DEFB \$B1, "SUB\$"
24E1	B2 4D 55 4C...		DEFB \$B2, "MUL\$"
24E6	B3 44 49 56...		DEFB \$B3, "DIV\$"
24EB	B4 43 4F 4D...		DEFB \$B4, "COMP\$"
24F1	FF		DEFB \$FF

3.61 FNTYPE: typer för funktionernas resultat och argument

\$24F2

Varje post:

[en eller flera token][resultat][argument ...]

Posten slutar vid nästa byte med bit 7 satt (nästa posts token, eller \$FF). Resultat- och argumentbyten:

bit 0-1 typ: 0 = flyttal, 1 = heltal, 2 = sträng
 bit 6 listan får sluta här (i resultatbyten: funktionen får skrivas utan argument)

Argumenten omvandlas till den typ som står (se [TYPENV](#)); heltal <-> flyttal görs automatiskt, sträng <-> tal ger fel.

Ur tabellen:

ABS	ATN	COS	EXP	FIX	INT	LOG	LOG10	SIN	SQR	TAN	(x)	-> flyttal
PI												-> flyttal
RND	eller	RND(x)										-> flyttal
SGN(x)												-> heltal
ASC(s\$)	LEN(s\$)											-> heltal
CHR\$(i[,i[,i[,i]]])	1-4 tecken											-> sträng
LEFT\$(s\$,i)	RIGHT\$(s\$,i)											-> sträng
MID\$(s\$,i,i)												-> sträng
INSTR(i,s\$,s\$)												-> heltal
TAB(i)	SPACE\$(i)	IEC\$(i)										-> sträng
CUR(i,i)	STRING\$(i,i)											-> sträng
NUM(x)												-> sträng
VAL(s\$)												-> flyttal
PEEK(i)	SWAP%(i)	INP(i)										-> heltal
ERRCODE												-> heltal
DOT(i,i)												-> heltal
ADD\$	SUB\$	MUL\$	DIV\$	(s\$,s\$,i)								-> sträng
COMP\$(s\$,s\$)												-> heltal

Efter \$FF följer två listor som används direkt med adress:

\$2548 \$41 \$41 \$41 \$FF DIM av sträng: 0-2 index
 \$254D \$01 \$41 \$41 \$FF DIM av tal: 1-2 index. Samma bytes är också posten för CALL(i[,i]) -> heltal (token \$9D står före).

24F2	81	82	83	84...	FNTYPE:	DEFB	\$81,\$82,\$83,\$84,\$85,\$86,\$87,\$88
24FA	8C	8D	8E	00...		DEFB	\$8C,\$8D,\$8E,\$00,\$40,\$89,\$40,\$8A
2502	40	40	8B	01...		DEFB	\$40,\$40,\$8B,\$01,\$40,\$8F,\$94,\$01
250A	42	90	02			DEFB	\$42,\$90,\$02
250D	41	41	41	41		DEFB	"AAAA"
2511	91	92	02	02...		DEFB	\$91,\$92,\$02,\$02,\$41,\$93,\$02,\$02
2519	01	41	95	01...		DEFB	\$01,\$41,\$95,\$01,\$01,\$02,\$42,\$A0
2521	96	9F	02	41...		DEFB	\$96,\$9F,\$02,\$41,\$A1,\$97,\$02,\$01
2529	41	98	02	40...		DEFB	\$41,\$98,\$02,\$40,\$99,\$00,\$42,\$9B
2531	9A	9C	01	41...		DEFB	\$9A,\$9C,\$01,\$41,\$9E,\$41,\$A3,\$01
2539	01	41	B0	B1...		DEFB	\$01,\$41,\$B0,\$B1,\$B2,\$B3,\$02,\$02
2541	02	41	B4	01...		DEFB	\$02,\$41,\$B4,\$01,\$02,\$42,\$FF,\$41
2549	41	41	FF	9D...		DEFB	\$41,\$41,\$FF,\$9D,\$01,\$41,\$41,\$FF

3.62 TYPENV: omvandla värdet till typen A (bit 0-1)

\$2551

B = typen på det kompillerade värdet. Heltal -> flyttal skriver \$DA, flyttal -> heltal \$D9. Sträng <-> tal ger fel 17 (OTILLÅTEN BLANDNING AV TAL OCH STRÄNGAR); ett tal där en sträng ska vara ger fel 13. Ut: B = den nya typen.

[TOINT](#)/[TOFLT](#): samma, men till heltal/flyttal.

2551	E6	03	TYPENV:	AND	\$03	
2553	B8		CP	B		redan rätt typ
2554	C8		RET	Z		
2555	FE	01	CP	\$01		
2557	28	08	JR	Z, TOINT		
2559	FE	00	CP	\$00		
255B	28	11	JR	Z, TOFLT		

255D	3E 8D		LD	A,\$8D	fel 13
255F	37		SCF		
2560	C9		RET		
2561	78	TOINT:	LD	A,B	
2562	06 01		LD	B,\$01	
2564	B8		CP	B	
2565	C8		RET	Z	
2566	FE 00		CP	\$00	
2568	20 0B		JR	NZ,L2575	
256A	3E D9		LD	A,\$D9	\$D9: flyttal -> heltal
256C	18 0D		JR	L257B	
256E	78	TOFLT:	LD	A,B	
256F	06 00		LD	B,\$00	
2571	B8		CP	B	
2572	C8		RET	Z	
2573	FE 01		CP	\$01	
2575	3E 91	L2575:	LD	A,\$91	fel 17
2577	37		SCF		
2578	C0		RET	NZ	
2579	3E DA		LD	A,\$DA	\$DA: heltal -> flyttal
257B	12	L257B:	LD	(DE),A	
257C	13		INC	DE	
257D	A7		AND	A	
257E	C9		RET		

3.63 PRIMARY: kompilera ett primärt uttryck

\$257F

siffra eller "." tal (CMPNUM)
 "(" uttryck ")" uttrycket följt av \$D3 (för LIST)
 " eller ' strängkonstant (CMPSTR)
 funktionsnamn CMPFNC (FN.. går till CMPFN)
 annars variabel (CMPVAR) och \$D4+typ (hämta värdet)

Ut: B = typen, DE efter koden, HL efter texten (blanktecken hoppas över), CY + A = felkod.

PRIMARY:					
257F	7E		LD	A,(HL)	
2580	FE 2E		CP	\$2E	","
2582	CA 89 26		JP	Z,CMPNUM	
2585	FE 30		CP	\$30	
2587	38 05		JR	C,L258E	
2589	FE 3A		CP	\$3A	
258B	DA 89 26		JP	C,CMPNUM	
258E	FE 28	L258E:	CP	\$28	"("
2590	28 1C		JR	Z,L25AE	
2592	FE 22		CP	\$22	citattecken
2594	28 30		JR	Z,CMPSTR	
2596	FE 27		CP	\$27	
2598	28 2C		JR	Z,CMPSTR	
259A	D5		PUSH	DE	
259B	11 31 24		LD	DE,FNKW	funktionsnamn?
259E	CD F5 09		CALL	KWSRCH	
25A1	D1		POP	DE	
25A2	28 78		JR	Z,CMPFNC	
25A4	CD EE 25		CALL	CMPVAR	
25A7	D8		RET	C	
25A8	3E D4		LD	A,\$D4	\$D4+typ: hämta värdet
25AA	80		ADD	A,B	
25AB	12		LD	(DE),A	
25AC	13		INC	DE	
25AD	C9		RET		
25AE	DF	L25AE:	RST	\$18	över (
25AF	CD 01 27		CALL	CMPEXP	från PREC1 igen
25B2	D8		RET	C	
25B3	7E		LD	A,(HL)	
25B4	FE 29		CP	\$29	")"

25B6	3E 92	LD	A, \$92	fel 18
25B8	37	SCF		
25B9	C0	RET	NZ	
25BA	3E D3	LD	A, \$D3	\$D3: parentesen
25BC	12	LD	(DE), A	
25BD	13	INC	DE	

SKIPSP (RST \$18): HL+1, hoppa över mellanslag. **SKIPS0** (RST \$20) utan HL+1.

25BE	23	SKIPSP:	INC	HL
25BF	7E	SKIPSP:	LD	A, (HL)
25C0	FE 20		CP	\$20
25C2	28 FA		JR	Z, SKIPSP
25C4	A7		AND	A
25C5	C9		RET	

CMPSTR: \$CB, citattecknet, längden, texten. Två citattecken efter varandra blir ett tecken i strängen. Strängen får sluta med radslutet utan ett avslutande citattecken.

25C6	23	CMPSTR:	INC	HL	
25C7	EB		EX	DE, HL	
25C8	36 CB		LD	(HL), \$CB	\$CB
25CA	23		INC	HL	
25CB	EB		EX	DE, HL	
25CC	4F		LD	C, A	
25CD	06 00		LD	B, \$00	
25CF	12		LD	(DE), A	
25D0	13		INC	DE	
25D1	D5		PUSH	DE	plats för längden
25D2	13		INC	DE	
25D3	7E	L25D3:	LD	A, (HL)	
25D4	FE 0D		CP	\$0D	radslut?
25D6	28 0F		JR	Z, L25E7	
25D8	23		INC	HL	
25D9	B9		CP	C	
25DA	28 05		JR	Z, L25E1	
25DC	12	L25DC:	LD	(DE), A	
25DD	13		INC	DE	
25DE	04		INC	B	
25DF	18 F2		JR	L25D3	
25E1	BE	L25E1:	CP	(HL)	dubbelt citattecken?
25E2	20 03		JR	NZ, L25E7	
25E4	23		INC	HL	
25E5	18 F5		JR	L25DC	
25E7	78	L25E7:	LD	A, B	A = längden
25E8	C1		POP	BC	
25E9	02		LD	(BC), A	
25EA	06 02		LD	B, \$02	typ sträng
25EC	18 D1		JR	SKIPS0	

3.64 CMPVAR: kompilera en variabel, med index

\$25EE

Koden blir 5 bytes:

[\$C0 + 4*antal index + typ] [namnbyte] [bokstav] [adress lo, hi]

Namnbyten: bit 4-7 = siffran (\$F om ingen), bit 2-3 = antal index, bit 0-1 = typen. Adressen fylls i vid förlänkningen (**LNKEXP**). Indexen (högst 2, heltal) kompileras först, före variabeln, med typlistan **D2618**.

Ut: B = variabelns typ.

25EE	CD CB 26	CMPVAR:	CALL	VARNAM	
25F1	D8		RET	C	
25F2	C5		PUSH	BC	
25F3	DD 21 18 26		LD	IX, D2618	högst 2 index
25F7	CD 4E 26		CALL	CMPARG	
25FA	38 45		JR	C, L2641	

25FC 79	L25FC:	LD	A, C	\$C0 + 4*antal index
25FD 87		ADD	A, A	
25FE 87		ADD	A, A	
25FF C6 C0		ADD	A, \$C0	
2601 12		LD	(DE), A	
2602 C1		POP	BC	
2603 3E 03		LD	A, \$03	
2605 A1		AND	C	
2606 EB		EX	DE, HL	
2607 B6		OR	(HL)	+ typ
2608 77		LD	(HL), A	
2609 23		INC	HL	
260A E6 0C		AND	\$0C	namnbyte med antal index
260C B1		OR	C	
260D 77		LD	(HL), A	
260E 23		INC	HL	
260F 70		LD	(HL), B	bokstaven
2610 23		INC	HL	
2611 23		INC	HL	
2612 23		INC	HL	
2613 E6 03		AND	\$03	
2615 47		LD	B, A	
2616 EB		EX	DE, HL	
2617 C9		RET		
2618 41 41 41 FF	D2618:	DEFB	\$41, \$41, \$41, \$FF	

CMPFNC: A = funktionens token. Posten letas upp i **FNTYPE**, och argumenten kompileras med **CMPARG**. Koden blir argument ... [\$CD + antal argument] [2*kod]

261C FE 80	CMPFNC:	CP	\$80	FN
261E CA B8 1A		JP	Z, CMPFN	
2621 E5		PUSH	HL	
2622 21 F2 24		LD	HL, FNTYPE	
2625 01 00 00		LD	BC, \$0000	
2628 ED B1		CPIR		hitta token i FNTYPE
262A 2B		DEC	HL	
262B 23	L262B:	INC	HL	
262C 4E		LD	C, (HL)	
262D 0C		INC	C	hoppa över token
262E FA 2B 26		JP	M, L262B	
2631 E5		PUSH	HL	
2632 DD E1		POP	IX	
2634 E1		POP	HL	
2635 87		ADD	A, A	2*kod
2636 4F		LD	C, A	
2637 DD 7E 00		LD	A, (IX+\$00)	resultatbyten
263A E6 03		AND	\$03	
263C 47		LD	B, A	
263D C5		PUSH	BC	
263E CD 4E 26		CALL	CMPARG	
2641 DA F7 1B	L2641:	JP	C, CMPERR	
2644 79		LD	A, C	
2645 C6 CD		ADD	A, \$CD	\$CD + antal
2647 12		LD	(DE), A	
2648 13		INC	DE	
2649 C1		POP	BC	
264A 79		LD	A, C	
264B 12		LD	(DE), A	2*kod
264C 13		INC	DE	
264D C9		RET		

CMPARG: kompilera argumentlistan "(a,b,...)". IX pekar på resultat-byten i en typlista (**FNTYPE**); för varje argument flyttas IX ett steg och argumentet omvandlas till den typen. En byte med bit 7 betyder att det inte får komma fler argument (fel 13), och listan får bara sluta där bit 6 är satt (annars fel 13). Ut: C = antal argument.

264E 0E 00	CMPARG:	LD	C, \$00
2650 3E 28		LD	A, \$28
2652 BE		CP	(HL)

2653	20 2A	JR	NZ, L267F	"("?
2655	DF	L2655:	RST \$18	över (eller ,
2656	DD 23	INC	IX	nästa typbyte
2658	0C	INC	C	
2659	C5	PUSH	BC	
265A	DD E5	PUSH	IX	
265C	CD 01 27	CALL	CMPEXP	
265F	DD E1	POP	IX	
2661	38 DE	JR	C, L2641	
2663	DD 7E 00	LD	A, (IX+\$00)	argumentets typ
2666	4F	LD	C, A	
2667	A7	AND	A	
2668	3E 8D	LD	A, \$8D	bit 7: för många argument, fel 13
266A	FA F7 1B	JP	M, CMPERR	
266D	79	LD	A, C	
266E	CD 51 25	CALL	TYPENV	
2671	C1	POP	BC	
2672	D8	RET	C	
2673	7E	LD	A, (HL)	
2674	FE 2C	CP	\$2C	","
2676	28 DD	JR	Z, L2655	
2678	FE 29	CP	\$29	")"
267A	3E 92	LD	A, \$92	
267C	37	SCF		
267D	C0	RET	NZ	
267E	DF	RST	\$18	över)
267F	DD 7E 00	L267F:	LD A, (IX+\$00)	
2682	E6 40	AND	\$40	får listan sluta här?
2684	C0	RET	NZ	
2685	3E 8D	LD	A, \$8D	fel 13
2687	37	SCF		
2688	C9	RET		

CMPNUM: siffror följda av "%" är ett heltal: \$C7 lo hi (fel 12 om talet är större än 65535; 65535% = -1). Blanka mellan siffrorna hoppas över: "1 2%" = 12, "1 2 3" = 123, och LIST skriver dem ihop. "1.5%" ger fel 11 (punkten gör det till ett flyttal, och "%" blir kvar). Annars ett flyttal: \$C3 och 5 bytes från GETFLT (fel 12 om talet är fel).

2689	E5	CMPNUM:	PUSH HL	
268A	2B		DEC HL	
268B	DF	L268B:	RST \$18	blanka hoppas över
268C	FE 25	CP	\$25	"%" efter siffrorna?
268E	28 1E	JR	Z, L26AE	
2690	FE 30	CP	\$30	
2692	38 04	JR	C, L2698	
2694	FE 3A	CP	\$3A	
2696	38 F3	JR	C, L268B	
2698	E1	L2698:	POP HL	inte %: flyttal
2699	EB	EX	DE, HL	
269A	36 C3	LD	(HL), \$C3	\$C3 flyttal
269C	01 05 00	LD	BC, \$0005	
269F	09	ADD	HL, BC	
26A0	E5	PUSH	HL	
26A1	CD 13 15	CALL	GETFLT	läs talet
26A4	EB	EX	DE, HL	
26A5	2B	DEC	HL	
26A6	D1	POP	DE	
26A7	13	INC	DE	
26A8	38 0C	JR	C, L26B6	
26AA	E7	RST	\$20	
26AB	06 00	LD	B, \$00	
26AD	C9	RET		
26AE	E1	L26AE:	POP HL	
26AF	D5	PUSH	DE	
26B0	CD 1E 18	CALL	GETLN	heltalet
26B3	42	LD	B, D	
26B4	4B	LD	C, E	
26B5	D1	POP	DE	
26B6	3E 8C	L26B6:	LD A, \$8C	fel 12

26B8	D8	RET	C	
26B9	EB	EX	DE,HL	
26BA	36 C7	LD	(HL), \$C7	\$C7 heltal
26BC	CD 28 10	CALL	WRBC1	
26BF	EB	EX	DE,HL	
26C0	7E	LD	A, (HL)	
26C1	FE 25	CP	\$25	% efter heltalet
26C3	37	SCF		
26C4	20 F0	JR	NZ, L26B6	
26C6	06 01	LD	B, \$01	typ heltal
26C8	C3 BE 25	JP	SKIPSP	

3.65 VARNAM: läs ett variabelnamn

\$26CB

Namnet är en bokstav (A-Z eller ÄÖÅ, små görs till stora), sedan en siffra om det finns, och sedan ""%" (heltal) eller "\$" (sträng).

Ut: B = bokstaven, C = namnbyte (siffra i bit 4-7, \$F0 utan siffra, typ i bit 0-1). CY + A = fel 11 om det inte börjar med en bokstav. Eftersom mellanslag hoppas över är "A 1" samma som "A1".

26CB	7E	VARNAME: LD	A, (HL)	
26CC	E6 DF	AND	\$DF	små bokstäver -> stora
26CE	47	LD	B, A	
26CF	FE 41	CP	\$41	
26D1	0E 8B	LD	C, \$8B	fel 11
26D3	79	LD	A, C	
26D4	D8	RET	C	
26D5	78	LD	A, B	
26D6	FE 5E	CP	\$5E	
26D8	3F	CCF		
26D9	79	LD	A, C	
26DA	D8	RET	C	
26DB	DF	RST	\$18	över bokstaven
26DC	0E F0	LD	C, \$F0	ingen siffra
26DE	7E	LD	A, (HL)	
26DF	D6 30	SUB	\$30	
26E1	38 0A	JR	C, L26ED	
26E3	FE 0A	CP	\$0A	
26E5	30 06	JR	NC, L26ED	
26E7	87	ADD	A, A	siffran i bit 4-7
26E8	87	ADD	A, A	
26E9	87	ADD	A, A	
26EA	87	ADD	A, A	
26EB	4F	LD	C, A	
26EC	DF	RST	\$18	över siffran
26ED	7E	L26ED: LD	A, (HL)	
26EE	FE 24	CP	\$24	""\$"?
26F0	28 0B	JR	Z, L26FD	
26F2	D6 25	SUB	\$25	""%""?
26F4	A7	AND	A	
26F5	C0	RET	NZ	
26F6	3E 01	LD	A, \$01	
26F8	B1	L26F8: OR	C	
26F9	4F	LD	C, A	
26FA	C3 BE 25	JP	SKIPSP	
26FD	3E 02	L26FD: LD	A, \$02	
26FF	18 F7	JR	L26F8	

3.66 CMPEXP: kompilera ett uttryck till omvänd polsk notation

\$2701

In: HL = texten, DE = var koden ska skrivas.

Ut: B = uttryckets typ, CY + A = felkod.

Kompilatorn går rekursivt genom prioritetsnivåerna **PREC1-PREC10**. **PRECLV** är den gemensamma rutinen för en nivå med binära operatorer:

1. vänster operand = nästa nivå (**NXTLEV**)
2. finns en operator ur nivåns tabell (**KWSRCH**)? nej: klart
3. höger operand = nästa nivå
4. typerna kontrolleras efter klassen, och operators token skrivs efter båda operanderna; tillbaka till 2.

Klasser:

- 1 aritmetik: typerna görs lika (**OPTYPE**), token = bas + typ.
Strängar bara för "+" (sammanfogning, \$F6).
- 2 jämförelse: typerna görs lika, token = bas + typ, resultat heltal
- 3 logisk: båda operanderna görs till heltal, resultat heltal
- 4 "^": flyttal^heltal får egen token (\$FF), annars som klass 1

Ett tal utan "%" är ett flyttal, så $7\%/2 = 3.5$ men $7\%/2\% = 3$, och $2^3\% = 8$ exakt men $2^3 = 8.00005$ (via flyttal); $(-2)^3$ fel 4. Heltalen slår runt utan fel: $32767\%+1\% = -32768$.

Om en operand måste omvandlas och det är den vänstra, skjuts omvandlingstoken in direkt efter den (**INSCNV**); den högra står sist och får token efter sig.

Stacken under steg 3 (IX): +3 = token, +4 = klass, +5 = vänster typ, +6/7 = var den högra operandens kod börjar.

2701	DD 21 B5 23	CMPEXP:	LD IX,PREC1	
2705	DD E5	PRECLV:	PUSH IX	
2707	CD E8 27		CALL NXTLEV	vänster operand
270A	DD E1		POP IX	
270C	D8		RET C	
270D	D5	L270D:	PUSH DE	
270E	DD 5E 04		LD E,(IX+\$04)	
2711	DD 56 05		LD D,(IX+\$05)	
2714	CD F5 09		CALL KWSRCH	operator?
2717	D1		POP DE	
2718	C0		RET NZ	
2719	DD 4E 06		LD C,(IX+\$06)	C = klass
271C	D5		PUSH DE	
271D	C5		PUSH BC	
271E	F5		PUSH AF	
271F	DD E5		PUSH IX	
2721	CD E8 27		CALL NXTLEV	höger operand
2724	38 5B		JR C,L2781	
2726	DD 21 00 00		LD IX,\$0000	IX = stacken
272A	DD 39		ADD IX,SP	
272C	DD 7E 03		LD A,(IX+\$03)	
272F	FE F4		CP \$F4	"+" (tillåter strängar)
2731	20 0F		JR NZ,L2742	
2733	CD A0 27		CALL OPTYPE	
2736	38 49	L2736:	JR C,L2781	
2738	79	L2738:	LD A,C	
2739	12		LD (DE),A	
273A	13		INC DE	
273B	DD E1		POP IX	
273D	F1		POP AF	
273E	F1		POP AF	
273F	F1		POP AF	
2740	18 CB		JR L270D	
2742	DD 35 04	L2742:	DEC (IX+\$04)	klass 1
2745	20 0B		JR NZ,L2752	
2747	78	L2747:	LD A,B	
2748	FE 02		CP \$02	sträng?
274A	3E 91		LD A,\$91	
274C	37		SCF	
274D	C4 A0 27		CALL NZ,OPTYPE	
2750	18 E4		JR L2736	

2752	DD 35 04	L2752:	DEC	(IX+\$04)	klass 2
2755	20 07		JR	NZ, L275E	
2757	CD A0 27		CALL	OPTYPE	
275A	06 01		LD	B, \$01	resultat heltal
275C	18 D8		JR	L2736	
275E	DD 35 04	L275E:	DEC	(IX+\$04)	klass 3
2761	20 24		JR	NZ, L2787	
2763	CD 61 25		CALL	TOINT	höger till heltal
2766	38 19		JR	C, L2781	
2768	DD 7E 05		LD	A, (IX+\$05)	vänster typ
276B	FE 01		CP	\$01	
276D	28 0D		JR	Z, L277C	
276F	FE 00		CP	\$00	
2771	3E 91		LD	A, \$91	
2773	20 0C		JR	NZ, L2781	
2775	3E D9		LD	A, \$D9	vänster till heltal
2777	CD CF 27		CALL	INSCNV	
277A	06 01		LD	B, \$01	
277C	DD 4E 03	L277C:	LD	C, (IX+\$03)	
277F	18 B7		JR	L2738	
2781	C1	L2781:	POP	BC	
2782	C1		POP	BC	
2783	C1		POP	BC	
2784	C1		POP	BC	
2785	37		SCF		
2786	C9		RET		
2787	DD 35 04	L2787:	DEC	(IX+\$04)	klass 4

Klass 5 och högre finns inte; här skulle datorn stå still.

278A	20 FE	L278A:	JR	NZ, L278A	
278C	78		LD	A, B	höger heltal och vänster flyttal?
278D	FE 01		CP	\$01	
278F	20 B6		JR	NZ, L2747	
2791	DD 7E 05		LD	A, (IX+\$05)	
2794	FE 00		CP	\$00	
2796	20 AF		JR	NZ, L2747	
2798	DD 4E 03		LD	C, (IX+\$03)	\$FF: flyttal^heltal
279B	0C		INC	C	
279C	0C		INC	C	
279D	47		LD	B, A	
279E	18 96		JR	L2736	

OPTYPE: gör typerna lika. B = höger typ, IX+5 = vänster typ. Heltal och flyttal blir flyttal. Ut: C = token + typ, B = typen. Fel 17 om en sträng blandas med ett tal.

27A0	78	OPTYPE:	LD	A, B	
27A1	DD BE 05		CP	(IX+\$05)	samma typ?
27A4	28 11		JR	Z, L27B7	
27A6	FE 00		CP	\$00	
27A8	28 13		JR	Z, L27BD	
27AA	FE 01		CP	\$01	
27AC	20 14		JR	NZ, L27C2	
27AE	DD 7E 05		LD	A, (IX+\$05)	
27B1	20 0F		JR	NZ, L27C2	
27B3	CD 6E 25		CALL	TOFLT	höger till flyttal
27B6	D8		RET	C	
27B7	78	L27B7:	LD	A, B	
27B8	DD 86 03		ADD	A, (IX+\$03)	token + typ
27BB	4F		LD	C, A	
27BC	C9		RET		
27BD	DD 7E 05	L27BD:	LD	A, (IX+\$05)	
27C0	FE 01		CP	\$01	
27C2	3E 91	L27C2:	LD	A, \$91	
27C4	37		SCF		
27C5	C0		RET	NZ	

27C6	3E DA	LD	A, \$DA	vänster till flyttal
27C8	CD CF 27	CALL	INSCNV	
27CB	06 00	LD	B, \$00	
27CD	18 E8	JR	L27B7	

INSCNV: skjut in byten A vid början av den högra operandens kod (IX+6/7), dvs. direkt efter den vänstra operanden. Koden efter flyttas en byte uppåt.

27CF	E5	INSCNV:	PUSH HL	
27D0	62	LD	H, D	
27D1	6B	LD	L, E	
27D2	DD 4E 06	LD	C, (IX+\$06)	
27D5	DD 46 07	LD	B, (IX+\$07)	
27D8	A7	AND	A	
27D9	ED 42	SBC	HL, BC	längden av högra operandens kod
27DB	44	LD	B, H	
27DC	4D	LD	C, L	
27DD	62	LD	H, D	
27DE	6B	LD	L, E	
27DF	2B	DEC	HL	
27E0	D5	PUSH	DE	
27E1	ED B8	LDDR		flytta upp en byte
27E3	12	LD	(DE), A	skjut in token
27E4	D1	POP	DE	
27E5	13	INC	DE	
27E6	E1	POP	HL	
27E7	C9	RET		

NXTLEV: anropa nästa nivå. IX = aktuell post; IX sätts till posten för nästa nivå, och dess rutin anropas.

27E8	E5	NXTLEV:	PUSH HL
27E9	DD 6E 02	LD	L, (IX+\$02)
27EC	DD 66 03	LD	H, (IX+\$03)
27EF	E3	EX	(SP), HL
27F0	DD E1	POP	IX
27F2	E5	PUSH	HL
27F3	DD 6E 00	LD	L, (IX+\$00)
27F6	DD 66 01	LD	H, (IX+\$01)
27F9	E3	EX	(SP), HL
27FA	C9	RET	

CMPPADD: nivån för + och -. Ett "-" först (unärt) gäller hela den följande termen: $-A*B = -(A*B)$. Koden blir termen följd av \$D7 (flyttal) eller \$D8 (heltal). Ett "+" först hoppas över. Sedan fortsätter det som en vanlig binär nivå.

Ett unärt tecken går bara först på nivån: efter * / ^ + - och efter ett annat unärt tecken är nästa nivå **PREC8/PRIMARY**, och det ger fel 11 (2^{-3} , $2+^{-3}$, 2^{-1} , $-^{-2}$; skriv $2^{*}(-3)$). Efter "(", "=", NOT, AND ... går det ($1=-1$).

27FB	7E	CMPPADD:	LD	A, (HL)	
27FC	FE 2D	CP	\$2D	unärt minus?	
27FE	28 08	JR	Z, L2808		
2800	FE 2B	CP	\$2B	unärt plus hoppas över	
2802	CC BE 25	CALL	Z, SKIPSP		
2805	C3 05 27	JP	PRECLV		
2808	DF	L2808:	RST	\$18	över -
2809	DD E5	PUSH	IX		
280B	CD E8 27	CALL	NXTLEV	termen	
280E	DD E1	POP	IX		
2810	D8	RET	C		
2811	3E 01	LD	A, \$01	heltal?	
2813	B8	CP	B		
2814	3E D8	LD	A, \$D8	\$D8: heltal	
2816	28 09	JR	Z, L2821		
2818	3E 00	LD	A, \$00		
281A	B8	CP	B		
281B	3E 91	LD	A, \$91	sträng: fel 17	
281D	37	SCF			
281E	C0	RET	NZ		
281F	3E D7	LD	A, \$D7	\$D7: flyttal	

```

2821 12      L2821: LD  (DE),A
2822 13      INC  DE
2823 C3 0D 27      JP  L270D

```

CMPNOT: "NOT" följt av nästa nivå (jämförelser), omvandlat till heltal, och \$E1. Utan NOT bara nästa nivå. NOT binder alltså svagare än jämförelser: NOT A=B = NOT (A=B), NOT 0+1 = -2. Efter NOT kommer **PREC6**, så NOT NOT 5 och 1+NOT 0 ger fel 11.

```

2826 D5      CMPNOT: PUSH DE
2827 DD 5E 04      LD  E,(IX+$04)
282A DD 56 05      LD  D,(IX+$05)
282D CD F5 09      CALL KWSRCH      "NOT"?
2830 D1          POP  DE
2831 C2 E8 27      JP  NZ,NXTLEV      inget NOT
2834 CD E8 27      CALL NXTLEV      PREC6: inget NOT till
2837 D8          RET  C
2838 CD 61 25      CALL TOINT      till heltal
283B D8          RET  C
283C 3E E1        LD  A,$E1      $E1
283E 12          LD  (DE),A
283F 13          INC  DE
2840 C9          RET

```

3.67 EXEXP: utför kompilerad uttryckskod (RST \$38)

\$2841

DE pekar på koden. Varje token \$B6-\$FF hämtas och rutinen tas ur **EXPTAB1** ($2 * (\text{token} - \$B6)$). Rutinerna arbetar med värden på Z80-stacken: ett heltal är 2 bytes, ett flyttal 5 bytes, och de flesta slutar med RST \$30 (hämta nästa token) eller RST \$28 (lägg HL på stacken och hämta nästa token). En sluttoken (**EXEND**) avslutar. **EXEXPH** (\$2841, RST \$28): PUSH HL, sedan som **EXEXP**.

```

2841 E5      EXEXPH: PUSH HL

```

3.68 Värdestacken när uttryckskoden körs

\$2842

DE pekar på koden och flyttas fram av varje rutin. Värdena ligger på Z80-stacken:

```

heltal  2 bytes
flyttal 5 bytes: SP+0..2 mantissa (BCD, mest signifikanta byten
             först), SP+3 tecken (0 = +, 1 = -), SP+4 exponent
             (0 = talet är noll). Flyttalsrutinerna får adressen till
             exponentbyten (den högsta), och en flyttalsvariabels
             adress pekar också på exponenten.
sträng  tre ord, överst först:
             [n = antal tillfälliga bytes] [adress] [längd]
             följda av n bytes. För en variabel är n = 0; en ny
             sträng (t.ex. från CHR$) ligger i de n byten, direkt
             under beskrivningen.
variabeladress 2 bytes (för tilldelning).

```

När en rutin startar är CY = 0 och B = 0 (**SUBI** och **EXFUNC** räknar med det). Typen i B vid slutet: 0 = flyttal, 1 = heltal, 2 = sträng.

```

2842 1A      EXEXP: LD  A,(DE)      token
2843 13      INC  DE
2844 D6 B6      SUB  $B6      index i EXPTAB1
2846 21 8B 28      LD  HL,EXPTAB1
2849 4F          LD  C,A
284A 06 00        LD  B,$00
284C 09          ADD  HL,BC      2*index (B = 0)
284D 09      L284D: ADD  HL,BC
284E 7E      L284E: LD  A,(HL)      läs rutinens adress
284F 23          INC  HL
2850 66          LD  H,(HL)
2851 6F          LD  L,A
2852 E9      JPHL: JP  (HL)

```

EXFUNC: \$CD-\$D1, nästa byte = $2 * \text{kod}$, rutinen tas ur **EXPTAB2**.

2853	21 1F 29	EXFUNC:	LD	HL, EXPTAB2	
2856	1A		LD	A, (DE)	2*kod
2857	13		INC	DE	
2858	4F		LD	C, A	
2859	18 F2		JR	L284D	
285B	E1	EXEND:	POP	HL	heltalet
285C	06 01		LD	B, \$01	
285E	C9		RET		

ENDF (\$B9): slut, flyttal. Flyttalet ska bli kvar på stacken, så returadressen under det flyttas upp: de 9 byten (flyttalet och två sparade ord) flyttas 2 bytes uppåt, och returadressen hamnar överst.

285F	21 05 00	ENDF:	LD	HL, \$0005	returadressen, under flyttalet
2862	39		ADD	HL, SP	
2863	4E		LD	C, (HL)	
2864	23		INC	HL	
2865	46		LD	B, (HL)	
2866	C5		PUSH	BC	sparas också i de 9 byten
2867	D5		PUSH	DE	
2868	EB		EX	DE, HL	
2869	21 FE FF		LD	HL, \$FFFE	
286C	19		ADD	HL, DE	
286D	01 09 00		LD	BC, \$0009	flyttal + 2 ord
2870	ED B8		LDDR		flytta 2 bytes uppåt
2872	D1		POP	DE	
2873	D1		POP	DE	
2874	06 00	L2874:	LD	B, \$00	B = 0: flyttal
2876	C9		RET		

ENDS (\$BB): slut, sträng. Returadressen under strängen räknas in bland de tillfälliga byten ($n + 2$), och det hoppas till den med JP (HL). Strängen blir kvar på stacken.

2877	E1	ENDS:	POP	HL	n
2878	23		INC	HL	returadressen räknas som tillfällig
2879	23		INC	HL	
287A	E5		PUSH	HL	
287B	39		ADD	HL, SP	HL = returadressens plats
287C	23		INC	HL	
287D	23		INC	HL	
287E	23		INC	HL	
287F	23		INC	HL	
2880	06 02		LD	B, \$02	B = 2, hoppa dit
2882	18 CA		JR	L284E	

ENDVF (\$BC): HL = variabelns adress, B = 0 (flyttal).

2884	E1	ENDVF:	POP	HL
2885	18 ED		JR	L2874

ENDVS (\$BE): HL = variabelns adress, B = 2 (sträng).

2887	E1	ENDVS:	POP	HL
2888	06 02		LD	B, \$02
288A	C9		RET	

3.69 EXPTAB1: rutiner för token \$B6-\$FF vid körning

\$288B

\$B6-\$B8	tilldelning flyttal/heltal/sträng	ASGNF ASGNI ASGNS
\$B9-\$BB	slut, värde av typ token-\$B9: flyttal och sträng blir kvar på stacken, heltal i HL	\$285F EXEND \$2877
\$BC-\$BE	slut, HL = variabelns adress och B = typ token-\$BC (efter CMPVR) \$BD avslutar också IF-villkoret	\$2884 EXEND \$2887
\$BF	slut på DIM-listan	RET
\$C0-\$C2	variabel utan index	VAR0
\$C3	flyttalskonstant	CONSTF
\$C4-\$C6	variabel med ett index	VAR1F VAR1I VAR1S
\$C7	heltalskonstant	CONSTI
\$C8-\$CA	variabel med två index	VAR2F VAR2I VAR2S
\$CB	strängkonstant	CONSTS
\$CC	DIM-post	\$1BFE
\$CD-\$D1	funktion, 0-4 argument	EXFUNC
\$D2	DIM av sträng utan längd: värdet 0	\$1C7F
\$D3	parentes: gör inget	EXEXP
\$D4-\$D6	hämta variabelns värde	GETF GETI GETS
\$D7 \$D8	unärt minus	NEGF NEGI
\$D9 \$DA	flyttal -> heltal, heltal -> flyttal	FTOI ITOF
\$DB	FN-argumentpost: hoppas över	\$1B85
\$DC-\$E1	EQV IMP OR XOR AND NOT	OPEQV ...
\$E2-\$F3	= <> < >= > <=, tre typer var	EQF EQI EQS ...
\$F4-\$FC	+ - * /, \$F6 = sammanfogning	ADDF ... CONCAT
\$FD-\$FF	^ flyttal, heltal, flyttal^heltal	POWF POWI POWFI

Namnen i tabellen är satta efter vilken token de hör till.

EXPTAB1:

288B	89 29	DEFW ASGNF
288D	B4 29	DEFW ASGNI
288F	C3 29	DEFW ASGNS
2891	5F 28	DEFW ENDF
2893	5B 28	DEFW EXEND
2895	77 28	DEFW ENDS
2897	84 28	DEFW ENDVF
2899	5B 28	DEFW EXEND
289B	87 28	DEFW ENDVS
289D	7E 1C	DEFW ENDDIM
289F	CD 2C	DEFW VAR0
28A1	CD 2C	DEFW VAR0
28A3	CD 2C	DEFW VAR0
28A5	36 2A	DEFW CONSTF
28A7	D9 2C	DEFW VAR1F
28A9	DF 2C	DEFW VAR1I
28AB	E4 2C	DEFW VAR1S
28AD	E3 2A	DEFW CONSTI
28AF	2B 2D	DEFW VAR2F
28B1	31 2D	DEFW VAR2I
28B3	36 2D	DEFW VAR2S
28B5	D7 2E	DEFW CONSTS
28B7	FE 1B	DEFW DIMELM
28B9	53 28	DEFW EXFUNC
28BB	53 28	DEFW EXFUNC
28BD	53 28	DEFW EXFUNC
28BF	53 28	DEFW EXFUNC
28C1	53 28	DEFW EXFUNC
28C3	7F 1C	DEFW PUSH0
28C5	42 28	DEFW EXEXP
28C7	A1 29	DEFW GETF
28C9	BD 29	DEFW GETI
28CB	22 2A	DEFW GETS
28CD	43 2A	DEFW NEGF
28CF	E9 2A	DEFW NEGI
28D1	E2 2D	DEFW FTOI
28D3	F0 2D	DEFW ITOF
28D5	85 1B	DEFW SKPARG

28D7	F5 2A	DEFW OPEQV
28D9	00 2B	DEFW OPIMP
28DB	0B 2B	DEFW OPOR
28DD	21 2B	DEFW OPXOR
28DF	14 2B	DEFW OPAND
28E1	1D 2B	DEFW OPNOT
28E3	4C 2A	DEFW EQF
28E5	2A 2B	DEFW EQI
28E7	E4 2F	DEFW EQS
28E9	55 2A	DEFW NEF
28EB	34 2B	DEFW NEI
28ED	ED 2F	DEFW NES
28EF	70 2A	DEFW LTF
28F1	3E 2B	DEFW LTI
28F3	08 30	DEFW LTS
28F5	75 2A	DEFW GEF
28F7	4A 2B	DEFW GEI
28F9	0D 30	DEFW GES
28FB	5E 2A	DEFW GTF
28FD	5A 2B	DEFW GTI
28FF	F6 2F	DEFW GTS
2901	67 2A	DEFW LEF
2903	56 2B	DEFW LEI
2905	FF 2F	DEFW LES
2907	B7 2A	DEFW ADDF
2909	5E 2B	DEFW ADDI
290B	47 2F	DEFW CONCAT
290D	BD 2A	DEFW SUBF
290F	62 2B	DEFW SUBI
2911	C3 2A	DEFW MULF
2913	67 2B	DEFW MULI
2915	C9 2A	DEFW DIVF
2917	73 2B	DEFW DIVI
2919	00 2E	DEFW POWF
291B	18 2E	DEFW POWI
291D	5E 2E	DEFW POWFI

3.70 EXPTAB2: funktionerna, index = kod (token i FNKW - \$80)

\$291F

0 FN	1 ABS	2 ATN	3 COS	4 EXP	5 FIX
6 INT	7 LOG	8 LOG10	9 PI	A RND	B SGN
C SIN	D SQR	E TAN	F ASC	10 CHR\$	11 LEFT\$
12 RIGHT\$	13 MID\$	14 LEN	15 INSTR	16 SPACE\$	17 STRING\$
18 NUM\$	19 VAL	1A SWAP%	1B PEEK	1C INP	1D CALL
1E ERRCODE	1F IEC\$	20 TAB	21 CUR	23 DOT	
30 ADD\$	31 SUB\$	32 MUL\$	33 DIV\$	34 COMP%	

Kod \$22 och \$24-\$2F har inget namn i FNKW och går till fel 8, FINNS EJ I DETTA SYSTEM (platser för fler funktioner (?)). IEC\$ (\$016B) ligger bland ingångarna till tilläggs-ROM:en.

EXPTAB2:

291F	2A 1B	DEFW FN
2921	7F 2D	DEFW EXABS
2923	AF 2D	DEFW EXATN
2925	A5 2D	DEFW EXCOS
2927	B4 2D	DEFW EXEXP1
2929	96 2D	DEFW EXFIX
292B	9B 2D	DEFW EXINT
292D	B9 2D	DEFW EXLOG
292F	BE 2D	DEFW EXLG10
2931	6E 2D	DEFW EXPI
2933	E0 31	DEFW EXRND
2935	86 2D	DEFW EXSGN
2937	A0 2D	DEFW EXSIN
2939	C3 2D	DEFW EXSQR
293B	AA 2D	DEFW EXTAN
293D	EC 2E	DEFW EXASC
293F	C1 2F	DEFW EXCHR

2941	07 2F	DEFW EXLEFT
2943	1C 2F	DEFW EXRGHT
2945	FD 2E	DEFW EXMID
2947	E3 2E	DEFW EXLEN
2949	75 30	DEFW EXINST
294B	F7 30	DEFW EXSPC
294D	FB 30	DEFW EXSTRG
294F	52 31	DEFW EXNUM
2951	72 31	DEFW EXVAL
2953	79 2B	DEFW EXSWAP
2955	7E 2B	DEFW EXPEEK
2957	C1 31	DEFW EXINP
2959	CC 31	DEFW EXCALL
295B	4F 20	DEFW EXERR
295D	6B 01	DEFW EXIEC
295F	26 31	DEFW EXTAB
2961	3B 31	DEFW EXCUR
2963	09 07	DEFW L0709
2965	D6 22	DEFW EXDOT
2967	09 07	DEFW L0709
2969	09 07	DEFW L0709
296B	09 07	DEFW L0709
296D	09 07	DEFW L0709
296F	09 07	DEFW L0709
2971	09 07	DEFW L0709
2973	09 07	DEFW L0709
2975	09 07	DEFW L0709
2977	09 07	DEFW L0709
2979	09 07	DEFW L0709
297B	09 07	DEFW L0709
297D	09 07	DEFW L0709
297F	9F 2B	DEFW EXADD
2981	A4 2B	DEFW EXSUB
2983	A9 2B	DEFW EXMUL
2985	AE 2B	DEFW EXDIV
2987	B3 2B	DEFW EXCOMP

3.71 ASGNF / ASGNI / ASGNS: tilldelning (\$B6-\$B8)

\$2989

Stacken: [värde] [variabelns adress]. Värdet skrivs till variabeln, båda tas bort, och RET går direkt till den som anropade RST \$38. Ut: HL = variabelns adress, B = typ.

ASGNS: variabelns adress ligger under strängens tillfälliga bytes.

- Tom sträng: bara längden (+4/5) sätts till 0.
- Är strängen inte reserverad (+0/1 = 0) reserveras 80 bytes, eller strängens längd om den är större (**DIMSTR**).
- Annars fel 10, TEXTEN FÅR EJ PLATS I STRÄNGEN, om texten är längre än den reserverade längden.
- Fel 10 även om variabelns post ligger ovanför SP, dvs. på stacken (?).

2989	21 06 00	ASGNF:	LD HL,\$0006	variabelns adress (SP+5/6)
298C	39		ADD HL,SP	
298D	D5		PUSH DE	koden
298E	56		LD D,(HL)	
298F	2B		DEC HL	
2990	5E		LD E,(HL)	
2991	2B		DEC HL	
2992	D5		PUSH DE	
2993	01 05 00		LD BC,\$0005	5 bytes, bakifrån
2996	ED B8		LDDR	
2998	E1		POP HL	HL = variabeln
2999	D1		POP DE	
299A	C1		POP BC	ta bort 7 bytes
299B	C1		POP BC	
299C	C1		POP BC	
299D	33		INC SP	

299E 06 00 LD B,\$00
29A0 C9 RET

GETF (\$D4): ersätt adressen på stacken med flyttalet (5 bytes).

29A1 C1 **GETF:** POP BC adressen
29A2 21 FB FF LD HL,\$FFFB plats för flyttalet
29A5 39 ADD HL,SP
29A6 F9 LD SP,HL
29A7 D5 PUSH DE
29A8 50 LD D,B
29A9 59 LD E,C
29AA 01 04 00 LD BC,\$0004
29AD 09 ADD HL,BC
29AE EB EX DE,HL
29AF 03 INC BC
29B0 ED B8 LDDR kopiera 5 bytes
29B2 D1 POP DE
29B3 F7 RST \$30

29B4 C1 **ASGNI:** POP BC värdet
29B5 E1 POP HL adressen
29B6 71 LD (HL),C
29B7 23 INC HL
29B8 70 LD (HL),B
29B9 2B DEC HL
29BA 06 01 LD B,\$01
29BC C9 RET

GETI (\$D5): ersätt adressen med heltalet.

29BD E1 **GETI:** POP HL
29BE 7E LD A,(HL)
29BF 23 INC HL
29C0 66 LD H,(HL)
29C1 6F LD L,A
29C2 EF RST \$28

29C3 DD E1 **ASGNS:** POP IX n
29C5 E1 POP HL textens adress
29C6 C1 POP BC längd
29C7 DD 39 ADD IX,SP IX = platsen för variabelns adress
29C9 79 LD A,C
29CA B0 OR B
29CB 28 46 JR Z,L2A13 tom sträng
29CD DD E5 PUSH IX
29CF D5 PUSH DE
29D0 E5 PUSH HL
29D1 DD 6E 00 LD L,(IX+\$00) variabelns adress
29D4 DD 66 01 LD H,(IX+\$01)
29D7 E5 PUSH HL
29D8 ED 72 SBC HL,SP på stacken?
29DA D2 0D 07 JP NC,L070D fel 10
29DD DD E1 POP IX IX = variabelns post
29DF DD 6E 00 LD L,(IX+\$00) reserverad längd
29E2 DD 66 01 LD H,(IX+\$01)
29E5 7D LD A,L
29E6 B4 OR H
29E7 20 10 JR NZ,L29F9 reserverad
29E9 21 50 00 LD HL,\$0050 80 bytes
29EC 54 LD D,H
29ED 5D LD E,L
29EE ED 42 SBC HL,BC längden större?
29F0 30 02 JR NC,L29F4
29F2 50 LD D,B
29F3 59 LD E,C
29F4 CD 38 33 **L29F4:** CALL DIMSTR reservera DE bytes
29F7 18 05 JR L29FE

29F9 ED 42 **L29F9:** SBC HL,BC får plats?

29FB	DA 0D 07		JP	C, L070D	fel 10
29FE	DD 71 04	L29FE:	LD	(IX+\$04), C	längden
2A01	DD 70 05		LD	(IX+\$05), B	
2A04	DD 5E 02		LD	E, (IX+\$02)	textens plats
2A07	DD 56 03		LD	D, (IX+\$03)	
2A0A	E1		POP	HL	
2A0B	ED B0		LDIR		kopiera texten
2A0D	D1		POP	DE	
2A0E	E1		POP	HL	
2A0F	F9		LD	SP, HL	ta bort strängen
2A10	E1		POP	HL	HL = variabeln
2A11	18 0C		JR	L2A1F	
2A13	DD F9	L2A13:	LD	SP, IX	ta bort strängen
2A15	E1		POP	HL	
2A16	E5		PUSH	HL	
2A17	DD E1		POP	IX	
2A19	DD 71 04		LD	(IX+\$04), C	längd 0
2A1C	DD 70 05		LD	(IX+\$05), B	
2A1F	06 02	L2A1F:	LD	B, \$02	B = 2: sträng
2A21	C9		RET		

GETS (\$D6): ersätt adressen med en strängbeskrivning [0] [adress] [längd] som pekar på variabelns text.

2A22	DD E1	GETS:	POP	IX	variabelns post
2A24	DD 4E 04		LD	C, (IX+\$04)	längd
2A27	DD 46 05		LD	B, (IX+\$05)	
2A2A	DD 6E 02		LD	L, (IX+\$02)	adress
2A2D	DD 66 03		LD	H, (IX+\$03)	
2A30	C5		PUSH	BC	
2A31	E5		PUSH	HL	
2A32	21 00 00		LD	HL, \$0000	n = 0
2A35	EF		RST	\$28	

CONSTF (\$C3): kopiera flyttalet (5 bytes) ur koden till stacken.

2A36	21 FB FF	CONSTF:	LD	HL, \$FFFB	plats för 5 bytes
2A39	39		ADD	HL, SP	
2A3A	F9		LD	SP, HL	
2A3B	EB		EX	DE, HL	
2A3C	01 05 00		LD	BC, \$0005	
2A3F	ED B0		LDIR		
2A41	EB		EX	DE, HL	
2A42	F7		RST	\$30	

NEGF (\$D7): byt tecken på flyttalet (tecknet i SP+3).

2A43	21 03 00	NEGF:	LD	HL, \$0003	tecknet
2A46	39		ADD	HL, SP	
2A47	3E 01		LD	A, \$01	
2A49	AE		XOR	(HL)	
2A4A	77		LD	(HL), A	
2A4B	F7		RST	\$30	

3.72 Jämförelse av flyttal, \$E2-\$F3

\$2A4C

FCMPST tar bort båda flyttalen och ger flaggorna: Z = lika, CY = vänster < höger. Resultatet är ett heltal: -1 (sant) eller 0.

2A4C	CD 7A 2A	EQF:	CALL FCMPST	
2A4F	28 09		JR Z, L2A5A	lika: sant
2A51	21 00 00	L2A51:	LD HL, \$0000	falskt
2A54	EF		RST \$28	
2A55	CD 7A 2A	NEF:	CALL FCMPST	
2A58	28 F7		JR Z, L2A51	
2A5A	21 FF FF	L2A5A:	LD HL, \$FFFF	sant
2A5D	EF		RST \$28	
2A5E	CD 7A 2A	GTF:	CALL FCMPST	
2A61	28 EE		JR Z, L2A51	
2A63	38 EC	L2A63:	JR C, L2A51	mindre: falskt
2A65	18 F3		JR L2A5A	
2A67	CD 7A 2A	LEF:	CALL FCMPST	
2A6A	28 EE		JR Z, L2A5A	
2A6C	38 EC	L2A6C:	JR C, L2A5A	mindre: sant
2A6E	18 E1		JR L2A51	
2A70	CD 7A 2A	LTF:	CALL FCMPST	
2A73	18 F7		JR L2A6C	
2A75	CD 7A 2A	GEF:	CALL FCMPST	
2A78	18 E9		JR L2A63	

FCMPST: jämför de två flyttalen överst på stacken (**FCMP**) och ta bort dem. Returadressen tas först bort och hoppet tillbaka görs med **JP (IX)**. Flaggorna sparas över **SP**-justeringen med **RLA/RRA**.

2A7A	DD E1	FCMPST:	POP IX	returadressen
2A7C	21 04 00		LD HL, \$0004	höger operand
2A7F	39		ADD HL, SP	
2A80	D5		PUSH DE	
2A81	11 05 00		LD DE, \$0005	
2A84	EB		EX DE, HL	
2A85	19		ADD HL, DE	vänster operand
2A86	CD 93 2A		CALL FCMP	
2A89	17		RLA	spara CY
2A8A	D1		POP DE	
2A8B	21 0A 00		LD HL, \$000A	ta bort båda
2A8E	39		ADD HL, SP	
2A8F	F9		LD SP, HL	
2A90	1F		RRA	
2A91	DD E9	JPIX:	JP (IX)	

3.73 FCMP: jämför flyttal

\$2A93

In: HL och DE pekar på exponenterna för a och b.

Ut: Z om a = b, annars CY om a < b.

Båda exponenterna 0 räknas som lika. Olika tecken: tecknet avgör. Samma tecken: exponenten, sedan mantissans 3 bytes från den mest signifikanta. För negativa tal byts a och b först.

Används också av **FOR/NEXT (FORTSF)**.

2A93	1A	FCMP:	LD A, (DE)	exponent b
2A94	47		LD B, A	
2A95	B6		OR (HL)	båda noll?
2A96	C8		RET Z	
2A97	4E		LD C, (HL)	exponent a
2A98	1B		DEC DE	
2A99	2B		DEC HL	
2A9A	1A		LD A, (DE)	tecken b
2A9B	BE		CP (HL)	olika tecken?

2A9C	C0		RET	NZ	
2A9D	B7		OR	A	positiva?
2A9E	79		LD	A,C	
2A9F	28 03		JR	Z, L2AA4	
2AA1	EB		EX	DE, HL	negativa: byt a och b
2AA2	78		LD	A,B	
2AA3	41		LD	B,C	
2AA4	B8	L2AA4:	CP	B	jämför exponenterna
2AA5	C0		RET	NZ	
2AA6	01 03 00		LD	BC, \$0003	till mantissan
2AA9	ED 42		SBC	HL, BC	
2AAB	EB		EX	DE, HL	
2AAC	ED 42		SBC	HL, BC	
2AAE	41		LD	B, C	
2AAF	1A	L2AAF:	LD	A, (DE)	3 bytes, mest signifikant först
2AB0	BE		CP	(HL)	
2AB1	C0		RET	NZ	
2AB2	23		INC	HL	
2AB3	13		INC	DE	
2AB4	10 F9		DJNZ	L2AAF	
2AB6	C9		RET		

3.74 ADDF / SUBF / MULF / DIVF

\$2AB7

IX = flyttalsrutinen. FARITH anropar den med DE = BC = vänster operands exponent och HL = höger operands. Resultatet hamnar i vänster operand, och höger (5 bytes) tas bort från stacken.

2AB7	DD 21 F7 33	ADDF:	LD	IX, FADD	
2ABB	18 10		JR	FARITH	
2ABD	DD 21 60 34	SUBF:	LD	IX, FSUB	
2AC1	18 0A		JR	FARITH	
2AC3	DD 21 DF 34	MULF:	LD	IX, FMUL	
2AC7	18 04		JR	FARITH	
2AC9	DD 21 EE 35	DIVF:	LD	IX, FDIV	
2ACD	21 04 00	FARITH:	LD	HL, \$0004	höger operand
2AD0	39		ADD	HL, SP	
2AD1	D5		PUSH	DE	
2AD2	EB		EX	DE, HL	
2AD3	21 05 00		LD	HL, \$0005	5 bytes att ta bort
2AD6	E5		PUSH	HL	
2AD7	19		ADD	HL, DE	
2AD8	EB		EX	DE, HL	
2AD9	42		LD	B, D	BC = DE = vänster
2ADA	4B		LD	C, E	
2ADB	CD 91 2A		CALL	JPIX	anropa rutinen (IX)
2ADE	E1		POP	HL	
2ADF	D1		POP	DE	
2AE0	39		ADD	HL, SP	ta bort höger operand
2AE1	F9		LD	SP, HL	
2AE2	F7		RST	\$30	

CONSTI (\$C7): lägg heltalet ur koden på stacken.

2AE3	EB	CONSTI:	EX	DE, HL	
2AE4	CD 1D 10		CALL	RDDE	heltalet ur koden
2AE7	EB		EX	DE, HL	
2AE8	EF		RST	\$28	

NEGI (\$D8): byt tecken på heltalet. Ingen kontroll av -32768: $-(-32768\%) = -32768$.

2AE9	E1	NEGI:	POP	HL	
2AEA	CD EE 2A		CALL	NEGHL	
2AED	EF		RST	\$28	

NEGHL: HL = -HL.

2AEE	AF	NEGHL:	XOR	A
2AEF	95		SUB	L
2AF0	6F		LD	L,A
2AF1	9F		SBC	A,A
2AF2	94		SUB	H
2AF3	67		LD	H,A
2AF4	C9		RET	

3.75 Logiska operatorer, \$DC-\$E1

\$2AF5

Bitvis på 16-bitars heltal (sant = -1, falskt = 0):

EQV: NOT (a XOR b) IMP: (NOT a) OR b OR, AND, XOR

NOT: lägger -1 på stacken och gör XOR.

2AF5	E1	OPEQV:	POP	HL	
2AF6	C1		POP	BC	
2AF7	78		LD	A,B	
2AF8	AC		XOR	H	
2AF9	2F		CPL		NOT (a XOR b)
2AFA	67		LD	H,A	
2AFB	79		LD	A,C	
2AFC	AD		XOR	L	
2AFD	2F		CPL		
2AFE	6F		LD	L,A	
2AFF	EF		RST	\$28	
2B00	E1	OPIMP:	POP	HL	
2B01	C1		POP	BC	
2B02	78		LD	A,B	
2B03	2F		CPL		NOT a OR b
2B04	B4		OR	H	
2B05	67		LD	H,A	
2B06	79		LD	A,C	
2B07	2F		CPL		
2B08	B5		OR	L	
2B09	6F		LD	L,A	
2B0A	EF		RST	\$28	
2B0B	E1	OPOR:	POP	HL	
2B0C	C1		POP	BC	
2B0D	79		LD	A,C	
2B0E	B5		OR	L	
2B0F	6F		LD	L,A	
2B10	78		LD	A,B	
2B11	B4		OR	H	
2B12	67		LD	H,A	
2B13	EF		RST	\$28	
2B14	E1	OPAND:	POP	HL	
2B15	C1		POP	BC	
2B16	79		LD	A,C	
2B17	A5		AND	L	
2B18	6F		LD	L,A	
2B19	78		LD	A,B	
2B1A	A4		AND	H	
2B1B	67		LD	H,A	
2B1C	EF		RST	\$28	
2B1D	21 FF FF	OPNOT:	LD	HL,\$FFFF	NOT x = x XOR -1
2B20	E5		PUSH	HL	
2B21	E1	OPXOR:	POP	HL	
2B22	C1		POP	BC	
2B23	79		LD	A,C	
2B24	AD		XOR	L	
2B25	6F		LD	L,A	
2B26	78		LD	A,B	
2B27	AC		XOR	H	
2B28	67		LD	H,A	
2B29	EF		RST	\$28	

3.76 Jämförelse av heltal, \$E2-\$F3

\$2B2A

HL - BC med SBC. Teckenflaggan avgör, och vid spill (P/V) gäller den omvända. Resultatet är -1 (sant) eller 0. **LEI** och **GTI** byter operanderna och använder samma kod som **GEI** och **LTI**.

2B2A	E1	EQI:	POP	HL	
2B2B	C1		POP	BC	
2B2C	ED 42		SBC	HL,BC	(CY = 0)
2B2E	28 0A		JR	Z, L2B3A	
2B30	21 00 00	L2B30:	LD	HL, \$0000	falskt
2B33	EF		RST	\$28	
2B34	E1	NEI:	POP	HL	
2B35	C1		POP	BC	
2B36	ED 42		SBC	HL,BC	
2B38	28 03		JR	Z, L2B3D	
2B3A	21 FF FF	L2B3A:	LD	HL, \$FFFF	sant
2B3D	EF	L2B3D:	RST	\$28	
2B3E	C1	LTI:	POP	BC	höger
2B3F	E1		POP	HL	vänster
2B40	ED 42	L2B40:	SBC	HL,BC	
2B42	EA 51 2B		JP	PE, L2B51	spill: omvänt
2B45	F2 30 2B	L2B45:	JP	P, L2B30	positivt: falskt
2B48	18 F0		JR	L2B3A	
2B4A	C1	GEI:	POP	BC	
2B4B	E1		POP	HL	
2B4C	ED 42	L2B4C:	SBC	HL,BC	
2B4E	EA 45 2B		JP	PE, L2B45	spill: omvänt
2B51	FA 30 2B	L2B51:	JP	M, L2B30	negativt: falskt
2B54	18 E4		JR	L2B3A	
2B56	E1	LEI:	POP	HL	byt operander
2B57	C1		POP	BC	
2B58	18 F2		JR	L2B4C	
2B5A	E1	GTI:	POP	HL	byt operander
2B5B	C1		POP	BC	
2B5C	18 E2		JR	L2B40	

ADDI / **SUBI**: heltalsaddition och -subtraktion utan kontroll av spill (resultatet blir modulo 65536).

2B5E	C1	ADDI:	POP	BC	
2B5F	E1		POP	HL	
2B60	09		ADD	HL,BC	inget spilltest
2B61	EF		RST	\$28	
2B62	C1	SUBI:	POP	BC	
2B63	E1		POP	HL	
2B64	ED 42		SBC	HL,BC	
2B66	EF		RST	\$28	

MULI / **DIVI**: via **SMULDV** med **IMUL** resp. **IDIV**. Division med 0 ger fel 7, FÖR STORT HELTAL, och division avrundar mot 0. Beloppen multipliceras utan tecken, så fel 7 kommer först när produkten i belopp är över 65535; 32768-65535 slår runt (200%*200% = -25536, -128%*256% = -32768).

2B67	DD 21 74 37	MULI:	LD	IX, IMUL	
2B6B	E1	IMULDV:	POP	HL	b
2B6C	EB		EX	DE, HL	
2B6D	E3		EX	(SP), HL	HL = a, koden på stacken
2B6E	CD 87 2B		CALL	SMULDV	
2B71	D1		POP	DE	
2B72	EF		RST	\$28	
2B73	DD 21 88 37	DIVI:	LD	IX, IDIV	
2B77	18 F2		JR	IMULDV	

EXSWAP: **SWAP%(x)** byter plats på den höga och den låga byten.

2B79	E1	EXSWAP:	POP	HL	
------	----	----------------	-----	----	--

2B7A	7C	LD	A,H
2B7B	65	LD	H,L
2B7C	6F	LD	L,A
2B7D	EF	RST	\$28

EXPEEK: PEEK(*a*) läser en byte.

2B7E	E1	EXPEEK:	POP	HL
2B7F	6E		LD	L,(HL)
2B80	26 00		LD	H,\$00
2B82	EF		RST	\$28

SMUL / SMULDV: multiplikation/division med tecken. *HL* = *a*, *DE* = *b*, *IX* = **IMUL** eller **IDIV** (utan tecken). Båda görs positiva, och resultatet byter tecken om *a* och *b* hade olika tecken. *CY* från rutinen (resultat över 65535) ger fel 7; 16-bitarspill prövas inte.

2B83	DD 21 74 37	SMUL:	LD	IX,IMUL	
2B87	06 02	SMULDV:	LD	B,\$02	
2B89	EB	L2B89:	EX	DE,HL	spara a och b
2B8A	E5		PUSH	HL	
2B8B	7C		LD	A,H	
2B8C	A7		AND	A	
2B8D	FC EE 2A		CALL	M,NEGHL	negativt: gör positivt
2B90	10 F7		DJNZ	L2B89	för a och b
2B92	CD 91 2A		CALL	JPIX	IMUL / IDIV
2B95	DA 07 07		JP	C,L0707	fel 7
2B98	C1		POP	BC	
2B99	F1		POP	AF	A = b:s höga byte
2B9A	A8		XOR	B	olika tecken?
2B9B	FC EE 2A		CALL	M,NEGHL	olika: byt tecken
2B9E	C9		RET		

3.77 ADD\$ SUB\$ MUL\$ DIV\$ COMP%: aritmetik på tal skrivna som text

\$2B9F

ADD\$(a\$,b\$,n) osv. ger en sträng med *n* decimaler; **COMP%(a\$,b\$)** ger -1, 0 eller 1. För **COMP%** läggs *n* = 0 på stacken. **BC** = rutinen: **AADD**, **ASUB**, **AMUL**, **ADIV** (\$37A6-) eller **ASCCMP**.

- ROOM:** det ska finnas 185 + 100 bytes ledigt över **HEAP**.
- n* till **ASDEC**, **DE** (koden) till **ASCODE**. 87 bytes buffert på stacken.
- ASCARG** kopierar *b\$* (överst) till bufferten och sedan *a\$*, som fylls ut till *n*+1 decimaler och högerjusteras med mellanslag (**ASARG1** = *a\$* i bufferten). Varje kopia slutar med \$83. Tal längre än 29 tecken ger fel 65, **SPILL** I ASCII-ARITMETIK, och en tom sträng fel 66, **STRÄNG** EJ NUMERISK.
- Rutinen anropas med *HL* = *a\$* (**ASARG1**) och *DE* = *b\$*. Resultatet hamnar i *a\$*:s plats, med *a\$*:s bredd och *n*+1 decimaler.
- n* > 26 ger fel 66. Under **ASEND**, där argumenten började, skrivs 29 nollor med en punkt före de *n* sista, och \$83. Resultatet adderas till det med **AADD**, som avrundar till *n* decimaler (exakt hälft uppåt för positiva, nedåt i belopp för negativa resultat). Se \$37A6.
- Inledande mellanslag hoppas över, och texten blir en tillfällig sträng på stacken: [längd+1] [adress] [längd] [text \$83].

CY från en rutin: fel 66 om bit 3 i *A* är satt, annars fel 65.

2B9F	01 A6 37	EXADD:	LD	BC,AADD	
2BA2	18 16		JR	ASCAR	
2BA4	01 AE 37	EXSUB:	LD	BC,ASUB	
2BA7	18 11		JR	ASCAR	
2BA9	01 B8 38	EXMUL:	LD	BC,AMUL	
2BAC	18 0C		JR	ASCAR	
2BAE	01 3B 38	EXDIV:	LD	BC,ADIV	
2BB1	18 07		JR	ASCAR	
2BB3	01 51 2C	EXCOMP:	LD	BC,ASCCMP	
2BB6	21 00 00		LD	HL,\$0000	<i>n</i> = 0
2BB9	E5		PUSH	HL	

2BBA	21 47 FF	ASCAR:	LD HL, \$FF47	185 bytes
2BBD	39		ADD HL, SP	
2BBE	CD 9B 33		CALL ROOM	+ 100 över HEAP?
2BC1	E1		POP HL	n
2BC2	22 EB FD		LD (ASDEC), HL	
2BC5	ED 53 ED FD		LD (ASCODE), DE	
2BC9	21 A9 FF		LD HL, \$FFA9	87 bytes buffert
2BCC	39		ADD HL, SP	
2BCD	F9		LD SP, HL	
2BCE	E5		PUSH HL	buffertens början
2BCF	EB		EX DE, HL	
2BD0	21 57 00		LD HL, \$0057	b\$:s beskrivning
2BD3	19		ADD HL, DE	
2BD4	C5		PUSH BC	
2BD5	AF		XOR A	Z: kopiera b\$
2BD6	CD 6C 2C		CALL ASCARG	
2BD9	D5		PUSH DE	a\$:s början i bufferten
2BDA	B2		OR D	NZ: fyll ut a\$
2BDB	CD 6C 2C		CALL ASCARG	
2BDE	22 E9 FD		LD (ASEND), HL	där argumenten började
2BE1	E1		POP HL	
2BE2	22 E7 FD		LD (ASARG1), HL	
2BE5	C1		POP BC	rutinen
2BE6	D1		POP DE	b\$
2BE7	FD E5		PUSH IY	
2BE9	CD 47 2C		CALL CALLBC	kör rutinen
2BEC	FD E1		POP IY	
2BEE	38 5B		JR C, ASCERR	
2BF0	2A EB FD		LD HL, (ASDEC)	n
2BF3	7C		LD A, H	
2BF4	A7		AND A	
2BF5	20 52		JR NZ, ERR66	
2BF7	7D		LD A, L	
2BF8	FE 1B		CP \$1B	
2BFA	30 4D		JR NC, ERR66	n > 26: fel 66
2BFC	2A E9 FD		LD HL, (ASEND)	
2BFF	E5		PUSH HL	
2C00	2B		DEC HL	
2C01	36 83		LD (HL), \$83	slutmarkering
2C03	2B		DEC HL	
2C04	36 30		LD (HL), \$30	
2C06	5D		LD E, L	
2C07	54		LD D, H	
2C08	1B		DEC DE	
2C09	01 1C 00		LD BC, \$001C	
2C0C	ED B8		LDDR	fyll med nollor
2C0E	E3		EX (SP), HL	
2C0F	ED 5B E7 FD		LD DE, (ASARG1)	
2C13	2B		DEC HL	
2C14	2B		DEC HL	
2C15	ED 4B EB FD		LD BC, (ASDEC)	
2C19	78		LD A, B	
2C1A	B1		OR C	
2C1B	28 04		JR Z, L2C21	
2C1D	ED 42		SBC HL, BC	n decimaler från slutet
2C1F	36 2E		LD (HL), \$2E	punkt före n decimaler
2C21	E1	L2C21:	POP HL	fältets början
2C22	FD E5		PUSH IY	
2C24	CD A6 37		CALL AADD	addera resultatet
2C27	FD E1		POP IY	
2C29	38 20		JR C, ASCERR	
2C2B	2A E9 FD		LD HL, (ASEND)	
2C2E	2B		DEC HL	
2C2F	01 1D 00		LD BC, \$001D	
2C32	ED 42		SBC HL, BC	
2C34	7E	L2C34:	LD A, (HL)	hoppa över mellanslag
2C35	FE 20		CP \$20	
2C37	20 04		JR NZ, L2C3D	
2C39	23		INC HL	

2C3A	0B		DEC	BC	
2C3B	18 F7		JR	L2C34	
2C3D	F9	L2C3D:	LD	SP,HL	texten ligger på stacken
2C3E	C5		PUSH	BC	längd
2C3F	E5		PUSH	HL	adress
2C40	03		INC	BC	n = längd + 1 (\$83)
2C41	C5	L2C41:	PUSH	BC	
2C42	ED 5B ED FD		LD	DE, (ASCII)	
2C46	F7		RST	\$30	

CALLBC: anropa rutinen i BC.

2C47	C5	CALLBC:	PUSH	BC	
2C48	C9		RET		
2C49	D7	ERR66:	RST	\$10	
2C4A	C2		DEFB	\$C2	fel 66: STRÄNG EJ NUMERISK
2C4B	CB 5F	ASCERR:	BIT	3,A	bit 3: ej numerisk
2C4D	20 FA		JR	NZ,ERR66	
2C4F	D7	ERR65:	RST	\$10	
2C50	C1		DEFB	\$C1	fel 65: SPILL I ASCII-ARITMETIK

ASCCMP: COMP%. Returadressen från **CALLBC** tas bort, och **ACOMP** jämför. A bit 6 = 0: lika (0). Annars bit 7 = 1 ger -1, bit 7 = 0 ger 1.

2C51	F1	ASCCMP:	POP	AF	ta bort returadressen
2C52	CD F6 37		CALL	ACOMP	
2C55	FD E1		POP	IY	
2C57	38 F2		JR	C,ASCERR	
2C59	2A E9 FD		LD	HL, (ASEND)	ta bort argumenten
2C5C	F9		LD	SP,HL	
2C5D	01 00 00		LD	BC, \$0000	
2C60	CB 77		BIT	6,A	lika: 0
2C62	28 DD		JR	Z, L2C41	
2C64	87		ADD	A,A	bit 7
2C65	38 02		JR	C, L2C69	
2C67	03		INC	BC	+1
2C68	03		INC	BC	
2C69	0B	L2C69:	DEC	BC	-1
2C6A	18 D5		JR	L2C41	

3.78 ASCARG: kopiera ett strängargument till bufferten (DE)

\$2C6C

In: HL = strängbeskrivningen på stacken, Z = kopiera som den är (b\$), NZ = fyll ut (a\$).

Ut: HL = nästa beskrivning, DE efter kopian och \$83.

Utfyllnad: k = antal tecken efter ".", eller 0 om punkt saknas. Det behövs z = n+1-k nollor (0 om negativt). Först 29-längd-z mellanslag, sedan texten, sedan "." om punkt saknas, och z nollor.

2C6C	08	ASCARG:	EX	AF,AF'	
2C6D	3E 03		LD	A, \$03	läs n, adress, längd
2C6F	CD 17 10	L2C6F:	CALL	RDBC	
2C72	C5		PUSH	BC	
2C73	3D		DEC	A	
2C74	20 F9		JR	NZ, L2C6F	
2C76	C1		POP	BC	längd
2C77	E5		PUSH	HL	
2C78	21 1D 00		LD	HL, \$001D	högst 29 tecken
2C7B	AF		XOR	A	
2C7C	ED 42		SBC	HL, BC	
2C7E	38 CF		JR	C,ERR65	fel 65
2C80	B1		OR	C	
2C81	28 C6		JR	Z,ERR66	tom: fel 66
2C83	7D		LD	A, L	A = 29 - längd
2C84	E1		POP	HL	
2C85	E3		EX	(SP), HL	HL = texten

2C86	08		EX	AF,AF'	
2C87	28	22	JR	Z,L2CAB	b\$: ingen utfyllnad
2C89	08		EX	AF,AF'	
2C8A	F5		PUSH	AF	
2C8B	C5		PUSH	BC	
2C8C	E5		PUSH	HL	
2C8D	3E	2E	LD	A,\$2E	
2C8F	ED	B1	CPIR		sök "."
2C91	3A	EB FD	LD	A,(ASDEC)	n + 1 - decimaler
2C94	3C		INC	A	
2C95	91		SUB	C	
2C96	E1		POP	HL	
2C97	C1		POP	BC	
2C98	30	01	JR	NC,L2C9B	negativt: 0
2C9A	AF		XOR	A	
2C9B	47	L2C9B:	LD	B,A	
2C9C	F1		POP	AF	
2C9D	90		SUB	B	mellanslag = 29 - längd - nollor
2C9E	38	AF	JR	C,ERR65	fel 65
2CA0	28	09	JR	Z,L2CAB	
2CA2	C5		PUSH	BC	
2CA3	47		LD	B,A	
2CA4	3E	20	LD	A,\$20	
2CA6	12	L2CA6:	LD	(DE),A	mellanslag
2CA7	13		INC	DE	
2CA8	10	FC	DJNZ	L2CA6	
2CAA	C1		POP	BC	
2CAB	78	L2CAB:	LD	A,B	
2CAC	06	00	LD	B,\$00	
2CAE	ED	B0	LDIR		kopiera texten
2CB0	A7		AND	A	
2CB1	28	12	JR	Z,L2CC5	
2CB3	47		LD	B,A	
2CB4	3A	EB FD	LD	A,(ASDEC)	
2CB7	3C		INC	A	
2CB8	B8		CP	B	saknas punkten?
2CB9	20	04	JR	NZ,L2CBF	
2CBB	3E	2E	LD	A,\$2E	
2CBD	12		LD	(DE),A	""
2CBE	13		INC	DE	
2CBF	3E	30	L2CBF:	LD	A,\$30
2CC1	12	L2CC1:	LD	(DE),A	nollor
2CC2	13		INC	DE	
2CC3	10	FC	DJNZ	L2CC1	
2CC5	3E	83	L2CC5:	LD	A,\$83
2CC7	12		LD	(DE),A	slutmarkering
2CC8	13		INC	DE	
2CC9	E1		POP	HL	
2CCA	C1		POP	BC	
2CCB	09		ADD	HL,BC	nästa beskrivning
2CCC	C9		RET		

3.79 VAR0 (\$C0-\$C2): variabel utan index

\$2CCD

Koden: token, namnbyte, bokstav, adress (2 bytes). Adressen läggs på stacken. Är adressen < \$100 är det en DEF FN-parameter, och FNPAR letar upp argumentet genom ramen FNFRAM.

2CCD	EB	VAR0:	EX	DE,HL	
2CCE	23		INC	HL	
2CCF	CD	1C 10	CALL	RDDE1	variabelns adress
2CD2	EB		EX	DE,HL	
2CD3	7C		LD	A,H	
2CD4	A7		AND	A	
2CD5	CC	AB 33	CALL	Z,FNPAR	< \$100: DEF FN-parameter
2CD8	EF		RST	\$28	

3.80 VAR1x / VAR2x: element i vektor eller matris

\$2CD9

BC = elementets storlek (C) och offset till värdet (B): flyttal 5/4 (exponenten), heltal 2/0, sträng 6/0. De skrivs in i posten (+4, +5). En variabel som inte är dimensionerad (+2/3 = 0) dimensioneras med 10 som högsta index (DIM1 / DIM2).

Index utanför ger fel 5, FÖR STORT INDEX. Ett negativt index blir ett stort tal utan tecken och ger också fel 5.

Adressen = +0/1 + elementnummer * storlek + offset. För en matris är elementnumret i * (max2 + 1) + j.

2CD9 06 04	VAR1F:	LD	B, \$04	
2CDB 0E 05		LD	C, \$05	
2CDD 18 08		JR	VAR1	
2CDF 01 02 00	VAR1I:	LD	BC, \$0002	
2CE2 18 03		JR	VAR1	
2CE4 01 06 00	VAR1S:	LD	BC, \$0006	
2CE7 CD 11 2D	VAR1:	CALL	VARIDX	
2CEA CC E6 32		CALL	Z, DIM1	odimensionerad: DIM (10)
2CED D1		POP	DE	
2CEE C1		POP	BC	index
2CEF DD 6E 02		LD	L, (IX+\$02)	högsta index
2CF2 DD 66 03		LD	H, (IX+\$03)	
2CF5 A7		AND	A	
2CF6 ED 42		SBC	HL, BC	
2CF8 38 55		JR	C, ERRIDX	för stort: fel 5
2CFA DD 7E 04	ELMADR:	LD	A, (IX+\$04)	storlek
2CFD DD 6E 00		LD	L, (IX+\$00)	elementens början
2D00 DD 66 01		LD	H, (IX+\$01)	
2D03 09	L2D03:	ADD	HL, BC	+ index * storlek
2D04 3D		DEC	A	
2D05 20 FC		JR	NZ, L2D03	
2D07 DD B6 05		OR	(IX+\$05)	offset
2D0A 28 04		JR	Z, L2D10	
2D0C 4F		LD	C, A	
2D0D 06 00		LD	B, \$00	
2D0F 09		ADD	HL, BC	
2D10 EF	L2D10:	RST	\$28	

VARIDX: IX = variabelns post (adressen i koden), DE = 10, skriv in storlek och offset. Koden-pekaren läggs under returadressen.

Ut: Z om variabeln inte är dimensionerad.

2D11 EB	VARIDX:	EX	DE, HL	
2D12 23		INC	HL	
2D13 CD 1C 10		CALL	RDDE1	variabelns post
2D16 D5		PUSH	DE	
2D17 DD E1		POP	IX	
2D19 E3		EX	(SP), HL	koden under returadressen
2D1A E5		PUSH	HL	
2D1B 11 0A 00		LD	DE, \$000A	förvalt högsta index
2D1E DD 71 04		LD	(IX+\$04), C	storlek
2D21 DD 70 05		LD	(IX+\$05), B	offset
2D24 DD 7E 02		LD	A, (IX+\$02)	dimensionerad?
2D27 DD B6 03		OR	(IX+\$03)	
2D2A C9		RET		
2D2B 06 04	VAR2F:	LD	B, \$04	
2D2D 0E 05		LD	C, \$05	
2D2F 18 08		JR	VAR2	
2D31 01 02 00	VAR2I:	LD	BC, \$0002	
2D34 18 03		JR	VAR2	
2D36 01 06 00	VAR2S:	LD	BC, \$0006	
2D39 CD 11 2D	VAR2:	CALL	VARIDX	
2D3C 6B		LD	L, E	HL = DE = 10
2D3D 62		LD	H, D	
2D3E CC 11 33		CALL	Z, DIM2	odimensionerad: DIM (10,10)
2D41 C1		POP	BC	
2D42 E1		POP	HL	j

2D43	D1		POP	DE	i
2D44	C5		PUSH	BC	
2D45	E5		PUSH	HL	
2D46	DD 6E 06		LD	L, (IX+\$06)	första indexets max
2D49	DD 66 07		LD	H, (IX+\$07)	
2D4C	A7		AND	A	
2D4D	ED 52		SBC	HL, DE	
2D4F	DA 03 07	ERRIDX:	JP	C, L0703	fel 5
2D52	DD 6E 08		LD	L, (IX+\$08)	andra indexets max
2D55	DD 66 09		LD	H, (IX+\$09)	
2D58	E5		PUSH	HL	
2D59	23		INC	HL	
2D5A	CD 74 37		CALL	IMUL	i * (max2 + 1)
2D5D	38 F0		JR	C, ERRIDX	spill: fel 5
2D5F	EB		EX	DE, HL	
2D60	E1		POP	HL	
2D61	C1		POP	BC	
2D62	ED 42		SBC	HL, BC	j för stort?
2D64	38 E9		JR	C, ERRIDX	
2D66	60		LD	H, B	
2D67	69		LD	L, C	
2D68	19		ADD	HL, DE	+ j
2D69	44		LD	B, H	
2D6A	4D		LD	C, L	
2D6B	D1		POP	DE	
2D6C	18 8C		JR	ELMADR	

EXPI: PI, 5 bytes från FPI.

2D6E	21 FB FF	EXPI:	LD	HL, \$FFFB	
2D71	39		ADD	HL, SP	
2D72	F9		LD	SP, HL	
2D73	D5		PUSH	DE	
2D74	EB		EX	DE, HL	
2D75	21 A6 3F		LD	HL, FPI	PI
2D78	01 05 00		LD	BC, \$0005	
2D7B	ED B0		LDIR		
2D7D	D1		POP	DE	
2D7E	F7		RST	\$30	

EXABS: nollställ tecknet.

2D7F	21 03 00	EXABS:	LD	HL, \$0003	
2D82	39		ADD	HL, SP	
2D83	36 00		LD	(HL), \$00	tecknet = +
2D85	F7		RST	\$30	

EXSGN: SGN(x) ger ett heltal: 0 om exponenten är 0, annars -1 eller 1.

2D86	C1	EXSGN:	POP	BC	mantissan
2D87	33		INC	SP	
2D88	C1		POP	BC	C = tecken, B = exponent
2D89	AF		XOR	A	
2D8A	6F		LD	L, A	
2D8B	67		LD	H, A	
2D8C	B0		OR	B	noll?
2D8D	28 06		JR	Z, L2D95	
2D8F	2B		DEC	HL	-1
2D90	0D		DEC	C	negativt?
2D91	28 02		JR	Z, L2D95	
2D93	23		INC	HL	+1
2D94	23		INC	HL	
2D95	EF	L2D95:	RST	\$28	

3.81 Flyttalsfunktioner med ett argument

\$2D96

BC = rutinen, som **FFUNC** anropar med HL = BC = argumentets exponent. Resultatet ersätter argumentet på stacken.

	FIX	FFIX	INT	FINT	SIN	FSIN	COS	FCOS	TAN	FTAN
	ATN	FATN	EXP	FEXP	LOG	FLOG	LOG10	FLOG10	SQR	FSQR
2D96	01 D5 2D		EXFIX:	LD	BC, FFIX					
2D99	18 2B			JR	FFUNC					
2D9B	01 7F 3C		EXINT:	LD	BC, FINT					
2D9E	18 26			JR	FFUNC					
2DA0	01 77 3D		EXSIN:	LD	BC, FSIN					
2DA3	18 21			JR	FFUNC					
2DA5	01 70 3D		EXCOS:	LD	BC, FCOS					
2DA8	18 1C			JR	FFUNC					
2DAA	01 AD 3E		EXTAN:	LD	BC, FTAN					
2DAD	18 17			JR	FFUNC					
2DAF	01 77 3B		EXATN:	LD	BC, FATN					
2DB2	18 12			JR	FFUNC					
2DB4	01 03 3C		EXEXP1:	LD	BC, FEXP					
2DB7	18 0D			JR	FFUNC					
2DB9	01 C4 3C		EXLOG:	LD	BC, FLOG					
2DBC	18 08			JR	FFUNC					
2DBE	01 B7 3C		EXLG10:	LD	BC, FLOG10					
2DC1	18 03			JR	FFUNC					
2DC3	01 F5 3D		EXSQR:	LD	BC, FSQR					
2DC6	21 04 00		FFUNC:	LD	HL, \$0004				argumentets exponent	
2DC9	39			ADD	HL, SP					
2DCA	C5			PUSH	BC					
2DCB	DD E1			POP	IX					
2DCD	44			LD	B, H					
2DCE	4D			LD	C, L					
2DCF	D5			PUSH	DE					
2DD0	CD 91 2A			CALL	JPIX				anropa rutinen	
2DD3	D1			POP	DE					
2DD4	F7			RST	\$30					

FFIX: $\text{FIX}(x) = \text{INT}(\text{ABS}(x))$ med x 's tecken, dvs. heltalsdelen mot 0.

2DD5	2B	FFIX:	DEC	HL	tecknet
2DD6	7E		LD	A, (HL)	
2DD7	36 00		LD	(HL), \$00	gör positivt
2DD9	E5		PUSH	HL	
2DDA	F5		PUSH	AF	
2ddb	CD 7F 3C		CALL	FINT	INT
2DDE	F1		POP	AF	
2DDF	E1		POP	HL	
2DE0	77		LD	(HL), A	tecknet tillbaka
2DE1	C9		RET		

FTOI (\$D9): flyttal till heltal (**FTOINT**, resultat i DE). Flyttalet tas bort och heltalet läggs på stacken.

2DE2	21 04 00	FTOI:	LD	HL, \$0004	
2DE5	39		ADD	HL, SP	
2DE6	D5		PUSH	DE	
2DE7	CD CA 3E		CALL	FTOINT	DE = heltalet
2DEA	E1		POP	HL	
2DEB	F1		POP	AF	ta bort 5 bytes
2DEC	F1		POP	AF	
2DED	33		INC	SP	
2DEE	EB		EX	DE, HL	
2DEF	EF		RST	\$28	

ITOF (\$DA): heltal till flyttal. Plats för 5 bytes görs på stacken och **INTTOF** skriver flyttalet där.

2DF0	E1	ITOF:	POP HL	
2DF1	F5		PUSH AF	plats för 5 bytes
2DF2	F5		PUSH AF	
2DF3	3B		DEC SP	
2DF4	D5		PUSH DE	
2DF5	EB		EX DE,HL	
2DF6	21 06 00		LD HL,\$0006	
2DF9	39		ADD HL,SP	
2DFA	EB		EX DE,HL	
2DFB	CD 18 3F		CALL INTTOF	skriv flyttalet
2DFE	D1		POP DE	
2DFF	F7		RST \$30	

3.82 Potens: **POWF (\$FD)**, **POWI (\$FE)**, **POWFI (\$FF)**

\$2E00

POWF a^b , båda flyttal. **FPOW** anropas via **FARITH** (BC = DE = a, HL = b; resultatet hamnar i a) och räknar $\text{EXP}(b * \text{LOG}(a))$. Om b = 0 hoppas LOG över, så $a^0 = 1$ för alla a, även 0 och negativa tal. Annars går a <= 0 till **FLOG**: 0^2 och $(-2)^2$ ger fel 4 (men $(-2)^3$ går via **POWFI**).

POWI a^b , båda heltal. Upprepad kvadrering: för varje bit i b, lägsta först, multipliceras resultatet med a om biten är satt, och a kvadreras. $a^0 = 1$ (även 0^0). Spill som i **SMUL**: fel 7 först över 65535, annars slår det runt ($200\%^2 = -25536$, $2\%^{15} = -32768$, $256\%^2$ fel 7).
Negativt b: a = 0 -> fel 4, a = 1 -> 1, a = -1 -> -1 om b är udda, annars 1, annars 0 ($1/a^{|b|}$ huggs av till 0).

POWFI a^n , a flyttal och n heltal. Negativt n: först a = 1/a och n = -n. Sedan upprepad kvadrering som i **POWI**, med **FMUL**. **FMUL** ger fel 4 redan när exponenternas summa blir för stor, före normaliseringen: $10^{125\%}$ går, $10^{126\%}$ ger fel 4, fast $1\text{E}126$ går att skriva in (likaså $1\text{E}100*1\text{E}26$ och $1\text{E}126*1$; se **FMUL \$34DF**).

2E00	DD 21 07 2E	POWF:	LD IX, FPOW	
2E04	C3 CD 2A		JP FARITH	
2E07	7E	FPOW:	LD A,(HL)	b:s exponent
2E08	A7		AND A	
2E09	E5		PUSH HL	
2E0A	C5		PUSH BC	
2E0B	C4 C4 3C		CALL NZ, FLOG	a = LOG a (inte om b = 0)
2E0E	C1		POP BC	
2E0F	E1		POP HL	
2E10	59		LD E,C	
2E11	50		LD D,B	
2E12	CD DF 34		CALL FMUL	a = a * b
2E15	C3 03 3C		JP FEXP	a = EXP a
2E18	C1	POWI:	POP BC	b
2E19	E1		POP HL	a
2E1A	D5		PUSH DE	
2E1B	78		LD A,B	
2E1C	87		ADD A,A	b negativt?
2E1D	38 21		JR C, L2E40	
2E1F	11 01 00		LD DE,\$0001	resultatet = 1
2E22	EB		EX DE,HL	HL = 1, DE = a
2E23	CB 38	L2E23:	SRL B	b >>= 1
2E25	CB 19		RR C	
2E27	30 07		JR NC, L2E30	biten 0: ingen multiplikation
2E29	D5		PUSH DE	
2E2A	C5		PUSH BC	
2E2B	CD 83 2B		CALL SMUL	resultat * a
2E2E	C1		POP BC	
2E2F	D1		POP DE	
2E30	78	L2E30:	LD A,B	b = 0: klart

2E31	B1		OR	C	
2E32	28 1A		JR	Z, L2E4E	
2E34	C5		PUSH	BC	
2E35	E5		PUSH	HL	
2E36	62		LD	H, D	
2E37	6B		LD	L, E	
2E38	CD 83 2B		CALL	SMUL	a * a
2E3B	D1		POP	DE	
2E3C	C1		POP	BC	
2E3D	EB		EX	DE, HL	HL = resultatet, DE = a
2E3E	18 E3		JR	L2E23	
2E40	7C	L2E40:	LD	A, H	a:s höga byte
2E41	B7		OR	A	
2E42	20 0C		JR	NZ, L2E50	a < 0 eller a > 255
2E44	B5		OR	L	
2E45	CA 01 07		JP	Z, L0701	0^n: fel 4
2E48	3D		DEC	A	a = 1?
2E49	28 03		JR	Z, L2E4E	1^n = 1
2E4B	21 00 00	L2E4B:	LD	HL, \$0000	annars 0
2E4E	D1	L2E4E:	POP	DE	koden
2E4F	EF		RST	\$28	
2E50	A5	L2E50:	AND	L	a = -1?
2E51	3C		INC	A	
2E52	20 F7		JR	NZ, L2E4B	
2E54	3E 01		LD	A, \$01	b udda?
2E56	A1		AND	C	
2E57	20 F5		JR	NZ, L2E4E	-1^udda = -1
2E59	21 01 00		LD	HL, \$0001	-1^jämnt = 1
2E5C	18 F0		JR	L2E4E	

POWFI: ett resultat $r = 1$ läggs på stacken under koden. För varje bit i i n (n skiftas på sin plats på stacken) blir $r = r * a$ om biten är satt, och $a = a * a$. Till sist kopieras r över a , och r och n tas bort. Stacken i slingan, $IX = r$:

	IX+0...4	r		IX+5,6	n		IX+7...11	a
2E5E	E1		POWFI:	POP	HL		n	
2E5F	E5			PUSH	HL			
2E60	7C			LD	A, H			
2E61	87			ADD	A, A		negativt?	
2E62	30 16			JR	NC, L2E7A			
2E64	44			LD	B, H			
2E65	4D			LD	C, L			
2E66	21 01 00			LD	HL, \$0001			
2E69	ED 42			SBC	HL, BC		1 - n - CY = -n	
2E6B	E3			EX	(SP), HL		ersätter n	
2E6C	D5			PUSH	DE			
2E6D	21 08 00			LD	HL, \$0008		a:s exponent	
2E70	39			ADD	HL, SP			
2E71	11 BB 3F			LD	DE, FONE		1	
2E74	44			LD	B, H			
2E75	4D			LD	C, L			
2E76	CD EE 35			CALL	FDIV		a = 1 / a	
2E79	D1			POP	DE			
2E7A	21 FB FF	L2E7A:		LD	HL, \$FFFB		plats för r	
2E7D	39			ADD	HL, SP			
2E7E	F9			LD	SP, HL			
2E7F	D5			PUSH	DE			
2E80	E5			PUSH	HL		r	
2E81	01 05 00			LD	BC, \$0005			
2E84	09			ADD	HL, BC			
2E85	2B			DEC	HL			
2E86	11 BB 3F			LD	DE, FONE			
2E89	EB			EX	DE, HL			
2E8A	ED B8			LDDR			r = 1	
2E8C	DD E1	L2E8C:		POP	IX		IX = r	
2E8E	DD CB 06 3E			SRL	(IX+\$06)		n >>= 1	
2E92	DD CB 05 1E			RR	(IX+\$05)			
2E96	30 13			JR	NC, L2EAB		biten 0?	

2E98	DD E5		PUSH IX	
2E9A	E1		POP HL	
2E9B	E5		PUSH HL	
2E9C	11 04 00		LD DE,\$0004	
2E9F	19		ADD HL,DE	
2EA0	1E 07		LD E,\$07	
2EA2	EB		EX DE,HL	
2EA3	19		ADD HL,DE	HL = a
2EA4	42		LD B,D	BC = DE = r
2EA5	4B		LD C,E	
2EA6	CD DF 34		CALL FMUL	r = r * a
2EA9	DD E1		POP IX	
2EAB	DD 7E 06	L2EAB:	LD A,(IX+\$06)	n = 0?
2EAE	DD B6 05		OR (IX+\$05)	
2EB1	28 11		JR Z,L2EC4	
2EB3	DD E5		PUSH IX	
2EB5	E1		POP HL	
2EB6	E5		PUSH HL	
2EB7	11 0B 00		LD DE,\$000B	
2EBA	19		ADD HL,DE	HL = a
2EBB	54		LD D,H	
2EBC	5D		LD E,L	
2EBD	44		LD B,H	
2EBE	4D		LD C,L	
2EBF	CD DF 34		CALL FMUL	a = a * a
2EC2	18 C8		JR L2E8C	
2EC4	DD E5	L2EC4:	PUSH IX	DE = r
2EC6	D1		POP DE	
2EC7	21 07 00		LD HL,\$0007	
2ECA	E5		PUSH HL	
2ECB	19		ADD HL,DE	
2ECC	EB		EX DE,HL	DE = a, HL = r
2ECD	01 05 00		LD BC,\$0005	
2ED0	ED B0		LDIR	a = r
2ED2	E1		POP HL	
2ED3	D1		POP DE	koden
2ED4	39		ADD HL,SP	ta bort r och n
2ED5	F9		LD SP,HL	
2ED6	F7		RST \$30	

CONSTS (\$CB): strängkonstant i koden: \$CB, citattecken, längd, text. Beskrivningen [*n* = 0][*adress*][*längd*] läggs på stacken; texten ligger kvar i programmet.

2ED7	EB	CONSTS:	EX DE,HL	
2ED8	23		INC HL	hoppa över citattecknet
2ED9	5E		LD E,(HL)	längd
2EDA	16 00		LD D,\$00	
2EDC	23		INC HL	
2EDD	D5		PUSH DE	längd
2EDE	E5		PUSH HL	adress
2EDF	19		ADD HL,DE	efter texten
2EE0	5A		LD E,D	n = 0
2EE1	EB		EX DE,HL	
2EE2	EF		RST \$28	

3.83 Strängfunktionerna

\$2EE3

Argumenten ligger på stacken, sista argumentet överst. En sträng är beskrivningen [n][adress][längd] följt av n tillfälliga bytes (se \$2842). En ny sträng byggs direkt på stacken och blir en tillfällig sträng med n = längden.

LEN(s\$)	längden
ASC(s\$)	första tecknets kod, 0 för en tom sträng
LEFT\$(s\$,j)	de j första tecknen; fel 9 om j > längden
RIGHT\$(s\$,i)	från tecken i till slutet (i = 1 ger hela strängen); fel 9 om i = 0 eller i > längden + 1
MID\$(s\$,i,j)	j tecken från tecken i: först som RIGHT\$, sedan som LEFT\$ på det som blev kvar

Negativa j och i blir stora tal och ger fel 9.

LEFT\$, RIGHT\$ och MID\$ kopierar inget: bara adress och längd i beskrivningen ändras, och de tillfälliga byten ligger kvar.

2EE3 DD E1	EXLEN: POP IX	n
2EE5 E1	POP HL	adress
2EE6 E1	POP HL	längd
2EE7 DD 39	ADD IX,SP	
2EE9 DD F9	LD SP,IX	ta bort de tillfälliga byten
2EEB EF	RST \$28	
2EEC DD E1	EXASC: POP IX	
2EEE E1	POP HL	
2EEF 4E	LD C,(HL)	första tecknet
2EF0 E1	POP HL	längd
2EF1 DD 39	ADD IX,SP	
2EF3 DD F9	LD SP,IX	
2EF5 7C	LD A,H	
2EF6 B5	OR L	
2EF7 28 03	JR Z,L2EFC	tom: 0
2EF9 26 00	LD H,\$00	
2EFB 69	LD L,C	
2EFC EF	L2EFC: RST \$28	
2EFD E1	EXMID: POP HL	j
2EFE C1	POP BC	i
2EFF E5	PUSH HL	
2F00 DD 21 04 00	LD IX,\$0004	
2F04 CD 25 2F	CALL STRSKP	som RIGHT\$, sedan LEFT\$
2F07 C1	EXLEFT: POP BC	j
2F08 21 04 00	LD HL,\$0004	
2F0B 39	ADD HL,SP	
2F0C 7E	LD A,(HL)	längd
2F0D 23	INC HL	
2F0E 66	LD H,(HL)	
2F0F 6F	LD L,A	
2F10 ED 42	SBC HL,BC	längd - j
2F12 38 1C	JR C,L2F30	j > längden: fel 9
2F14 E1	POP HL	n
2F15 DD E1	POP IX	adress
2F17 F1	POP AF	gamla längden
2F18 C5	PUSH BC	ny längd = j
2F19 DD E5	PUSH IX	
2F1B EF	RST \$28	
2F1C C1	EXRGHT: POP BC	i
2F1D DD 21 02 00	LD IX,\$0002	
2F21 CD 25 2F	CALL STRSKP	
2F24 F7	RST \$30	

STRSKP: hoppa över de i-1 första tecknen. BC = i, IX = avståndet från SP (efter CALL) till beskrivningen: 4 från MID\$ (j ligger emellan), 2 från RIGHT\$. Fel 9 om i-1 är större än längden (också för i = 0, som blir \$FFFF).

2F25 0B	STRSKP: DEC BC	i - 1
2F26 DD 39	ADD IX,SP	
2F28 DD 6E 04	LD L,(IX+\$04)	
2F2B DD 66 05	LD H,(IX+\$05)	

2F2E	ED 42		SBC	HL,BC	längd - (i - 1)
2F30	DA 0B 07	L2F30:	JP	C, L070B	fel 9
2F33	DD 75 04		LD	(IX+\$04), L	ny längd
2F36	DD 74 05		LD	(IX+\$05), H	
2F39	DD 6E 02		LD	L, (IX+\$02)	
2F3C	DD 66 03		LD	H, (IX+\$03)	
2F3F	09		ADD	HL,BC	adress + i - 1
2F40	DD 75 02		LD	(IX+\$02), L	
2F43	DD 74 03		LD	(IX+\$03), H	
2F46	C9		RET		

3.84 CONCAT (\$F6): a\$ + b\$

\$2F47

b\$ ligger överst, a\$ under. Den nya strängen byggs först under stacken ([ROOM](#) kontrollerar att det finns plats): a\$ och sedan b\$ kopieras dit. Sedan flyttas den med LDDR uppåt så att den slutar där a\$:s tillfälliga bytes slutade, och ersätter båda strängarna. Den blir en tillfällig sträng [n = längd][adress][längd].

[STRPSH](#) (\$2FBB) är gemensamt slut för [CONCAT](#) och [CHR\\$](#): HL = strängen (och ny SP), BC = längden; beskrivningen läggs överst.

2F47	DD 21 06 00	CONCAT:	LD	IX, \$0006	
2F4B	DD 39		ADD	IX, SP	
2F4D	DD 6E FE		LD	L, (IX-\$02)	b\$:s längd
2F50	DD 66 FF		LD	H, (IX-\$01)	
2F53	C1		POP	BC	b\$:s n
2F54	C5		PUSH	BC	
2F55	DD 09		ADD	IX, BC	IX = a\$:s beskrivning
2F57	DD 4E 04		LD	C, (IX+\$04)	a\$:s längd
2F5A	DD 46 05		LD	B, (IX+\$05)	
2F5D	09		ADD	HL, BC	summan
2F5E	44		LD	B, H	
2F5F	4D		LD	C, L	
2F60	21 00 00		LD	HL, \$0000	
2F63	39		ADD	HL, SP	
2F64	ED 42		SBC	HL, BC	SP - summan
2F66	DA FF 06		JP	C, L06FF	fel 3
2F69	CD 9B 33		CALL	ROOM	minst 100 bytes över HEAP ?
2F6C	F9		LD	SP, HL	plats för den nya strängen
2F6D	D5		PUSH	DE	
2F6E	C5		PUSH	BC	längd
2F6F	EB		EX	DE, HL	DE = nya strängen
2F70	DD 46 05		LD	B, (IX+\$05)	
2F73	DD 4E 04		LD	C, (IX+\$04)	
2F76	79		LD	A, C	
2F77	B0		OR	B	
2F78	28 08		JR	Z, L2F82	
2F7A	DD 66 03		LD	H, (IX+\$03)	
2F7D	DD 6E 02		LD	L, (IX+\$02)	
2F80	ED B0		LDIR		kopiera a\$
2F82	DD E1	L2F82:	POP	IX	
2F84	DD E5		PUSH	IX	
2F86	DD 39		ADD	IX, SP	IX+4 = b\$:s beskrivning
2F88	DD 46 09		LD	B, (IX+\$09)	
2F8B	DD 4E 08		LD	C, (IX+\$08)	
2F8E	79		LD	A, C	
2F8F	B0		OR	B	
2F90	28 08		JR	Z, L2F9A	
2F92	DD 66 07		LD	H, (IX+\$07)	
2F95	DD 6E 06		LD	L, (IX+\$06)	
2F98	ED B0		LDIR		kopiera b\$
2F9A	1B	L2F9A:	DEC	DE	sista byten
2F9B	C1		POP	BC	
2F9C	21 02 00		LD	HL, \$0002	
2F9F	7D		LD	A, L	A = 2 strängar
2FA0	39		ADD	HL, SP	SP+2 = nya strängen
2FA1	09		ADD	HL, BC	HL = b\$:s beskrivning
2FA2	C5		PUSH	BC	

2FA3	4E	L2FA3:	LD	C, (HL)	n
2FA4	23		INC	HL	
2FA5	46		LD	B, (HL)	
2FA6	09		ADD	HL, BC	+ n
2FA7	01 05 00		LD	BC, \$0005	
2FAA	09		ADD	HL, BC	+ 6: nästa värde
2FAB	3D		DEC	A	
2FAC	20 F5		JR	NZ, L2FA3	
2FAE	2B		DEC	HL	sista byten i a\$:s plats
2FAF	C1		POP	BC	
2FB0	C5		PUSH	BC	
2FB1	EB		EX	DE, HL	
2FB2	78		LD	A, B	
2FB3	B1		OR	C	
2FB4	28 02		JR	Z, L2FB8	
2FB6	ED B8		LDDR		flytta upp strängen
2FB8	C1	L2FB8:	POP	BC	
2FB9	13		INC	DE	
2FBA	EB		EX	DE, HL	HL = strängen
2FBB	D1	STRPSH:	POP	DE	koden
2FBC	F9		LD	SP, HL	strängen överst på stacken
2FBD	C5		PUSH	BC	längd
2FBE	E5		PUSH	HL	adress
2FBF	C5		PUSH	BC	n = längd
2FC0	F7		RST	\$30	

EXCHR: CHR\$(i[,i[,i[,i]]]) ger en sträng med 1-4 tecken. Antalet k tas ur token före funktionskoden (\$CD + k). Heltalen ligger överst, det sista först. Deras låga bytes packas ihop i början av området (sista argumentet först), och skrivs sedan baklänges efter dem, så att tecknen i rätt ordning hamnar i den övre halvan av de 2k bytes som heltalen tog. Den halvan blir strängen (**STRPSH**). Bara den låga byten räknas (CHR\$(321) = "A"); fem argument ger fel 13 redan när raden skrivs in.

2FC1	D5	EXCHR:	PUSH	DE	
2FC2	1B		DEC	DE	
2FC3	1B		DEC	DE	
2FC4	1A		LD	A, (DE)	token \$CD + k
2FC5	D6 CD		SUB	\$CD	k = antal argument
2FC7	4F		LD	C, A	
2FC8	21 02 00		LD	HL, \$0002	B = 0
2FCB	44		LD	B, H	
2FCC	39		ADD	HL, SP	sista argumentet
2FCD	54		LD	D, H	
2FCE	5D		LD	E, L	
2FCF	ED A0	L2FCF:	LDI		packa de låga byten
2FD1	23		INC	HL	hoppa över höga byten
2FD2	EA CF 2F		JP	PE, L2FCF	
2FD5	4F		LD	C, A	
2FD6	D5		PUSH	DE	strängens början
2FD7	62		LD	H, D	
2FD8	6B		LD	L, E	
2FD9	2B	L2FD9:	DEC	HL	skriv dem baklänges
2FDA	ED A0		LDI		
2FDC	2B		DEC	HL	
2FDD	EA D9 2F		JP	PE, L2FD9	
2FE0	E1		POP	HL	HL = strängen
2FE1	4F		LD	C, A	BC = k
2FE2	18 D7		JR	STRPSH	

3.85 Strängjämförelser: EQS NES GTS LES LTS GES

\$2FE4

STRCMP ger $Z = \text{lika}$ och $CY = a\$ < b\$,$ och tar bort båda strängarna. Resultatet är ett heltal: -1 (**STRUE**) = sant, 0 (**SFALSE**) = falskt. Teckenkoderna jämförs som de är, så "a" > "A", och "" < "A".

2FE4	CD 12 30	EQS:	CALL STRCMP	
2FE7	28 09		JR Z,STRUE	lika: sant
2FE9	21 00 00	SFALSE:	LD HL,\$0000	falskt
2FEC	EF		RST \$28	
2FED	CD 12 30	NES:	CALL STRCMP	
2FF0	28 F7		JR Z,SFALSE	lika: falskt
2FF2	21 FF FF	STRUE:	LD HL,\$FFFF	sant
2FF5	EF		RST \$28	
2FF6	CD 12 30	GTS:	CALL STRCMP	
2FF9	28 EE		JR Z,SFALSE	lika: falskt
2FFB	38 EC	L2FFB:	JR C,SFALSE	mindre: falskt
2FFD	18 F3		JR STRUE	större: sant
2FFF	CD 12 30	LES:	CALL STRCMP	
3002	28 EE		JR Z,STRUE	lika: sant
3004	38 EC	L3004:	JR C,STRUE	mindre: sant
3006	18 E1		JR SFALSE	större: falskt
3008	CD 12 30	LTS:	CALL STRCMP	
300B	18 F7		JR L3004	mindre: sant
300D	CD 12 30	GES:	CALL STRCMP	
3010	18 E9		JR L2FFB	mindre: falskt, annars sant

STRCMP: jämför a\$ och b\$ (b\$ överst, under returadressen) tecken för tecken. Ut: Z = lika, CY = a\$ < b\$. Om den ena strängen är början av den andra är den kortare minst. Båda strängarna tas bort. CY sparas i bit 7 av A (RRA ... RLA) medan ADD HL,SP ändrar CY. Stacken efter PUSH DE: SP+2 returadress, SP+4 b\$:s beskrivning, sedan b\$:s tillfälliga bytes och a\$:s beskrivning (IX+\$0C/\$0E).

3012	D5	STRCMP:	PUSH DE	
3013	21 09 00		LD HL,\$0009	b\$:s längd
3016	39		ADD HL,SP	
3017	46		LD B,(HL)	
3018	2B		DEC HL	
3019	4E		LD C,(HL)	
301A	2B		DEC HL	
301B	56		LD D,(HL)	b\$:s adress
301C	2B		DEC HL	
301D	5E		LD E,(HL)	
301E	2B		DEC HL	
301F	7E		LD A,(HL)	b\$:s n
3020	2B		DEC HL	
3021	6E		LD L,(HL)	
3022	67		LD H,A	
3023	39		ADD HL,SP	hoppa över b\$:s bytes
3024	E5		PUSH HL	
3025	DD E1		POP IX	
3027	DD 6E 0E		LD L,(IX+\$0E)	a\$:s längd
302A	DD 66 0F		LD H,(IX+\$0F)	
302D	ED 42		SBC HL,BC	a\$:s längd - b\$:s
302F	DD 6E 0C		LD L,(IX+\$0C)	a\$:s adress
3032	DD 66 0D		LD H,(IX+\$0D)	
3035	EB		EX DE,HL	DE = a\$, HL = b\$
3036	38 23		JR C,L305B	a\$ kortare
3038	20 1C		JR NZ,L3056	a\$ längre
303A	78	L303A:	LD A,B	lika långa: jämför allt
303B	B1		OR C	
303C	28 07		JR Z,L3045	
303E	1A		LD A,(DE)	
303F	13		INC DE	
3040	BE		CP (HL)	
3041	23		INC HL	
3042	0B		DEC BC	

3043	28 F5	JR	Z, L303A	
3045	1F	L3045: RRA		spara CY
3046	D1	POP	DE	koden
3047	DD E1	POP	IX	returadressen
3049	E1	POP	HL	b\$:s n
304A	C1	POP	BC	
304B	C1	POP	BC	
304C	39	ADD	HL, SP	ta bort b\$
304D	F9	LD	SP, HL	
304E	E1	POP	HL	a\$:s n
304F	C1	POP	BC	
3050	C1	POP	BC	
3051	39	ADD	HL, SP	ta bort a\$
3052	F9	LD	SP, HL	
3053	17	RLA		CY tillbaka
3054	DD E9	JP	(IX)	
3056	CD 68 30	L3056: CALL	STRCPN	jämför b\$:s längd
3059	18 0A	JR	L3065	lika början: a\$ större
305B	DD 4E 0E	L305B: LD	C, (IX+\$0E)	a\$:s längd
305E	DD 46 0F	LD	B, (IX+\$0F)	
3061	CD 68 30	CALL	STRCPN	
3064	37	SCF		lika början: a\$ mindre
3065	3C	L3065: INC	A	NZ
3066	18 DD	JR	L3045	

STRCPN: jämför BC tecken, DE = a\$, HL = b\$. Lika: RET med Z.

Olika: returadressen tas bort och STRCMP slutar med flaggorna från CP.

3068	78	STRCPN: LD	A, B	
3069	B1	OR	C	
306A	C8	RET	Z	
306B	1A	LD	A, (DE)	
306C	BE	CP	(HL)	
306D	13	INC	DE	
306E	23	INC	HL	
306F	0B	DEC	BC	
3070	28 F6	JR	Z, STRCPN	
3072	E1	POP	HL	olika: ta bort returadressen
3073	18 D0	JR	L3045	

3.86 EXINST: INSTR(i, a\$, b\$)

\$3075

Ger positionen (1 = första tecknet) där b\$ finns i a\$, sökt från tecken i. 0 om b\$ inte finns, om i = 0, eller om i är större än längden + 1. Första tecknet i b\$ söks med CPIR, resten jämförs med CPI. Hittas det inte går sökningen vidare efter det första tecknet. Med tom b\$ hoppar koden till \$30DB med HL = antal tecken kvar i a\$ i stället för en adress, så resultatet blir (längd - i + 1) minus a\$:s adress. Det är ett fel i ROM:en: INSTR(1,A\$, "") ger ett tal som beror på var A\$ ligger i minnet, inte 1.

3075	DD E1	EXINST: POP	IX	b\$:s n
3077	E1	POP	HL	b\$:s adress
3078	C1	POP	BC	b\$:s längd
3079	DD 39	ADD	IX, SP	IX = a\$:s beskrivning
307B	D5	PUSH	DE	koden
307C	E5	PUSH	HL	
307D	C5	PUSH	BC	
307E	DD 4E 00	LD	C, (IX+\$00)	a\$:s n
3081	DD 46 01	LD	B, (IX+\$01)	
3084	DD 6E 02	LD	L, (IX+\$02)	a\$:s adress
3087	DD 66 03	LD	H, (IX+\$03)	
308A	DD 5E 04	LD	E, (IX+\$04)	a\$:s längd
308D	DD 56 05	LD	D, (IX+\$05)	
3090	DD E5	PUSH	IX	
3092	DD 09	ADD	IX, BC	hoppa över a\$:s bytes
3094	DD 4E 06	LD	C, (IX+\$06)	i
3097	DD 46 07	LD	B, (IX+\$07)	

309A	DD E1	POP	IX	
309C	0B	DEC	BC	i - 1
309D	09	ADD	HL, BC	sökningens början
309E	EB	EX	DE, HL	
309F	ED 42	SBC	HL, BC	tecken kvar = längd - (i - 1)
30A1	38 4E	JR	C, L30F1	i för stort: 0
30A3	C1	POP	BC	b\$:s längd
30A4	78	LD	A, B	
30A5	B1	OR	C	
30A6	28 33	JR	Z, L30DB	tom b\$: fel resultat, se blocket
30A8	C5	PUSH	BC	
30A9	ED 42	SBC	HL, BC	- b\$:s längd
30AB	38 44	JR	C, L30F1	b\$ längre än resten: 0
30AD	23	INC	HL	antal möjliga startlägen
30AE	44	LD	B, H	
30AF	4D	LD	C, L	
30B0	E1	POP	HL	
30B1	E3	EX	(SP), HL	HL = b\$:s adress
30B2	EB	EX	DE, HL	DE = b\$, HL = a\$
30B3	78	L30B3:	LD A, B	inga lägen kvar: 0
30B4	B1	OR	C	
30B5	28 3B	JR	Z, L30F2	
30B7	1A	LD	A, (DE)	b\$:s första tecken
30B8	ED B1	CPIR		sök det
30BA	20 36	JR	NZ, L30F2	finns inte: 0
30BC	C5	PUSH	BC	
30BD	D5	PUSH	DE	
30BE	E5	PUSH	HL	
30BF	21 06 00	LD	HL, \$0006	
30C2	39	ADD	HL, SP	
30C3	4E	LD	C, (HL)	b\$:s längd
30C4	23	INC	HL	
30C5	46	LD	B, (HL)	
30C6	0B	DEC	BC	resten av b\$
30C7	78	LD	A, B	
30C8	B1	OR	C	
30C9	28 0B	JR	Z, L30D6	ett tecken: hittat
30CB	E1	POP	HL	
30CC	E5	PUSH	HL	
30CD	13	L30CD:	INC DE	
30CE	1A	LD	A, (DE)	
30CF	ED A1	CPI		jämför nästa tecken
30D1	20 03	JR	NZ, L30D6	
30D3	EA CD 30	JP	PE, L30CD	
30D6	E1	L30D6:	POP HL	
30D7	D1	POP	DE	
30D8	C1	POP	BC	
30D9	20 D8	JR	NZ, L30B3	olika: sök vidare
30DB	DD 5E 02	L30DB:	LD E, (IX+\$02)	a\$:s adress
30DE	DD 56 03	LD	D, (IX+\$03)	
30E1	ED 52	SBC	HL, DE	position = efter första tecknet - a\$
30E3	D1	L30E3:	POP DE	b\$:s längd
30E4	D1	POP	DE	koden
30E5	DD F9	LD	SP, IX	a\$:s beskrivning
30E7	DD E1	POP	IX	a\$:s n
30E9	C1	POP	BC	
30EA	C1	POP	BC	
30EB	DD 39	ADD	IX, SP	
30ED	DD F9	LD	SP, IX	ta bort a\$ och b\$
30EF	C1	POP	BC	ta bort i
30F0	EF	RST	\$28	
30F1	E1	L30F1:	POP HL	
30F2	21 00 00	L30F2:	LD HL, \$0000	0
30F5	18 EC	JR	L30E3	

EXSPC: SPACE\$(n). EXSTRG: STRING\$(n,c): n tecken med koden c. STRFIL (\$30FE): BC = längd, A = tecknet. Strängen byggs på stacken och blir tillfällig (n = längden). Fel 3 om den inte får plats.

30F7	3E 20	EXSPC:	LD	A, \$20	mellanslag
30F9	18 02		JR	L30FD	
30FB	E1	EXSTRG:	POP	HL	c
30FC	7D		LD	A, L	
30FD	C1	L30FD:	POP	BC	n
30FE	21 00 00	STRFIL:	LD	HL, \$0000	
3101	39		ADD	HL, SP	
3102	ED 42		SBC	HL, BC	SP - n
3104	DA FF 06		JP	C, L06FF	fel 3
3107	F5		PUSH	AF	
3108	CD 9B 33		CALL	ROOM	minst 100 bytes över HEAP?
310B	F1		POP	AF	
310C	F9		LD	SP, HL	plats för strängen
310D	C5		PUSH	BC	längd
310E	E5		PUSH	HL	adress
310F	C5		PUSH	BC	n = längd
3110	F5		PUSH	AF	
3111	78		LD	A, B	
3112	B1		OR	C	
3113	28 0F		JR	Z, L3124	tom sträng
3115	F1		POP	AF	
3116	77		LD	(HL), A	första tecknet
3117	0B		DEC	BC	
3118	78		LD	A, B	
3119	B1		OR	C	
311A	28 07		JR	Z, L3123	
311C	D5		PUSH	DE	
311D	54		LD	D, H	
311E	5D		LD	E, L	
311F	13		INC	DE	
3120	ED B0		LDIR		füll resten
3122	D1		POP	DE	
3123	F7	L3123:	RST	\$30	
3124	C1	L3124:	POP	BC	
3125	F7		RST	\$30	

EXTAB: TAB(n) ger en sträng med mellanslag från aktuell kolumn (GETPOS) fram till kolumn n, tom om man redan är där eller längre. Bara den låga byten av skillnaden används.

3126	D5	EXTAB:	PUSH	DE	
3127	CD EF 08		CALL	GETPOS	A = kolumnen
312A	D1		POP	DE	
312B	E1		POP	HL	n
312C	4F		LD	C, A	
312D	06 00		LD	B, \$00	
312F	A7		AND	A	
3130	ED 42		SBC	HL, BC	n - kolumnen
3132	4D		LD	C, L	
3133	3E 20		LD	A, \$20	
3135	30 C7		JR	NC, STRFIL	
3137	0E 00		LD	C, \$00	redan förbi: tom
3139	18 C3		JR	STRFIL	

EXCUR: CUR(r,k) ger en sträng på 4 bytes: ESC "=" r+32 k+32, som flyttar markören när den skrivs ut (se \$0202). Det är samma markörstyrning som terminalen ADM-3A använder, ett arv från terminalerna (noterat i litteraturen om ABC80). \$3D1B är bara konstanten ESC "=" (bytes \$1B \$3D).

313B	C1	EXCUR:	POP	BC	k
313C	E1		POP	HL	r
313D	61		LD	H, C	H = k, L = r
313E	01 20 20		LD	BC, \$2020	
3141	09		ADD	HL, BC	+ 32
3142	E5		PUSH	HL	
3143	21 1B 3D		LD	HL, \$3D1B	ESC "="
3146	E5		PUSH	HL	
3147	21 00 00		LD	HL, \$0000	
314A	39		ADD	HL, SP	strängen ligger på SP

314B	01 04 00	LD	BC,\$0004	
314E	C5	PUSH	BC	längd 4
314F	E5	PUSH	HL	adress
3150	C5	PUSH	BC	n = 4
3151	F7	RST	\$30	

EXNUM: NUM\$(x). Talet skrivs med **FLTAS6** (6 siffror, som **PRINT**) i en buffert på 15 bytes under x. Resultatet blir en tillfällig sträng med n = 20 (bufferten + x:s 5 bytes).

3152	21 F1 FF	EXNUM:	LD	HL,\$FFF1	15 bytes buffert
3155	39		ADD	HL,SP	
3156	CD 9B 33		CALL	ROOM	minst 100 bytes över HEAP?
3159	F9		LD	SP,HL	
315A	D5		PUSH	DE	koden
315B	EB		EX	DE,HL	
315C	21 13 00		LD	HL,\$0013	
315F	19		ADD	HL,DE	HL = x:s exponent
3160	D5		PUSH	DE	
3161	CD 77 16		CALL	FLTAS6	skriv talet
3164	E1		POP	HL	buffertens början
3165	7B		LD	A,E	längd = slut - början
3166	95		SUB	L	
3167	4F		LD	C,A	
3168	06 00		LD	B,\$00	
316A	D1		POP	DE	koden
316B	C5		PUSH	BC	längd
316C	E5		PUSH	HL	adress
316D	01 14 00		LD	BC,\$0014	n = 15 + 5
3170	C5		PUSH	BC	
3171	F7		RST	\$30	

3.87 EXVAL: VAL(s\$)

\$3172

Texten kopieras till en buffert på 80 bytes på stacken, avslutas med CR och läses med **GETSFL** ("+" eller "-" först är tillåtet).

längd 0, eller inget tal	-> fel 12
längd 78 eller mer	-> fel 3
något annat än blanka efter talet	-> fel 12 ("12X")

Blanka före, efter och inne i talet hoppas över (**GETFLT**): VAL(" 1 2 ") = 12. RST \$20 på \$31B3 läser från HL och gör ingenting som används; blanka efter talet har **GETSFL** redan hoppat över.

Resultatet skrivs i de översta 5 byten av strängens plats på stacken (beskrivningen + n bytes, minst 6), och allt under tas bort.

Stacken: IX = bufferten, IX+\$50 adress, IX+\$52 längd, IX+\$54... s\$:s tillfälliga bytes.

3172	C1	EXVAL:	POP	BC	n
3173	21 B0 FF		LD	HL,\$FFB0	80 bytes buffert
3176	39		ADD	HL,SP	
3177	CD 9B 33		CALL	ROOM	minst 100 bytes över HEAP?
317A	F9		LD	SP,HL	
317B	D5		PUSH	DE	koden
317C	C5		PUSH	BC	
317D	E5		PUSH	HL	IX = bufferten
317E	DD E1		POP	IX	
3180	EB		EX	DE,HL	DE = bufferten
3181	DD 7E 53		LD	A,(IX+\$53)	längd, hög byte
3184	A7		AND	A	
3185	20 08		JR	NZ,L318F	>= 256: fel 3
3187	DD B6 52		OR	(IX+\$52)	längd, låg byte
318A	CA 13 07		JP	Z,L0713	tom: fel 12
318D	FE 4E		CP	\$4E	>= 78?
318F	D2 FF 06	L318F:	JP	NC,L06FF	fel 3
3192	4F		LD	C,A	
3193	06 00		LD	B,\$00	
3195	DD 6E 50		LD	L,(IX+\$50)	adress
3198	DD 66 51		LD	H,(IX+\$51)	

319B	ED B0	LDIR	kopiera texten
319D	3E 0D	LD A,\$0D	
319F	12	LD (DE),A	CR sist
31A0	7B	LD A,E	CR:s adress, låg byte
31A1	DD E5	PUSH IX	
31A3	D1	POP DE	
31A4	E1	POP HL	n
31A5	19	ADD HL,DE	
31A6	01 53 00	LD BC,\$0053	
31A9	09	ADD HL,BC	översta byten i s\$:s plats
31AA	E5	PUSH HL	= resultatets exponent
31AB	F5	PUSH AF	
31AC	CD FB 14	CALL GETSFL	läs talet
31AF	DA 13 07	JP C,L0713	inget tal: fel 12
31B2	1B	DEC DE	
31B3	E7	RST \$20	verkningslös: HL används inte sedan
31B4	F1	POP AF	
31B5	BB	CP E	slutade talet vid CR?
31B6	C2 13 07	JP NZ,L0713	nej: fel 12
31B9	D1	POP DE	resultatets exponent
31BA	21 FC FF	LD HL,\$FFFC	
31BD	19	ADD HL,DE	resultatets början
31BE	D1	POP DE	koden
31BF	F9	LD SP,HL	ta bort resten
31C0	F7	RST \$30	

EXINP: INP(*p*). Först skrivs det sparade värdet för port 1 (\$FE2B, från OUT 1,*v*) till port 1, sedan läses port *p*. OUT gör likadant (OUTST). Port 1 väljer enhet på ABC-bussen (så även i abc80sim).

31C1	C1	EXINP: POP BC	p
31C2	FD 7E 15	LD A,(IY+\$15)	sparat värde
31C5	D3 01	OUT (\$01),A	till port 1
31C7	ED 68	IN L,(C)	läs port p
31C9	26 00	LD H,\$00	
31CB	EF	RST \$28	

EXCALL: CALL(*a*,*d*) anropar maskinkod på adress *a* med DE = *d* (0 om *d* saknas). HL vid återhoppet blir funktionens värde (heltal).

31CC	D5	EXCALL: PUSH DE	
31CD	1B	DEC DE	
31CE	1B	DEC DE	
31CF	1A	LD A,(DE)	token \$CD + antal argument
31D0	FE CF	CP \$CF	två argument?
31D2	C1	POP BC	koden
31D3	11 00 00	LD DE,\$0000	
31D6	20 01	JR NZ,L31D9	
31D8	D1	POP DE	d
31D9	E1	L31D9: POP HL	a
31DA	C5	PUSH BC	
31DB	CD 52 28	CALL JPHL	JP (HL)
31DE	D1	POP DE	koden
31DF	EF	RST \$28	resultatet i HL

3.88 EXRND: RND och RND(x)

\$31E0

Argumentet x används inte; dess 5 bytes blir en del av arbetsytan (13 bytes reserveras med argument, 18 utan, så ytan är alltid 18 bytes). Slumptalsfröet RNDSED (\$FE39-\$FE3D) är ett heltal på 40 bitar med den minst signifikanta byten på \$FE3D.

1. Nytt frö (linjär kongruens, modulo 2^{40}):

$$\text{frö} = \text{frö} * \$F73B40E64D + \$00A205B065$$

Multiplikationen görs bit för bit. Faktorns bytes tas från RNDMUL (\$32E0) och nedåt, lägsta biten först. Är biten satt adderas fröet till summan på IX+0.4, och fröet skiftas ett steg åt vänster (på sin plats; det är 0 efteråt). Efter varje byte blir additionen en byte kortare, eftersom fröets lägsta byte då är 0. Faktorns högsta byte, \$F7, är RST \$30 på \$32DC, den sista instruktionen i rutinen. Att den används visar följden: från fröet 0 ger RND .079112, .106671, .452764 ..., som med faktorn \$F73B40E64D (med \$3B40E64D blir andra talet .731671).

Sedan skrivs summan + konstanten (RNDADD, \$32E5 och nedåt) tillbaka till fröet.

2. De 35 lägsta bitarna av fröet blir ett decimalt bråk 0.ddddddd: för varje bit, lägsta först, blir bråket (bråk + bit) / 2, räknat i BCD: skift höger, +5 i översta siffran för en inskiftad bit, och -3 för en siffra som fick bit 3 från siffran över. Siffrorna efter den sjätte faller bort (avhuggning, inte avrundning: .452764 där bråket är .4527646...).
3. Bråket normaliseras (inledande nollor bort, exponenten minskas) och blir ett flyttal $0 \leq x < 1$. Om alla sex siffror är 0 (bråket under 0.000001, ungefär en gång på en miljon) slutar normaliseringen efter tre hela nollbytes med exponenten \$7A i stället för 0. Talet skrivs ut som 0 men är ingen riktig nolla: X=0 är falskt, X>0 sant, SGN(X) = 1 och X<1E-100 falskt.

Arbetsytan: IX+0.4 summan, IX+\$0A/\$0B pekare till fröet, IX+\$0C antal bytes kvar i faktorn, IX+\$0D..\$11 resultatet.

31E0	1B	EXRND:	DEC	DE	
31E1	1B		DEC	DE	
31E2	1A		LD	A, (DE)	token före funktionskoden
31E3	13		INC	DE	
31E4	13		INC	DE	
31E5	DD 21 EE FF		LD	IX, \$FFEE	18 bytes
31E9	FE CD		CP	\$CD	RND utan argument?
31EB	28 04		JR	Z, L31F1	
31ED	DD 21 F3 FF		LD	IX, \$FFF3	13 bytes + x:s 5
31F1	DD 39	L31F1:	ADD	IX, SP	
31F3	DD F9		LD	SP, IX	arbetsytan
31F5	D5		PUSH	DE	koden
31F6	DD E5		PUSH	IX	
31F8	E1		POP	HL	
31F9	01 00 12		LD	BC, \$1200	18 bytes, C = 0
31FC	71	L31FC:	LD	(HL), C	nollställ
31FD	23		INC	HL	
31FE	10 FC		DJNZ	L31FC	
3200	11 27 00		LD	DE, \$0027	IY + \$27 = \$FE3D
3203	FD E5		PUSH	IY	
3205	E1		POP	HL	
3206	19		ADD	HL, DE	
3207	EB		EX	DE, HL	fröets lägsta byte
3208	DD 73 0A		LD	(IX+\$0A), E	pekare till fröet
320B	DD 72 0B		LD	(IX+\$0B), D	
320E	21 E0 32		LD	HL, RNDMUL	faktorns lägsta byte
3211	DD 36 0C 05		LD	(IX+\$0C), \$05	5 bytes
3215	0E 08	L3215:	LD	C, \$08	8 bitar
3217	7E		LD	A, (HL)	nästa byte i faktorn
3218	E5		PUSH	HL	
3219	D5	L3219:	PUSH	DE	
321A	0F		RRCA		nästa bit
321B	F5		PUSH	AF	
321C	30 1A		JR	NC, L3238	0: ingen addition
321E	DD E5		PUSH	IX	
3220	26 00		LD	H, \$00	
3222	DD 6E 0C		LD	L, (IX+\$0C)	antal bytes
3225	45		LD	B, L	
3226	EB		EX	DE, HL	
3227	DD 19		ADD	IX, DE	IX = summan + antal

3229	7E	L3229:	LD A, (HL)	summa += frö
322A	DD 8E FF		ADC A, (IX-\$01)	
322D	DD 77 FF		LD (IX-\$01), A	
3230	2B		DEC HL	
3231	DD 2B		DEC IX	
3233	10 F4		DJNZ L3229	
3235	DD E1		POP IX	
3237	AF		XOR A	CY = 0
3238	DD 6E 0A	L3238:	LD L, (IX+\$0A)	fröet
323B	DD 66 0B		LD H, (IX+\$0B)	
323E	06 05		LD B, \$05	
3240	CB 16	L3240:	RL (HL)	frö <= 1
3242	2B		DEC HL	
3243	10 FB		DJNZ L3240	
3245	F1		POP AF	
3246	D1		POP DE	
3247	0D		DEC C	
3248	20 CF		JR NZ, L3219	
324A	E1		POP HL	
324B	1B		DEC DE	nästa byte i fröet
324C	2B		DEC HL	nästa byte i faktorn
324D	DD 35 0C		DEC (IX+\$0C)	en byte kortare
3250	20 C3		JR NZ, L3215	
3252	06 05		LD B, \$05	5 bytes
3254	11 E5 32		LD DE, RNDADD	konstantens lägsta byte
3257	DD 6E 0A		LD L, (IX+\$0A)	fröet
325A	DD 66 0B		LD H, (IX+\$0B)	
325D	AF		XOR A	
325E	DD E5		PUSH IX	
3260	1A	L3260:	LD A, (DE)	frö = summa + konstant
3261	DD 8E 04		ADC A, (IX+\$04)	
3264	77		LD (HL), A	
3265	2B		DEC HL	
3266	1B		DEC DE	
3267	DD 2B		DEC IX	
3269	10 F5		DJNZ L3260	
326B	DD E1		POP IX	
326D	FD E5		PUSH IY	
326F	11 0D 00		LD DE, \$000D	
3272	DD 19		ADD IX, DE	IX = resultatet
3274	DD E3		EX (SP), IX	IX = IY, resultatet på stacken
3276	1E 23		LD E, \$23	35 bitar
3278	DD 56 27	L3278:	LD D, (IX+\$27)	nästa byte i fröet
327B	CB 1A	L327B:	RR D	nästa bit, lägsta först
327D	E1		POP HL	resultatets mantissa
327E	E5		PUSH HL	
327F	06 03		LD B, \$03	3 bytes
3281	7E	L3281:	LD A, (HL)	
3282	30 02		JR NC, L3286	ingen bit in
3284	C6 A0		ADD A, \$A0	+5 i översta siffran efter RRA
3286	1F	L3286:	RRA	halvera
3287	CB 11		RL C	spara utskiftad bit
3289	CB 5F		BIT 3, A	fick låga siffran bit 3?
328B	28 02		JR Z, L328F	
328D	C6 FD		ADD A, \$FD	-3
328F	CB 19	L328F:	RR C	biten tillbaka i CY
3291	77		LD (HL), A	
3292	23		INC HL	
3293	10 EC		DJNZ L3281	
3295	1D		DEC E	alla bitar?
3296	28 0A		JR Z, L32A2	
3298	0C		INC C	bitar i byten
3299	79		LD A, C	
329A	E6 07		AND \$07	
329C	20 DD		JR NZ, L327B	
329E	DD 2B		DEC IX	nästa byte i fröet
32A0	18 D6		JR L3278	
32A2	E1	L32A2:	POP HL	mantissan

32A3	0E 80		LD	C, \$80	exponent 0 (\$80)
32A5	7E	L32A5:	LD	A, (HL)	
32A6	A7		AND	A	
32A7	28 13		JR	Z, L32BC	hel byte 0
32A9	E6 F0		AND	\$F0	första siffran 0?
32AB	20 26		JR	NZ, L32D3	nej: klart
32AD	11 02 00		LD	DE, \$0002	
32B0	06 03		LD	B, \$03	
32B2	19		ADD	HL, DE	
32B3	ED 6F	L32B3:	RLD		skifta en siffra
32B5	2B		DEC	HL	
32B6	10 FB		DJNZ	L32B3	
32B8	0D		DEC	C	exponent - 1
32B9	23		INC	HL	
32BA	18 E9		JR	L32A5	
32BC	79	L32BC:	LD	A, C	
32BD	FE 7A		CP	\$7A	tre nollbytes: exponent \$7A, ingen riktig nolla
32BF	28 12		JR	Z, L32D3	
32C1	11 02 00		LD	DE, \$0002	
32C4	06 03		LD	B, \$03	
32C6	19		ADD	HL, DE	
32C7	AF		XOR	A	
32C8	5E	L32C8:	LD	E, (HL)	skifta en byte
32C9	77		LD	(HL), A	
32CA	7B		LD	A, E	
32CB	2B		DEC	HL	
32CC	10 FA		DJNZ	L32C8	
32CE	0D		DEC	C	exponent - 2
32CF	0D		DEC	C	
32D0	23		INC	HL	
32D1	18 D2		JR	L32A5	
32D3	11 04 00	L32D3:	LD	DE, \$0004	
32D6	EB		EX	DE, HL	
32D7	19		ADD	HL, DE	
32D8	71		LD	(HL), C	exponenten
32D9	EB		EX	DE, HL	
32DA	D1		POP	DE	koden
32DB	F9		LD	SP, HL	resultatet överst
32DC	F7		RST	\$30	
32DD	3B 40 E6		DEFB	\$3B, \$40, \$E6	faktorns byte 3-1; byte 4 = \$F7 på \$32DC
32E0	4D	RNDMUL:	DEFB	\$4D	RNDMUL: lägsta byten i \$F73B40E64D
32E1	00 A2 05 B0		DEFB	\$00, \$A2, \$05, \$B0	konstanten \$00A205B065, baklänges
32E5	65	RNDADD:	DEFB	\$65	

DIM1: dimensionera en vektor med högsta index DE (IX = posten). Finns den redan är det bara tillåtet att minska (annars fel 0), och inget händer: de gamla elementen går fortfarande att nå. Annars reserveras (DE+1) * elementstorlek bytes, nollställda.

32E6	DD 6E 02	DIM1:	LD	L, (IX+\$02)	
32E9	DD 66 03		LD	H, (IX+\$03)	
32EC	7D		LD	A, L	
32ED	B4		OR	H	
32EE	28 05		JR	Z, L32F5	inte dimensionerad än
32F0	ED 52		SBC	HL, DE	
32F2	38 50		JR	C, L3344	större: fel 0
32F4	C9		RET		
32F5	D5	L32F5:	PUSH	DE	
32F6	13		INC	DE	antal element
32F7	DD 46 04		LD	B, (IX+\$04)	B = elementets storlek
32FA	19	L32FA:	ADD	HL, DE	(DE+1) * storleken
32FB	38 24		JR	C, L3321	spill: fel 3
32FD	10 FB		DJNZ	L32FA	
32FF	EB		EX	DE, HL	
3300	CD 74 33		CALL	ALLOC	reservera, nollställt
3303	D1		POP	DE	
3304	DD 73 02		LD	(IX+\$02), E	
3307	DD 72 03		LD	(IX+\$03), D	

```

330A DD 75 00      LD  (IX+$00),L
330D DD 74 01      LD  (IX+$01),H
3310 C9            RET

```

DIM2: två index, DE = första, HL = andra. Antal element = (DE+1)*(HL+1); för stort ger fel 3, liksom index -1. Sedan **DIM1** och +6..+9. **DIM1** jämför bara antalet element, och +6..+9 skrivs även när fältet redan finns: **DIM A(3,2)** efter **DIM A(2,3)** (12 element båda) ger inget fel utan byter formen, och elementen ligger kvar radvis (A(1,2) blir A(2,0)). **DIM A(1,1)** krymper formen på samma sätt.

```

3311 E5      DIM2:  PUSH HL
3312 D5              PUSH DE
3313 23              INC  HL
3314 13              INC  DE
3315 7D              LD   A,L
3316 B4              OR   H
3317 28 02          JR   Z,L331B      index -1: fel 3
3319 7B              LD   A,E
331A B2              OR   D
331B CA FF 06      L331B: JP   Z,L06FF      fel 3
331E CD 74 37      CALL IMUL      (DE+1) * (HL+1)
3321 DA FF 06      L3321: JP   C,L06FF
3324 2B              DEC  HL      DE = antal - 1
3325 EB              EX   DE,HL
3326 CD E6 32      CALL DIM1
3329 D1              POP  DE
332A E1              POP  HL
332B DD 75 08      LD   (IX+$08),L      formen, även när fältet fanns
332E DD 74 09      LD   (IX+$09),H
3331 DD 73 06      LD   (IX+$06),E
3334 DD 72 07      LD   (IX+$07),D
3337 C9            RET

```

DIMSTR: reservera DE bytes för strängen (IX = posten). Finns den redan får den bara minska (annars fel 0). **DIM A\$=0** reserverar ingenting och lämnar +0/+1 = 0: strängen räknas som odimensionerad och får plats efter första tilldelningen. Minnesbrist ger fel 3.

```

3338 DD 6E 00      DIMSTR: LD   L,(IX+$00)
333B DD 66 01      LD   H,(IX+$01)
333E 7D              LD   A,L
333F B4              OR   H
3340 28 06          JR   Z,L3348
3342 ED 52          SBC  HL,DE
3344 DA F9 06      L3344: JP   C,L06F9      större: fel 0
3347 C9            RET

3348 CD 5A 33      L3348: CALL MEMRES      reservera
334B 38 D4          JR   C,L3321      ryms inte: fel 3
334D DD 73 00      LD   (IX+$00),E
3350 DD 72 01      LD   (IX+$01),D
3353 DD 75 02      LD   (IX+$02),L
3356 DD 74 03      LD   (IX+$03),H
3359 C9            RET

```

MEMRES: reservera DE bytes på heapen. **HEAP** + DE + 100 måste vara under SP, annars CY = 1. Ut: HL = det reserverade området, och **HEAP** har flyttats fram.

ALLOC: samma sak men området nollställs. Minnesbrist ger fel 3. **STKCHK:** fel 3 om SP före anropet ligger högst 100 bytes över **HEAP** (94 + returadressen och de två PUSH), samma marginal som i **MEMRES** och **ROOM**.

```

335A 2A 20 FE      MEMRES: LD   HL,(HEAP)
335D 19              ADD  HL,DE
335E D8              RET   C
335F D5              PUSH DE
3360 11 64 00      LD   DE,$0064
3363 19              ADD  HL,DE      minst 100 bytes marginal mot stacken
3364 D1              POP  DE
3365 D8              RET   C
3366 ED 72          SBC  HL,SP
3368 3F              CCF

```

```

3369 D8          RET C
336A 2A 20 FE    LD HL, (HEAP)
336D E5          PUSH HL
336E 19          ADD HL, DE
336F 22 20 FE    LD (HEAP), HL
3372 E1          POP HL
3373 C9          RET

3374 CD 5A 33    ALLOC: CALL MEMRES
3377 38 1F       JR C, MEMFUL
3379 E5          PUSH HL
337A D5          PUSH DE
337B C5          PUSH BC
337C 42          LD B, D
337D 4B          LD C, E
337E 54          LD D, H
337F 5D          LD E, L
3380 0B          DEC BC          BC = antal - 1
3381 13          INC DE
3382 36 00       LD (HL), $00    första byten, LDIR sprider nollan
3384 ED B0       LDIR
3386 C1          POP BC
3387 18 0C       JR L3395

3389 E5          STKCHK: PUSH HL
338A D5          PUSH DE
338B 2A 20 FE    LD HL, (HEAP)
338E 11 5E 00    LD DE, $005E    94 + 6 bytes på stacken = 100
3391 19          ADD HL, DE
3392 ED 72       SBC HL, SP
3394 3F          CCF
3395 D1          L3395: POP DE
3396 E1          POP HL
3397 D0          RET NC
3398 C3 FF 06    MEMFUL: JP L06FF

```

ROOM: fel 3 om HL (den nya SP) ligger mindre än 100 bytes över HEAP. Används innan en sträng eller buffert byggs på stacken.

```

339B E5          ROOM: PUSH HL
339C D5          PUSH DE
339D 11 9C FF    LD DE, $FF9C
33A0 19          ADD HL, DE      HL - 100
33A1 7D          LD A, L
33A2 FD 96 0A    SUB (IY+$0A)    under HEAP?
33A5 7C          LD A, H
33A6 FD 9E 0B    SBC A, (IY+$0B)
33A9 18 EA       JR L3395

```

3.89 FNPAR: adressen till en DEF FN-parameter

\$33AB

In: HL = \$00nn från koden (LNKEXP), nn = antalet parametrar efter den här. Ut: HL = adressen till motsvarande argument på stacken. Argumenten ligger på stacken med det sista överst, vid FNFRAM. Under FNFRAM ligger kodpekaren från FN, som pekar på avståndsbyten sist i anropet. Därifrån nås argumentposterna bakåt (se CMPFN):

IX+0 avstånd till förra posten (0 = den första)
IX+2 parameterns typ

ARGWLK hoppar över nn argument uppifrån, med storleken enligt typen: flyttal 5, heltal 2, sträng 6 + n. För ett flyttal pekar HL till sist på exponenten (+4), som för vanliga variabler.

FNRES använder ARGWLK med nn = \$FF för att hoppa över alla.

```

33AB D5          FNPAR: PUSH DE
33AC EB          EX DE, HL      E = nn
33AD 2A 3E FE    LD HL, (FNFRAM)  ramen
33B0 2B          ARGWLK: DEC HL
33B1 2B          DEC HL
33B2 CD 17 10    CALL RDBC      kodpekaren från FN
33B5 C5          PUSH BC      IX = avståndsbyten

```

33B6	DD E1	POP	IX	
33B8	7B	L33B8:	LD A,E	framme?
33B9	A7	AND	A	
33BA	06 FF	LD	B,\$FF	avstånden är negativa
33BC	DD 4E 00	LD	C,(IX+\$00)	
33BF	28 23	JR	Z,L33E4	
33C1	79	LD	A,C	
33C2	A7	AND	A	
33C3	28 30	JR	Z,L33F5	ingen post till: klart
33C5	DD 09	ADD	IX,BC	förra posten
33C7	1D	DEC	E	
33C8	DD 7E 02	LD	A,(IX+\$02)	parametrernas typ
33CB	FE 02	CP	\$02	
33CD	28 0D	JR	Z,L33DC	sträng
33CF	0E 02	LD	C,\$02	heltal: 2 bytes
33D1	FE 01	CP	\$01	
33D3	28 02	JR	Z,L33D7	
33D5	0E 05	LD	C,\$05	flyttal: 5 bytes
33D7	06 00	L33D7:	LD B,\$00	
33D9	09	ADD	HL,BC	hoppa över argumentet
33DA	18 DC	JR	L33B8	
33DC	4E	L33DC:	LD C,(HL)	n
33DD	23	INC	HL	
33DE	46	LD	B,(HL)	
33DF	09	ADD	HL,BC	+ n
33E0	0E 05	LD	C,\$05	+ 1 + 5 = 6 + n
33E2	18 F3	JR	L33D7	
33E4	79	L33E4:	LD A,C	
33E5	A7	AND	A	
33E6	28 0D	JR	Z,L33F5	inga argument
33E8	DD 09	ADD	IX,BC	argumentets post
33EA	DD 7E 02	LD	A,(IX+\$02)	
33ED	FE 00	CP	\$00	flyttal?
33EF	20 04	JR	NZ,L33F5	
33F1	01 04 00	LD	BC,\$0004	peka på exponenten
33F4	09	ADD	HL,BC	
33F5	D1	L33F5:	POP DE	
33F6	C9	RET		

3.90 Flyttalsaritmetik: FADD FSUB FMUL FDIV

\$33F7

Talformat, 5 bytes. Adressen pekar på exponenten e:

e-4..e-2 mantissan, 6 BCD-siffror d1..d6 (d1 högst i e-4)
e-1 tecken, 0 = +, 1 = -
e exponent med förskjutningen \$80; 0 = talet är noll
värdet = 0,d1d2d3d4d5d6 * 10^(e - \$80), normaliserat (d1 <> 0)
Exempel: 2 = \$20 \$00 \$00 \$00 \$81, PI = \$31 \$41 \$59 \$00 \$81

In: DE = BC = a (vänster), HL = b (höger). Resultatet skrivs över a (BC). HL, DE och BC bevaras.

FENTER (\$3766) sparar HL, DE och BC och ger IX = SP - \$27. Rutinen sätter sedan SP = IX, så att arbetsytan ligger ovanför stacken:

IX+0..4 **FDIV:** kvoten (mantissa, tecken, exponent)
IX+0..\$1F **FMUL:** a gånger 1, 2, 4, 8, 10, 20, 40, 80, 4 bytes var
IX+\$20 **FMUL:** räknare (annars 0)
IX+\$21..\$24 mantissan, med en extra byte (IX+\$21) överst
IX+\$25 tecken
IX+\$26 exponent
IX+\$27 **FADD:** <> 0 om förskjutningen är ett udda antal siffror
IX+\$28 **FADD:** byten med den första siffran som skiftas ut

FSTORE (\$3434) kopierar IX+\$22..\$26 till resultatet, tar bort arbetsytan och återställer registren. **FDIV** går in på **FSTOR1** med HL = 4, så att kvoten på IX+0.4 kopieras i stället.

FOVFL (\$3758): fel 4, FÖR STORT FLYTTAL (också för division med 0). **FZERO (\$375B)**: resultatet blir 0. Det går via **FMULE**, som nollställer mantissan och sätter exponenten till 0.

33F7	CD 66 37	FADD:	CALL FENTER	spara registren, IX = arbetsytan
33FA	DD F9		LD SP, IX	SP under arbetsytan
33FC	C5		PUSH BC	resultatets adress
33FD	CD BB 36		CALL FALIGN	DE = störst exponent
3400	0E 00		LD C, \$00	addition: tecknet byts inte

FADD / FSUB. FALIGN har låtit DE peka på det tal som har störst exponent. Det andra talet ligger i IX+\$22..\$25, skiftat åt höger med skillnaden i exponent (högst 6 siffror), och C = 1 om talen bytte plats. **FSUB** har bytt tecken på kopian. Har talen bytt plats (C = 1) byts också resultatets tecken, eftersom $b - a = -(a - b)$.

Den första siffran som skiftades ut avrundar: ADD A, \$B0 ger CY om den är ≥ 5 , och den CY:n går in i additionen (ADC) eller subtraktionen (SBC) som en etta i sista siffran. RRA/RLA tar bit 0 av B (olika tecken?) utan att tappa CY.

Samma tecken: **MADD3** adderar. Spill ger en etta i IX+\$21; mantissan skiftas en siffra åt höger och exponenten ökar (fel 4 vid \$FF).

Egenheter (provat):

- Skillnaden begränsas till 6, så ett hur litet tal som helst kan avrunda: $1 + 5E-51 = 1.00001$, men $1 + 4E-51 = 1$.
- Bara den mindre operanden avrundas: $1 - .999999 = 0$, eftersom $.999999$ blir 1.00000 före subtraktionen. $1 - 5E-6 = .99999$.

3402	1B	FADDC:	DEC DE	DE = teckenbyten
3403	EB		EX DE, HL	
3404	DD 7E 25		LD A, (IX+\$25)	kopians tecken
3407	AE		XOR (HL)	
3408	47		LD B, A	B bit 0 = olika tecken
3409	EB		EX DE, HL	
340A	1A		LD A, (DE)	tecknet för talet vid DE
340B	1B		DEC DE	DE = lägsta mantissabyten
340C	A9		XOR C	byt om talen bytte plats
340D	DD 77 25		LD (IX+\$25), A	resultatets tecken
3410	DD 7E 27		LD A, (IX+\$27)	udda förskjutning?
3413	B7		OR A	
3414	DD 7E 28		LD A, (IX+\$28)	byten med utskiftad siffra
3417	28 04		JR Z, L341D	
3419	07		RLCA	den låga halvan upp
341A	07		RLCA	
341B	07		RLCA	
341C	07		RLCA	
341D	C6 B0	L341D:	ADD A, \$B0	CY = första utskiftade siffran ≥ 5
341F	78		LD A, B	
3420	1F		RRA	CY = olika tecken, A bit 7 = avrundning
3421	38 50		JR C, FSUBM	olika tecken: subtrahera
3423	17		RLA	CY = avrundning
3424	CD 47 34		CALL MADD3	addera
3427	30 0B		JR NC, FSTORE	inget spill
3429	06 04		LD B, \$04	en siffra
342B	CD 14 37		CALL MSHR	skifta åt höger
342E	DD 34 26		INC (IX+\$26)	exponent + 1
3431	CA 58 37		JP Z, FOVFL	\$FF + 1: fel 4
3434	21 26 00	FSTORE:	LD HL, \$0026	mantissa, tecken, exponent vid IX+\$22..\$26
3437	D1	FSTOR1:	POP DE	resultatets adress
3438	39		ADD HL, SP	
3439	01 05 00		LD BC, \$0005	5 bytes
343C	ED B8		LDDR	
343E	21 29 00		LD HL, \$0029	förbi arbetsytan
3441	39		ADD HL, SP	
3442	F9		LD SP, HL	
3443	C1		POP BC	återställ registren
3444	D1		POP DE	
3445	E1		POP HL	
3446	C9		RET	

MADD3: IX+\$22..\$24 += de 3 byten vid DE (DE pekar på den lägsta), BCD med ADC och DAA. CY in läggs till i sista siffran. Spill går in i IX+\$21. Ut: CY = spill, DE 3 bytes lägre.

3447	DD E5	MADD3:	PUSH IX	
3449	06 03		LD B, \$03	3 bytes
344B	1A	L344B:	LD A, (DE)	
344C	DD 8E 24		ADC A, (IX+\$24)	+ CY

344F	27		DAA	
3450	DD 77 24		LD (IX+\$24), A	
3453	DD 2B		DEC IX	
3455	1B		DEC DE	
3456	10 F3		DJNZ L344B	
3458	30 03		JR NC, L345D	
345A	DD 34 24		INC (IX+\$24)	spill till IX+\$21
345D	DD E1	L345D:	POP IX	
345F	C9		RET	
3460	CD 66 37	FSUB:	CALL FENTER	spara registren, IX = arbetsytan
3463	DD F9		LD SP, IX	
3465	C5		PUSH BC	
3466	CD BB 36		CALL FALIGN	DE = störst exponent
3469	DD 7E 25		LD A, (IX+\$25)	byt tecken på kopian
346C	EE 01		XOR \$01	
346E	DD 77 25		LD (IX+\$25), A	
3471	18 8F		JR FADDC	C = 1 om talen bytte plats

FSUBM: olika tecken, subtrahera. Blir det lån var b större än a (bara möjligt vid samma exponent): tecknet byts och mantissan byts mot 0 - mantissan (tiokomplement; XOR nollställer CY först).

FNORM (\$3494): räkna inledande nollor i $IX+\$22..\24 (C = antal siffror). Är alla noll blir resultatet 0. Annars minskas exponenten med C (blir den ≤ 0 blir resultatet 0) och mantissan skiftas åt vänster. Ingen siffra skiftas in, så en avrundad siffra från den mindre operanden kommer aldrig tillbaka.

3473	17	FSUBM:	RLA	CY = avrundning
3474	CD CB 34		CALL MSUB3	subtrahera
3477	30 1B		JR NC, FNORM	inget lån
3479	DD 7E 25		LD A, (IX+\$25)	byt tecken
347C	EE 01		XOR \$01	
347E	DD 77 25		LD (IX+\$25), A	
3481	DD E5		PUSH IX	
3483	06 03		LD B, \$03	3 bytes
3485	3E 00	L3485:	LD A, \$00	0 - mantissan (XOR gav CY = 0)
3487	DD 9E 24		SBC A, (IX+\$24)	
348A	27		DAA	
348B	DD 77 24		LD (IX+\$24), A	
348E	DD 2B		DEC IX	
3490	10 F3		DJNZ L3485	
3492	DD E1		POP IX	
3494	DD E5	FNORM:	PUSH IX	
3496	01 00 03		LD BC, \$0300	B = 3 bytes, C = 0 nollor
3499	DD 7E 22	L3499:	LD A, (IX+\$22)	första siffran?
349C	B7		OR A	
349D	20 0E		JR NZ, L34AD	
349F	DD 23		INC IX	
34A1	0C		INC C	två nollor till
34A2	0C		INC C	
34A3	10 F4		DJNZ L3499	
34A5	DD E1		POP IX	
34A7	AF		XOR A	alla noll: resultatet 0
34A8	DD 77 26		LD (IX+\$26), A	
34AB	18 87		JR FSTORE	
34AD	DD E1	L34AD:	POP IX	
34AF	FE 10		CP \$10	första siffran också 0?
34B1	30 01		JR NC, L34B4	
34B3	0C		INC C	
34B4	DD 7E 26	L34B4:	LD A, (IX+\$26)	exponent - antal nollor
34B7	91		SUB C	
34B8	CA 5B 37		JP Z, FZERO	≤ 0 : resultatet 0
34BB	DA 5B 37		JP C, FZERO	
34BE	DD 77 26		LD (IX+\$26), A	
34C1	79		LD A, C	4 bitar per siffra
34C2	07		RLCA	
34C3	07		RLCA	
34C4	47		LD B, A	
34C5	CD 36 37		CALL MSHL	skifta bort nollorna
34C8	C3 34 34		JP FSTORE	

MSUB3: $IX + \$22..\$24 = \text{de 3 byten vid DE minus } IX + \$22..\$24, \text{BCD med SBC och DAA. CY in dras också av. Ut: CY = lån.}$

34CB	DD E5	MSUB3:	PUSH IX	
34CD	06 03		LD B, \$03	3 bytes
34CF	1A	L34CF:	LD A, (DE)	
34D0	DD 9E 24		SBC A, (IX+\$24)	- CY
34D3	27		DAA	
34D4	DD 77 24		LD (IX+\$24), A	
34D7	DD 2B		DEC IX	
34D9	1B		DEC DE	
34DA	10 F3		DJNZ L34CF	
34DC	DD E1		POP IX	
34DE	C9		RET	

FMUL: $\text{exponenten} = ea + eb - \80 (fel 4 om den blir $> \$FF$, 0 om den blir < 1). Är något tal 0 blir resultatet 0. Tecknet = XOR.

En tabell med a gånger 1, 2, 4, 8, 10, 20, 40 och 80 byggs på $IX + 0.. \$1F$, med BCD-dubbling (ADC A,A och DAA) och $10a = 8a + 2a$. b tas en byte (två siffror) i taget, den lägsta först. Varje bit i byten, den högsta först, motsvarar ett tabellvärde (80, 40 ... 2, 1), och **MADD3** lägger till det i $IX + \$21..\24 . Efter varje byte skiftas summan en byte åt höger (**MSHR**); den utskiftade byten finns kvar i D.

Efter tre bytes är mantissan klar (**FMRND**).

Exponenten prövas före multiplikationen, så fel 4 kommer också när produkten efter normaliseringen hade fått plats (provat: $.1E127 * .1E1 = \text{fel 4, fast } .1E127 \text{ finns}$). Åt andra hållet går det rätt: $.1E-127 * .1$ blir 0.

34DF	CD 66 37	FMUL:	CALL FENTER	spara registren, IX = arbetsytan
34E2	DD F9		LD SP, IX	
34E4	C5		PUSH BC	
34E5	7E		LD A, (HL)	eb
34E6	B7		OR A	
34E7	28 11		JR Z, FMULE	b = 0: resultatet 0
34E9	1A		LD A, (DE)	ea
34EA	B7		OR A	
34EB	28 0D		JR Z, FMULE	a = 0: resultatet 0
34ED	86		ADD A, (HL)	ea + eb
34EE	38 05		JR C, L34F5	
34F0	F2 5B 37		JP P, FZERO	< \$80: resultatet 0
34F3	18 03		JR L34F8	
34F5	FA 58 37	L34F5:	JP M, FOVFL	för stort: fel 4
34F8	D6 80	L34F8:	SUB \$80	- \$80
34FA	DD 77 26	FMULE:	LD (IX+\$26), A	exponenten
34FD	1B		DEC DE	
34FE	2B		DEC HL	
34FF	1A		LD A, (DE)	teckenbyten
3500	AE		XOR (HL)	tecknet = XOR
3501	2B		DEC HL	
3502	1B		DEC DE	
3503	DD 77 25		LD (IX+\$25), A	
3506	DD E5		PUSH IX	
3508	AF		XOR A	nollställ $IX + \$20..\24
3509	06 05		LD B, \$05	
350B	DD 77 24	L350B:	LD (IX+\$24), A	
350E	DD 2B		DEC IX	
3510	10 F9		DJNZ L350B	
3512	DD E1		POP IX	
3514	DD 7E 26		LD A, (IX+\$26)	exponent 0?
3517	B7		OR A	ja: resultatet 0
3518	CA 34 34		JP Z, FSTORE	
351B	01 03 00		LD BC, \$0003	3 bytes
351E	DD E5		PUSH IX	(SP) = b:s mantissa
3520	E3		EX (SP), HL	
3521	09		ADD HL, BC	HL = IX+3
3522	1A	L3522:	LD A, (DE)	a:s mantissa till $IX + 1..3$
3523	77		LD (HL), A	
3524	2B		DEC HL	
3525	1B		DEC DE	
3526	0D		DEC C	
3527	20 F9		JR NZ, L3522	
3529	71		LD (HL), C	$IX + 0 = 0$

352A	2B		DEC	HL	
352B	06 FA		LD	B, \$FA	-6: 2a, 4a, 8a
352D	11 04 00	L352D:	LD	DE, \$0004	4 bytes per tabellvärde
3530	4B		LD	C, E	
3531	19		ADD	HL, DE	
3532	EB		EX	DE, HL	
3533	19		ADD	HL, DE	DE = förra värdet, HL = nästa
3534	04		INC	B	
3535	F2 68 35		JP	P, L3568	färdig
3538	1A	L3538:	LD	A, (DE)	dubbla: ADC A,A och DAA
3539	8F		ADC	A, A	
353A	27		DAA		
353B	77		LD	(HL), A	
353C	1B		DEC	DE	
353D	2B		DEC	HL	
353E	0D		DEC	C	
353F	20 F7		JR	NZ, L3538	
3541	04		INC	B	
3542	20 E9		JR	NZ, L352D	
3544	23		INC	HL	
3545	EB		EX	DE, HL	
3546	21 14 00		LD	HL, \$0014	IX+\$10
3549	39		ADD	HL, SP	
354A	EB		EX	DE, HL	
354B	0E 04		LD	C, \$04	kopiera 8a till IX+\$10
354D	41		LD	B, C	
354E	7E	L354E:	LD	A, (HL)	
354F	12		LD	(DE), A	
3550	23		INC	HL	
3551	13		INC	DE	
3552	0D		DEC	C	
3553	20 F9		JR	NZ, L354E	
3555	21 0B 00		LD	HL, \$000B	2a
3558	39		ADD	HL, SP	
3559	1B		DEC	DE	
355A	1A	L355A:	LD	A, (DE)	10a = 8a + 2a
355B	8E		ADC	A, (HL)	
355C	27		DAA		
355D	12		LD	(DE), A	
355E	1B		DEC	DE	
355F	2B		DEC	HL	
3560	05		DEC	B	
3561	20 F7		JR	NZ, L355A	
3563	06 F9		LD	B, \$F9	-7: 20a, 40a, 80a
3565	EB		EX	DE, HL	
3566	18 C5		JR	L352D	
3568	EB	L3568:	EX	DE, HL	
3569	23		INC	HL	
356A	36 04		LD	(HL), \$04	IX+\$20: räknare, 3 bytes
356C	C1	FMLOOP:	POP	BC	BC = b:s mantissa
356D	DD 35 20		DEC	(IX+\$20)	alla bytes?
3570	28 2B		JR	Z, FMRND	
3572	0A		LD	A, (BC)	nästa byte i b, lägsta först
3573	0B		DEC	BC	
3574	21 21 00		LD	HL, \$0021	80a
3577	39		ADD	HL, SP	
3578	C5		PUSH	BC	
3579	EB	L3579:	EX	DE, HL	
357A	87	L357A:	ADD	A, A	nästa bit, högsta först
357B	38 08		JR	C, L3585	biten satt
357D	28 17		JR	Z, L3596	inga bitar kvar
357F	21 FC FF		LD	HL, \$FFFC	nästa tabellvärde
3582	19		ADD	HL, DE	
3583	18 F4		JR	L3579	
3585	4F	L3585:	LD	C, A	spara bitarna
3586	B7		OR	A	CY = 0
3587	CD 47 34		CALL	MADD3	summan += tabellvärdet

358A	1A		LD	A, (DE)	översta byten
358B	DD 86 21		ADD	A, (IX+\$21)	
358E	27		DAA		
358F	DD 77 21		LD	(IX+\$21), A	
3592	79		LD	A, C	bitarna
3593	1B		DEC	DE	nästa tabellvärde
3594	18 E4		JR	L357A	
3596	06 08	L3596:	LD	B, \$08	en byte
3598	CD 14 37		CALL	MSHR	skifta åt höger, D = utskiftad byte
359B	18 CF		JR	FML00P	

FMRND: avrunda med de två siffrorna i D. Är d1 <> 0 avrundar D:s höga siffra. Är d1 = 0 skiftas mantissan en siffra åt vänster (exponenten minskar) och A = D med halvorna bytta: den låga halvan (D:s höga siffra) fyller den tomma sista siffran, och den höga halvan (D:s låga siffra) avrundar. Avrundningen: >= \$50 ger INC A (+1), sedan läggs A:s låga halva till sista siffran. Spill ger mantissan .100000 och exponenten + 1. Provat: .111111 * .111111 = .123457E-1.

359D	DD 7E 22	FMRND:	LD	A, (IX+\$22)	d1
35A0	E6 F0		AND	\$F0	
35A2	28 05		JR	Z, L35A9	d1 = 0: normalisera
35A4	7A		LD	A, D	avrundning: D:s höga siffra
35A5	E6 F0		AND	\$F0	
35A7	18 10		JR	L35B9	
35A9	06 04	L35A9:	LD	B, \$04	
35AB	DD 35 26		DEC	(IX+\$26)	exponent - 1
35AE	CA 5B 37		JP	Z, FZERO	0: resultatet 0
35B1	CD 36 37		CALL	MSHL	skifta åt vänster
35B4	7A		LD	A, D	låg halva: D:s höga siffra, hög: den låga
35B5	0F		RRCA		
35B6	0F		RRCA		
35B7	0F		RRCA		
35B8	0F		RRCA		
35B9	FE 50	L35B9:	CP	\$50	avrunda uppåt?
35BB	38 26		JR	C, L35E3	
35BD	3C		INC	A	+1
35BE	E6 0F		AND	\$0F	siffran till sista platsen
35C0	06 03		LD	B, \$03	
35C2	DD E5		PUSH	IX	
35C4	DD 8E 24	L35C4:	ADC	A, (IX+\$24)	lägg till
35C7	27		DAA		
35C8	DD 77 24		LD	(IX+\$24), A	
35CB	3E 00		LD	A, \$00	
35CD	DD 2B		DEC	IX	
35CF	10 F3		DJNZ	L35C4	
35D1	DD E1		POP	IX	
35D3	D2 34 34		JP	NC, FSTORE	inget spill
35D6	DD 36 22 10		LD	(IX+\$22), \$10	mantissan .100000
35DA	DD 34 26		INC	(IX+\$26)	exponent + 1
35DD	C2 34 34		JP	NZ, FSTORE	
35E0	C3 58 37		JP	FOVFL	fel 4
35E3	E6 0F	L35E3:	AND	\$0F	bara siffran till sista platsen
35E5	DD 86 24		ADD	A, (IX+\$24)	
35E8	DD 77 24		LD	(IX+\$24), A	
35EB	C3 34 34		JP	FSTORE	

FDIV: division med 0 ger fel 4, a = 0 ger 0. Exponenten blir ea - eb + \$81 (fel 4 eller 0 om den hamnar utanför). a:s mantissa är resten i IX+\$21..\$24 (FCOPY), och tecknet = XOR läggs i IX+3. **FDLOOP** (\$3622): för varje siffra dras b från resten tills det blir lån. Då läggs b tillbaka (MADD3), och antalet lyckade subtraktioner är siffran. Resten skiftas en siffra åt vänster. Blir den första siffran 0 minskas exponenten och siffran räknas om.

L räknar siffrorna från 7 och ned. Siffrorna packas två och två i IX+0..2 (udda L: låg halva, jämnt L: föregående siffra flyttas upp). Den sjunde siffran avrundar (FDRND).

Egenhet (provat): exponenten provas bara före \$81 läggs till. Är ea - eb = \$7F blir den \$00 utan fel. Blir första siffran 0 räknas den ned till \$FF och kvoten blir rätt (1E126 / .2 = 5E126). Annars står exponenten kvar som 0 med mantissan ifylld: 1E126 / .1 ger ett tal som räknas som 0 (X = 0 är sant, SGN 0, X * 2 = 0) men skrivs ut som 1 (9E126 / .1 som 9), i st. f. fel 4. Likadant i 9913.

35EE	CD 66 37	FDIV:	CALL	FENTER	spara registren, IX = arbetsytan
------	----------	-------	------	--------	----------------------------------

35F1	DD F9		LD SP,IX	
35F3	C5		PUSH BC	
35F4	7E		LD A,(HL)	eb
35F5	B7		OR A	
35F6	CA 58 37		JP Z,FOVFL	division med 0: fel 4
35F9	1A		LD A,(DE)	ea
35FA	B7		OR A	
35FB	CA FA 34		JP Z,FMULE	a = 0: resultatet 0
35FE	96		SUB (HL)	ea - eb
35FF	38 05		JR C,L3606	
3601	FA 58 37		JP M,FOVFL	för stort: fel 4
3604	18 03		JR L3609	
3606	F2 5B 37	L3606:	JP P,FZERO	för litet: 0
3609	C6 81	L3609:	ADD A,\$81	+ \$81
360B	DD 77 04		LD (IX+\$04),A	kvotens exponent
360E	EB		EX DE,HL	
360F	D5		PUSH DE	
3610	CD FC 36		CALL FCOPY	a:s mantissa blir resten
3613	D1		POP DE	
3614	EB		EX DE,HL	
3615	DD 7E 25		LD A,(IX+\$25)	a:s tecken
3618	2B		DEC HL	
3619	AE		XOR (HL)	XOR b:s tecken
361A	DD 77 03		LD (IX+\$03),A	kvotens tecken
361D	EB		EX DE,HL	
361E	1B		DEC DE	DE = b:s lägsta byte
361F	DD E5		PUSH IX	
3621	C1		POP BC	BC = IX: kvotens första byte
3622	2E 07	FDLOOP:	LD L,\$07	7 siffror
3624	C5	L3624:	PUSH BC	
3625	E5		PUSH HL	
3626	0E 00		LD C,\$00	C = siffran
3628	A7	L3628:	AND A	CY = 0
3629	DD E5		PUSH IX	
362B	06 03		LD B,\$03	
362D	DD 7E 24	L362D:	LD A,(IX+\$24)	resten - b
3630	EB		EX DE,HL	
3631	9E		SBC A,(HL)	
3632	EB		EX DE,HL	
3633	27		DAA	
3634	DD 77 24		LD (IX+\$24),A	
3637	DD 2B		DEC IX	
3639	1B		DEC DE	
363A	10 F1		DJNZ L362D	
363C	DD E1		POP IX	
363E	DD 7E 21		LD A,(IX+\$21)	lån från översta byten
3641	DE 00		SBC A,\$00	
3643	DD 77 21		LD (IX+\$21),A	
3646	1F		RRA	spara CY
3647	21 03 00		LD HL,\$0003	DE tillbaka
364A	19		ADD HL,DE	
364B	EB		EX DE,HL	
364C	0C		INC C	en subtraktion till
364D	17		RLA	CY = lån
364E	30 D8		JR NC,L3628	inget lån: igen
3650	B7		OR A	CY = 0
3651	CD 47 34		CALL MADD3	lägg tillbaka b
3654	21 03 00		LD HL,\$0003	
3657	19		ADD HL,DE	
3658	EB		EX DE,HL	
3659	C5		PUSH BC	
365A	06 04		LD B,\$04	en siffra
365C	CD 36 37		CALL MSHL	resten * 10
365F	C1		POP BC	
3660	0D		DEC C	siffran
3661	E1		POP HL	
3662	61		LD H,C	H = siffran, L = siffror kvar
3663	C1		POP BC	

3664	7D	LD	A, L	första siffran och 0?
3665	20 0C	JR	NZ, L3673	
3667	FE 07	CP	\$07	
3669	20 08	JR	NZ, L3673	
366B	DD 35 04	DEC	(IX+\$04)	exponent - 1
366E	CA 5B 37	JP	Z, FZERO	0: resultatet 0
3671	18 AF	JR	FDLOOP	börja om
3673	FE 01	L3673: CP	\$01	sjunde siffran?
3675	28 12	JR	Z, FDRND	
3677	1F	RRA		CY = udda L
3678	7C	LD	A, H	
3679	38 0A	JR	C, L3685	udda: ny byte
367B	0A	LD	A, (BC)	förra siffran
367C	07	RLCA		
367D	07	RLCA		
367E	07	RLCA		
367F	07	RLCA		upp till höga halvan
3680	84	ADD	A, H	+ den nya
3681	02	LD	(BC), A	
3682	03	INC	BC	
3683	18 01	JR	L3686	
3685	02	L3685: LD	(BC), A	siffran i låga halvan
3686	2D	L3686: DEC	L	siffror kvar
3687	18 9B	JR	L3624	

FDRND: sjunde siffran > 5 avrundar uppåt, < 5 nedåt. Exakt 5 avrundar uppåt om inte bit 7 i IY+3 (\$FE19) är satt. Inget annat i ROM:en läser eller skriver \$FE19, och **INIT** nollställer den inte, så det som RAM-kretsarna råkar innehålla efter start avgör (?). Spill ger .100000 och exponent + 1.

Provot: .246913 / 2 = .123456 med \$FE19 = \$FF, .123457 med \$7F (**POKE** 65049,0 i BASIC).

3689	7C	FDRND: LD	A, H	sjunde siffran
368A	FE 05	CP	\$05	
368C	38 27	JR	C, L36B5	< 5: klart
368E	20 06	JR	NZ, L3696	> 5: avrunda
3690	FD B6 03	OR	(IY+\$03)	= 5: bit 7 i \$FE19 hindrar
3693	FA B5 36	JP	M, L36B5	
3696	DD E5	L3696: PUSH	IX	
3698	11 03 00	LD	DE, \$0003	IX+3
369B	DD 19	ADD	IX, DE	
369D	DD E3	EX	(SP), IX	
369F	E1	POP	HL	
36A0	06 03	LD	B, \$03	3 bytes
36A2	37	SCF		+1
36A3	2B	L36A3: DEC	HL	
36A4	7E	LD	A, (HL)	
36A5	CE 00	ADC	A, \$00	
36A7	27	DAA		
36A8	77	LD	(HL), A	
36A9	10 F8	DJNZ	L36A3	
36AB	30 08	JR	NC, L36B5	inget spill
36AD	DD 34 04	INC	(IX+\$04)	exponent + 1
36B0	CA 58 37	JP	Z, FOVFL	fel 4
36B3	36 10	LD	(HL), \$10	mantissan .100000
36B5	21 04 00	L36B5: LD	HL, \$0004	kvoten vid IX+0..4
36B8	C3 37 34	JP	FSTOR1	

FALIGN: DE och HL pekar på två tals exponenter. Ut: DE = talet med störst exponent (C = 1 om DE och HL bytte plats), IX+\$26 = dess exponent. Det andra talet kopieras (**FCOPY**) och skiftas åt höger med skillnaden, högst 6 siffror. B = 4 * skillnaden (bitar). IX+\$28 får byten där den första utskiftade siffran låg, och IX+\$27 säger om den är den låga halvan (udda skillnad). Vid skillnaden 0 är IX+\$28 = 0.

36BB	1A	FALIGN: LD	A, (DE)	skillnad i exponent
36BC	96	SUB	(HL)	
36BD	0E 00	LD	C, \$00	C = 0: ingen byte
36BF	30 04	JR	NC, L36C5	
36C1	0C	INC	C	DE mindre: byt
36C2	EB	EX	DE, HL	

36C3	2F		CPL		skillnaden positiv
36C4	3C		INC	A	
36C5	47	L36C5:	LD	B, A	B = skillnaden
36C6	1A		LD	A, (DE)	den största exponenten
36C7	DD 77 26		LD	(IX+\$26), A	
36CA	78		LD	A, B	
36CB	FE 06		CP	\$06	högst 6 siffror
36CD	38 02		JR	C, L36D1	
36CF	3E 06		LD	A, \$06	
36D1	07	L36D1:	RLCA		4 bitar per siffra
36D2	07		RLCA		
36D3	47		LD	B, A	
36D4	E6 04		AND	\$04	udda antal siffror?
36D6	DD 77 27		LD	(IX+\$27), A	
36D9	D5		PUSH	DE	
36DA	C5		PUSH	BC	
36DB	CD FC 36		CALL	FCOPY	kopiera det mindre talet
36DE	C1		POP	BC	
36DF	C5		PUSH	BC	
36E0	3E 28		LD	A, \$28	vilken byte skiftas ut först
36E2	90		SUB	B	
36E3	FE 28		CP	\$28	ingen förskjutning
36E5	28 0F		JR	Z, L36F6	
36E7	E6 F8		AND	\$F8	
36E9	1F		RRA		byte = (40 - B) / 8
36EA	1F		RRA		
36EB	1F		RRA		
36EC	83		ADD	A, E	DE = IX+\$20 + byte
36ED	5F		LD	E, A	
36EE	7A		LD	A, D	
36EF	CE 00		ADC	A, \$00	
36F1	57		LD	D, A	
36F2	1A		LD	A, (DE)	byten med första utskiftade siffran
36F3	DD 77 28		LD	(IX+\$28), A	
36F6	CD 14 37	L36F6:	CALL	MSHR	skifta åt höger
36F9	C1		POP	BC	
36FA	D1		POP	DE	
36FB	C9		RET		

FCOPY: kopiera tecken och mantissa för talet vid HL till IX+\$22..\$25. IX+\$20, IX+\$21 och IX+\$28 nollställs. Ut: DE = IX+\$20.

36FC	DD E5	FCOPY:	PUSH	IX	
36FE	11 25 00		LD	DE, \$0025	DE = IX+\$25
3701	DD 19		ADD	IX, DE	
3703	DD E3		EX	(SP), IX	
3705	D1		POP	DE	
3706	01 04 00		LD	BC, \$0004	tecken och mantissa
3709	2B		DEC	HL	HL = teckenbyten
370A	ED B8		LDDR		
370C	AF		XOR	A	IX+\$21 = 0
370D	12		LD	(DE), A	
370E	1B		DEC	DE	
370F	12		LD	(DE), A	IX+\$20 = 0
3710	DD 77 28		LD	(IX+\$28), A	IX+\$28 = 0
3713	C9		RET		

MSHR: skifta IX+\$21..\$24 åt höger, B/4 siffror (B = 4 per siffra).

Hela bytes flyttas först; den sista utskiftade byten ligger sedan kvar i D. En siffra till skiftas med RRD. Utskiftade siffror försvinner.

3714	0E 04	MSHR:	LD	C, \$04	4 bytes
3716	21 21 00		LD	HL, \$0021	IX+\$21
3719	CD 5F 37		CALL	IXOFF	
371C	78		LD	A, B	minst en hel byte?
371D	D6 08		SUB	\$08	
371F	30 0A		JR	NC, L372B	
3721	05		DEC	B	B = 0: klart
3722	F8		RET	M	

3723	AF		XOR	A	en siffra, 0 in
3724	ED 67	L3724:	RRD		halv byte åt höger
3726	23		INC	HL	
3727	0D		DEC	C	
3728	20 FA		JR	NZ, L3724	
372A	C9		RET		
372B	47	L372B:	LD	B, A	B - 8
372C	AF		XOR	A	0 in
372D	56	L372D:	LD	D, (HL)	hel byte åt höger
372E	77		LD	(HL), A	
372F	7A		LD	A, D	
3730	23		INC	HL	
3731	0D		DEC	C	
3732	20 F9		JR	NZ, L372D	
3734	18 DE		JR	MSHR	

MSHL: skifta $IX + \$21..\24 åt vänster på samma sätt, med RLD. Nollor skiftas in.

3736	0E 04	MSHL:	LD	C, \$04	4 bytes
3738	21 24 00		LD	HL, \$0024	$IX + \$24$
373B	CD 5F 37		CALL	IXOFF	
373E	78		LD	A, B	minst en hel byte?
373F	D6 08		SUB	\$08	
3741	30 0A		JR	NC, L374D	
3743	05		DEC	B	B = 0: klart
3744	F8		RET	M	
3745	AF		XOR	A	en siffra, 0 in
3746	ED 6F	L3746:	RLD		halv byte åt vänster
3748	2B		DEC	HL	
3749	0D		DEC	C	
374A	20 FA		JR	NZ, L3746	
374C	C9		RET		
374D	47	L374D:	LD	B, A	B - 8
374E	AF		XOR	A	
374F	56	L374F:	LD	D, (HL)	hel byte åt vänster
3750	77		LD	(HL), A	
3751	7A		LD	A, D	
3752	2B		DEC	HL	
3753	0D		DEC	C	
3754	20 F9		JR	NZ, L374F	
3756	18 DE		JR	MSHL	
3758	C3 01 07	FOVFL:	JP	L0701	fel 4, FÖR STORT FLYTTAL
375B	AF	FZERO:	XOR	A	exponent 0
375C	C3 FA 34		JP	FMULE	via FMUL: resultatet 0
375F	D5	IXOFF:	PUSH	DE	
3760	DD E5		PUSH	IX	
3762	D1		POP	DE	
3763	19		ADD	HL, DE	HL = IX + HL
3764	D1		POP	DE	
3765	C9		RET		

FENTER: början på flyttalsrutinerna. Returadressen tas bort och läggs tillbaka efter HL, DE och BC. Ut: $IX = SP - \$27$ (före RET), dvs. arbetsytan $IX + 0..\$28$ slutar precis under de sparade registren.

3766	DD E1	FENTER:	POP	IX	returadressen
3768	E5		PUSH	HL	spara registren
3769	D5		PUSH	DE	
376A	C5		PUSH	BC	
376B	DD E5		PUSH	IX	returadressen tillbaka
376D	DD 21 D9 FF		LD	IX, \$FFD9	$IX = \text{arbetsytan}$
3771	DD 39		ADD	IX, SP	
3773	C9		RET		

3.91 IMUL / IDIV: 16-bitars heltal utan tecken

\$3774

IMUL: HL = HL * DE. Skift och addition, högsta biten först. CY = 1 om produkten inte ryms i 16 bitar.

IDIV: HL = HL / DE, DE = resten. Återställande division, 16 varv (A = 17 räknar också det sista skiftet av kvoten). CY = 1 om DE = 0. Anropas via **SMULDV** (\$2B87), som sköter tecknen, och direkt från **\$2D5A**, **\$331E** och **\$3F44**.

3774	42	IMUL:	LD	B,D	BC = b
3775	4B		LD	C,E	
3776	11 00 00		LD	DE,\$0000	DE = produkten
3779	3E 10		LD	A,\$10	16 bitar
377B	29	L377B:	ADD	HL,HL	nästa bit i a
377C	EB		EX	DE,HL	
377D	30 02		JR	NC,L3781	0: ingen addition
377F	09		ADD	HL,BC	produkten += b
3780	D8		RET	C	spill
3781	3D	L3781:	DEC	A	
3782	C8		RET	Z	klart, HL = produkten
3783	29		ADD	HL,HL	produkten * 2
3784	EB		EX	DE,HL	
3785	30 F4		JR	NC,L377B	
3787	C9		RET		spill

IDIV: HL / DE. Kvoten skiftas in i HL och resten byggs upp i DE (växlas med EX).

3788	7A	IDIV:	LD	A,D	b = 0?
3789	B3		OR	E	
378A	37		SCF		CY: division med 0
378B	C8		RET	Z	
378C	42		LD	B,D	BC = b
378D	4B		LD	C,E	
378E	11 00 00		LD	DE,\$0000	
3791	EB		EX	DE,HL	DE = a, HL = resten = 0
3792	3E 11		LD	A,\$11	16 bitar + sista skiftet
3794	37		SCF		
3795	3F	L3795:	CCF		kvotbiten = NOT CY
3796	EB		EX	DE,HL	
3797	ED 6A		ADC	HL,HL	in i kvoten, ut ur a
3799	3D		DEC	A	
379A	C8		RET	Z	klart, HL = kvoten
379B	EB		EX	DE,HL	
379C	ED 6A		ADC	HL,HL	in i resten
379E	A7		AND	A	
379F	ED 42		SBC	HL,BC	resten - b
37A1	30 F2		JR	NC,L3795	gick: kvotbit 1
37A3	09		ADD	HL,BC	lägg tillbaka: kvotbit 0
37A4	18 EF		JR	L3795	

3.92 ASCII-aritmetik: AADD ASUB ACOMP ADIV AMUL

\$37A6

Talen är text: mellanslag, tecken (+ eller -), siffror och högst en punkt, avslutade med \$83 (samma format som **ASCARG** lägger i bufferten). Mellanslag, tecken och punkt räknas som siffran 0 när siffrorna läses, och punkten hoppas över.

In: HL = a, DE = b.

Ut: resultatet skrivs över a, i a:s format: lika många positioner och lika många decimaler. Ledande nollor blir mellanslag och ett minustecken skrivs närmast före första siffran.

A: bit 7 = negativt, bit 6 = inte noll, bit 5 = spill.

CY = 1 vid fel. A = 8 (bit 3): texten är inte ett tal, annars spill. (**ASCERR**, **\$2C4B**, gör fel 66 resp. 65 av det.)

ACOMP skriver också över a (med skräp) och ger bara A.

ASENTR (\$396F) ger en arbetsyta under stacken (SP0 = SP vid anropet av rutinen, HL' = SP0 för felutgång):

IY = SP0-20: IY+4 a:s längd IY+5 a:s decimaler IY+6 flaggor
(bit 7 tecken, 6 inte noll, 5 spill)

IX = SP0-10: IX+4 b:s längd IX+5 b:s decimaler IX+6 b:s tecken

IX+7 räknare IX+8 se [ASWIDE](#)
 ASBBEG/ASBEND (\$FDD0/\$FDD2) = b:s början och \$83,
 ASABEG/ASAEND (\$FDD4/\$FDD6) = a:s början och \$83.

”Längd” är antalet positioner utom punkten, alltså med ledande mellanslag och tecken. Högst 31, annars fel (A = 8).

Z från [ASENTR](#): a och b har samma tecken.

Felutgången [ASBAD](#) återställer SP = HL' och returnerar direkt till den som anropade rutinen, så RET C efter CALL [ASENTR](#) tas aldrig.

Addition och subtraktion görs på beloppen: [AADD](#) med samma tecken och [ASUB](#) med olika tecken adderar ([ASMADD](#)), annars subtraheras ([ASMSUB](#)). Resultatet får a:s tecken; blir det lån tiokomplementeras a och tecknet vänds ([ASNEG](#)).

Avrundning: har b fler decimaler än a kapas b, och första kapade siffran avgör. > 5, eller 5 följd av någon siffra <> 0: beloppet avrundas uppåt. Exakt 5 (resten nollor): uppåt bara om b är positivt. [AADD](#) avrundar därför exakt hälft mot +oändligheten, [ASUB](#) mot -oändligheten. Ex: 1.00 + 0.005 = 1.01, -1.00 + -0.005 = -1.00, 1.0 - 1.05 = -1.

37A6	CD 6F 39	AADD:	CALL ASENTR	Z: samma tecken
37A9	D8		RET C	(tas aldrig)
37AA	28 08		JR Z, ASMADD	addera beloppen
37AC	18 1B		JR ASMSUB	subtrahera beloppen
37AE	CD 6F 39	ASUB:	CALL ASENTR	Z: samma tecken
37B1	D8		RET C	
37B2	28 15		JR Z, ASMSUB	subtrahera beloppen
37B4	CD 12 3A	ASMADD:	CALL ASALGN	ställ upp, B = avrundningen
37B7	C4 0D 39		CALL NZ, ASADDN	a += b
37BA	CD CD 3A		CALL ASREST	a:s övriga siffror
37BD	C4 1A 39		CALL NZ, ASCARN	för minnet vidare
37C0	30 04		JR NC, L37C6	minne ut ur a
37C2	FD CB 06 EE		SET 5, (IY+\$06)	spill
37C6	C3 00 3B	L37C6:	JP ASFIN	
37C9	CD 12 3A	ASMSUB:	CALL ASALGN	ställ upp, B = avrundningen
37CC	C4 3B 39		CALL NZ, ASSUBN	a -= b
37CF	CD CD 3A		CALL ASREST	a:s övriga siffror
37D2	C4 48 39		CALL NZ, ASBORN	för lånet vidare
37D5	38 0A		JR C, L37E1	lån: a < b
37D7	DD 7E 08		LD A, (IX+\$08)	b:s ledande 1 (ASWIDE)?
37DA	A7		AND A	
37DB	28 E9		JR Z, L37C6	
37DD	FD CB 06 AE		RES 5, (IY+\$06)	får plats ändå
37E1	CD D6 3A	L37E1:	CALL ASNEG	tiokomplement, vänd tecknet
37E4	18 E0		JR L37C6	
37E6	FD 46 04	ASZERA:	LD B, (IY+\$04)	a:s längd
37E9	CD 72 3B		CALL ASALST	
37EC	CD 4D 3B	L37EC:	CALL ASDIG	füll a med "0"
37EF	36 30		LD (HL), \$30	
37F1	2B		DEC HL	
37F2	10 F8		DJNZ L37EC	
37F4	18 BE		JR ASMADD	a += b (produkten/kvoten)

[ACOMP](#): jämför a och b. Olika tecken: beloppen adderas bara för att se om båda är noll; resultatet får a:s tecken. Samma tecken: beloppen subtraheras ([ASCMPS](#)). Ut: A bit 6 = 0 lika, annars bit 7 = 1 för a < b. CY = 0 om texterna är tal.

Fel i ROM:en: vid samma tecken och inget lån vänds tecknet också när a är negativt (ADD A,A lägger tecknet i CY); rätt vore att vända bara vid lån. Två olika negativa tal ger därför alltid "a > b": COMP%("-2","-1") = 1 och COMP%("-10","-5") = 1 (men COMP%("-1","-2") = 1 är rätt). Positiva tal och olika tecken jämförs rätt. Lika i 9913.

37F6	CD 6F 39	ACOMP:	CALL ASENTR	
37F9	D8		RET C	
37FA	28 1B		JR Z, ASCMPS	samma tecken
37FC	CD 12 3A		CALL ASALGN	olika tecken: addera
37FF	C4 0D 39		CALL NZ, ASADDN	
3802	CD CD 3A		CALL ASREST	
3805	C4 1A 39		CALL NZ, ASCARN	
3808	FD 4E 06		LD C, (IY+\$06)	flaggorna
380B	CB 69		BIT 5,C	spill: inte noll

380D	20 04	JR	NZ, L3813	
380F	CB 18	RR	B	minne ut: inte noll
3811	30 1D	JR	NC, L3830	
3813	CB F1	L3813: SET	6, C	inte noll
3815	18 19	JR	L3830	
3817	CD 12 3A	ASCMP: CALL	ASALGN	subtrahera beloppen
381A	C4 3B 39	CALL	NZ, ASSUBN	
381D	CD CD 3A	CALL	ASREST	
3820	C4 48 39	CALL	NZ, ASBORN	
3823	FD 7E 06	LD	A, (IY+\$06)	flaggorna, bit 7 = a:s tecken
3826	4F	LD	C, A	
3827	38 03	JR	C, L382C	lån: a < b
3829	87	ADD	A, A	CY = a:s tecken: vänder fel
382A	30 04	JR	NC, L3830	
382C	3E 80	L382C: LD	A, \$80	vänd tecknet
382E	A9	XOR	C	
382F	4F	LD	C, A	
3830	CB A9	L3830: RES	5, C	inget spill
3832	CB 71	BIT	6, C	noll: inget tecken
3834	20 02	JR	NZ, L3838	
3836	CB B9	RES	7, C	
3838	C3 3E 3B	L3838: JP	ASRET	A = flaggorna

3.93 ADIV: a = a / b

\$383B

ASWORK lägger 62 nollor + \$83 (W) under arbetsytan och ger kvotens tecken i IY+6. a kopieras till W, förskjuten b:s decimaler åt vänster. Sedan vanlig återställande lång division: b dras från W så länge det går, antalet blir kvotsiffran och skrivs där fönstret började. Kvoten får a:s längd + b:s decimaler siffror med a:s + b:s decimaler. Blir en kvotsiffran 10 (t.ex. b = 0) sätts spill. a:s längd + b:s längd + 2 * b:s decimaler >= 63 ger A = 8.

Kvoten blir sedan b (**ASSETB**), a nollställs och **ASMADD** lägger in den, med avrundning till a:s decimaler. Har b inga decimaler finns inga extra siffror, så kvoten kapas: $2.00/3 = .66$ men $2.00/3.0 = .67$.

383B	CD 6F 39	ADIV: CALL	ASENTR	
383E	D8	RET	C	
383F	CD 9D 3A	CALL	ASWORK	W, kvotens tecken
3842	DD 7E 05	LD	A, (IX+\$05)	b:s decimaler
3845	4F	LD	C, A	
3846	FD 86 04	ADD	A, (IY+\$04)	+ a:s längd = antal kvotsiffror
3849	CD FC 38	CALL	ASLEN	+ b:s längd
384C	81	ADD	A, C	+ b:s decimaler; HL = W - b:s decimaler
384D	CD 02 39	CALL	ASLEN1	
3850	EB	EX	DE, HL	DE = W
3851	FD 4E 04	LD	C, (IY+\$04)	a:s längd
3854	CD 72 3B	CALL	ASALST	
3857	CD 0D 39	CALL	ASADDN	kopiera a till W
385A	DD 7E 04	LD	A, (IX+\$04)	b:s heltalssiffror
385D	DD 96 05	SUB	(IX+\$05)	
3860	4F	LD	C, A	
3861	AF	XOR	A	
3862	47	LD	B, A	
3863	EB	EX	DE, HL	DE = fönstrets början
3864	ED 42	SBC	HL, BC	
3866	EB	EX	DE, HL	
3867	13	INC	DE	
3868	3E 30	ASDLP: LD	A, \$30	kvotsiffran = "0"
386A	08	L386A: EX	AF, AF'	
386B	DD 4E 04	LD	C, (IX+\$04)	b:s längd
386E	AF	XOR	A	
386F	47	LD	B, A	
3870	EB	EX	DE, HL	DE = fönstrets slut
3871	09	ADD	HL, BC	
3872	EB	EX	DE, HL	
3873	CD 6D 3B	CALL	ASBLST	
3876	CD 3B 39	CALL	ASSUBN	W -= b
3879	1A	LD	A, (DE)	lånet i siffran till vänster

387A	90		SUB	B	
387B	12		LD	(DE),A	
387C	FE 30		CP	\$30	
387E	38 0B		JR	C,L388B	negativt: klart
3880	08		EX	AF,AF'	kvotsiffran + 1
3881	3C		INC	A	
3882	FE 3A		CP	\$3A	högst "9"
3884	38 E4		JR	C,L386A	
3886	3D		DEC	A	"9"
3887	FD CB 06 EE		SET	5,(IY+\$06)	spill
388B	CD BA 3A	L388B:	CALL	ASADDB	lägg tillbaka b
388E	12		LD	(DE),A	kvotsiffran
388F	13		INC	DE	nästa fönster
3890	DD 35 07		DEC	(IX+\$07)	fler kvotsiffror?
3893	20 D3		JR	NZ,ASDLP	
3895	EB		EX	DE,HL	
3896	36 83		LD	(HL),\$83	\$83 efter kvoten
3898	DD 7E 05		LD	A,(IX+\$05)	kvotens längd
389B	FD 86 04		ADD	A,(IY+\$04)	
389E	DD 77 04		LD	(IX+\$04),A	
38A1	4F		LD	C,A	
38A2	FD 7E 05		LD	A,(IY+\$05)	kvotens decimaler
38A5	DD 86 05		ADD	A,(IX+\$05)	
38A8	DD 77 05		LD	(IX+\$05),A	
38AB	22 D2 FD	ASSETB:	LD	(ASBEND),HL	b = W
38AE	AF		XOR	A	
38AF	47		LD	B,A	
38B0	ED 42		SBC	HL,BC	
38B2	22 D0 FD		LD	(ASBBEG),HL	
38B5	C3 E6 37		JP	ASZERA	a = 0, a += b

3.94 AMUL: a = a * b

\$38B8

Skolmultiplikation i W (se ADIV): för varje siffra k i a, från höger, adderas b k gånger (ASADDB) på rätt plats. Produkten har a:s + b:s längd och a:s + b:s decimaler; ≥ 63 positioner ger A = 8. Sedan som i ADIV: produkten blir b, a nollställs och ASMADD avrundar in den. Exakt hälft avrundas uppåt om det ursprungliga b är positivt:

1.00 * 0.125 = .13 men 1.00 * -0.125 = -.12 .

38B8	CD 6F 39	AMUL:	CALL	ASENTR	
38BB	D8		RET	C	
38BC	CD 9D 3A		CALL	ASWORK	W, produktens tecken
38BF	FD 7E 04		LD	A,(IY+\$04)	a:s längd
38C2	DD 4E 04		LD	C,(IX+\$04)	b:s längd
38C5	CD FC 38		CALL	ASLEN	IX+7 = a:s längd; HL = W - b:s längd
38C8	EB		EX	DE,HL	DE = W
38C9	CD 72 3B		CALL	ASALST	a:s sista siffra
38CC	CD 4D 3B	ASMLP:	CALL	ASDIG	siffran k
38CF	E5		PUSH	HL	
38D0	D6 30		SUB	\$30	k = 0?
38D2	28 07		JR	Z,L38DB	
38D4	08	L38D4:	EX	AF,AF'	W += b, k gånger
38D5	CD BA 3A		CALL	ASADDB	
38D8	3D		DEC	A	
38D9	20 F9		JR	NZ,L38D4	
38DB	E1	L38DB:	POP	HL	nästa siffra i a
38DC	2B		DEC	HL	
38DD	1B		DEC	DE	W en plats åt vänster
38DE	DD 35 07		DEC	(IX+\$07)	fler siffror i a?
38E1	20 E9		JR	NZ,ASMLP	
38E3	FD 7E 05		LD	A,(IY+\$05)	produktens decimaler
38E6	DD 86 05		ADD	A,(IX+\$05)	
38E9	DD 77 05		LD	(IX+\$05),A	
38EC	FD 7E 04		LD	A,(IY+\$04)	produktens längd
38EF	DD 86 04		ADD	A,(IX+\$04)	
38F2	DD 77 04		LD	(IX+\$04),A	
38F5	21 3E 00		LD	HL,\$003E	HL = W:s \$83

```

38F8 39          ADD HL,SP
38F9 4F          LD  C,A
38FA 18 AF      JR   ASSETB

```

ASLEN: $IX+7 = A$, $A += b$:s längd. **ASLEN1:** $A \geq 63$ ger **ASBAD**, annars $HL -= C$ ($B = 0$).

```

38FC DD 77 07    ASLEN: LD  (IX+$07),A
38FF DD 86 04    ADD  A,(IX+$04)
3902 FE 3F      ASLEN1: CP  $3F
3904 D2 BF 39    JP   NC,ASBAD      för långt: A = 8
3907 A7          AND  A
3908 06 00      LD  B,$00
390A ED 42      SBC  HL,BC
390C C9          RET

```

Siffer-slingor. $HL = x$, $DE = y$ pekar på sista siffran. Resultatet skrivs i y , från höger. $C =$ antal siffror, $B =$ minnessiffran ($0/1$).

ASADDN: $y = y + x + B$, C siffror. Ut: $HL = y$, $DE = x$ (nästa). **ASCARN:** för minnet vidare genom HL , C siffror. $CY =$ minnessiffran. **ASSUBN** / **ASBORN:** samma med subtraktion ($y - x - B$) och lån.

ASPUT / **ASPUTB:** skriv siffran och ge nytt minne/lån i B ; en siffra

$\langle \rangle 0$ sätter bit 6 i $IY+6$. Mellanslag m.m. i y blir alltså "0".

ASDIG2: $A = y$ -siffran, $B = x$ -siffran + B .

```

390D CD 5E 39    ASADDN: CALL ASDIG2
3910 80          ADD  A,B           y + x + minne
3911 D6 30      SUB  $30
3913 CD 26 39    CALL ASPUT        skriv, B = minnet
3916 EB          EX   DE,HL
3917 C8          RET  Z           C = 0: klart
3918 18 F3      JR   ASADDN

391A CD 4D 3B    ASCARN: CALL ASDIG      siffran
391D 80          ADD  A,B           + minnet
391E CD 26 39    CALL ASPUT
3921 20 F7      JR   NZ,ASCARN
3923 78          LD   A,B           CY = minnet
3924 0F          RRCA
3925 C9          RET

3926 06 00      ASPUT: LD   B,$00
3928 FE 3A      CP   $3A           > 9?
392A 38 03      JR   C,L392F
392C D6 0A      SUB  $0A           - 10
392E 04          L392E: INC  B           minne/lån
392F 77          L392F: LD   (HL),A      skriv siffran
3930 FE 30      CP   $30
3932 28 04      JR   Z,L3938
3934 FD CB 06 F6 SET  6,(IY+$06)      inte noll
3938 2B          L3938: DEC  HL
3939 0D          DEC  C
393A C9          RET

393B CD 5E 39    ASSUBN: CALL ASDIG2
393E 90          SUB  B           y - x - lån
393F C6 30      ADD  A,$30
3941 CD 54 39    CALL ASPUTB
3944 EB          EX   DE,HL
3945 C8          RET  Z
3946 18 F3      JR   ASSUBN

3948 CD 4D 3B    ASBORN: CALL ASDIG
394B 90          SUB  B
394C CD 54 39    CALL ASPUTB
394F 20 F7      JR   NZ,ASBORN
3951 78          LD   A,B           CY = lånet
3952 0F          RRCA
3953 C9          RET

3954 06 00      ASPUTB: LD   B,$00
3956 FE 30      CP   $30           < 0?
3958 30 D5      JR   NC,L392F

```

395A	C6 0A	ADD	A, \$0A	+ 10
395C	18 D0	JR	L392E	
395E	7E	ASDIG2: LD	A, (HL)	x-siffran
395F	FE 30	CP	\$30	
3961	DC 47 3B	CALL	C, ASDOT	punkt m.m.
3964	80	ADD	A, B	+ minnet
3965	47	LD	B, A	
3966	2B	DEC	HL	
3967	EB	EX	DE, HL	
3968	7E	LD	A, (HL)	y-siffran
3969	FE 30	CP	\$30	
396B	D0	RET	NC	
396C	C3 47 3B	JP	ASDOT	
396F	C1	ASENTR: POP	BC	returadressen
3970	D9	EXX		
3971	21 00 00	LD	HL, \$0000	HL' = SP0
3974	39	ADD	HL, SP	
3975	EB	EX	DE, HL	
3976	DD 21 F6 FF	LD	IX, \$FFF6	IX = SP0-10
397A	FD 21 EC FF	LD	IY, \$FFEC	IY = SP0-20
397E	DD 19	ADD	IX, DE	
3980	FD 19	ADD	IY, DE	
3982	FD F9	LD	SP, IY	SP under arbetsytan
3984	EB	EX	DE, HL	
3985	D9	EXX		
3986	C5	PUSH	BC	
3987	ED 53 D0 FD	LD	(ASBBEG), DE	b:s början
398B	22 D4 FD	LD	(ASABEG), HL	a:s början
398E	D5	PUSH	DE	
398F	CD C5 39	CALL	ASPARS	läs a
3992	22 D6 FD	LD	(ASAEND), HL	a:s \$83
3995	FD 70 06	LD	(IY+\$06), B	a:s tecken
3998	FD 73 04	LD	(IY+\$04), E	a:s längd
399B	FD 72 05	LD	(IY+\$05), D	a:s decimaler
399E	E1	POP	HL	
399F	38 1E	JR	C, ASBAD	
39A1	CD C5 39	CALL	ASPARS	läs b
39A4	38 19	JR	C, ASBAD	
39A6	DD 70 06	LD	(IX+\$06), B	b:s tecken
39A9	22 D2 FD	LD	(ASBEND), HL	b:s \$83
39AC	DD 72 05	LD	(IX+\$05), D	
39AF	DD 73 04	LD	(IX+\$04), E	
39B2	AF	XOR	A	IX+8 = 0
39B3	DD 77 08	LD	(IX+\$08), A	
39B6	DD 7E 06	LD	A, (IX+\$06)	Z: samma tecken
39B9	FD AE 06	XOR	(IY+\$06)	
39BC	E6 80	AND	\$80	
39BE	C9	RET		

ASBAD: fel, A = 8 och CY. SP = HL' (SP0) och RET direkt till den som anropade AADD osv.

39BF	3E 08	ASBAD: LD	A, \$08	inte ett tal
39C1	37	SCF		
39C2	D9	EXX		
39C3	F9	LD	SP, HL	SP = SP0
39C4	C9	RET		

3.95 ASPARS: läs ett tal i text

\$39C5

In: HL = texten. Ut: HL på \$83, E = antal positioner (mellanslag, tecken och siffror, inte punkten), D = antal decimaler, B bit 7 = minustecken. CY = inte ett tal: annat tecken än mellanslag före talet, en andra punkt, tom text eller fler än 31 positioner (ASCNT). Tecknet måste stå direkt före första siffran eller punkten.

39C5	11 00 00	ASPARS:	LD	DE, \$0000	
39C8	43		LD	B, E	
39C9	48		LD	C, B	
39CA	7E	L39CA:	LD	A, (HL)	
39CB	FE 20		CP	\$20	mellanslag
39CD	20 08		JR	NZ, L39D7	
39CF	CD 68 3B		CALL	ASCNT	räknas
39D2	28 34		JR	Z, L3A08	
39D4	23		INC	HL	
39D5	18 F3		JR	L39CA	
39D7	FE 2B	L39D7:	CP	\$2B	+
39D9	28 06		JR	Z, L39E1	
39DB	FE 2D		CP	\$2D	-
39DD	20 09		JR	NZ, L39E8	
39DF	CB F8		SET	7, B	minustecken
39E1	CD 68 3B	L39E1:	CALL	ASCNT	tecknet räknas
39E4	28 22		JR	Z, L3A08	
39E6	23		INC	HL	
39E7	7E		LD	A, (HL)	
39E8	FE 2E	L39E8:	CP	\$2E	punkt
39EA	28 1E		JR	Z, L3A0A	
39EC	FE 30		CP	\$30	inte en siffra: fel
39EE	38 18		JR	C, L3A08	
39F0	FE 3A		CP	\$3A	
39F2	30 14		JR	NC, L3A08	
39F4	CD 68 3B		CALL	ASCNT	siffran räknas
39F7	28 0F		JR	Z, L3A08	
39F9	23	L39F9:	INC	HL	
39FA	7E		LD	A, (HL)	
39FB	FE 83		CP	\$83	slut?
39FD	20 E9		JR	NZ, L39E8	
39FF	AF		XOR	A	E = 0: tom
3A00	B3		OR	E	
3A01	28 05		JR	Z, L3A08	
3A03	92		SUB	D	positioner efter punkten
3A04	0D		DEC	C	C = 1: fanns en punkt
3A05	C0		RET	NZ	
3A06	57		LD	D, A	D = decimalerna
3A07	C9		RET		
3A08	37	L3A08:	SCF		
3A09	C9		RET		
3A0A	AF	L3A0A:	XOR	A	andra punkten: fel
3A0B	B1		OR	C	
3A0C	20 FA		JR	NZ, L3A08	
3A0E	0C		INC	C	
3A0F	53		LD	D, E	D = positioner före punkten
3A10	18 E7		JR	L39F9	

3.96 ASALGN: ställ upp a och b så att decimalerna står under varandra

\$3A12

a har minst lika många decimaler: a:s extra decimaler går igenom med $B = 0$ (de står kvar, men mellanslag blir "0"). b har fler decimaler ([ASALG1](#)): b:s extra decimaler kapas och ger avrundningen i B (se ovan), och b:s längd minskas.

ASWIDE: jämför de återstående längderna. $IX+7$ = hur många siffror a har utöver b. Är b längre sätts $IX+7 = 0$, och b:s extra ledande siffror måste vara 0; annars spill. Undantag: är den enda en "1" närmast a:s bredd sätts $IX+8$, för en subtraktion kan ändå få plats ([ASMSUB](#)). Ut: HL = b, DE = a (sista siffran), C = antal par, Z om 0.

3A12	FD 7E 05	ASALGN:	LD A, (IX+\$05)	a:s decimaler - b:s
3A15	DD 96 05		SUB (IX+\$05)	
3A18	38 18		JR C, ASALG1	b har fler
3A1A	4F		LD C, A	
3A1B	FD 7E 04		LD A, (IX+\$04)	a:s längd - skillnaden
3A1E	91		SUB C	
3A1F	08		EX AF, AF'	
3A20	CD 72 3B		CALL ASALST	a:s sista siffra
3A23	AF		XOR A	
3A24	47		LD B, A	
3A25	B1		OR C	
3A26	C4 1A 39		CALL NZ, ASCARN	gå förbi a:s extra decimaler
3A29	23		INC HL	
3A2A	E5		PUSH HL	a:s plats
3A2B	CD 6D 3B		CALL ASBLST	b:s sista siffra
3A2E	5D		LD E, L	
3A2F	54		LD D, H	
3A30	18 35		JR ASWIDE	
3A32	ED 44	ASALG1:	NEG	C = b:s extra decimaler
3A34	4F		LD C, A	
3A35	AF		XOR A	
3A36	47		LD B, A	
3A37	CD 6D 3B		CALL ASBLST	b:s sista siffra
3A3A	23		INC HL	
3A3B	ED 42		SBC HL, BC	HL = första kapade siffran
3A3D	54		LD D, H	
3A3E	5D		LD E, L	
3A3F	DD 7E 04		LD A, (IX+\$04)	b:s längd - C
3A42	91		SUB C	
3A43	DD 77 04		LD (IX+\$04), A	
3A46	0D		DEC C	
3A47	7E		LD A, (HL)	kapad siffra - 5
3A48	D6 35		SUB \$35	
3A4A	38 11		JR C, L3A5D	< 5: B = 0
3A4C	20 0D		JR NZ, L3A5B	> 5: B = 1
3A4E	B1		OR C	
3A4F	23		INC HL	
3A50	C4 53 3B		CALL NZ, ASNZ	fler siffror <> 0?
3A53	20 06		JR NZ, L3A5B	
3A55	DD CB 06 7E		BIT 7, (IX+\$06)	exakt hälft: b negativt?
3A59	20 02		JR NZ, L3A5D	
3A5B	06 01	L3A5B:	LD B, \$01	avrunda uppåt
3A5D	1B	L3A5D:	DEC DE	DE = b:s sista kvarvarande siffra
3A5E	FD 7E 04		LD A, (IX+\$04)	a:s längd
3A61	08		EX AF, AF'	
3A62	CD 72 3B		CALL ASALST	a:s sista siffra
3A65	23		INC HL	
3A66	E5		PUSH HL	a:s plats
3A67	08	ASWIDE:	EX AF, AF'	
3A68	4F		LD C, A	C = a:s återstående längd
3A69	DD 96 04		SUB (IX+\$04)	- b:s
3A6C	38 08		JR C, L3A76	b längre
3A6E	DD 77 07		LD (IX+\$07), A	a:s siffror utöver b
3A71	DD 4E 04		LD C, (IX+\$04)	par = b:s längd
3A74	18 21		JR L3A97	
3A76	ED 44	L3A76:	NEG	b:s extra siffror
3A78	C5		PUSH BC	

3A79	4F	LD	C, A	
3A7A	DD 36 07 00	LD	(IX+\$07), \$00	inga utöver b i a
3A7E	2A D0 FD	LD	HL, (ASBBEG)	b:s början
3A81	CD 53 3B	CALL	ASNZ	någon <> 0?
3A84	28 10	JR	Z, L3A96	
3A86	FE 31	CP	\$31	en "1"...
3A88	20 08	JR	NZ, L3A92	
3A8A	79	LD	A, C	
3A8B	FE 01	CP	\$01	...närmast a:s bredd?
3A8D	20 03	JR	NZ, L3A92	
3A8F	DD 77 08	LD	(IX+\$08), A	IX+8 = 1
3A92	FD CB 06 EE	L3A92: SET	5, (IY+\$06)	spill (kanske)
3A96	C1	L3A96: POP	BC	C = par, B = avrundningen
3A97	E1	L3A97: POP	HL	a:s plats
3A98	EB	EX	DE, HL	HL = b, DE = a
3A99	1B	DEC	DE	
3A9A	AF	XOR	A	
3A9B	B1	OR	C	
3A9C	C9	RET		

ASWORK: $IY+6 = \text{produktens/kvotens tecken (a:s XOR b:s)}$. Flyttar SP 63 bytes nedåt och fyller W med 62 "0" och \$83. HL = W:s sista siffra.

3A9D	FD CB 06 BE	ASWORK: RES	7, (IY+\$06)	a:s tecken bort
3AA1	FD B6 06	OR	(IY+\$06)	XOR b:s
3AA4	FD 77 06	LD	(IY+\$06), A	
3AA7	C1	POP	BC	returadressen
3AA8	21 C1 FF	LD	HL, \$FFC1	63 bytes
3AAB	39	ADD	HL, SP	
3AAC	F9	LD	SP, HL	
3AAD	C5	PUSH	BC	
3AAE	06 3E	LD	B, \$3E	62 "0"
3AB0	36 30	L3AB0: LD	(HL), \$30	
3AB2	23	INC	HL	
3AB3	10 FB	DJNZ	L3AB0	
3AB5	36 83	LD	(HL), \$83	\$83
3AB7	2B	DEC	HL	W:s sista siffra
3AB8	AF	XOR	A	
3AB9	C9	RET		

ASADDB: $W(DE) += b$. DE flyttas b:s längd åt höger och b adderas med **ASADDN**; minnet läggs i siffran till vänster. A' tillbaka till A (räknaren i **AMUL/ADIV**).

3ABA	AF	ASADDB: XOR	A	
3ABB	47	LD	B, A	
3ABC	DD 4E 04	LD	C, (IX+\$04)	b:s längd
3ABF	EB	EX	DE, HL	
3AC0	09	ADD	HL, BC	
3AC1	EB	EX	DE, HL	
3AC2	CD 6D 3B	CALL	ASBLST	b:s sista siffra
3AC5	CD 0D 39	CALL	ASADDN	W += b
3AC8	1A	LD	A, (DE)	minnet till vänster
3AC9	80	ADD	A, B	
3ACA	12	LD	(DE), A	
3ACB	08	EX	AF, AF'	A = räknaren
3ACC	C9	RET		

ASREST: $C = IX+7$ (a:s siffror utöver b), Z om inga; HL = a; CY = minnet i B.

3ACD	DD 4E 07	ASREST: LD	C, (IX+\$07)	
3AD0	79	LD	A, C	
3AD1	A7	AND	A	
3AD2	EB	EX	DE, HL	
3AD3	78	LD	A, B	
3AD4	0F	RRCA		CY = minnet
3AD5	C9	RET		

ASNEG: tiokomplement av a och vänt tecken i IY+6. Nollor längst till höger står kvar, den första siffran c <> 0 blir 10-c (\$6A - c)

och resten 9-c (\$69 - c).

3AD6	FD 46 04	ASNEG:	LD B, (IY+\$04)	
3AD9	3E 80		LD A, \$80	vänd tecknet
3ADB	FD AE 06		XOR (IY+\$06)	
3ADE	FD 77 06		LD (IY+\$06), A	
3AE1	CD 72 3B		CALL ASALST	a:s sista siffra
3AE4	CD 4D 3B	L3AE4:	CALL ASDIG	
3AE7	FE 30		CP \$30	nollor till höger står kvar
3AE9	20 04		JR NZ, L3AEF	
3AEB	2B		DEC HL	
3AEC	10 F6		DJNZ L3AE4	
3AEE	C9		RET	
3AEF	4F	L3AEF:	LD C, A	
3AF0	3E 6A		LD A, \$6A	10 - c
3AF2	91	L3AF2:	SUB C	
3AF3	77		LD (HL), A	
3AF4	10 01		DJNZ L3AF7	
3AF6	C9		RET	
3AF7	2B	L3AF7:	DEC HL	
3AF8	CD 4D 3B		CALL ASDIG	
3AFB	4F		LD C, A	
3AFC	3E 69		LD A, \$69	9 - c
3AFE	18 F2		JR L3AF2	

3.97 ASFIN: gör resultatet i a läsbart

\$3B00

Blev resultatet 0 (bit 6 = 0) tas minustecknet bort. Ledande "0" blir mellanslag. Är allt noll, eller står bara ". \$83" kvar, sätts en "0" tillbaka före \$83 eller punkten (heltal: "0", annars ".00"). Negativt: "-" skrivs direkt före första siffran; saknas plats blir det spill (bit 5).

ASRET: A = flaggorna, SP = SP0, CY = bit 5 (spill).

3B00	FD 4E 06	ASFIN:	LD C, (IY+\$06)	
3B03	CB 71		BIT 6, C	noll?
3B05	20 02		JR NZ, L3B09	
3B07	CB B9		RES 7, C	inget minustecken
3B09	2A D4 FD	L3B09:	LD HL, (ASABEG)	a:s början
3B0C	7E	L3B0C:	LD A, (HL)	
3B0D	FE 30		CP \$30	
3B0F	20 05		JR NZ, L3B16	ledande nolla
3B11	36 20		LD (HL), \$20	blir mellanslag
3B13	23		INC HL	
3B14	18 F6		JR L3B0C	
3B16	FE 83	L3B16:	CP \$83	allt noll
3B18	28 0B		JR Z, L3B25	
3B1A	FE 2E		CP \$2E	punkt...
3B1C	20 0A		JR NZ, L3B28	
3B1E	23		INC HL	
3B1F	7E		LD A, (HL)	
3B20	2B		DEC HL	
3B21	FE 83		CP \$83	...sist?
3B23	20 03		JR NZ, L3B28	
3B25	2B	L3B25:	DEC HL	
3B26	36 30		LD (HL), \$30	en "0" tillbaka
3B28	CB 79	L3B28:	BIT 7, C	negativt?
3B2A	28 12		JR Z, ASRET	
3B2C	2B		DEC HL	platsen före
3B2D	E5		PUSH HL	
3B2E	ED 5B D4 FD		LD DE, (ASABEG)	
3B32	A7		AND A	
3B33	ED 52		SBC HL, DE	
3B35	E1		POP HL	
3B36	38 04		JR C, L3B3C	före a: ingen plats
3B38	36 2D		LD (HL), \$2D	minustecken
3B3A	18 02		JR ASRET	

3B3C	CB E9	L3B3C:	SET	5,C	spill
3B3E	AF	ASRET:	XOR	A	
3B3F	79		LD	A,C	A = flaggorna
3B40	D9		EXX		
3B41	F9		LD	SP,HL	SP = SP0
3B42	CB 6F		BIT	5,A	spill?
3B44	C8		RET	Z	
3B45	37		SCF		
3B46	C9		RET		

ASDOT / ASDIG: A = siffran vid HL. Punkten hoppas över (DEC HL), och allt under "0" (mellanslag, tecken) läses som "0".

3B47	FE 2E	ASDOT:	CP	\$2E	punkt?
3B49	3E 30		LD	A,\$30	annars "0"
3B4B	C0		RET	NZ	
3B4C	2B		DEC	HL	hoppa över punkten
3B4D	7E	ASDIG:	LD	A,(HL)	
3B4E	FE 30		CP	\$30	
3B50	D0		RET	NC	
3B51	18 F4		JR	ASDOT	

ASNZ: sök framåt efter en siffra <> 0 bland C positioner (punkten hoppas över). NZ och A = siffran om den finns; Z vid \$83 eller slut.

3B53	7E	ASNZ:	LD	A,(HL)	siffra?
3B54	FE 30		CP	\$30	
3B56	38 06		JR	C,L3B5E	
3B58	C0		RET	NZ	<> "0": NZ
3B59	23	L3B59:	INC	HL	
3B5A	0D		DEC	C	
3B5B	20 F6		JR	NZ,ASNZ	
3B5D	C9		RET		
3B5E	FE 83	L3B5E:	CP	\$83	slut: Z
3B60	C8		RET	Z	
3B61	FE 2E		CP	\$2E	
3B63	20 F4		JR	NZ,L3B59	
3B65	23		INC	HL	
3B66	18 EB		JR	ASNZ	

ASCNT: E += 1; Z om E blir 32 (för långt).

3B68	1C	ASCNT:	INC	E	
3B69	3E 1F		LD	A,\$1F	
3B6B	A3		AND	E	E = 32: Z
3B6C	C9		RET		

ASBLST / ASALST: HL = b:s resp. a:s sista siffra (före \$83).

3B6D	2A D2 FD	ASBLST:	LD	HL,(ASBEND)	
3B70	2B		DEC	HL	
3B71	C9		RET		
3B72	2A D6 FD	ASALST:	LD	HL,(ASAEND)	
3B75	2B		DEC	HL	
3B76	C9		RET		

3.98 Flyttalsfunktioner: ATN EXP INT LOG LOG10 SIN COS TAN SQR

\$3B77

FFUNC (\$2DC6) anropar rutinen med HL = BC = argumentets exponent på värdestacken, och resultatet skrivs över argumentet. **FPOW** (\$2E07) använder **FLOG** och **FEXP**. **FTOI** och **ITOF** använder **FTOINT** och **INTTOF**. Arbetsminnet är samma bytes som ASCII-aritmetiken använder (därför namnen **ASBBEG** och **ASEND**):

```
$FDD0-$FDD4  x, kopia av argumentet (FLDX), exponenten i $FDD4
$FDD5-$FDD9  ett tal till: ATN x+1, LOG n*LN 10, SIN först
               INT(q), sedan vinkeln
$FDDA-$FDDE  ATN: t^2
$FDDF-$FDE3  TAN: kopia av argumentet, sedan COS
$FDE4        SINQ: SIN/COS-kvadranten
$FDE9        ASEND: adressen där resultatet ska skrivas (FLDX)
```

Serierna räknas med **FPOLY** (\$3F61). Konstanterna har bara 6 siffror och varje steg avrundas, så den sista siffran är ofta fel.

Provat (rätt värde inom parentes):

```
ATN(.001) = .000999 (.001)      ATN(1E30) = 1.57079 (1.57080)
EXP(-1) = .367883 (.367879)     EXP(2.302585) = 9.99987 (10)
LOG(2.718282) = 1.00001 (1)     LOG10(1E-5) = -5.00003 (-5)
SIN(100) = -.50633 (-.506366)   COS(10000) = -.95106 (-.952155)
```

FATN: $ATN(-x) = -ATN(x)$. Negativt x görs positivt (**FNEGX**), **ATNPOS** räknar och resultatets tecken byts sedan. $x = 0$ går till **ATNPOS**.

3B77	CD C5 3B	FATN:	CALL FLDX	x = argumentet
3B7A	F2 8A 3B		JP P, ATNPOS	x >= 0
3B7D	CD BD 3B		CALL FNEGX	x = -x
3B80	23		INC HL	HL = x
3B81	CD 8A 3B		CALL ATNPOS	ATN(-x)
3B84	2A E9 FD		LD HL, (ASEND)	resultatets exponent
3B87	2B		DEC HL	byt tecken
3B88	18 36		JR FNEG	

ATNPOS: $x \geq 0$ i \$FDD4. Med $t = (x-1)/(x+1)$, som ligger i $[-1, 1]$, är

$ATN(x) = \pi/4 + ATN(t)$, och $ATN(t) = t * P(t^2)$ med serien **ATNSER** (.999996 -.333174 .198079 -.132335 .0796263 -.0336063 .00681241, en approximation för hela $[-1, 1]$, inte Taylorserien). Nära 0 tar $\pi/4$ och $ATN(t)$ nästan ut varandra, så **ATN** av små tal tappar siffror: $ATN(.001) = .000999$.

3B8A	EB	ATNPOS:	EX DE, HL	DE = x
3B8B	21 BB 3F		LD HL, FONE	1
3B8E	01 D9 FD		LD BC, \$FDD9	\$FDD9
3B91	C5		PUSH BC	
3B92	D5		PUSH DE	
3B93	CD F7 33		CALL FADD	\$FDD9 = x + 1
3B96	C1		POP BC	
3B97	CD 60 34		CALL FSUB	x = x - 1
3B9A	E1		POP HL	
3B9B	C5		PUSH BC	
3B9C	CD EE 35		CALL FDIV	$t = (x-1) / (x+1)$
3B9F	E1		POP HL	
3BA0	01 DE FD		LD BC, \$FDDE	\$FDDE
3BA3	C5		PUSH BC	
3BA4	CD DF 34		CALL FMUL	\$FDDE = t^2
3BA7	D1		POP DE	
3BA8	01 02 3C		LD BC, ATNSER	serien
3BAB	CD 61 3F		CALL FPOLY	$P(t^2)$
3BAE	44		LD B, H	BC = t
3BAF	4D		LD C, L	
3BB0	CD DF 34		CALL FMUL	$t = t * P(t^2)$
3BB3	11 B4 3F		LD DE, FPIQ	$\pi/4$
3BB6	ED 4B E9 FD		LD BC, (ASEND)	resultatet
3BBA	C3 F7 33		JP FADD	= $\pi/4 + ATN(t)$

FNEGX: byt tecken på x (\$FDD3). **FNEG**: byt tecken på talet vars teckenbyte HL pekar på (bit 0).

3BBD	21 D3 FD	FNEGX:	LD HL, \$FDD3	x:s tecken
3BC0	3E 01	FNEG:	LD A, \$01	
3BC2	AE		XOR (HL)	byt bit 0

3BC3 77 LD (HL),A
3BC4 C9 RET

FLDX: HL = BC = argumentet. **ASEND** = argumentet (där resultatet ska skrivas) och argumentet kopieras till x (\$FDD0-\$FDD4).

Ut: HL = \$FDD4. Z om x = 0. Annars NZ, och S = tecknet (M om x < 0, genom att RRCA lägger teckenbiten i bit 7).

3BC5 60 **FLDX:** LD H,B
3BC6 69 LD L,C
3BC7 22 E9 FD LD (ASEND),HL resultatet skrivs här
3BCA 11 D4 FD LD DE,\$FDD4 x
3BCD 01 05 00 LD BC,\$0005
3BD0 D5 PUSH DE
3BD1 ED B8 LDDR kopiera 5 bytes
3BD3 E1 POP HL
3BD4 7E LD A,(HL) exponent
3BD5 A7 AND A
3BD6 C8 RET Z Z: x = 0
3BD7 2B DEC HL
3BD8 7E LD A,(HL) tecknet
3BD9 0F RRCA bit 0 -> bit 7
3BDA F6 01 OR \$01 NZ
3BDC 23 INC HL
3BDD C9 RET

ATN-serien, 7 koefficienter c0..c6. **FPOLY** går baklänges från **ATNSER** (c6); \$FF där nästa posts tecken skulle ligga avslutar.

3BDE FF FF DEFB \$FF,\$FF
3BE0 99 99 96 00... DEFB \$99,\$99,\$96,\$00,\$80 0.999996
3BE5 33 31 74 01... DEFB \$33,\$31,\$74,\$01,\$80 -0.333174
3BEA 19 80 79 00... DEFB \$19,\$80,\$79,\$00,\$80 0.198079
3BEF 13 23 35 01... DEFB \$13,\$23,\$35,\$01,\$80 -0.132335
3BF4 79 62 63 00... DEFB \$79,\$62,\$63,\$00,\$7F 0.0796263
3BF9 33 60 63 01... DEFB \$33,\$60,\$63,\$01,\$7F -0.0336063
3BFE 68 12 41 00 DEFB \$68,\$12,\$41,\$00 0.00681241
3C02 7E **ATNSER:** DEFB \$7E

FXP: $\text{EXP}(x) = 10^y$ med $y = x * \text{LOG10}(e)$ (**FLGE** = .434294). y delas i $n = \text{INT}(y)$ och $f = y - n$ i $[0, 1)$. 10^f räknas med serien **EXPSE** (Taylorserien för 10^f : $\text{LN}(10)^k/k!$) och blir 1..10. Sedan läggs n till exponenten. Över \$FF: fel 4. Under 1: resultatet 0. $|y| >= 65536$ ger fel 7 i **FTOINT**, så $\text{EXP}(1E6)$ och $\text{EXP}(-1E6)$ ger FÖR STORT HELTAL i stället för fel 4 resp. 0. Provat: $\text{EXP}(292) = 6.51628E126$, $\text{EXP}(300)$ fel 4, $\text{EXP}(-300) = 0$.

3C03 50 **FXP:** LD D,B DE = x
3C04 59 LD E,C
3C05 21 31 3D LD HL,FLGE $\text{LOG10}(e)$
3C08 CD DF 34 CALL FMUL $y = x * \text{LOG10}(e)$
3C0B C5 PUSH BC
3C0C CD C5 3B CALL FLDX kopiera y till \$FDD4
3C0F E5 PUSH HL
3C10 CD CA 3E CALL FTOINT $\$FDD4 = \text{INT}(y)$, DE = n
3C13 E1 POP HL
3C14 C1 POP BC
3C15 D5 PUSH DE n
3C16 50 LD D,B
3C17 59 LD E,C
3C18 CD 60 34 CALL FSUB $f = y - \text{INT}(y)$
3C1B D5 PUSH DE
3C1C 01 7E 3C LD BC,EXPSE serien
3C1F CD 61 3F CALL FPOLY 10^f
3C22 E1 POP HL HL = resultatet
3C23 C1 POP BC BC = n
3C24 5E LD E,(HL) $E = 10^f$ s exponent
3C25 AF XOR A
3C26 57 LD D,A
3C27 B3 OR E 0?
3C28 28 0F JR Z,FCLR5
3C2A EB EX DE,HL
3C2B 09 ADD HL,BC + n
3C2C EB EX DE,HL

3C2D	7A	LD	A,D	
3C2E	87	ADD	A,A	
3C2F	38 08	JR	C,FCLR5	< 0: resultatet 0
3C31	C2 01 07	JP	NZ,L0701	> \$FF: fel 4
3C34	B3	OR	E	
3C35	28 02	JR	Z,FCLR5	= 0: resultatet 0
3C37	73	LD	(HL),E	ny exponent
3C38	C9	RET		

FCLR5: nollställ de 5 bytes som slutar vid HL (talet blir 0).

3C39	06 05	FCLR5:	LD	B,\$05
3C3B	AF		XOR	A
3C3C	77	L3C3C:	LD	(HL),A
3C3D	2B		DEC	HL
3C3E	10 FC		DJNZ	L3C3C
3C40	C9		RET	

EXP-serien, 12 koefficienter: 1 2.30259 2.65095000241667 ($\ln(10)^k/k!$, $k = 0..11$), sista ($k = 11$) vid **EXPSE**.

3C41	FF FF	DEFB	\$FF,\$FF	
3C43	10 00 00 00...	DEFB	\$10,\$00,\$00,\$00,\$81	1
3C48	23 02 59 00...	DEFB	\$23,\$02,\$59,\$00,\$81	2.30259
3C4D	26 50 95 00...	DEFB	\$26,\$50,\$95,\$00,\$81	2.65095
3C52	20 34 68 00...	DEFB	\$20,\$34,\$68,\$00,\$81	2.03468
3C57	11 71 25 00...	DEFB	\$11,\$71,\$25,\$00,\$81	1.17125
3C5C	53 93 83 00...	DEFB	\$53,\$93,\$83,\$00,\$80	0.539383
3C61	20 69 96 00...	DEFB	\$20,\$69,\$96,\$00,\$80	0.206996
3C66	68 08 94 00...	DEFB	\$68,\$08,\$94,\$00,\$7F	0.0680894
3C6B	19 59 77 00...	DEFB	\$19,\$59,\$77,\$00,\$7F	0.0195977
3C70	50 13 93 00...	DEFB	\$50,\$13,\$93,\$00,\$7E	0.00501393
3C75	11 54 50 00...	DEFB	\$11,\$54,\$50,\$00,\$7E	0.0011545
3C7A	24 16 67 00	DEFB	\$24,\$16,\$67,\$00	0.000241667
3C7E	7D	EXPSE:	DEFB	\$7D

FINT: $\text{INT}(x)$, det största heltal $\leq x$ (golv). BC = talets exponent, talet ändras på plats.

Antalet siffror efter kommat är 6 - ($e - \$80$). $e \geq \$86$: talet är redan ett heltal. $e \leq \$80$ ($|x| < 1$): exponenten sätts till 0 och alla 6 siffror tas bort. Siffrorna nollställs bakifrån, och E samlar dem (OR), så att $E < 0$ om något togs bort. Var talet negativt och något togs bort dras 1 av: $\text{INT}(-2.5) = -3$, $\text{INT}(-.5) = -1$.

Provot: $\text{INT}(2.5) = 2$, $\text{INT}(-3) = -3$, $\text{INT}(-12345.6) = -12346$.

3C7F	0A	FINT:	LD	A,(BC)	exponenten
3C80	D6 81		SUB	\$81	- \$81
3C82	F2 8B 3C		JP	P,FINT1	≥ 1
3C85	AF		XOR	A	
3C86	02		LD	(BC),A	$ x < 1$: exponenten 0
3C87	3E FA		LD	A,\$FA	alla 6 siffror bort
3C89	18 03		JR	FINT2	
3C8B	D6 05	FINT1:	SUB	\$05	siffror efter kommat - 6
3C8D	D0		RET	NC	≥ 6 siffror före: heltal
3C8E	57	FINT2:	LD	D,A	D = -antalet
3C8F	1E 00		LD	E,\$00	E = borttagna siffror
3C91	0B		DEC	BC	tecknet
3C92	C5		PUSH	BC	
3C93	0B	FINTLP:	DEC	BC	sista kvarvarande byten
3C94	0A		LD	A,(BC)	
3C95	E6 0F		AND	\$0F	låga siffran
3C97	B3		OR	E	
3C98	5F		LD	E,A	
3C99	0A		LD	A,(BC)	
3C9A	E6 F0		AND	\$F0	behåll den höga
3C9C	02		LD	(BC),A	
3C9D	14		INC	D	räkna
3C9E	28 07		JR	Z,FINTFL	
3CA0	B3		OR	E	och den höga
3CA1	5F		LD	E,A	
3CA2	AF		XOR	A	hela byten bort

3CA3	02	LD	(BC), A	
3CA4	14	INC	D	
3CA5	20 EC	JR	NZ, FINTLP	
3CA7	C1	FINTFL: POP	BC	BC = tecknet
3CA8	7B	LD	A, E	
3CA9	A7	AND	A	inget borttaget
3CAA	C8	RET	Z	
3CAB	0A	LD	A, (BC)	positivt
3CAC	A7	AND	A	
3CAD	C8	RET	Z	
3CAE	03	INC	BC	exponenten
3CAF	59	LD	E, C	
3CB0	50	LD	D, B	
3CB1	21 BB 3F	LD	HL, FONE	1
3CB4	C3 60 34	JP	FSUB	x = x - 1

FLOG10: $\text{LOG}_{10}(x) = \text{LOG}(x) * \text{LOG}_{10}(e)$. Provat: $\text{LOG}_{10}(1000) = 3$, $\text{LOG}_{10}(2) = .301031$ (.301030).

3CB7	C5	FLOG10: PUSH	BC	
3CB8	CD C4 3C	CALL	FLOG	
3CBB	C1	POP	BC	
3CBC	59	LD	E, C	
3CBD	50	LD	D, B	
3CBE	21 31 3D	LD	HL, FLGE	$\text{LOG}_{10}(e)$
3CC1	C3 DF 34	JP	FMUL	$* \text{LOG}(x)$

FLOG: $\text{LOG}(x)$, naturliga logaritmen. $x \leq 0$ ger fel 4.

Reduktion: x dubblas (FADD med sig själv) k gånger tills första siffran är 1 ($k = 0..3$: 1 ger 0, 2 ger 3, 3-4 ger 2, 5-9 ger 1). Med exponenten d efter dubblingen sätts exponenten till \$81, så att $y = 1,d2d3\dots$ ligger i $[1, 2)$. Då är

$$\text{LOG}(x) = \text{LOG}(y) - k * \text{LN } 2 + (d - \$81) * \text{LN } 10.$$

$\text{LOG}(y) = P(y - 1)$ med serien LOGSER ($c0 = 0, .999981 - .499471 \dots$). $-k * \text{LN } 2$ hämtas ur tabellen FMLN2 ($-\text{LN } 2, -2 \text{ LN } 2, -3 \text{ LN } 2$).

3CC4	CD C5 3B	FLOG: CALL	FLDX	$x = \text{argumentet}$
3CC7	CA 01 07	JP	Z, L0701	LOG 0: fel 4
3CCA	FA 01 07	JP	M, L0701	negativt: fel 4
3CCD	58	LD	E, B	$E = k = 0$
3CCE	3A D0 FD	LOGDBL: LD	A, (ASBBEG)	första siffran
3CD1	E6 F0	AND	\$F0	
3CD3	FE 10	CP	\$10	= 1?
3CD5	28 0C	JR	Z, LOGRED	
3CD7	1C	INC	E	$k + 1$
3CD8	D5	PUSH	DE	
3CD9	54	LD	D, H	
3CDA	44	LD	B, H	
3CDB	5D	LD	E, L	
3CDC	4D	LD	C, L	
3CDD	CD F7 33	CALL	FADD	$x = x + x$
3CE0	D1	POP	DE	
3CE1	18 EB	JR	LOGDBL	
3CE3	56	LOGRED: LD	D, (HL)	D = exponenten
3CE4	36 81	LD	(HL), \$81	$y = x$ med exponent \$81
3CE6	D5	PUSH	DE	
3CE7	44	LD	B, H	
3CE8	4D	LD	C, L	
3CE9	EB	EX	DE, HL	
3CEA	21 BB 3F	LD	HL, FONE	1
3CED	CD 60 34	CALL	FSUB	$u = y - 1$
3CF0	01 6F 3D	LD	BC, LOGSER	serien
3CF3	CD 61 3F	CALL	FPOLY	$\text{LN}(1+u)$
3CF6	D1	POP	DE	
3CF7	1D	DEC	E	$k - 1$
3CF8	FA 0D 3D	JP	M, LOGEXP	$k = 0$
3CFB	D5	PUSH	DE	
3CFC	21 3B 3D	LD	HL, FMLN2	$-\text{LN } 2$
3CFF	16 00	LD	D, \$00	
3D01	3E 05	LD	A, \$05	$+ 5 * (k - 1)$

3D03	19	LOGK:	ADD	HL,DE	
3D04	3D		DEC	A	
3D05	20 FC		JR	NZ,LOGK	
3D07	50		LD	D,B	
3D08	59		LD	E,C	
3D09	CD F7 33		CALL	FADD	LN(y) - k LN 2
3D0C	D1		POP	DE	

LOGEXP: (d - \$81) blir ett flyttal (INTTOF) i \$FDD9, gånger LN 10, och läggs till LOG(y) i \$FDD4. Resultatet skrivs vid ASEND.

3D0D	C5	LOGEXP:	PUSH	BC	
3D0E	6A		LD	L,D	HL = d
3D0F	26 00		LD	H,\$00	
3D11	11 7F FF		LD	DE,\$FF7F	- \$81
3D14	19		ADD	HL,DE	
3D15	11 D9 FD		LD	DE,\$FDD9	\$FDD9
3D18	D5		PUSH	DE	
3D19	CD 18 3F		CALL	INTTOF	n = d - \$81
3D1C	D1		POP	DE	
3D1D	42		LD	B,D	
3D1E	4B		LD	C,E	
3D1F	21 36 3D		LD	HL,FLN10	LN 10
3D22	CD DF 34		CALL	FMUL	* n
3D25	ED 4B E9 FD		LD	BC,(ASEND)	resultatet
3D29	E1		POP	HL	
3D2A	C3 F7 33		JP	FADD	summan

Konstanter: LOG10(e) = .434294 (FLGE), LN 10 (FLN10), och FMLN2: -LN 2, -2 LN 2, -3 LN 2 (steg om 5 bytes).

3D2D	43 42 94 00		DEFB	\$43,\$42,\$94,\$00	0.434294
3D31	80	FLGE:	DEFB	\$80	
3D32	23 02 59 00		DEFB	\$23,\$02,\$59,\$00	2.30259
3D36	81	FLN10:	DEFB	\$81	
3D37	69 31 47 01		DEFB	\$69,\$31,\$47,\$01	-0.693147
3D3B	80	FMLN2:	DEFB	\$80	
3D3C	13 86 29 01...		DEFB	\$13,\$86,\$29,\$01,\$81	-1.38629
3D41	20 79 44 01...		DEFB	\$20,\$79,\$44,\$01,\$81	-2.07944

LOG-serien för LN(1+u), u i [0, 1): 8 koefficienter, c0 = 0, c7 vid LOGSER.

3D46	FF FF		DEFB	\$FF,\$FF	
3D48	00 00 00 00...		DEFB	\$00,\$00,\$00,\$00,\$00	0
3D4D	99 99 81 00...		DEFB	\$99,\$99,\$81,\$00,\$80	0.999981
3D52	49 94 71 01...		DEFB	\$49,\$94,\$71,\$01,\$80	-0.499471
3D57	32 82 33 00...		DEFB	\$32,\$82,\$33,\$00,\$80	0.328233
3D5C	22 58 73 01...		DEFB	\$22,\$58,\$73,\$01,\$80	-0.225873
3D61	13 46 39 00...		DEFB	\$13,\$46,\$39,\$00,\$80	0.134639
3D66	55 12 00 01...		DEFB	\$55,\$12,\$00,\$01,\$7F	-0.05512
3D6B	10 75 74 00		DEFB	\$10,\$75,\$74,\$00	0.0107574
3D6F	7F	LOGSER:	DEFB	\$7F	

3.99 FSIN / FCOS

\$3D70

FCOS: COS(x) = SIN(x + PI/2): argumentets tecken nollställs (COS är jämn) och kvadranten börjar på 1. **FSIN:** kvadranten börjar på 0. Negativt x: kvadranten + 2, och x görs positivt (SIN(-x) = SIN(x + PI)). q = |x| * 2/PI, n = INT(q), kvadranten + n (bara de två lägsta bitarna används). Vinkeln v = (q - n) * PI/2 i [0, PI/2).

Udda kvadrant: COS(v) = C(v^2) (COSSER), annars SIN(v) = v * S(v^2) (SINSER). Kvadrant 2 och 3 (bit 1): tecknet byts. q räknas med 2/PI = .63662, och n måste rymmas i 16 bitar (FTOINT), så |x| > ca 102940 ger fel 7. Provat: SIN(103000) fel 7, SIN(102900) = .309017. COS(1.570796) (läst som 1.5708) blir 00 00 00 01 00, en negativ nolla (rätt värde -.00000367), som uppför sig som 0: X=0 är sant och SGN(X) = 0. TAN(1.570796) ger därför fel 4.

3D70	AF	FCOS:	XOR	A	FCOS: kvadrant 1
3D71	0B		DEC	BC	
3D72	02		LD	(BC),A	COS(-x) = COS(x)
3D73	03		INC	BC	
3D74	3C		INC	A	

3D75	18 01	JR	FSINC	
3D77	AF	FSIN:	XOR A	FSIN: kvadrant 0
3D78	F5	FSINC:	PUSH AF	
3D79	CD C5 3B		CALL FLDX	x = argumentet
3D7C	F1		POP AF	
3D7D	2B		DEC HL	tecknet
3D7E	86		ADD A, (HL)	
3D7F	86		ADD A, (HL)	+ 2 om negativt
3D80	32 E4 FD		LD (SINQ), A	kvadranten
3D83	36 00		LD (HL), \$00	x
3D85	01 D4 FD		LD BC, \$FDD4	
3D88	C5		PUSH BC	
3D89	50		LD D, B	
3D8A	59		LD E, C	
3D8B	21 A5 3F		LD HL, F2DPI	2/PI
3D8E	CD DF 34		CALL FMUL	q = x * 2/PI
3D91	EB		EX DE, HL	
3D92	11 D9 FD		LD DE, \$FDD9	\$FDD9
3D95	D5		PUSH DE	
3D96	D5		PUSH DE	
3D97	01 05 00		LD BC, \$0005	
3D9A	ED B8		LDDR	kopiera q
3D9C	E1		POP HL	
3D9D	CD CA 3E		CALL FTOINT	INT(q), DE = n
3DA0	21 E4 FD		LD HL, \$FDE4	
3DA3	7E		LD A, (HL)	
3DA4	83		ADD A, E	kvadranten + n
3DA5	77		LD (HL), A	
3DA6	E1		POP HL	HL = INT(q)
3DA7	C1		POP BC	BC = q
3DA8	E5		PUSH HL	
3DA9	59		LD E, C	
3DAA	50		LD D, B	
3DAB	CD 60 34		CALL FSUB	q - INT(q)
3DAE	21 A0 3F		LD HL, FPIH	PI/2
3DB1	CD DF 34		CALL FMUL	v = * PI/2
3DB4	EB		EX DE, HL	
3DB5	D1		POP DE	
3DB6	E5		PUSH HL	
3DB7	E5		PUSH HL	
3DB8	E5		PUSH HL	
3DB9	01 05 00		LD BC, \$0005	
3DBC	ED B8		LDDR	kopiera v till \$FDD9
3DBE	E1		POP HL	
3DBF	D1		POP DE	
3DC0	C1		POP BC	
3DC1	CD DF 34		CALL FMUL	\$FDD4 = v^2
3DC4	3A E4 FD		LD A, (SINQ)	
3DC7	0F		RRCA	udda kvadrant?
3DC8	38 10		JR C, SINCOS	
3DCA	01 D4 3F		LD BC, SINSER	SIN-serien
3DCD	CD 61 3F		CALL FPOLY	S(v^2)
3DD0	21 D9 FD		LD HL, \$FDD9	v
3DD3	42		LD B, D	
3DD4	4B		LD C, E	
3DD5	CD DF 34		CALL FMUL	SIN(v) = v * S(v^2)
3DD8	18 06		JR SINSGN	
3DDA	01 F4 3F	SINCOS:	LD BC, COSSER	COS-serien
3DDD	CD 61 3F		CALL FPOLY	COS(v) = C(v^2)
3DE0	EB	SINSGN:	EX DE, HL	HL = resultatet
3DE1	3A E4 FD		LD A, (SINQ)	
3DE4	2B		DEC HL	tecknet
3DE5	CB 4F		BIT 1, A	kvadrant 2, 3:
3DE7	C4 C0 3B		CALL NZ, FNEG	byt tecken
3DEA	23		INC HL	
3DEB	ED 5B E9 FD		LD DE, (ASEND)	argumentets plats
3DEF	01 05 00		LD BC, \$0005	

3DF2 ED B8
3DF4 C9

LDDR
RET

kopiera

3.100 FSQR: kvadratroten, siffra för siffra med udda tal

\$3DF5

Arbetsyta IX+0..\$0F (SP = IX):

IX+0 antal kvarvarande bytes i roten (3, 2, 1)
IX+3.. o, udda tal (BCD) som dras från resten
IX+8..IX+10 roten, 6 siffror
IX+\$0C..\$0F resten; talets mantissa läggs i IX+\$0D..\$0F

Exponenten: $e = (e-\$80)/2$ avrundat uppåt, + \$80 (SRA och RL C ger C = 1 om $e-\$80$ var udda). Negativt tal: fel 50.

För varje siffra: dra o från resten, lägg 1 till rotens siffra och 2 till o (o = 1, 3, 5, ... i siffrans position C = \$10 eller \$01) tills det blir lån. Då läggs o tillbaka, o minskas med 1 i positionen C och får en etta i positionen till höger (nästa byte när C = \$01), och resten skiftas en siffra åt vänster (RLD över 4 bytes). Udda exponent börjar med o = 1 i den höga halvan, så att talet läses som .0d1d2... Efter 6 siffror kopieras roten till mantissan. Sista siffran huggs av: SQR(10) = 3.16227 (3.162278).

SQR(0): exponenten blir \$40 med mantissan 0 (inte \$00). Talet skrivs 0 och fungerar som 0 i FADD och FMUL ($X+1 = 1$, $X*5 = 0$), men $X=0$ är falskt och SGN(X) = 1. $5/SQR(0)$ fastnar i FDIV, som aldrig får lån när den drar av 0, och CTRL-C bryter inte. 9913 ger 0 direkt.

3DF5 21 FF FF	FSQR:	LD HL, \$FFFF	HL = SP0 - 1 = IX+\$0F
3DF8 39		ADD HL, SP	
3DF9 DD 21 F0 FF		LD IX, \$FFF0	IX = SP0 - 16
3DFD DD 39		ADD IX, SP	
3DFF DD F9		LD SP, IX	SP = IX
3E01 DD 36 00 03		LD (IX+\$00), \$03	3 bytes = 6 siffror
3E05 EB		EX DE, HL	DE = IX+\$0F
3E06 60		LD H, B	
3E07 69		LD L, C	
3E08 7E		LD A, (HL)	exponenten
3E09 EE 80		XOR \$80	med tecken
3E0B CB 2F		SRA A	/ 2
3E0D 0E 00		LD C, \$00	
3E0F CB 11		RL C	C = udda
3E11 81		ADD A, C	avrunda uppåt
3E12 EE 80		XOR \$80	
3E14 77		LD (HL), A	ny exponent
3E15 79		LD A, C	A = udda
3E16 2B		DEC HL	tecknet
3E17 CB 46		BIT 0, (HL)	
3E19 C2 25 07		JP NZ, L0725	negativt: fel 50
3E1C 2B		DEC HL	sista mantissabyten
3E1D E5		PUSH HL	resultatets plats
3E1E D5		PUSH DE	
3E1F 01 03 00		LD BC, \$0003	
3E22 ED B8		LDDR	mantissan till IX+\$0D..\$0F
3E24 13		INC DE	IX+\$0D
3E25 D5		PUSH DE	
3E26 1B		DEC DE	IX+\$0C
3E27 EB		EX DE, HL	
3E28 06 0A		LD B, \$0A	nollställ IX+3..IX+\$0C
3E2A 71	SQRCLR:	LD (HL), C	
3E2B 2B		DEC HL	
3E2C 10 FC		DJNZ SQRCLR	
3E2E 23		INC HL	HL = IX+4
3E2F 23		INC HL	
3E30 D1		POP DE	DE = IX+\$0D
3E31 1F		RRA	udda exponent?
3E32 30 59		JR NC, SQRBYT	
3E34 36 10		LD (HL), \$10	o = 1 i höga halvan
3E36 4E		LD C, (HL)	
3E37 3E 05	SQRLP:	LD A, \$05	antal bytes
3E39 DD 96 00		SUB (IX+\$00)	
3E3C 47		LD B, A	

3E3D	E5		PUSH HL	
3E3E	D5		PUSH DE	
3E3F	A7		AND A	CY = 0
3E40	1A	SQRSUB:	LD A, (DE)	
3E41	9E		SBC A, (HL)	resten - o
3E42	27		DAA	
3E43	12		LD (DE), A	
3E44	2B		DEC HL	
3E45	1B		DEC DE	
3E46	10 F8		DJNZ SQRSUB	
3E48	D1		POP DE	
3E49	38 1D		JR C, SQRBOR	lån: siffran klar
3E4B	E1		POP HL	
3E4C	E5		PUSH HL	
3E4D	7E		LD A, (HL)	o + C
3E4E	81		ADD A, C	
3E4F	27		DAA	
3E50	47		LD B, A	
3E51	30 08		JR NC, SQRDIG	ingen minnessiffra
3E53	2B	SQRCY:	DEC HL	
3E54	7E		LD A, (HL)	
3E55	C6 01		ADD A, \$01	minnet till bytes före
3E57	27		DAA	
3E58	77		LD (HL), A	
3E59	38 F8		JR C, SQRCY	
3E5B	21 FB FF	SQRDIG:	LD HL, \$FFFB	rotens byte (DE - 5)
3E5E	19		ADD HL, DE	
3E5F	79		LD A, C	
3E60	86		ADD A, (HL)	roten + C
3E61	77		LD (HL), A	
3E62	E1		POP HL	
3E63	78		LD A, B	o + 2C
3E64	81		ADD A, C	
3E65	77		LD (HL), A	
3E66	18 CF		JR SQRLP	
3E68	7D	SQRBOR:	LD A, L	A = L efter subtraktionen
3E69	E1		POP HL	
3E6A	E5		PUSH HL	
3E6B	D5		PUSH DE	
3E6C	2F		CPL	B = antal bytes
3E6D	3C		INC A	
3E6E	85		ADD A, L	
3E6F	A7		AND A	
3E70	47		LD B, A	
3E71	1A	SQRADD:	LD A, (DE)	
3E72	8E		ADC A, (HL)	lägg tillbaka o
3E73	27		DAA	
3E74	12		LD (DE), A	
3E75	2B		DEC HL	
3E76	1B		DEC DE	
3E77	10 F8		DJNZ SQRADD	
3E79	D1		POP DE	
3E7A	E1		POP HL	
3E7B	7E		LD A, (HL)	
3E7C	91		SUB C	o - C
3E7D	77		LD (HL), A	
3E7E	0D		DEC C	var C = \$01?
3E7F	28 05		JR Z, SQRNXT	
3E81	0E 01		LD C, \$01	nu låga halvan
3E83	34		INC (HL)	o + 1
3E84	18 0A		JR SQRSHL	
3E86	DD 35 00	SQRNXT:	DEC (IX+\$00)	nästa byte
3E89	28 13		JR Z, SQREND	alla 6 siffror klara
3E8B	23		INC HL	
3E8C	13		INC DE	
3E8D	0E 10	SQRBYT:	LD C, \$10	o + 1 i höga halvan
3E8F	71		LD (HL), C	

3E90	E3	SQRSHL:	EX (SP),HL	HL = IX+\$0F
3E91	E5		PUSH HL	
3E92	06 04		LD B,\$04	4 bytes
3E94	AF		XOR A	
3E95	ED 6F	SQRRLD:	RLD	resten en siffra åt vänster
3E97	2B		DEC HL	
3E98	10 FB		DJNZ SQRRLD	
3E9A	E1		POP HL	
3E9B	E3		EX (SP),HL	
3E9C	18 99		JR SQRLP	
3E9E	01 04 00	SQREND:	LD BC,\$0004	roten = HL + 4
3EA1	09		ADD HL,BC	
3EA2	D1		POP DE	
3EA3	D1		POP DE	DE = resultatets mantissa
3EA4	0B		DEC BC	
3EA5	ED B8		LDDR	kopiera 3 bytes
3EA7	21 10 00		LD HL,\$0010	ta bort arbetsytan
3EAA	39		ADD HL,SP	
3EAB	F9		LD SP,HL	
3EAC	C9		RET	

FTAN: $\text{TAN}(x) = \text{SIN}(x) / \text{COS}(x)$. x kopieras till \$FDDF-\$FDE3, där COS räknas, och SIN räknas på argumentets plats. $\text{COS} = 0$ ger fel 4 ($\text{TAN}(1.570796)$).

3EAD	60	FTAN:	LD H,B	
3EAE	69		LD L,C	
3EAF	11 E3 FD		LD DE,\$FDE3	\$FDE3
3EB2	D5		PUSH DE	
3EB3	C5		PUSH BC	
3EB4	D5		PUSH DE	
3EB5	01 05 00		LD BC,\$0005	
3EB8	ED B8		LDDR	kopiera x
3EBA	C1		POP BC	
3EBB	CD 70 3D		CALL FCOS	\$FDE3 = COS(x)
3EBE	C1		POP BC	
3EBF	C5		PUSH BC	
3EC0	CD 77 3D		CALL FSIN	argumentet = SIN(x)
3EC3	C1		POP BC	
3EC4	E1		POP HL	
3EC5	50		LD D,B	
3EC6	59		LD E,C	
3EC7	C3 EE 35		JP FDIV	SIN / COS

3.101 FT0INT: flyttalet vid HL till heltal i DE

\$3ECA

Talet ändras först till INT(x) på plats (**FINT**), så -2.7 ger -3. Sedan läses e - \$80 siffror med **MUL10A** (DE = DE*10 + siffran). C = e - 1 räknas ned; när den blir positiv (från \$80 till \$7F) är alla siffror lästa. Mer än 16 bitar utan tecken: fel 7. Negativt: DE = -DE. Intervallet kontrolleras bara som 0..65535 före tecknet, så 40000 ger DE = \$9C40 (-25536) och -40000 ger \$63C0, 65535 ger -1, -65535 ger 1 och 65536 fel 7 (provat).

3ECA	44	FT0INT:	LD B,H	
3ECB	4D		LD C,L	
3ECC	E5		PUSH HL	
3ECD	CD 7F 3C		CALL FINT	INT(x) på plats
3ED0	E1		POP HL	
3ED1	4E		LD C,(HL)	exponenten
3ED2	2B		DEC HL	
3ED3	46		LD B,(HL)	tecknet
3ED4	11 FD FF		LD DE,\$FFFD	
3ED7	19		ADD HL,DE	HL = första mantissabyten
3ED8	AF		XOR A	
3ED9	5F		LD E,A	DE = 0
3EDA	57		LD D,A	
3EDB	B1		OR C	x = 0
3EDC	C8		RET Z	
3EDD	0D		DEC C	C = e - 1

3EDE	7E	FTILP:	LD	A, (HL)	
3EDF	0F		RRCA		höga siffran
3EE0	0F		RRCA		
3EE1	0F		RRCA		
3EE2	0F		RRCA		
3EE3	CD 00 3F		CALL	MUL10A	DE * 10 + siffran
3EE6	38 09		JR	C, FTIERR	
3EE8	0D		DEC	C	fler?
3EE9	F2 F8 3E		JP	P, FTISGN	
3EEC	7E		LD	A, (HL)	låga siffran
3EED	CD 00 3F		CALL	MUL10A	
3EF0	23		INC	HL	nästa byte
3EF1	DA 07 07	FTIERR:	JP	C, L0707	fel 7
3EF4	0D		DEC	C	
3EF5	FA DE 3E		JP	M, FTILP	
3EF8	05	FTISGN:	DEC	B	negativt?
3EF9	C0		RET	NZ	
3EFA	EB		EX	DE, HL	
3EFB	CD EE 2A		CALL	NEGHL	DE = -DE
3EFE	EB		EX	DE, HL	
3EFF	C9		RET		

MUL10A: $DE = DE * 10 + (A \text{ AND } \$0F)$. *CY = spill (DE är då oförändrat).*

3F00	E5	MUL10A:	PUSH	HL	
3F01	62		LD	H, D	
3F02	6B		LD	L, E	
3F03	29		ADD	HL, HL	* 2
3F04	38 10		JR	C, MUL10X	
3F06	29		ADD	HL, HL	* 4
3F07	38 0D		JR	C, MUL10X	
3F09	19		ADD	HL, DE	* 5
3F0A	38 0A		JR	C, MUL10X	
3F0C	29		ADD	HL, HL	* 10
3F0D	38 07		JR	C, MUL10X	
3F0F	16 00		LD	D, \$00	
3F11	E6 0F		AND	\$0F	
3F13	5F		LD	E, A	
3F14	19		ADD	HL, DE	+ siffran
3F15	EB		EX	DE, HL	
3F16	E1	MUL10X:	POP	HL	
3F17	C9		RET		

3.102 INTTOF: heltalet i HL (med tecken) som flyttal vid DE (exponenten)

\$3F18

De 5 byten nollställs. 0 blir 0. Negativt: HL = -HL och tecknet 1. **INTDIG** delar med 10 (**IDIV**) rekursivt, så att siffrorna kommer ut den högsta först. B räknar från \$80 och blir exponenten: udda B lägger siffran i den höga halvan (RRD), jämnt B i den låga (OR), och går till nästa byte.

3F18	D5	INTTOF:	PUSH	DE	
3F19	E5		PUSH	HL	
3F1A	62		LD	H, D	
3F1B	6B		LD	L, E	
3F1C	1B		DEC	DE	DE = HL - 1
3F1D	36 00		LD	(HL), \$00	exponenten 0
3F1F	01 04 00		LD	BC, \$0004	
3F22	ED B8		LDDR		nollställ 5 bytes
3F24	EB		EX	DE, HL	DE = första mantissabyten
3F25	E1		POP	HL	HL = heltalet
3F26	C1		POP	BC	BC = exponentens plats
3F27	7C		LD	A, H	
3F28	B5		OR	L	
3F29	C8		RET	Z	0
3F2A	7C		LD	A, H	
3F2B	87		ADD	A, A	CY = tecknet

3F2C	C5		PUSH BC	
3F2D	30 07		JR NC,ITFPOS	
3F2F	CD EE 2A		CALL NEGHL	HL = -HL
3F32	0B		DEC BC	
3F33	3E 01		LD A,\$01	
3F35	02		LD (BC),A	tecknet 1
3F36	06 80	ITFPOS:	LD B,\$80	exponenten från \$80
3F38	CD 3F 3F		CALL INTDIG	
3F3B	78		LD A,B	
3F3C	C1		POP BC	
3F3D	02		LD (BC),A	exponenten
3F3E	C9		RET	
3F3F	D5	INTDIG:	PUSH DE	
3F40	C5		PUSH BC	
3F41	11 0A 00		LD DE,\$000A	
3F44	CD 88 37		CALL IDIV	HL = HL / 10, E = resten
3F47	7C		LD A,H	
3F48	B5		OR L	Z: kvoten 0
3F49	C1		POP BC	
3F4A	7B		LD A,E	A = siffran
3F4B	D1		POP DE	
3F4C	F5		PUSH AF	
3F4D	C4 3F 3F		CALL NZ,INTDIG	högre siffror först
3F50	04		INC B	exponenten + 1
3F51	78		LD A,B	
3F52	0F		RRCA	udda: höga halvan
3F53	EB		EX DE,HL	
3F54	38 06		JR C,INTDHI	
3F56	F1		POP AF	
3F57	B6		OR (HL)	låga halvan
3F58	77		LD (HL),A	
3F59	EB		EX DE,HL	
3F5A	13		INC DE	nästa byte
3F5B	C9		RET	
3F5C	F1	INTDHI:	POP AF	
3F5D	ED 67		RRD	siffran till höga halvan
3F5F	EB		EX DE,HL	
3F60	C9		RET	

3.103 FPOLY: $P(z) = c_0 + c_1 z + \dots + c_n z^n$ med Horners metod

\$3F61

In: DE = z (skrivs över med P(z)), BC = cn:s exponent (tabellens slut). HL bevaras. z kopieras till 5 bytes på stacken och cn till DE. Sedan

$$s = s * z + c(k-1)$$

för varje post bakåt i tabellen (5 bytes per post). Tabellen slutar när byten där nästa posts tecken skulle ligga har bit 7 satt (\$FF). Ut: BC = DE = P(z).

3F61	E5	FPOLY:	PUSH HL	spara HL
3F62	21 FB FF		LD HL,\$FFFB	
3F65	39		ADD HL,SP	
3F66	F9		LD SP,HL	5 bytes på stacken
3F67	C5		PUSH BC	tabellen
3F68	01 04 00		LD BC,\$0004	
3F6B	09		ADD HL,BC	stackplatsens exponent
3F6C	E5		PUSH HL	
3F6D	D5		PUSH DE	
3F6E	EB		EX DE,HL	
3F6F	03		INC BC	
3F70	ED B8		LDDR	kopiera z dit
3F72	D1		POP DE	
3F73	E1		POP HL	
3F74	E3		EX (SP),HL	HL = tabellen, (SP) = z
3F75	D5		PUSH DE	
3F76	0E 05		LD C,\$05	
3F78	ED B8		LDDR	s = cn

3F7A	D1	POP	DE	
3F7B	42	LD	B,D	BC = DE = s
3F7C	4B	LD	C,E	
3F7D	E3	POLYLP: EX	(SP),HL	HL = z
3F7E	CD DF 34	CALL	FMUL	s = s * z
3F81	E3	EX	(SP),HL	HL = tabellen
3F82	CD F7 33	CALL	FADD	s = s + ck
3F85	C5	PUSH	BC	
3F86	01 FA FF	LD	BC,\$FFFA	
3F89	09	ADD	HL,BC	HL - 6: nästa posts tecken
3F8A	C1	POP	BC	
3F8B	7E	LD	A,(HL)	
3F8C	23	INC	HL	HL - 5: nästa post
3F8D	87	ADD	A,A	bit 7: slut
3F8E	30 ED	JR	NC,POLYLP	
3F90	21 07 00	LD	HL,\$0007	ta bort 7 bytes
3F93	39	ADD	HL,SP	
3F94	F9	LD	SP,HL	
3F95	E1	POP	HL	HL tillbaka
3F96	C9	RET		

Konstanter: 2 och LN 2 (.693147, \$3FAB) används inte (ingen instruktion pekar dit). PI/2 (FPIH), 2/PI (F2DPI), PI (FPI; EXPI kopierar de 5 byten från \$3FA6), PI/4 (FPIQ). \$FF avslutar SIN-serien, och FONE (1) är dess c0.

3F97	20 00 00 00...	DEFB	\$20,\$00,\$00,\$00,\$81	2
3F9C	15 70 80 00	DEFB	\$15,\$70,\$80,\$00	1.5708
3FA0	81	FPIH: DEFB	\$81	
3FA1	63 66 20 00	DEFB	\$63,\$66,\$20,\$00	0.63662
3FA5	80	F2DPI: DEFB	\$80	
3FA6	31 41 59 00...	FPI: DEFB	\$31,\$41,\$59,\$00,\$81	3.14159
3FAB	69 31 47 00...	DEFB	\$69,\$31,\$47,\$00,\$80	0.693147
3FB0	78 53 98 00	DEFB	\$78,\$53,\$98,\$00	0.785398
3FB4	80	FPIQ: DEFB	\$80	
3FB5	FF 00	DEFB	\$FF,\$00	
3FB7	10 00 00 00	DEFB	\$10,\$00,\$00,\$00	1
3FBB	81	FONE: DEFB	\$81	

SIN-serien för $\sin(v)/v$ i v^2 : 1 (FONE) -1/3! 1/5! ... -1/11!, sista vid SINSER.

3FBC	16 66 67 01...	DEFB	\$16,\$66,\$67,\$01,\$80	-0.166667
3FC1	83 33 33 00...	DEFB	\$83,\$33,\$33,\$00,\$7E	0.00833333
3FC6	19 84 13 01...	DEFB	\$19,\$84,\$13,\$01,\$7D	-0.000198413
3FCB	27 55 73 00...	DEFB	\$27,\$55,\$73,\$00,\$7B	2.75573e-06
3FD0	25 05 21 01	DEFB	\$25,\$05,\$21,\$01	-2.50521e-08
3FD4	79	SINSER: DEFB	\$79	

COS-serien i v^2 : 1 -1/2! 1/4! ... -1/10!, sista vid COSSER.

3FD5	FF 00	DEFB	\$FF,\$00	
3FD7	10 00 00 00...	DEFB	\$10,\$00,\$00,\$00,\$81	1
3FDC	50 00 00 01...	DEFB	\$50,\$00,\$00,\$01,\$80	-0.5
3FE1	41 66 67 00...	DEFB	\$41,\$66,\$67,\$00,\$7F	0.0416667
3FE6	13 88 89 01...	DEFB	\$13,\$88,\$89,\$01,\$7E	-0.00138889
3FEB	24 80 16 00...	DEFB	\$24,\$80,\$16,\$00,\$7C	2.48016e-05
3FF0	27 55 73 01	DEFB	\$27,\$55,\$73,\$01	-2.75573e-07
3FF4	7A	COSSER: DEFB	\$7A	

Oanvänt (\$FF) till slutet av ROM:en.

3FF5	FF FF FF FF...	DEFB	\$FF,\$FF,\$FF,\$FF,\$FF,\$FF,\$FF,\$FF	
3FFD	FF FF FF	DEFB	\$FF,\$FF,\$FF	

Interpretatorns arkitektur

Två steg: kompilering och körning

En BASIC-rad kompileras redan när den matas in. Varje sats har därför två rutiner: en kompileringsrutin i `STTAB1/STTAB2` som gör om texten till intern kod, och en körrutin i `STEXE1/STEXE2` som utför koden. Uttryck kompileras till omvänd polsk notation (operanderna först, operatoren efter), och vid `LIST` görs koden om till text igen (`DETOK`, `DETEXP`). Programmet lagras alltså aldrig som text.

```
inmatad rad -> TOKLIN -> INSLIN (med radnummer)
                        -> RUNSTM (utan radnummer, direkt)
RUN          -> PRELNK -> RUNLIN -> RUNSTM -> EXESTM
LIST, ED     -> DETOK / DETEXP -> text
```

Mellan inmatningen och körningen ligger ett tredje steg: när `RUN` ges går `PRELNK` en gång genom hela programmet och byter radnummer och variabelnamn mot adresser. Det är Johan Finnveds uppläggning, och han beskriver den i sitt examensarbete om BASIC:en (Finnved 1979). Koden tar mer plats än texten, men vid körningen behöver ingenting läsas om, sökas eller jämföras med nyckelord.

Minnet

Vid kallstart letar `INIT` efter den lägsta sidan om 256 bytes med skrivbart minne, nedåt från `$FA00` men inte under `$8000`. Där börjar programmet, `BOFA`; med maskinens 16 KB är det `$C000`. Sedan ligger minnet så här:

```
BOFA - EOFA    programmet; EOFA pekar på slutmarkeringen $01
EOFA+1 - HEAP  variablerna, FOR-blocken och filernas FCB
HEAP - SP      ledigt
SP - $FAFF     Z80-stacken, som börjar på STKTOP = $FB00
$FB00 - $FCFF  kassetts två blockbuffertar
$FDD0 - $FEFF  systemvariablerna (IY = $FE16) och radbufferten
                BUF1 på $FE40
```

Heapen växer uppåt från programmet och stacken nedåt mot den. `MEMRES` reserverar plats på heapen och kräver att det finns 100 bytes kvar till `SP`; `STKCHK` gör samma prov när stacken växer, och båda ger fel 3 när det inte räcker. Variablerna har ingen egen plats: de ligger direkt efter programmet, och när programmet ändras flyttas de inte utan raderas. `CLEAR` sätter `HEAP = EOFA + 1`, tömmer variabellistan (`VARLST`) och stänger alla filer, och det görs vid varje ändring av en rad, vid `RUN` och när programmet tar slut.

Kommandoloopen

`CMDLP` sätter stacken till `STKTOP` och läser en rad med `INLINE` till `BUF1`, högst 120 tecken. Börjar raden med ett direktkommando (`CMDKW`: `RUN`, `LIST`, `LOAD`, `SAVE`, `ED`, `REN`, `NEW` ...) körs det via `CMDTAB` med `READY` som återhopp. Annars görs raden om till intern kod av `TOKLIN`. En rad med radnummer läggs in i programmet av `INSLIN`, som flyttar resten av programmet och gör `CLEAR`, eftersom variablerna ligger direkt efter programmet. Bara ett radnummer raderar raden.

En rad utan radnummer, eller med numret 0, kompileras på samma sätt men körs direkt: `LNKELM` länkar den (variablerna får plats på heapen som vid `RUN`) och `RUNSTM` utför den. Efter `STOP`, `BREAK` eller ett fel finns variablerna kvar, så `PRINT A` visar vad `A` var; `END` tömmer dem. `GOTO`, `GOSUB` och `STOP` ger fel 2 i direktläge, och det finns inget `CON`: ett program kan bara startas från början med `RUN`.

`ED n` gör om rad `n` till text (`DETOK`) och lägger den som mall för `INLINE`. Varje `TAB` hämtar mallens nästa tecken, och den nya raden går sedan samma väg som en inskriven rad, så den kan lika gärna vara ett kommando eller få ett annat radnummer.

Kompilering av en sats

TOKLIN kompilar satserna, åtskilda av ":", en i taget. **TOKKW** letar upp nyckelordet med **KWSRCH**, först i **STKW1** och sedan i **STKW2**, skriver satsens token och anropar kompileringsrutinen. Hittas inget nyckelord tolkas satsen som en tilldelning (underförstådd **LET**, token \$83). **KWSRCH** jämför texten med orden i tabellen i tur och ordning och hoppar över blanktecken, så att "ON ERROR GOTO" och "ONERRORGOTO" ger samma token.

Satserna i **STKW1** får token \$80-\$96. Där finns de vanligaste satserna, och några token har ingen egen text: \$81 är **GOTO** efter **THEN** eller **ELSE**, \$83 **LET** utan ordet, \$85 ett ";" som betyder **PRINT**, och \$87 följd av en längd är hoppet förbi **THEN**- eller **ELSE**-delen i en **IF**. De övriga satserna (**DIM**, **POKE**, **OPEN**, **REM**, **END** ...) står i **STKW2** och skrivs som \$86 följd av \$80 + satsens nummer i tabellen. Varje kompileringsrutin skriver satsens kod och ger **CY** och en felkod om texten är fel. Koderna består av några få slags element (**SKPELM**):

\$0D	radens slut
\$86 \$80+k	sats k i tabell 2
\$87 n	hopp framåt n bytes (IF , ELSE)
\$BF n1 nh al ah	radreferens: radnumret och målradens adress
\$BE k + k*4 bytes	lista med k radreferenser (ON ... GOTO)
\$96 ...	DEF FN : namnet och parametrarna
\$C0-\$FF ...	ett uttryck
\$00-\$7F	tecken som kopieras, t.ex. ", " och "; "

Ett uttryck är alltså ett element bland andra, och satserna själva har nästan ingen egen kod: **PRINT A;"X"** är token \$84, ett uttryck, ett ";" och ett uttryck till. En rad som inte går att kompilera när ett program läses från en fil sparas som token \$95 följd av den råa texten, och ger fel 53 om den körs.

Uttryckskompilatorn

CMPEXP går rekursivt genom prioritetsnivåerna **PREC1-PREC10** (se bilagan om tokens), från **EQV** med lägst prioritet över **IMP**, **OR** och **XOR**, **AND**, **NOT**, jämförelserna, + och -, * och / till ^, och sist de primära uttrycken. **PRECLV** hanterar en nivå med binära operatorer: den kompilar den vänstra operanden på nästa nivå, letar upp operatoren med **KWSRCH**, kompilar den högra operanden och skriver sedan operators token. Alla nivåer räknas därför från vänster: **A-B-C** är **(A-B)-C**. **PRIMARY** kompilar konstanter, variabler, funktioner och parenteser.

Typerna kontrolleras redan här. Varje operator har en token per typ (bas + 0 flyttal, + 1 heltal, + 2 sträng), och där heltal och flyttal blandas skjuts omvandlingstoken \$D9 (flyttal till heltal) eller \$DA (heltal till flyttal) in (**TYPCNV**, **INSCNV**). Ett tal utan "%" är ett flyttal, så $7\%/2 = 3.5$ men $7\%/2\% = 3$. Sträng och tal blandade ger fel 17 redan när raden skrivs in. Vilka argument en funktion tar och vilken typ den ger står i tabellen **FNTYPE**, och argumenten omvandlas efter den. De logiska operatorerna räknar med heltal, där -1 är sant och 0 falskt.

En variabel blir fem bytes: en token ($\$C0 + 4 * \text{antal index} + \text{typ}$), en namnbyte med siffran, antalet index och typen, bokstaven och två bytes för adressen, som fylls i vid **RUN**. Indexen kompileras före variabeln. Ett värde som ska läsas, och inte tilldelas, följs av \$D4 + typ. Parenteserna finns kvar som token \$D3, som inte gör något vid körning men gör att **LIST** kan skriva uttrycket med samma parenteser som användaren skrev. Ett uttryck slutar med en sluttoken \$B6-\$BE, som anger resultatets typ och vad som ska göras med det: tilldelas (\$B6-\$B8), lämnas som värde (\$B9-\$BB) eller lämnas som en variabls adress (\$BC-\$BE, för **INPUT** och **READ**).

Ett program i minnet

Programmet

```
10 A=B+1
30 PRINT A;"X"
40 END
```

ligger efter **RUN** så här från \$C000 (rad 30 börjar på \$C018):

18 0A 00	rad 10, 24 bytes
83	LET utan ordet
C0 F0 41 39 C0	variabeln A, värdet på \$C039
C0 F0 42 42 C0	variabeln B, värdet på \$C042
D4	hämta B:s värde
C3 10 00 00 00 81	flyttalskonstanten 1
F4	+ för flyttal
B6	tilldela flyttalet
0D	radens slut
12 1E 00	rad 30, 18 bytes

```

84                                PRINT
C0 F0 41 39 C0 D4 B9          A:s värde, slut: flyttal
3B                                ","
CB 22 01 58 BB                strängen "X", slut: sträng
0D
06 28 00 86 8A 0D            rad 40: END, nummer $0A i tabell 2
01                            programmets slut (EOFA = $C030)
F0 41 3A C0 10 00 00 00 81    variabeln A = 1, nästa post $C03A
F0 42 00 00 00 00 00 00      variabeln B = 0, sista posten

```

Koden för $A=B+1$ är omvänd polsk notation: B, 1, +, och sist tilldelningen till A, vars adress kom först. En variabelpost på heapen har namnbyten och bokstaven, en länk till nästa post och värdet; adressen i koden pekar på flyttalets sista byte, exponenten, som alla flyttalsrutiner utgår från. Vid END gör CLEAR heapen tom igen, men byten står kvar tills något skriver över dem.

Förlänkning vid RUN

Innan programmet startar går PRELNK igenom hela programmet en gång, element för element (LNKELM):

- Radreferenser (\$BF, och listorna \$BE för ON) får adressen till målraden. Finns raden inte blir det fel 6.
- Variablerna i uttrycken slås upp i variabellistan, som börjar i VARLST. En ny variabel får en nollställd post på heapen (LNKNEW), och adressen till värdet skrivs in i koden (LNKEXP).
- FOR och NEXT paras ihop. Varje FOR får ett block på heapen, och dess adress skrivs in både i FOR- och i NEXT-satsen (LNKFOR, LNKXNT). En FOR utan NEXT ger fel 24.
- Ett FN-anrop får adressen till sin DEF-rad (LNKFN), och parametrarnas namn känns igen inne i DEF-uttrycket.

Inget körs förrän hela programmet är länkat: ett fel i sista raden stoppar programmet innan första raden har körts. Vid körning behövs sedan ingen sökning efter rader eller variabler. Länkningen görs om vid varje RUN, eftersom CLEAR har tömt heapen; adresserna i koden skrivs då över med nya. REN använder samma länkning (LNKREF): först får alla referenser målradens adress, och sedan får varje referens det nya numret från raden som adressen pekar på.

Körning

RUNLIN går från rad till rad och RUNSTM från sats till sats. För varje rad sparas adressen i CURLIN (för felmeddelandet) och SP i STMSP, och mellan satserna provas BREAK. EXESTM utför en sats: token \$80-\$96 är index i STEXE1, och \$86 följt av en kod är index i STEXE2. Uttryckskoden körs av EXEXP, som nås med RST \$38: varje token \$B6-\$FF slår upp sin rutin i EXPTAB1, och funktionerna har sina rutiner i EXPTAB2. Rutinerna slutar med RST \$30 (nästa token) eller RST \$28 (lägg HL på stacken och ta nästa token). Värdena ligger på Z80-stacken:

```

heltal    2 bytes
flyttal   5 bytes: mantissa, tecken, exponent
sträng    [antal tillfälliga bytes] [adress] [längd] + bytes

```

En sträng som räknas fram, till exempel av CHR\$ eller +, byggs direkt på stacken under sin beskrivning. LEFT\$, RIGHT\$ och MID\$ kopierar ingenting utan ändrar bara adressen och längden.

Hoppen är enkla, eftersom adresserna redan står i koden: GOTO tar adressen ur radreferensen och fortsätter i RUNLIN där. GOSUB anropar RUNLIN med CALL, och RETURN tar bort RUNSTM:s och RUNLIN:s återhoppadresser och gör RET, tillbaka till satsen efter GOSUB. Varje GOSUB-nivå binder alltså en del av Z80-stacken. Längst ned lägger RUN adressen till fel 29, så att en RETURN utan GOSUB hamnar där.

FOR räknar fram startvärdet, gränsen och steget en gång och lägger gränsen och steget i FOR-blocket, som också har variabelns adress, adressen till slingans början och adressen efter NEXT. NEXT behöver därför bara addera steget, jämföra med gränsen och fortsätta på en av de två adresserna. Är startvärdet redan förbi gränsen hoppar FOR direkt förbi NEXT.

En egen funktion, DEF FNA(X)=..., körs av FN: argumenten räknas fram på stacken, FNFRAM pekar på dem, och RST \$38 räknar ut DEF-uttrycket. En parameter har i koden en liten "adress" under \$100, som FNPAR översätter till argumentets plats på stacken. Funktionerna kan ha högst fyra parametrar och bara ge tal.

Variablerna

Variabelnamnet är en bokstav (A-Z eller ÄÖÅ), kanske en siffra, och "%" för heltal eller "\$" för sträng. Namnbyten skiljer också på hur många index variabeln har, så A, A%, A\$, A(..) och A(,...) är fem olika variabler. Posterna på heapen har ett huvud på fyra bytes och sedan värdet:

flyttal	5 bytes
heltal	2 bytes
sträng	6 bytes: reserverad längd, adress, längd
vektor, matris	6 eller 10 bytes: elementens adress, antal, elementets storlek och de högsta indexen

En sträng får sin plats vid första tilldelningen: 80 bytes, eller textens längd om den är längre. Platsen stannar hos variabeln, och en längre text senare ger fel 10. Med DIM A\$=n går längden att välja. Ingen skräpsamling behövs alltså. Ett fält som används utan DIM får 10 som högsta index (DIM1, DIM2), och indexen börjar på 0.

LIST

LIST gör om varje rad till text med DETOK. Radnumret skrivs med PRNUM, nyckelorden hämtas ur STKW1 och STKW2, och uttrycken skrivs av DETEXP, som gör om den omvända polska notationen till vanlig infix. Den skriver operanderna i den ordning de står och flyttar sedan operatorns text bakåt till sin plats mellan dem (ROTTXT); token \$D3 sätter tillbaka parenteserna. Mellanslag sparas inte, så "PRINT A; B" listas "PRINT A;B", och talkonstanterna skrivs som FLTASC skriver dem: "1.50" blir "1.5". Text i REM och i strängar står som den skrevs.

Tal

Flyttalen är decimala: 6 BCD-siffror i mantissan, ett teckenbyte och en exponent med förskjutningen \$80 (se FADD-avsnittet). Därför blir till exempel 0,1 exakt. Inläsningen (GETFLT) tar de sex första siffrorna och hugger av resten, och utskriften (FLTASC) avrundar till sex siffror och väljer själv mellan vanlig form och E-form. Talen går från .1E-127 till .999999E127. De matematiska funktionerna räknas med polynom (FPOLY) och med konstanter om sex siffror, så den sista siffran är ofta fel. Heltalen är 16 bitar med tecken och slår runt utan fel: $32767\% + 1\% = -32768$.

ADD\$, SUB\$, MUL\$, DIV\$ och COMP% räknar med tal skrivna som text, siffra för siffra, med upp till 29 tecken och ett valt antal decimaler (ASCII-aritmetik).

Felhantering

Ett fel ges med RST \$10 följt av en byte med felkoden (\$80 + nummer), eller genom ett hopp till en av felgångarna, där varje felkod har en egen RST \$10. Är felet ett av de 16 i listan ERRTRP och ON ERROR GOTO är aktivt sparas koden (ERRCODE ger den), stacken återställs från STMSP och programmet fortsätter på ON ERROR-raden. ON ERROR är då avstängt och måste köras igen. Annars stoppar ERRSTP programmet, och ERRMSG skriver "ERR n LINE m". Fel som upptäcks när en rad skrivs in skrivs ut direkt. ROM:en innehåller inga feltexter; texterna i bilagan kommer ur ABC80:s fellista, och en tilläggs-ROM kan skriva ut dem.

In- och utmatning

Skärmen (PRTXT) har 24 rader om 40 tecken i bildminnet på \$7C00. Tangentbordet läses av avbrottsrutinen KEYINT, och INLINE läser en rad. Filer går genom enhetslistan (DEVHEAD): varje enhet har ett namn och en hopptabell med nio rutiner (OPEN, PREPARE, CLOSE, INPUT, PRINT, GET, PUT ...), som anropas via DVCALL. Varje öppna fil har ett FCB på heapen med en pekare till enhetens post. ROM:en har själv bara kassetten, CAS:, som också är standardenhet. Kassetten läses med avbrott, ett per bit (CASINT). PR: och IEC: finns i tilläggskretsar, och en tilläggs-ROM kan lägga till satser, funktioner och enheter genom ingångarna EXTJP1/EXTJP2.

SAVE skriver programmet som det står i minnet, den kompillerade koden rad för rad, till en BAC-fil. LOAD läser först filen som text och kompilerar varje rad som om den hade skrivits in; ger det fel 58 (tecken över \$7F) är det en BAC-fil, och raderna läggs in som de är. LIST till en fil skriver texten, så att programmet kan läsas av en annan dator eller en annan version.

Hårdvaran

Minneskarta

Så som ROM-koden använder adressrymden:

\$0000-\$3FFF	BASIC-ROM:en (16 KB)
\$6000-\$6FFF	tilläggs-ROM (skivminne); anropas via \$604B m.fl.
\$7000-\$73FF	tilläggs-ROM (IEC); anropas via \$7000-\$7006
\$7800-\$7BFF	tilläggs-ROM för skrivaren (enheten "PR ")
\$7C00-\$7FFF	bildminne, 24 rader x 40 tecken
\$8000-\$BFFF	RAM-utbyggnad
\$C000-\$FFFF	RAM (16 KB)

I RAM:et:

BOFA-	programmet, därefter variablerna (HEAP)
-\$FAFF	stacken, som börjar på \$FB00 (STKTOP)
\$FB00-\$FCFF	kassetts två blockbuffertar
\$FDD0-\$FEFF	systemvariabler (IY = \$FE16), klockan \$FDF0, radbufferten BUF1 på \$FE40

Samma uppdelning finns i Markesjös minneskarta: \$0000-\$3FFF BASIC, \$4000 fritt, \$6000 4 KB ROM för skivminnet, \$7000 1 KB ROM för IEC, \$7400 fritt, \$7800 1 KB ROM för skrivaren, \$7C00 1 KB bildminne, \$8000-\$BFFF plats för extra RAM och \$C000-\$FFFF det inbyggda dynamiska RAM:et (Markesjö 1978, s. 96, fig 5.6). Vid RESET letar INIT efter minnets början: med början på \$FA00 och sedan nedåt skriver den det inverterade värdet till första byten i varje sida om 256 bytes, jämför och återställer byten. Den lägsta sidan i en obruten följd där det går blir BOFA, dock aldrig under \$8000. Med bara det inbyggda minnet blir BOFA alltså \$C000. Tilläggs-ROM:arna startas inte vid RESET, utom den på \$6000: står det JP (\$C3) på \$604B anropas den. De andra nås via EXTJP1/EXTJP2 och enhetslistan, och saknas de blir det fel 8.

I/O-portar

Alla portar som ROM:en använder, med de ställen där de används:

port	riktning	användning	ställe
\$01	ut	ABC-bussen: kanalval	INP, OUT
\$06	ut	ljudgeneratorn	INIT, pip, KEYINT
\$07	in	ABC-bussen: återställning	INIT
\$38	in	PIO A data: tangentbordet	GETKEY, KEYINT
\$39	ut	PIO A styrning	INIT
\$3A	in/ut	PIO B data: kassetten	CASxxx, RCVSTA
\$3B	ut	PIO B styrning	INIT, RCVSTA, RCVBLK

Satserna INP och OUT skriver först kanalvalet (PORT1V) till port \$01 och läser eller skriver sedan den angivna porten, så att de når ett kort på ABC-bussen. Övriga portar används inte av ROM:en själv. Enligt Markesjö är portarna på ABC-bussen (Markesjö 1978, s. 96, fig 5.6):

IN 0	data in	OUT 0	data ut
IN 1	status	OUT 1	kanalval
IN 7	återställning	OUT 2-5	kommando C1-C4
		OUT 6	ljudgeneratorn

PIO:n väljs när adressbit 4 är 1 (portarna \$38-\$3B); bit 0 väljer port A eller B och bit 1 data eller styrning (Markesjö 1978, s. 120). IN (\$07) ger signalen RSTB, som återställer bara enheterna på ABC-bussen, inte processorn (Markesjö 1978, s. 122). INIT gör det som det första den gör.

Avbrott

Z80:an går i läge 2 med I = 0, så vektortabellen ligger i ROM:en på \$0034, mellan RST-vektorerne:

```
$0034 KEYINT tangentbordet, PIO A (vektor $34)
$0036 CASINT kassetten, PIO B (vektor $36)
```

PIO A får sin vektor vid RESET, PIO B först när en mottagning från kassetten startar (RCVSTA). Andra avbrott har ingen vektor i tabellen. Ett kort på ABC-bussen som begär avbrott får vektorn \$FF, eftersom databussen då dras upp till \$FF (Markesjö 1978, s. 120); ROM:en har ingen rutin för det.

Klockan (NMI)

NMI (\$0066) hoppar till NMICKL. Den räknar ned CLOCK, ett tal om 24 bitar på \$FDF0-\$FDF2, med 1 per NMI: den låga byten med DJNZ, som inte ändrar flaggorna, och vid noll de två övre med DEC BC. Rutinen slutar med RETN. Pulserna kommer från bildräknaren, 50 per sekund, en per bild (Markesjö 1978, s. 221-222). Klockan räknas alltså ned, och adresserna 65008-65010 som Markesjö anger är desamma som \$FDF0-\$FDF2.

Tangentbordet (PIO A, port \$38/\$39)

INIT ställer in PIO A:

```
$34 avbrottsvektor $34
$B7 avbrott på, aktivt hög, mask följer
$7F masken: bara bit 7 ger avbrott
$FF läge 3 (bitvis)
$FF alla bitar ingångar
```

Bit 7 är strobesignalen (1 när en tangent är nedtryckt), och bit 0-6 är teckenkoden. Tangentbordet sitter på port A (Markesjö 1978, s. 122); koderna för varje tangent med SHIFT och CTRL står i fig 6.2 (s. 134). KEYINT läser porten vid avbrottet, sätter BRKFLG om koden är \$83 (CTRL-C med strobebiten), skriver \$80 till ljudporten och sätter KEYFLG. GETKEY väntar på KEYFLG och läser tecknet ur porten; samtidigt läser den bit 7 direkt för autorepetitionen (en lång fördröjning, \$46, sedan en kortare, 8).

Skärmen

Bildminnet på \$7C00-\$7FFF har 24 rader om 40 tecken, men raderna ligger inte i ordning. Radtabellen ROWTAB (\$0374) ger radernas adresser:

```
rad 0-7    $7C00, $7C80, $7D00 ... $7F80
rad 8-15   $7C28, $7CA8, $7D28 ... $7FA8
rad 16-23  $7C50, $7CD0, $7D50 ... $7FD0
```

De sista 8 bytes i varje grupp om \$80 (\$7C78-\$7C7F osv.) används inte. Uppställningen kommer av att videokretsen räknar fram adresserna i rutor om 8 x 8 tecken, så att 24 x 40 tecken får plats i 1 KB (Markesjö 1978, s. 148-150, fig 6.16-6.18). CURADR räknar ut markörens adress via ROWTAB. CLS och rullningen arbetar rad för rad. Markören är bit 7 i tecknet i bildminnet; GETKEY sätter den medan den väntar. ROM:en programmerar inga videokretsar; den skriver bara i bildminnet.

Ljudet (port \$06)

Port \$06 går till en latch (74LS273), vars utgångar styr ljudgeneratoren SN76477 (Markesjö 1978, s. 97). Bitarna betyder (Markesjö 1978, s. 114, fig 5.19):

```
bit 0      1 = på, 0 = av
bit 1-2    VCO: 00 hög, 01 låg, 10 och 11 styrd av SLF
bit 3-5    blandning: 000 VCO, 001 brus, 010 SLF, 011 VCO+brus,
             100 SLF+brus, 101 SLF+VCO, 110 alla tre, 111 tyst
bit 6-7    envelope: 00 VCO, 01 rakt igenom, 10 monovippa,
             11 VCO med växlande polaritet
```

ROM:en använder tre värden:

```
$00 INIT, och före pipet      av
```

\$83 pipet (BEL, \$07) på, låg VCO, monovippa
\$80 KEYINT, varje tangent av, monovippa

Pipet är alltså en ton som monovippan gör kort, och \$00 före \$83 ger ett nytt startögonblick för den (?). Varför KEYINT skriver \$80 framgår inte av koden; det stänger av en ton som fortfarande låter (?).

PIO B (port \$3A/\$3B): V24 och kassetten

INIT ställer PIO B i läge 3 (\$CF) med bit 0-2 och 7 som ingångar och bit 3-6 som utgångar (\$87). Bitarna är kopplade så här (Markesjö 1978, s. 122-124 och 164-166):

bit 0	in	V24: RxD, data in (stift 3)
bit 1	in	V24: CTS (stift 5)
bit 2	in	V24: DCD (stift 8)
bit 3	ut	V24: TxD, data ut (stift 2)
bit 4	ut	V24: RTS (stift 4)
bit 5	ut	kassetten: motorreläet
bit 6	ut	kassetten: skrivsignalen, och nollställning av flankklatchen vid läsning
bit 7	in	kassetten: läst signal (flankklatchen)

V24-bitarna går genom linjekretsarna MC1489 (in) och MC1488 (ut), som inverterar. DTR (stift 1) ligger fast på +9 V (Markesjö 1978, s. 122-123, fig 5.24, och s. 208). ABC80-ROM:en använder inte V24-bitarna; det finns ingen V24-enhet i ROM:en.

Kassetten

Motorn startas med \$20 till port \$3A (bit 5) och stoppas med \$00. Vid skrivning väntar CASPRE en stund efter motorstarten innan namnblocket skrivs. Skrivningen gör programmet självt: CASHLF väntar och vänder bit 6. En nollbit är en halv period om $2 \times \$96$ DJNZ-varv, en etta är två halva om $2 \times \$4B$, så varje bitcell slutar med en flank och en etta har en flank till i mitten. Byten skrivs med den lägsta biten först. Det är FM-metoden med en klockfrekvens på ca 800 Hz (Markesjö 1978, s. 167). Räknat ur koden tar en nolla 3993 T och en etta 4065 T (anropet, CASHLF, RR C och hoppet tillbaka), 1,333 resp. 1,357 ms vid 2,9952 MHz, alltså 750 resp. 737 bitar/s; Luxor anger 700 baud (servicemanualen s. 2). Ett block är $256 + 8 \times 264 = 2368$ bitar, ca 3,2 s. Motorväntan i CASPRE är 12×65536 varv à 26 T, ca 6,8 s. NMI-klockan stjälar ca 110 T per 20 ms och ändrar inte detta märkbart. I 9913 är andra halvan av en etta LD A,\$4B i st. f. LD A,\$96 / SRL A,8 T kortare. Varje block har samma form (CASWBL):

```
256 nollbitar
3 x SYN ($16), STX ($02)
256 bytes data
ETX ($03)
checksumma, 2 bytes (summan av data och ETX)
$00 (CASWBL faller igenom in i CASWBY med C = 0)
```

Namnblocket börjar med \$FF \$FF \$FF och namnet (11 tecken) och är resten nollor. Ett datablock börjar med \$00 och blocknumret (2 bytes, från 0). Samma format beskriver Markesjö (Markesjö 1978, s. 193-194, fig 7.8). Läsningen sker med avbrott, ett per bitcell. RCVSTA ställer in PIO B:

```
$36 avbrottsvektor $36
$97 avbrott på, aktivt låg, mask följer
$7F masken: bara bit 7 ger avbrott
```

och skriver \$20 och sedan \$60 till port \$3A (motorn på, bit 6 = 1). Inspelningen passerar en deriverande länk och två komparatorer, som sätter en latch vid varje flank; latchen är kopplad till bit 7 (Markesjö 1978, s. 166). CASINT kommer på flanken i början av bitcellen, sätter bit 6 till 0 och sedan till 1 (nollställer latchen), väntar ungefär 3/4 bitcell och läser bit 7: har latchen satts igen under tiden fanns en flank i mitten, och biten är en etta. Biten är bit 7 inverterad. Rutinen tar sedan emot biten i en korutin (RCVBLK, RCVBYT, RCVBIT), som letar SYN bit för bit, tar emot STX, 256 bytes, ETX och checksumman och lägger blocket i en av buffertarna \$FB00/\$FC00 (växelvis). Till sist slås avbrotten från PIO B av (\$07 till \$3B). Markesjö beskriver samma förlopp: latchen nollställs med bit 6 ca 0,4 ms efter avbrottet, och biten avgörs av om en ny flank kommer (Markesjö 1978, s. 167, fig 6.31).

ABC-bussen

ROM:en använder bara ABC-bussens återställning (IN (\$07)) och kanalvalet i INP och OUT. Kortet väljs med ett nummer till port \$01; sedan är port 0 data och 1 status, och 2-5 ger kommandon (Markesjö 1978, s. 96). Skrivaren, IEC och skivminnet sköts av tilläggs-ROM:arna.

Felkoder

Felkoden är \$80+n. Den ges med RST \$10 följt av koden, eller med ett hopp till en felgång (en RST \$10 med koden efter). ROM:en skriver bara ut ERR n; texterna är ABC80:s fellista. *Fångas* betyder att felet står i **ERRTRP** och kan tas om hand av ON ERROR GOTO. Kolumnen *ges i* är de namngivna rutiner där koden ger felet (enligt kommentarerna och hoppen till felgångarna); listan är inte nödvändigtvis fullständig.

Nr	Kod	Text	Fångas	Ingång	Ges i
0	\$80	EJ TILLÅTET ÖKA "DIM"		L06F9	DIM1, DIMSTR
1	\$81	FEL ANTAL INDEX		L06FB	LNKSR
2	\$82	OTILLÅTET SOM KOMMANDO		L06FD	NODIR, LNKFOR, RETURN
3	\$83	MINNET FULLT		L06FF	INSLIN, CMDMERG, FN, CONCAT, STRFIL, EXVAL, DIM1, DIM2, DIMSTR, MEMFUL
4	\$84	FÖR STORT FLYTTAL	ja	L0701	POWI, FADDC, FMUL, FMRND, FDIV, FDRND, FOVFL, FEXP, FLOG
5	\$85	FÖR STORT INDEX		L0703	VAR1, ERRIDX
6	\$86	HITTAR EJ DETTA RADNUMMER		L0705	CMDDED, REFADR, LNKELM
7	\$87	FÖR STORT HELTAL	ja	L0707	SMULDV, FTIERR
8	\$88	FINNS EJ I DETTA SYSTEM		L0709	EXTJP1, EXTJP2, STXE1, STXE2, LNKELM, STTAB1, STTAB2, FN, EXPTAB2
9	\$89	INDEX UTANFÖR STRÄNGEN		L070B	EXLEFT, STRSKP
10	\$8A	TEXTEN FÅR EJ PLATS I STRÄNGEN		L070D	ASGNS
11	\$8B	FÖRSTÅR EJ			TOKKW, CMPPR, VARNAM
12	\$8C	FELAKTIGT TAL	ja	L0713	RDVAL, CMPNUM, EXVAL
13	\$8D	FEL ANTAL ELLER TYP AV ARGUMENT			LNKFN, CMPDEF, CMPFN, CMPINP, CMPPOK, TYPCNV, CMPARG
14	\$8E	OTILLÅTET TECKEN EFTER SATSEN			TOKLIN
15	\$8F	"=" SAKNAS ELLER PÅ FEL PLATS			TOKKW, CMPDEF, CMPLT
16	\$90	RADNUMMER SAKNAS			CMDMERG, CMPREF, CMPON
17	\$91	OTILLÅTEN BLANDNING AV TAL OCH STRÄNGAR			CMPSV, CMPGET, TOFLT, CMPADD
18	\$92	")" SAKNAS ELLER PÅ FEL PLATS			CMPDEF, CMPFN, PRIMARY
19	\$93	KAN EJ ÖPPNA FLER FILER	ja		
20	\$94	FÖR LÅNG RAD			TXTINP
21	\$95	HITTAR EJ FILEN	ja	L070F	OPEN
22	\$96	OTILLÅTEN SATS		L0711	
23	\$97	"TO" SAKNAS			CMPFOR
24	\$98	"NEXT" SAKNAS		L0715	PRELNK
25	\$99	FELAKTIG SATS EFTER "ON"			CMPON
26	\$9A	FEL I ON-UTTRYCK		L071D	ON
27	\$9B	"NEXT" UTAN "FOR"		L0719	LNKNXT
28	\$9C	FEL VARIABEL EFTER "NEXT"		L0717	LNKNXT
29	\$9D	"RETURN" UTAN "GOSUB"		L071B	
30	\$9E	DATA SLUT	ja	L071F	READ
31	\$9F	FEL DATA TILL KOMMANDO		L0721	CMDDED, CMDLIST, CMDREN
32	\$A0	FILEN EJ ÖPPNAD		L0723	CURFIL
33	\$A1	"AS FILE" SAKNAS			CMPPOP
34	\$A2	SLUT PÅ FILEN	ja		INPUT
35	\$A3	CHECKSUMMAFEL VID LÄSNING	ja		RCVBLK
37	\$A5	FELAKTIGT RECORDFORMAT	ja		CASOPN, CASGET, RCVBLK
38	\$A6	RECORDNUMMER UTANFÖR FILEN	ja		
50	\$B2	KVADRATROT UR NEGATIVT TAL		L0725	FSQR
51	\$B3	ENHETEN UPPTAGEN			
52	\$B4	EJ TILL DENNA ENHET		L0727	CASTAB, CMDMERG, CMDSAVE
53	\$B5	FELAKTIG RAD		L06F7	STXE1

Nr	Kod	Text	Fångas	Ingång	Ges i
57	\$B9	FUNKTIONEN EJ DEFINIERAD		L0729	LNKFN
58	\$BA	OGILTIGT TECKEN INLÄST			TXTINP
59	\$BB	FEL PROGRAMFORMAT			CMDMERG
60	\$BC	BIT ADRESS 16 BITAR		L072B	
61	\$BD	KOMMA SAKNAS			
62	\$BE	DOT-ADRESS UTANFÖR SKÄRMEN			DOTARG
63	\$BF	"AS" SAKNAS			
64	\$C0	FELAKTIG "RENAME"			
65	\$C1	SPILL I ASCII-ARITMETIK		ERR65	ASCARG
66	\$C2	STRÄNG EJ NUMERISK		ERR66	ASCAR, ASCERR, ASCARG

Tokens och tabeller

Direktkommandon

Ur [CMDKW](#) och [CMDTAB](#). Koderna är index i CMDTAB.

Kommando	Kod	Rutin
BYE	\$80	L017A (tillägg \$604E)
RUN	\$81	CMDRUN
LIST	\$82	CMDLIST
NEW	\$83	NEW
SCR	\$83	NEW
CLEAR	\$84	CLEAR
LOAD	\$85	CMDLOAD
MERGE	\$86	CMDMERG
REN	\$87	CMDREN
ED	\$88	CMDED
SAVE	\$89	CMDSAVE
UNSAVE	\$8A	L014C (tillägg \$605A)

Satser

Token \$80–\$96 ur [STKW1](#). Kompileringsrutinen står i [STTAB1](#) och körrutinen i [STEXE1](#), båda med index token – \$80. Satser i tabell 2 skrivs som \$86 följt av koden i [STKW2](#) och har rutinerna i [STTAB2](#) och [STEXE2](#).

Token	Ord	Kompilering	Körning
\$80	GOTO	CMPREF	GOTO
\$81	(GOTO efter THEN/ELSE)	CMPREF	GOTO
\$82	LET	CMPIET	LET
\$83	(LET utan ordet)	CMPIET	LET
\$84	PRINT	CMPPRT	PRINTS
\$85	;	CMPPRT	PRINTS
\$86	(tabell 2, kod följer)	RESET	EXEST2
\$87	(hopp framåt, IF)	RESET	EXESKP
\$88	IF	CMPPIF	IF
\$89	INPUT	CMPINP	INPT
\$8A	INPUTLINE	CMPINL	INPLIN
\$8B	FOR	CMPFOR	FOR
\$8C	NEXT	CMPNXT	NEXT
\$8D	(inget ord)	fel 8	fel 8
\$8E	READ	CMPRD	READ
\$8F	RESTORE	CMPRES	RESTOR
\$90	GOSUB	CMPREF	GOSUB
\$91	RETURN	NOCODE	RETURN
\$92	ON	CMPON	ON
\$93	DATA	CMPREM	DATA
\$94	ONERRORGOTO	CMPREF	ONERRG
\$95	(felaktig rad)	RESET	fel 53
\$96	DEFFN	CMPDEF	DEF

Satser, tabell 2

Kod	Ord	Kompilering	Körning
\$86 \$00	DIM	CMPDIM	DIM
\$86 \$01	POKE	CMPPOK	POKE
\$86 \$02	OUT	CMPOUT	OUTST
\$86 \$03	(inget ord)	CMPINX	fel 8
\$86 \$04	REM	CMPREM	REM
\$86 \$05	OPEN	CMPOPEN	OPENST
\$86 \$06	PREPARE	CMPOPEN	PREPST
\$86 \$07	CLOSE	CMPCLS	CLOSST
\$86 \$08	RANDOMIZE	CMPNOP	RANDOM
\$86 \$09	STOP	NOCODE	STOP
\$86 \$0A	END	NOCODE	ENDPRG
\$86 \$0B	CMD	L0147 (tillägg \$7000)	L0166 (tillägg \$7003)
\$86 \$0C	(inget ord)	fel 8	fel 8
\$86 \$0D	(inget ord)	fel 8	fel 8
\$86 \$0E	SETDOT	CMPDOT	SETDOT
\$86 \$0F	CLRDOT	CMPDOT	CLRDOT
\$86 \$10	GET	CMPGET	GET
\$86 \$11	CHAIN	CMPSTX	CHAIN
\$86 \$12	KILL	CMPSTX	L0170 (tillägg \$605D)
\$86 \$13	NAME	L0151 (tillägg \$6054)	L0175 (tillägg \$6057)
\$86 \$14	TRACE	NOCODE	TRACE
\$86 \$15	NOTRACE	NOCODE	NOTRAC

Övriga nyckelord

Tabeller med ett ord var, som kompileringsrutinerna söker i. Token skrivs i satsens kod och listas genom STKW1.

Tabell	Token	Ord
KWTHEN	\$BD	THEN
KWELSE	\$BC	ELSE
KWSTEP	\$BB	STEP
KWTO	\$BA	TO
KWASF	\$B8	ASFILE
KWAS	\$B7	AS

Operatorer

Nivåerna i uttryckskompilatorn, från lägst till högst prioritet (PREC1). Aritmetik och jämförelser har en token per typ: bas + 0 flyttal, + 1 heltal, + 2 sträng; körrutinerna står i EXPTAB1.

Nivå	Operator	Token	Klass	Kompilering	Körning
PREC1	EQV	\$DC	logisk	PRECLV	OPEQV
PREC2	IMP	\$DD	logisk	PRECLV	OPIMP
PREC3	OR	\$DE	logisk	PRECLV	OPOR
PREC3	XOR	\$DF	logisk	PRECLV	OPXOR
PREC4	AND	\$E0	logisk	PRECLV	OPAND
PREC5	NOT	\$E1	unär	CMPNOT	OPNOT
PREC6	<=	\$F1	jämförelse	PRECLV	LEF LEI LES
PREC6	<>	\$E5	jämförelse	PRECLV	NEF NEI NES
PREC6	<	\$E8	jämförelse	PRECLV	LTF LTI LTS
PREC6	>=	\$EB	jämförelse	PRECLV	GEF GEI GES
PREC6	>	\$EE	jämförelse	PRECLV	GTF GTI GTS
PREC6	=	\$E2	jämförelse	PRECLV	EQF EQI EQS
PREC7	+	\$F4	aritmetik	CMPADD	ADDF ADDI CONCAT
PREC7	-	\$F7	aritmetik	CMPADD	SUBF SUBI
PREC8	*	\$F9	aritmetik	PRECLV	MULF MULI
PREC8	/	\$FB	aritmetik	PRECLV	DIVF DIVI
PREC9	^, **	\$FD	potens	PRECLV	POWF POWI POWFI
PREC10			primärt uttryck	PRIMARY	

Funktioner

Token \$80 + k ur FNKW; koden k är index i EXPTAB2. Formen och typerna är ur FNTYPE: x flyttal, i heltal, s\$ sträng, [] får utelämnas.

Kod	Form	Resultat	Rutin
\$00	FN	–	FN
\$01	ABS(x)	flyttal	EXABS
\$02	ATN(x)	flyttal	EXATN
\$03	COS(x)	flyttal	EXCOS
\$04	EXP(x)	flyttal	EXEXP1
\$05	FIX(x)	flyttal	EXFIX
\$06	INT(x)	flyttal	EXINT
\$07	LOG(x)	flyttal	EXLOG
\$08	LOG10(x)	flyttal	EXLG10
\$09	PI	flyttal	EXPI
\$0A	RND[(x)]	flyttal	EXRND
\$0B	SGN(x)	heltal	EXSGN
\$0C	SIN(x)	flyttal	EXSIN
\$0D	SQR(x)	flyttal	EXSQR
\$0E	TAN(x)	flyttal	EXTAN
\$0F	ASC(s\$)	heltal	EXASC
\$10	CHR\$(i[,i[,i[,i]]])	sträng	EXCHR
\$11	LEFT\$(s\$,i)	sträng	EXLEFT
\$12	RIGHT\$(s\$,i)	sträng	EXRGHT
\$13	MID\$(s\$,i,i)	sträng	EXMID
\$14	LEN(s\$)	heltal	EXLEN
\$15	INSTR(i,s\$,s\$)	heltal	EXINST
\$16	SPACE\$(i)	sträng	EXSPC
\$17	STRING\$(i,i)	sträng	EXSTRG
\$18	NUM\$(x)	sträng	EXNUM
\$19	VAL(s\$)	flyttal	EXVAL
\$1A	SWAP%(i)	heltal	EXSWAP
\$1B	PEEK(i)	heltal	EXPEEK
\$1C	INP(i)	heltal	EXINP
\$1D	CALL(i[,i])	heltal	EXCALL
\$1E	ERRCODE	heltal	EXERR
\$1F	IEC\$(i)	sträng	EXIEC (tillägg \$7006)
\$20	TAB(i)	sträng	EXTAB
\$21	CUR(i,i)	sträng	EXCUR
\$23	DOT(i,i)	heltal	EXDOT
\$30	ADD\$(s\$,s\$,i)	sträng	EXADD
\$31	SUB\$(s\$,s\$,i)	sträng	EXSUB
\$32	MUL\$(s\$,s\$,i)	sträng	EXMUL
\$33	DIV\$(s\$,s\$,i)	sträng	EXDIV
\$34	COMP\$(s\$,s\$)	heltal	EXCOMP

Koder utan namn i FNKW: \$22, \$24, \$25, \$26, \$27, \$28, \$29, \$2A, \$2B, \$2C, \$2D, \$2E, \$2F (fel 8).

Uttryckskod

Token \$B6–\$FF i den kompillerade uttryckskoden och rutinen i EXPTAB1 som utför den. Vad varje token betyder beskrivs vid tabellen i listningen.

\$B6	ASGNF	\$BF	ENDDIM	\$C8	VAR2F
\$B7	ASGNI	\$C0	VAR0	\$C9	VAR2I
\$B8	ASGNS	\$C1	VAR0	\$CA	VAR2S
\$B9	ENDF	\$C2	VAR0	\$CB	CONSTS
\$BA	EXEND	\$C3	CONSTF	\$CC	DIMELM
\$BB	ENDS	\$C4	VAR1F	\$CD	EXFUNC
\$BC	ENDVF	\$C5	VAR1I	\$CE	EXFUNC
\$BD	EXEND	\$C6	VAR1S	\$CF	EXFUNC
\$BE	ENDVS	\$C7	CONSTI	\$D0	EXFUNC

\$D1	EXFUNC		\$E1	OPNOT	NOT	\$F1	LEF	<=
\$D2	PUSH0		\$E2	EQF	=	\$F2	LEI	<=
\$D3	EXEXP		\$E3	EQI	=	\$F3	LES	<=
\$D4	GETF		\$E4	EQS	=	\$F4	ADDF	+
\$D5	GETI		\$E5	NEF	<>	\$F5	ADDI	+
\$D6	GETS		\$E6	NEI	<>	\$F6	CONCAT	+
\$D7	NEGF		\$E7	NES	<>	\$F7	SUBF	-
\$D8	NEGI		\$E8	LTF	<	\$F8	SUBI	-
\$D9	FTOI		\$E9	LTI	<	\$F9	MULF	*
\$DA	ITOF		\$EA	LTS	<	\$FA	MULI	*
\$DB	SKPARG		\$EB	GEF	>=	\$FB	DIVF	/
\$DC	OPEQV	EQV	\$EC	GEI	>=	\$FC	DIVI	/
\$DD	OPIMP	IMP	\$ED	GES	>=	\$FD	POWF	^
\$DE	OPOR	OR	\$EE	GTF	>	\$FE	POWI	^
\$DF	OPXOR	XOR	\$EF	GTI	>	\$FF	POWFI	^
\$E0	OPAND	AND	\$F0	GTS	>			

Skillnader i version 9913

Allmänt

Version 9913 är en senare version av samma BASIC. Nästan all kod är densamma, och de flesta skillnaderna i en jämförelse byte för byte är adresser som har flyttats 1-13 bytes. De ingångar som andra program använder ligger kvar på samma adresser: RST-vektorer och hoppen på \$0000-\$0047, IM 2-vektorer, [ERROR](#) på \$062E (en NOP på \$062D håller den kvar), satstabellerna [STEXE1](#) och [STEXE2](#) (\$0E14, \$0E42) samt [PRELNK](#) och [LNKELM](#) (NOP på \$0E91-\$0E94 i stället för den borttagna tilldelningen till LNKOK). Nedan anges först de rättade felen, sedan kassetts ändringar och sist de små ändringarna.

Sist i den nya ROM:en, där den gamla har \$FF, står CR LF och texten "JF DIAB T[BY '79", dvs. "JF DIAB TÄBY '79" (ABC80 har Å på []).

Rättade fel

Utskrift av tal: FLTASC (\$1787 -> \$1785)

När avrundningen ger en siffra till ([FAOVFL](#)) ökade den gamla ROM:en alltid E med 1, också för tal under 1. Därför skrevs .0999999 som ".001". Den nya ROM:en har LD E,A (E = IX+2) i stället för INC E.

Blandade typer: OPTYPE (\$27AE -> \$27AB)

När den högra operanden är ett heltal och den vänstra inte är det testas [OPTYPE](#) om den vänstra är ett flyttal. I den gamla ROM:en står LD A,(IX+5) direkt före JR NZ, men LD sätter inte flaggorna, och Z är redan satt av CP \$01. Därför gjordes den högra operanden alltid om till flyttal, även när den vänstra var en sträng: A\$+1 och A\$=1 gav inte fel 17 vid kompileringen, utan blev en flyttalsoperation på en sträng. Den nya ROM:en har AND A efter LD. (CP \$00 på \$27A6 har också bytts mot AND A, med samma verkan.)

LIST av REM och DATA: DETOK (\$11C1 -> \$11CD)

I den gamla ROM:en listades texten efter [REM](#) och [DATA](#) av samma slinga som resten av raden, så ett ":" i texten fick ett mellanslag framför sig (" :"), och raden ändrades varje gång den listades och lästes in igen. Den nya ROM:en skriver nyckelordet och kopierar sedan texten oförändrad till CR med [CMPREM](#) (token \$93 [DATA](#), och \$86 \$84 [REM](#)). [FNKWTX](#) har fått en ingång (\$13F1 i den nya ROM:en) där tabellen ges i BC, och används nu också för satsernas nyckelord och för [DEF FN](#), så [KWDEFN](#)-slingan behövs inte längre.

DIM av sträng utan index och längd: CMPDMI (\$1BDA -> \$1BD9)

I den gamla ROM:en kompilerades [DIM A\\$](#) (utan index och utan "=längd") till längden 0. En sådan rad går att köra, men LIST av den skriver sönder minnet (se \$1337). Den nya ROM:en ger fel 15 när en sträng varken har index eller längd (C = 0 efter [CMPARG](#)). Med index och utan längd blir det som förut \$D2. Strängtestet är nu BIT 1,C i stället för AND 3 / XOR 2.

UNSAVE utan tillägg (\$014C)

UNSAVE hoppar till ett tillägg på \$605A. Den gamla ROM:en gick via [EXTJP1](#), som vid saknat tillägg bara returnerar med CY = 1 och A = \$88; returen går till [READY](#), som inte tittar på CY, så kommandot gjorde ingenting. Den nya ROM:en går via [EXTJP2](#), som ger fel 8.

SQR(0): FSQR (\$3DF5 -> \$3DEC)

Den nya FSQR börjar med LD A,(BC) / AND A / RET Z: är exponenten 0 (talet är 0) blir resultatet 0 direkt. Den gamla ROM:en räknade på och gav exponenten \$40 med mantissan 0: talet skrivs 0, men X=0 är falskt, och 5/SQR(0) hänger i FDIV (provat).

VAL: EXVAL (\$31A0 -> \$319E)

Den gamla ROM:en sparade CR:s adress och gav fel 12 om talet inte slutade vid CR. Kontrollen använde DE efter RST \$20, som arbetar med HL (?). I den nya ROM:en är kontrollen borttagen; text efter talet ignoreras.

DIV\$: ADIV (\$384C -> \$3843)

ADD A,C före ASLEN1 är ersatt med NOP. Längdkontrollen av arbetsytan (högst 62 siffror) räknade b:s decimaler två gånger, så DIV\$ kunde avbrytas som för långt (ASBAD) för tal som ryms (?).

Kassetten

Den gamla ROM:en skrev fasta värden till port \$3A (PIO B, data): \$20 för motor på, \$60 för motor och skrivsignal, \$00 för motor av. Den nya ROM:en läser porten och ändrar bara de bitar som ska ändras (bit 5 = motor, bit 6 = skrivsignal), och har några nya smårutiner:

```
$0438 motor av: IN, RES 5, OUT; sedan $043E
$043E PIO B: avbrott av ($03 till port $3B)
$0586 motor på: IN, SET 5, RES 6, OUT, SET 6, OUT (RCVSTA)
$040D DE = CASBLK + 1, sparas i CASBLK (CASPUT och CASGET)
```

CASPRE, CASOPN och CASCLO anropar dem i stället för OUT (\$3A). CASCLO stänger nu också av kassettavbrotten. När ett block är läst skrivs \$03 till port \$3B i stället för \$07; båda stänger av avbrotten, men \$03 ändrar bara spärren och inte övriga avbrottsinställningar (?). Tidsgränsen i CASRBL när inget block kommer är längre (\$19 -> \$3C varv i den inre slingan), ca 18,4 s i st. f. 8,5 s (65536 varv à 841 resp. 386 T-perioder vid 2,9952 MHz). Blockets data börjar alltid på byte 3 i bufferten, så den låga byten av buffertpekaren (IX+8) sätts inte längre i CASGET, och CASPUT använder L = 3. LD A,\$96 / SRL A i CASWBI är nu LD A,\$4B, och i CASOPN har LD A,(HL) / INC A bytts mot INC (HL) och LD BC mot LD C.

Små ändringar

Ny systemvariabel \$FE0C: den byte som SAVE skriver först i filen (filtypen). NEW sätter den till \$82, och den gamla ROM:en skrev alltid \$82. Med \$83 blir programmet skyddat när det laddas (READY, bit 5 i FLAGS) (?).

NEW nollställer nu hela FLAGS (LD (IY+\$0F),0) i stället för bit 4 och 5. Flaggan LNKOK (\$FE26), som den gamla ROM:en satte i PRELNK och nollställde i CLEAR men aldrig läste, används inte längre.

GETKEY nollställer hela KEYFLG (LD (IX+2),0) i stället för bit 7.

Byten på \$0065, före NMI, är \$45 i den gamla ROM:en och \$00 i den nya. Den används inte.

Adresser

Tabellen visar var varje del av 11273 hamnar i 9913. Det mesta är samma kod som bara har flyttats några bytes. Kolumnen Flyttad anger hur många; minus betyder att koden ligger lägre i 9913. Läger man till talet till en adress i 11273 får man adressen i 9913: FLTASC ligger på \$1787 i 11273 och på \$1787 - 2 = \$1785 i 9913. Där det står "ändrad" är koden omskriven, och där går adresserna inte att räkna om.

I 11273	I 9913	Flyttad	Innehåll
\$0000-\$03D4	\$0000-\$03D4	0	start, RST-vektorer, skärm, tangentbord
\$03D5-\$058D	\$03D5-\$058D	ändrad	kassetten, CASPUT-RCVSTA (−4 till +10)
\$058E-\$0A72	\$058E-\$0A72	0	felen, enheterna, PRINT , INPUT , NEW ; dock −1 på \$060F-\$062D
\$0A73-\$0A92	\$0A73-\$0A90	ändrad	slutet av NEW , CLEAR
\$0A93-\$0DDD	\$0A91-\$0DDB	−2	kommandona LIST , LOAD , RUN m.fl.
\$0DDE-\$11C0	\$0DDE-\$11C0	0	CMDSAVE , STEXE1 , STEXE2 , PRELNK och förlänkningen
\$11C1-\$13FF	\$11C1-\$13FD	ändrad	DETOK , FNKWTX (LIST av raderna)
\$1400-\$1BA9	\$13FE-\$1BA7	−2	variabelnamn, utskrift och inläsning av tal
\$1BAA-\$1BF6	\$1BA8-\$1BF4	ändrad	CMPDIM , CMPDMI
\$1BF7-\$319F	\$1BF5-\$319D	−2	satserna; dock −3 i OPTYPE på \$27A8-\$27B0
\$31A0-\$31B9	\$319E-\$31B0	ändrad	EXVAL
\$31BA-\$3DF4	\$31B1-\$3DEB	−9	funktionerna, minnet, flyttalsräkningen
\$3DF5-\$3E08	\$3DEC-\$3E01	ändrad	början av FSQR (3 bytes nya först)
\$3E09-\$3FF4	\$3E02-\$3FED	−7	resten av FSQR , FTAN , FPOLY m.fl.
\$3FF5-\$3FFF	\$3FEE-\$3FFF	ny	\$FF i 11273; CR LF och texten ovan i 9913

Oklara ställen

Ställen i listningen där tolkningen är osäker, markerade med (?) i kommentarerna, och det som enligt kommentarerna inte används. Adressen är kodraden som kommentaren hör till (för en blockkommentar raden efter den) och är en länk dit.

Osäkra tolkningar

Adress	Rutin	Kommentar
\$0594	CASINT	CY = inverterad bit 7 = den mottagna biten (?). Biten skiftas in i RCVBC (C).
\$09A1	TXTINP	LD (IX+\$0D), \$01: lediga = 1 (?)
\$1703	FATINY	Med bit 7: "." och nollor (?).
\$1737	FACARY	OR A: kan inte bli 0 (?)
\$1998	KWAS	AS (\$B7): ingen söker i den här tabellen, och \$B7 i uttryckskoden betyder tilldelning (typ 1), inte AS. Posten verkar oanvänd (?).
\$1A08	STTAB1	Kod 3, C och D har inga nyckelord och kan inte skrivas in; de är nog platser för satser som inte finns i denna version (?).
\$1D26	NEXT	Grenen passar inte med FOR -blocket, och LNKNXT tillåter inte NEXT utan variabel. Den används nog aldrig (?).
\$291F	EXPTAB2	Kod \$22 och \$24-\$2F har inget namn i FNKW och går till fel 8, FINNS EJ I DETTA SYSTEM (platser för fler funktioner (?)).
\$2989	ASGNF	- Fel 10 även om variabelns post ligger ovanför SP, dvs. på stacken (?).
\$3689	FDRND	Inget annat i ROM:en läser eller skriver \$FE19, och INIT nollställer den inte, så det som RAM-kretsarna råkar innehålla efter start avgör (?).

Oanvänt

Adress	Rutin	Kommentar
\$018C	DEVLST	7, 8 används inte av CAS (fel 52)
\$10B8	VARSIZ	VARSIZ : postens storlek för namnbyte AND \$0F (4 bytes huvud + värdet): flyttal 9, heltal 6, sträng 10 (strängpost 6 bytes), med ett index 10 (vektorpost 6 bytes), med två index 14. Typ 3 används inte.
\$1388	DETTAB	Ingången \$138C (flagga 2, unär utan mellanslag) används inte i denna ROM.
\$239F	CMPIVR	CMPIVR kräver heltal (anropas inte i denna ROM), CMPGET kräver sträng (GET , INPUT LINE). Annars fel 17.
\$31B3	EXVAL	RST \$20: verkningslös: HL används inte sedan
\$31E0	EXRND	Argumentet x används inte; dess 5 bytes blir en del av arbetsytan (13 bytes reserveras med argument, 18 utan, så ytan är alltid 18 bytes).
\$3F97	POLYLP	Konstanter: 2 och LN 2 (.693147, \$3FAB) används inte (ingen instruktion pekar dit).

Källor

Källorna citeras i texten med nyckeln inom parentes, t.ex. (Finnved 1979); nyckeln står i fetstil först i posten.

Tryckt litteratur och press

ABC-bladet ABC-klubben: *ABC-bladet*. ABC-klubbens medlemstidning, Stockholm 1980-2010.

Om Superbasics upphovsman: Magnus "Nyfiken" Lundberg skrev artikelserien "Nyfiken" (del 3 i 1981 nr 3, om skivformatet, signerad Nyfors, Rosersberg; i samma nummer "Maskinnära programmering" från Nyfors Data Science), 1984 nr 2, s. 9 (jobbströmmar med EXEC i Superbasic "som Magnus Lundberg kan tillhandahålla"), 1985 nr 2, s. 34 (Magnus Lundberg, Nyfors Data i Rosersberg, har ett UFD-DOS till ABC80). Om 80 tecken per rad: 1981 nr 2, s. 16 (CAT80 från CAT Ingenjörbyrå och Marin-Data, kort för flexskiveenheten, drivrutin "CRT:"), 1981 nr 4-5, s. 2 (annons för MYAB:s TKN-80, UNI-80 och Expand ABC) och s. 23 ("80 kolumners bildskärm för ABC-80": CAT-80, GeJo och MyAB, luckorna i bildminnet).

Inskannad: <https://www.abc80.net/archive/luxor/ABC-bladet/>

Alm 1983 Torbjörn Alm: *CP/M på ABC-datorer*. ABC-bladet 1983:4, s. 25.

Luxors och MyAB:s CP/M för ABC 800/802/806 och ABC80, programutbudet och kostnaderna.

Inskannad: <https://www.abc80.net/archive/luxor/ABC-bladet/>

Andersson 1985 David Andersson: *TERM100.DOC. Terminalprogram för ABC80. Beskrivning av TERM100 Version 1.4, med TERM100.LAT, Lathund för TERM100 Version 1.4*. ABC-bladet (ABC-klubben) 1985 nr 2, s. 6-11.

VT100- och VT52-emulator över V24: hastigheter 50-1200 bit/s, keypad-läget med prefixtangenter (CTRL-É, CTRL-Q eller CTRL-Ö), NO SCROLL med XON/XOFF, inställningslägena R, T, Y och U, filformaten TEXT och DATA, styrsekvenserna och begränsningarna (understrykning och omvänd video visas blinkande). Kallstart CALL (53000), varmstart CALL (53003), samma adresser som texttelefonens program i RAM (\$CF08, \$CF0B). Lathundens tabell har Y och U ombytta mot beskrivningen och programmet.

Inskannad: <https://www.abc80.net/archive/luxor/ABC-bladet/>

Apple 1977 Apple Computer: [annons för Apple II]. Byte, vol. 2 nr 10, oktober 1977, s. 14-16.

Apple II för 1 298 dollar med 8 KB ROM och 4 KB RAM (till 48 KB), grafik i 15 färger; färg-tv och vanlig kassetbandspelare ansluts av köparen (s. 15).

Inskannad: <https://archive.org/details/byte-magazine-1977-10>

Blank 1980 George Blank, Roger Fuller, John Hartford, John T. Phillipp och Robert M. Richardson: *Pathways Through the ROM. Guide to Level II BASIC and DOS*. SoftSide Publications, Milford, New Hampshire, 1980.

Genomgång av TRS-80:s BASIC-ROM. Förordet: Level II BASIC är Microsofts BASIC, och Microsoft är upphovsman till bl.a. Apples och PET:s BASIC. S. 44: ROM:en lägger vid start in hopp till felet L3 för DOS-kommandona; s. 66-67: DOS-ingångarna i RAM (\$4152 CVI...\$41A3 LINE, bl.a. CMD \$4173) och hopp för BASIC-rutiner (\$41A6-\$41E2) som kan läggas om.

Inskannad: https://archive.org/details/Pathways_Through_the_ROM_1980_SouthSide_Publications

Borg 1985 Bjarne Borg: *En smartaid födelse*. ABC-bladet (ABC-klubben) 1985 nr 3, s. 20.

Smartaid-seriens historia, skriven av OWOCO: programmet AUTO (Henrik och Felix Burton, klart hösten 1980) via tangentbordsavbrottet; hårdvaran till den första Smartaid (Bengt Sjöqvist och Bernt Lindgren), 1 kilobyte, bara för en version av BASIC-tolken; Smartaid II (David Andersson och Ulf Söderman) för alla versioner; Smartaid III och Schoolaid våren 1982 på begäran av Luxor och Liber, ny hårdvara av Christer Ekman (svart låda på bussen), 5 resp. 4 kilobyte, osynliga bytes i bildminnet, fyra DOS och TKN-80; Super Smartaid (10 kilobyte + 2 kilobyte CMOS, bankväxling, hårdvara Ekman, programvara Andersson); Smartaid 800; Smartaid Magnum hösten 1985 (debuggern av Magnus Stråle och Andersson, miniassembler).

Inskannad: <https://www.abc80.net/archive/luxor/ABC-bladet/>

Bray, Dickens och Holmes 1983 Andrew C. Bray, Adrian C. Dickens och Mark A. Holmes: *The Advanced User Guide for the BBC Microcomputer*. Cambridge Microcomputer Centre, Cambridge 1983.

Utbytbara ROM:ar (paged ROMs) på &8000-&BFFF (s. 317) och väljaren på &FE30 (s. 395).

Inskannad: <https://archive.org/details/manualcomputergame-bbc-cambridgemicrocomputercentre-theadvanceduserguideforthe>

1983-c048

- Byte 1977** Byte: *Commodore's New PET Computer*. Byte, vol. 2 nr 10, oktober 1977, s. 50; april 1977, s. 10.
PET på NCC i Dallas, "an excellent example of the true appliance computer". I aprilnumret ("A Nybble on the Apple", Helmers) sägs Apple II vara kanske den första som "fully qualify as the 'appliance computer'".
Inskannad: <https://archive.org/details/byte-magazine-1977-10>
Inskannad: <https://archive.org/details/byte-magazine-1977-04>
- Byte 1978** Byte: [reportage från Personal Computer Expo, New York, oktober 1977]. Byte, vol. 3 nr 2, februari 1978, s. 179.
"the new appliance computers, such as the Commodore PET and the Radio Shack TRS-80".
Inskannad: <https://archive.org/details/byte-magazine-1978-02>
- Christensen 1979** Børge R. Christensen: *ABC 80 - en solid, men noget naiv svensker*. Populær Elektronik nr 9 1979, s. 60-61, fortsättning i nästa nummer (s. 46-47).
Dansk recension av COMAL:s upphovsman. Beröm för bygget och farten ("den hurtigste Basic, jeg nogensinde har set", halvöversatt kod i omvänd polsk notation); kritik av teckensnittet, grafikkoderna, att CON och RUN radnr saknas, INPUT som tar bort blanktecken och kassetts 10 s väntan (alla fyra stämmer med koden, se CMDRUN, RDVAL och CASRBL; väntan är ca 8,5 s i 11273 och 18,4 s i 9913).
- DI 1976** Dagens Industri: [artikel om Lars Karlssons mikrodatorsystem 4680]. Dagens Industri 1976-06-10.
DIAB:s modulsystem på Europakort med en egen buss för olika processorer; "hemligheten" i adresseringen av in- och utgångar.
Inskannad: <https://www.df.lth.se/~triad/diab/archive/>
- DI 1979a** Dagens Industri: *Vem valde rätt?* Dagens Industri 1979-04-03.
Krönika av en ledamot i juryn för Stora Industripriset: Data Future AB:s terminaldator Seven S fick priset 1978; ramavtal i början av 1978 om 500 system för 25 Mkr med Kommundata, konceptet sålt till Datasaab, avtalet hävt ett år senare. Dataindustrier bland dem som utvecklat konceptet.
Inskannad: <https://www.df.lth.se/~triad/diab/archive/Seven%20S/>
- DI 1979b** BG Wennersten: *Lars Karlsson och hans gäng*. Dagens Industri 1979-04-05.
Om DIAB: utvecklingen av ABC80 från mars 1978, serietillverkning hos Luxor från november, över 2 000 sålda.
Inskannad: <https://www.df.lth.se/~triad/diab/archive/>
- DI 1980** BG Wennersten: *Ensam svensk utställare lockar kunder med schackspelande robot*. Dagens Industri 1980-02-07, s. 8.
Sattco på Microsystems 80 i London; DataBoard 4680 och ABC80 "utvecklats av ett och samma företag: Dataindustrier AB i Täby".
Inskannad: <https://www.df.lth.se/~triad/diab/archive/>
- DN 1981** Mats Hallgren: *Idéer finns, men låt oss få tjäna pengar på dem*. Dagens Nyheter 1981-08-26.
Intervju med Lars Karlsson: DIAB utvecklar datorer och säljer rätten att tillverka dem; Luxor tillverkar ABC 800, som Facit säljer.
Inskannad: <https://www.df.lth.se/~triad/diab/archive/>
- Ekman 1982** Christer Ekman: *Bygg ut din ABC 80 till 32K RAM*. MikroDatorn nr 7 1982, s. 34-35.
Åtta 4116 lödda ovanpå de gamla kapslarna och en ELLER-grind före adressavkodaren, så att RAM finns även på 32-48 kilobyte; för under 200 kr.
Inskannad: <https://www.abc80.net/archive/luxor/ABC80/ABC80-32K-mod-Mikrodatorn.pdf>
- Ekman 1983a** Christer Ekman: *Bygg själv: ABC 80 till 64 kbyte*. MikroDatorn nr 9 1983, s. 58- (första delen av tre).
Byter minneskapslar och avkodaren så att ABC80 får 64 KB RAM, valt med OUT 7; ROM:en kopieras till RAM på samma adress, så att BASIC-tolken kan ändras.
Inskannad: <https://www.abc80.net/archive/luxor/ABC80/ABC80-64k-bygg-sj%C3%A4lv-MD9-1983.pdf>
- Ekman 1983b** Christer Ekman: *Bygg själv: ABC 80 till 64 kbyte (del 2)*. MikroDatorn nr 10 1983, s. 71-74.
Ändringar i BASIC-tolken i RAM: TOLKMOVE och BLCKMOVE flyttar ROM:en, sedan ny prompt, felhantering, kassetts fart, klockrutin och svenska nyckelord.
Inskannad: <https://www.abc80.net/archive/luxor/ABC80/ABC80-64k-bygg-sj%C3%A4lv-MD9-1983.pdf>
- Ekman 1984** Christer Ekman: *Bygg själv: ABC 80 till 64 kbyte (del 3)*. MikroDatorn nr 1 1984, s. 58-62.
ABC 800:s BASIC i ABC80: tolken sparas som datafil på ABC 800 och läses in och ändras på ABC80 (tangenterbord, skärm, ljud, NMI); byte mellan tolkarna med OUT 7. Byggsats och PROM till salu.
Inskannad: <https://www.abc80.net/archive/luxor/ABC80/ABC80-64k-bygg-sj%C3%A4lv-MD9-1983.pdf>
- Emanuel 2009** Martin Emanuel (red.): *ABC 80 i pedagogikens tjänst: Exempel på tidig användning av mikrodatorer i den svenska skolan. Transkript av ett vittnesseminarium vid Cloetta Center i Linköping den 23 september 2008*. KTH, Avdelningen för teknik- och vetenskapshistoria, TRITA-HST 2008/32, Stockholm 2009. ISBN 978-91-7415-062-9.

Projektledarens berättelse: styrgruppen för projektet HD80 (Börjesson, Karlsson, Björklund) bildades 27 februari 1978, sex månader från idé till produktion, prototypen till Markesjö vid midsommar 1978, pressvisning 24 augusti 1978, namnet "Advanced Basic Computer".

Inskannad: <https://www.diva-portal.org/smash/get/diva2:214183/FULLTEXT01.pdf>

Finnved 1979 Johan Finnved: *Programvaran i persondatorn ABC80*. Examensarbete, Institutionen för tillämpad elektronik, KTH, godkänt 1979-01-29 (utfört vid Dataindustrier AB, Täby).

Uppdraget var att ta fram BASIC:en (påbörjat 1978-02-15, handledare Gunnar Markesjö). Beskrivning av den: kompilering till postfix, länkning vid RUN, LIST, enheter.

Inskannad: <https://dflund.se/~triad/diab/archive/ABC%2080/>

Franzén 1985 Anders Franzén: *Nya kommandon på ABC80*. ABC-bladet (ABC-klubben) 1985 nr 3, s. 23-25.

Assemblerprogram (CALL(61440)) som via tangentbordsavbrottet och I-registret byter återhoppadressen till direktlägrets slinga på stacken (240 = \$00F0) mot en egen kommandotolk; känner själv av kontrollsumman 11273/9913. Samma knep som exemplen i band 2.

Inskannad: <https://www.abc80.net/archive/luxor/ABC-bladet/>

Fylstra 1978a Dan Fylstra: *User's Report: The PET 2001*. Byte, vol. 3 nr 3, mars 1978, s. 114 ff.

Provning av PET 2001: inbyggd svartvit skärm på 9 tum (22,9 cm) med 25 rader om 40 tecken (s. 116); 64 grafiktecken (s. 118).

Inskannad: <https://archive.org/details/byte-magazine-1978-03>

Fylstra 1978b Dan Fylstra: *The Radio Shack TRS-80: An Owner's Report*. Byte, vol. 3 nr 4, april 1978, s. 49 ff.

TRS-80 för 599,95 dollar med 4 KB, bildskärm och kassetbandspelare (s. 49); minnet kan byggas ut till 16 KB i tangentbordsenheten (s. 50); 12 tums (30,5 cm) skärm med 16 rader om 64 tecken (s. 51); grafik 128 x 48 punkter (s. 52).

Inskannad: <https://archive.org/details/byte-magazine-1978-04>

Gruber 1981 Tad Gruber: *ABC 800 är här!* MikroDatorn nr 2 (mars) 1981, s. 6-10.

Presentationen av ABC 800 (projektnamnet X2/1), utvecklad av Luxor tillsammans med Data Industrier AB: Z80 3 MHz, 32 KB RAM, 24 KB ROM med BASIC, 4 KB DOS, 4 KB tillval; skärm med 40 eller 80 tecken, högupplösningsskortet (s. 8), BASIC II (s. 9), minneskarta (s. 10).

Inskannad: <https://dflund.se/~triad/diab/archive/ABC%20800%20Facit%20DTC/>

GT 1978 Göteborgs-Tidningen: [artikel om ABC80 som "den första svenska hemdatorn"]. GT 1978-09-10.

Presentationen före försäljningen: Luxor, Dataindustrier AB och Scandia Metric, idén från Karl-Johan Börjesson, sex månaders utveckling, pris 7 200 kr.

Inskannad: <https://www.df.lth.se/~triad/diab/archive/>

Gustavsson 1985 Börje Gustavsson: *Jämförelse mellan Superbasic, Lökaren, SmartAid 3 och SupersmartAid*. ABC-bladet (ABC-klubben) 1985 nr 1, s. 12-13.

Tabell med typ, adressområde, tillverkare och pris (Superbasic: ROM 16384-22527 och 28672-30719, Nyfors Data, ca 1000 kr), skärmeditorernas CTRL-tangenter och alla kommandon med vilken krets som har dem. Till Superbasic följer en skiva med program, och ett ändrat DOS (autostart, filstorlek och datum i katalogen) kan köpas till.

Inskannad: <https://www.abc80.net/archive/luxor/ABC-bladet/>

Helmers 1977 Carl Helmers: *The Appliance Computer, Circa 1977*. Byte, vol. 2 nr 1, januari 1977, s. 4 (ledare).

Förutsäger att 1977 blir "the year of the Appliance Computer": en färdig dator som köps hos handlaren och programmeras direkt.

Inskannad: <https://archive.org/details/byte-magazine-1977-01>

Helmers 1978 Carl Helmers: *An Apple to Byte*. Byte, vol. 3 nr 3, mars 1978, s. 18 ff.

Redaktörens rapport om Apple II. Tabell 2 (s. 30): 6502 med 1,023 MHz, 4-48 KB RAM, 8 KB ROM installerat (monitor 2 KB, hjälprutiner 1 KB, BASIC för heltal 5 KB), bandspelare med omkring 1 500 bit/s.

Inskannad: <https://archive.org/details/byte-magazine-1978-03>

Isaksson och Kärrsgård 1980 Anders Isaksson och Örjan Kärrsgård: *Avancerad programmering på ABC80*. Studentlitteratur, Lund 1980 (Scandia Metric). ISBN 91-44-17451-9.

Programmering i BASIC och assembler. Avsnitt 5.4 (s. 79-81) beskriver ljudgeneratoren: styrordet på port 6 och en tabell med omkring 34 användbara ljud (fig 5.5-5.6).

Inskannad: <https://www.abc80.net/archive/luxor/ABC80/>

Kullmar 1984 Bo Kullmar: *Historik*. ABC-bladet (ABC-klubben) 1984 nr 1, s. 12.

ABC-klubbens och datorernas historia: ABC800 presenterad våren 1981, ABC802 och ABC806 våren 1983.

Inskannad: <https://www.abc80.net/archive/luxor/ABC-bladet/>

- Kullmar 1985** Bo Kullmar: *Nätverk. ABC, CAT och LUX-NET*. ABC-bladet (ABC-klubben) 1985 nr 1, s. 5.
Jämförelse av näten: ABC-NET (ca 19 200 bit/s), CAT-NET (153 600 bit/s, centralen frågar noderna, nodkort utan processor, över 600 installationer, även för ABC80) och LUX-NET (från december 1984, HDLC 500 000 bit/s, processor på nodkortet, inget DOS för ABC80).
Inskannad: <https://www.abc80.net/archive/luxor/ABC-bladet/>
- Mansfield 1982** Richard Mansfield: *Review: SoftBox – CP/M For PET/CBM*. Compute! nr 26, juli 1982, s. 185.
Small Systems Engineerings SoftBox: Z80, 64 KB RAM, IEEE och RS-232, CP/M 2.2.
Inskannad: <https://archive.org/details/1982-07-compute-magazine>
- Markesjö 1978** Gunnar Markesjö: *Mikrodatorns ABC. Elektroniken i ett mikrodatorsystem*. Esselte Studium, 1978. ISBN 91-24-29008-4.
Lärobok som bygger på ABC80: elektroniken, PIO:ns inkoppling, grafiken, kassetten och realtidsklockan.
- Mattsson 1983** Rune Mattsson: *ABC80 som RAM-dator*. ABC-bladet (ABC-klubben) 1983 nr 3, s. 12.
Användarrapport om GeJo:s kort med 128 kilobyte i tangentbordsenheten, med utdrag ur kortets användarhandledning: port 7, sidor om 16 kilobyte, adressbitar för statiskt RAM, RAM från adress 0, klocka och bildminne; RAM-skiva DR6.
Inskannad: <https://www.abc80.net/archive/luxor/ABC80/minnesexpansion-gejo128k.pdf>
Inskannad: <https://www.abc80.net/archive/luxor/ABC-bladet/>
- MikroDatorn 1982** Karl-David Höglund: *Hur snabb är ABC 80?* MikroDatorn nr 9 1982, s. 65 f. (ABC-Forum).
Rugg/Feldmans testprogram BM1-BM8 (Kilobaud 1977, Personal Computer World) körda på ABC80 och jämförda med andra datorer.
<https://github.com/Feyerabend/ABC80>
- MikroDatorn 1983** Christer Åkerman: *DIAB sprider ut sig*. MikroDatorn nr 6 1983.
DIAB byter namn, köper Sattco och går till börsen; Luxor blir delägare.
Inskannad: <https://www.df.lth.se/~triad/diab/archive/>
- NEECO 1981** NEECO: *NEECO introduces the CBM VIC-20 computer! [annons]*. Compute!, nr 15, augusti 1981, s. 29.
VIC-20 för 299,95 dollar, 5 KB minne som kan byggas ut till 32 KB, åtta färger, ansluts till en tv; bandstationen C2N 74,95 dollar.
Inskannad: <https://archive.org/details/1981-08-compute-magazine>
- Olwig 1981** Bengt Olwig: *ABC800 – den nya svenska datorn*. ABC-bladet (ABC-klubben) 1981 nr 2, s. 4-5.
De två modellerna ABC 800 M och ABC 800 C och vad de var tänkta för, Basic II, produktionsstart, leveranser och pris.
Inskannad: <https://www.abc80.net/archive/luxor/ABC-bladet/>
- Pålsson 1985** Ture Pålsson: *Det där med HJÄLPARE...* ABC-bladet (ABC-klubben) 1985 nr 3, s. 22.
Hur en hjälpare kopplar in en egen avbrottsrutin för tangentbordet, läser tangenten med IN A,(56) och går vidare till ROM:ens rutin för andra tangenter än CTRL-É.
Inskannad: <https://www.abc80.net/archive/luxor/ABC-bladet/>
- Rolander 1983** Odd Rolander: *CP/M till ABC80 ABC800/FACIT DTC*. ABC-bladet 1983:4, s. 18-19.
CP/M-korten från MyAB, CAT och Facit, priserna.
Inskannad: <https://www.abc80.net/archive/luxor/ABC-bladet/>
- Sather 1983** James Sather: *Understanding the Apple II*. Quality Software, Chatsworth 1983.
Apple II:s 16K RAM-kort: RAM i stället för ROM på \$D000-\$FFFF (s. 5-26) och två banker à 4 KB på \$D000-\$DFFF (s. 5-28).
Inskannad: <https://archive.org/details/utaii>
- Severance 1998** Charles Severance: *IEEE 754: An Interview with William Kahan*. IEEE Computer, vol. 31 nr 3, mars 1998 (förkortad; hela texten daterad 20 februari 1998).
Flyttalsstandarden IEEE 754-1985: förhistorien (varje datorserie med eget omfång, egen precision och egen avrundning), Kahans arbete med Intels 8087, förslaget från Kahan, Coonen och Stone, gradvist underspill.
Inskannad: <https://people.eecs.berkeley.edu/~wkahan/ieee754status/754story.html>
- Sjöström 1996** Roland Sjöström: *Positionering under strategisk osäkerhet. Vol. 2: Luxor Datorer och persondatorbranschen*. Linköping Studies in Management and Economics, Dissertations No. 30, Linköpings universitet, 1996. ISBN 91-7871-699-3.
Doktorsavhandling; band 2 följer Luxor Datorer år för år 1978-1986, byggt på intervjuer, interna PM och fackpressen. Sidnumren i hänvisningarna är bandets tryckta.
- Stockman 1981** Arne Stockman: *Programvaran i ABC80. Disassemblering med kommentarer*. ABC-klubben, Rapport nr 1, april 1981, 172 s.

Kommenterad disassemblering av ABC80:s BASIC-ROM, handskrivna kommentarer på A3-uppslag. Detta arbete bygger delvis på den.

Inskannad: <https://www.abc80.net/archive/luxor/ABC80/>

Wahlén 1982 Robert Wahlén: *Bygg ut ABC 80 och ABC 800/Facit DTC med DataBoard 4680!* Liber, 1982. ISBN 91-40-20486-3.

4680-bussen som ABC-datorernas buss: kortval, portar, expansionsenheten och kontakterna (appendix B för ABC80).

Inskannad: <https://www.df.lth.se/~triad/diab/archive/>

Westh 1980 Åke Westh: *Styr och mät med ABC80*. Studentlitteratur, Lund 1980. ISBN 91-44-17631-7.

V24-snittet (användning, programmering bit för bit via PIO B, tillämpningskopplingar), ABC-bussen och minnesexpansion.

Inskannad: <https://www.abc80.net/archive/luxor/ABC80/>

Wickberg 1982 Sven Wickberg: *Sätt autopilot på din ABC80! Användarrapport*. ABC-bladet (ABC-klubben) 1982 nr 2, s. 28.

Superbasic från Nyfors Data Science (Magnus Lundberg och Magnus Jacobsson): kommandona, HOLD, STEP, EDIT, EXEC; moduler på skiva (ca 500 kr, EDIT och EXEC 700 kr) i RAM under taket eller PROM på ett kort i flexskiveenheten; DOS med datum och filstorlek (500 kr).

Inskannad: <https://www.abc80.net/archive/luxor/ABC80/Superbasic-review.pdf>

Inskannad: <https://www.abc80.net/archive/luxor/ABC-bladet/>

Wickberg 1983 Sven Wickberg: *Mera super(b)basic*. ABC-bladet (ABC-klubben) 1983 nr 3, s. 10.

MEDIT, EDIT, "den svarta lådan" på busskontakten som startas med CMD, LIST med tio rader, underprogrammen.

Inskannad: <https://www.abc80.net/archive/luxor/ABC80/Superbasic-review.pdf>

Inskannad: <https://www.abc80.net/archive/luxor/ABC-bladet/>

Williams 1982 Gregg Williams: *A Closer Look at the IBM Personal Computer*. Byte, vol. 7 nr 1, januari 1982, s. 36 ff.

Intel 8088, 40 KB ROM med Microsofts BASIC, upp till 64 KB på moderkortet, kassettanslutning (s. 36); text 25 x 40 eller 25 x 80, grafik 320 x 200 med fyra färger och 640 x 200 (s. 39); 48 KB med kassett och färggrafikkort 1 745 dollar (s. 40, tabell 1a); 4,77 MHz (s. 42); 16 KB RAM som standard (s. 45).

Inskannad: <https://archive.org/details/byte-magazine-1982-01>

Worth och Lechner 1981 Don Worth och Pieter Lechner: *Beneath Apple DOS*. Quality Software, Reseda 1981 (fjärde tryckningen maj 1982).

Apple II:s DOS 3.3 och skivminnet Disk II: styrkortet, bitarnas kodning i programmet, spår och sektorer. Sektorens ordning på spåret (s. 3-22): i följd skulle nästa sektor kräva ett helt varv.

Inskannad: https://archive.org/details/Beneath_Apple_DOS_OCR

Wozniak 1977 Steve Wozniak: *System Description: The Apple-II*. Byte, vol. 2 nr 5, maj 1977, s. 34-43.

Konstruktörens beskrivning av Apple II med dess egen BASIC för heltal ("interpretive integer BASIC", s. 41).

Inskannad: <https://archive.org/details/byte-magazine-1977-05>

Handböcker, datablad och trycksaker

ABC 80 Bruksanvisning, komplettering Luxor: *ABC 80 Bruksanvisning, komplettering*. Luxor, Motala, art. nr 66 79589-12.

Tillägg till bruksanvisningen; teckningen av maskinen bakifrån (s. 3) är underlag för skissen av baksidan.

Inskannad: <https://www.abc80.net/archive/luxor/ABC80/>

ABC 800 Bruksanvisning Luxor: *ABC 800 Bruksanvisning*. Luxor Datorer AB, Motala 1982.

S. 12: ljudgeneratoren styrs med INP(5), som ger en puls till högtalaren.

Inskannad: <https://www.abc80.net/archive/luxor/ABC80x/>

ABC 802 Servicemanual Luxor: *ABC 802 Servicemanual*. Luxor Datorer AB, Motala, art. nr 66 78088-40 (odaterad).

S. 4-6: översikt över ABC 802 (datorn i bildskärmen, tangentbord med egen processor, 64 KB RAM, ROM, ABC NET).

Inskannad: <https://www.abc80.net/archive/luxor/ABC80x/>

ABC 806 Bruksanvisning Luxor: *ABC 806 Bruksanvisning*. Luxor Datorer AB, Motala 1983.

ABC 806 som vidareutveckling av ABC 800, grafiskt minne 128 KB.

Inskannad: <https://www.abc80.net/archive/luxor/ABC80x/>

ABC80L u.å. Okänd utgivare: *Extra kommandon i ABC80L*. Maskinskrivet blad, 2 s., odaterat.

Beskriver kretsen ABC80L på \$5000: start med CALL (20480), kommandona IPR, EXIT, CAT, DEL, LIST, RUN, VAR, CROSS, LEN m.fl. och skrivardrivrutinen (V24, 1200 baud, jämn paritet, M-filnamn ger vänstermarginal).

Inskannad: <https://www.abc80.net/archive/luxor/ABC80/ABC80L.pdf>

Acorn 1982 Acorn Computers: *The Econet System User Guide*. Acorn Computers Ltd, Cambridge 1982 (403,100 Issue 1, 29 november 1982).

Econet för BBC Microcomputer, Atom och System 3-5: gränssnitt och programvara i EPROM (s. 11), filservern (s. 16), *VIEW och *REMOTE (s. 51-52).

Inskannad: https://chrisacorns.computinghistory.org.uk/docs/Acorn/Manuals/Acorn_The_Econet_System_UG.pdf

Acorn 1983 Acorn Computers: *Econet Advanced user guide*. Acorn Computers Ltd, Cambridge 1983 (412,019 Issue 1, oktober 1983).

Programmering av nätet, direkta operationer (s. 28), MC68B54 och fyrvägshandskakningen (s. 30).

Inskannad: https://chrisacorns.computinghistory.org.uk/docs/Acorn/Manuals/Acorn_Econet_AdvancedUG.pdf

Acorn 1984a Acorn Computers: *Econet Installation Guide*. Acorn Computers Ltd, Cambridge 1984 (0436,101 Issue 1, november 1984).

Kabeln (s. 8), stationsnumren (s. 12), klocklådan och hastigheten (s. 14), bryggor (s. 17), skolnät (s. 20).

Inskannad: https://chrisacorns.computinghistory.org.uk/docs/Acorn/Manuals/Acorn_Econet_Installation_Guide.pdf

Acorn 1984b Acorn Computers: *Z80 second processor for the BBC Microcomputer. User guide*. Acorn Computers Ltd, Cambridge 1984 (409,010 Issue 1, februari 1984).

Z80 med 64 KB i egen låda, kopplad till BBC-datorns uttag "tube" (s. 11), start (s. 14-15), arbetsfördelningen mellan Z80:an och BBC-datorns 6502 (s. 38).

Inskannad: https://chrisacorns.computinghistory.org.uk/docs/Acorn/Manuals/Acorn_Z802ndprocUG.pdf

Anvisning UFD-DOS Luxor Datorer: *Anvisning UFD-DOS*. Luxor Datorer AB, Motala, (c) 1983, 77 s.

UFD-DOS för ABC 800-serien: styrkortet A5/A6/A8 med kortnummer, klusterstorlek och bitkartans sektor, DOSGEN, systemprogrammen.

Inskannad: <https://www.abc80.net/archive/luxor/sw/doc/>

Apple 1978 Apple Computer: *Apple II Reference Manual*. Apple Computer Inc., januari 1978.

Handboken till Apple II: Integer BASIC, monitorn och hårdvaran. S. 127: åtta kortplatser (Peripheral Connectors), med 16 adresser för varje plats (DEVICE SELECT) och 256 (I/O SELECT), t.ex. för en ROM på kortet.

Inskannad: <https://archive.org/details/applerefjan78>

Bit för bit med ABC 800 Luxor: *Bit för bit med ABC 800*. Luxor Datorer AB, Motala 1982 (framtagen av NANCO Elektronik).

Avancerad programmering på ABC800. Kap. 4.1 (s. 52-53): ljudgeneratoren är en vippra som slås om av inport 5, till skillnad från ABC80:s programmerbara krets; rutiner för toner.

Inskannad: <https://www.devili.iki.fi/pub/Luxor/docs/books/>

CAT 1984 CAT Ingenjörbyrå AB: *CAT-NET. Användarbeskrivning, bruksanvisning, systemprogram, monteringsanvisning, felmeddelanden*. CAT Ingenjörbyrå AB, Sundbyberg, pärm med blad daterade 1983-1984 (262 s. inskannade).

Nät för ABC80, ABC800, ABC802 och ABC806 med en centralenhet och terminalkort (RS422A) i varje dator; kommunikationsrutinen ligger i terminalkortets PROM och anropas med CALL. Med lokal flexskiva körs programmet CAT80L (ABC80) eller CAT800L.

Inskannad: https://www.abc80.net/archive/luxor/misc_hw/catnet-manualer.pdf

Coll 1984 John Coll: *The BBC Microcomputer User Guide*. Red. David Allen. British Broadcasting Corporation 1982; utgåvan oktober 1984.

PAGE med och utan Econet (s. 383), minneskartan med NMI-rutinens område (s. 495).

Inskannad: <https://archive.org/details/BBCUG>

Commodore 1978 Commodore: *PET 2001-8 Personal Computer User Manual*. Commodore Business Machines, oktober 1978.

Handboken till PET 2001: MCS 6502 (s. 3); "7167 BYTES FREE" i 8 KB-modellen (s. 6); minneskartan (s. 8); skärmen 25 rader om 40 tecken (s. 19); flyttal med omkring nio siffror (s. 21).

Inskannad: https://archive.org/details/PET_2001-8_Personal_Computer_User_Manual_1978-10_Commodore

Commodore 1979 Commodore: *CBM Dual Drive Floppy Disk Model 2040. User Manual*. Commodore Business Machines, juli 1979 (P/N 329830-4).

Skivenheten 2040 till PET/CBM på IEEE-488-bussen, en "intelligent" enhet som inte tar något av datorns minne (s. 1). Två processorer, 6504 för styrningen och 6502 för bussen, med program i ROM (s. 3); DOS:et s. 39-.

Inskannad: <https://archive.org/details/commodore-cbm-dual-drive-floppy-disk-model-2040-user-manual-july-1979-pn-329830-4>

Commodore 1982 Commodore: *Commodore 64 Programmer's Reference Guide*. Commodore Business Machines 1982 (ISBN 0-672-22056-3).

Minneshanteringen: processorns port på adress 1 kopplar in och ur BASIC, KERNAL och teckengeneratoren (s. 259-260).

Inskannad: <https://archive.org/details/c64-programmer-ref>

Commodore 1982b Commodore: *VIC 20 Programmer's Reference Guide*. Commodore Business Machines och Howard W. Sams, 1982.

Handboken till VIC-20: drygt 5 KB RAM (s. 27); teckengeneratoren 4 KB ROM (s. 82); 23 rader om 22 tecken, med egna tecken 176 x 184 punkter (s. 88); 8 eller 16 färger (s. 92); 6502 (s. 109); BASIC 8 KB och KERNAL 8 KB i ROM (s. 116).

Inskannad: <https://archive.org/details/vic-20-programmers-reference-guide>

Commodore 1983 Commodore: *Commodore 64 CP/M Operating System User's Guide*. Commodore Business Machines 1983.

CP/M-kassetten: 48 KB och 6510:an som I/O-processor (s. 8), växlingen via \$DE00/\$CE00 (s. 137).

Inskannad: https://archive.org/details/Commodore_64_CPM_Operating_System_Users_Guide_1983_Commodore

DataBoard 4025 Sattco: *DataBoard 4025, IEC Bus Interface*. Sattco AB, Solna, datablad 4025-H OKT 79 (22 s.).

Det äldre IEC-kortet (drivrutin 1 KB i 2708 på DOS-PROM-modulen, kanal 49). Satsformatet IEC adress\$, data\$..., IEC\$(n), OPEN/CLOSE med IFC och REN.

Inskannad: https://www.abc80.net/archive/luxor/misc_hw/databoard/

DataBoard 4027 DataSweden: *DataBoard 4027, GPIB Interface*. DataSweden AB, Täby, datablad 87-4027.10 NOV 83 (16 s.); Data Industrier AB, Täby, 4027-20 SEP 84 och MAR 86 (25 s.).

Kortet för IEC-kretsen 4027: TMS 9914A, 75160A/75162A, kortval 49, portarna 0-2, 5 och 7, kodpluggen 5B med bussadressen; för ABC80 en PROM på \$7000-\$73FF.

Inskannad: https://www.abc80.net/archive/luxor/misc_hw/databoard/

DataBoard 4038, 4001 Sattco: *DataBoard 4038 och 4001, Printer Interface Centronics*. Sattco AB, datablad, 4038 JAN 80 (5 s.) och 4001 87-4001.10 MAJ 83 (6 s.).

Centronicskort för DataBoard 4680-bussen: statusordet D3 SLCT, D4 BUSY, D5 PE, D6 FAULT, D7 READY, data med OUT, kortval med kodplugg.

Inskannad: https://www.abc80.net/archive/luxor/misc_hw/databoard/

Digital Research 1982 Digital Research: *CP/M Operating System Manual*. Digital Research, Pacific Grove 1982 (första tryckningen juli 1982; CP/M 2.2).

Delarna BIOS, BDOS, CCP och TPA (s. 1-2); de inbyggda kommandona och filnamnen (s. 3); USER 0-15 (s. 8); de tillfälliga kommandona STAT, ASM, LOAD, PIP, ED, SYSGEN, SUBMIT, DUMP, MOVCPM (s. 9-28).

Digital Research 1983 Digital Research: *CP/M Plus (CP/M Version 3) Operating System. User's Guide, Programmer's Guide, System Guide*. Digital Research Inc., Monterey 1983.

Bankat CP/M 3: minst 96 KB, upp till 16 banker, ett gemensamt område överst, normalt 16 KB (Programmer's Guide, s. 1-1-1-5).

Inskannad: https://archive.org/details/CP-M_Plus_Users_Programmers_and_System_Guides_1983_Digital_Research

Flexskiveenhet ABC Luxor: *Flexskiveenhet ABC. Bruksanvisning*. Luxor AB, Motala, (c) 1980, 64 s. + register.

Flexskiveenheten till ABC 80 (två drivenheter MPI 51 eller BASF 6106, expansionsenhet med kontrollkort och minneskort): skivan, spår och sektorer, söktid, buffertar, DOS-kommandon, tekniska data.

Inskannad: <https://www.abc80.net/archive/luxor/ABC80/>

Hitachi 1985 Hitachi: *HD64180 8-Bit High Integration CMOS Microprocessor. Data Book (Advance Information)*. Hitachi Ltd., februari 1985.

Z80-kompatibel processor med MMU: 64 KB logiska adresser i tre områden, översatta till 512 KB (s. 27-28).

Inskannad: https://archive.org/details/bitsavers_hitachi641aBookFeb85_9635829

Intel 1979 Intel: *The 8086 Family User's Manual*. Intel Corporation, Santa Clara, oktober 1979 (9800722-03).

Segmentregistren (s. 2-7) och segmenteringen av en megabyte (s. 2-10-2-11).

Inskannad: https://archive.org/details/bitsavers_intel80869lyUsersManualOct79_62967963

Liber 1984a Liber: *BASIC II till ABC 80*. Produktblad, Datorutrustning från Liber, Liber Teknikutbildning, Sala, september 1984.

BASIC II-kortet för ABC80 (art. nr 10651): ABC 802:s tolk, UFD-DOS, 64 KB RAM varav 48 KB RAM-skiva, förberett för CP/M. Sidan 21 i samlingen av Sala Teknikutbildnings prislistor och produktblad.

Inskannad: https://www.abc80.net/archive/luxor/marknad/prislistor/Prislista_Salateknikutbildning_AB.pdf

Liber 1984b Liber: *Färggrafik till ABC 80*. Produktblad, Datorutrustning från Liber, Liber Teknikutbildning, Sala, september 1984.

Grafikkortet HR-80 (art. nr 10650 och 10652): egen processor, 256 x 256 punkter i fyra färger, styrkoder och makron. Sidan 22 i samlingen av Sala Teknikutbildnings prislistor och produktblad.

Inskannad: https://www.abc80.net/archive/luxor/marknad/prislistor/Prislista_Salateknikutbildning_AB.pdf

Luxor 1983 Luxor Datorer: *Länge leve ABC 80*. Reklambroschyr, 4 s., tryckkod DR 082 - 8306 (1983 (?)).

Över 20 000 installerade, nära 80 % av skolmarknaden, hemdatorn ca 4 500 kr, ny videoenhet (HF-utgång kanal 36); tekniska data och BASIC-ordlista, CHR\$(151) = "START GRAFIK", CHR\$(135) = "SLUT GRAFIK".

Inskannad: <https://abc80.dk/wp-content/uploads/2025/03/Lange-leve-ABC-80.pdf>

Luxor u.å. Luxor: *Bruksanvisning Expansionsenhet ABC*. Luxor, u.å., 66 79207-11 (4 s., svenska, engelska, tyska). Låda för fyra I/O-kort och tre minneskort till ABC80.

Inskannad: <https://www.abc80.net/archive/luxor/ABC80/ABC80-expansionsl%C3%A5da-ABC-bruksanvisning.pdf>

Luxor, cirkaprislista 1-83 Luxor: *Cirkaprislista Nr 1-83*. Luxor Datorer AB, 2 s.

Priser exkl. moms för datorer, kringutrustning och program, bl.a. ABC 80, ABC800C, ABC800M, ABC 802 och ABC 806.

Inskannad: <https://www.abc80.net/archive/luxor/marknad/prislistor/>

Luxor, flexskiveenheten Luxor: *Flexskiveenhet ABC, ABC 830, ABC 832, Expansionsenhet ABC 890. Servicemanual*. Luxor Datorer, art. nr 66 79207-40 (odaterad).

Drivenheterna, bakplanet och kap. 6 om kontrollerkortet (Z80, PROM 2716, FD1791, PIO, portarna och statusregistret, förkompensering och dataseparator).

Inskannad: <https://www.abc80.net/archive/luxor/diskdrives/>

Luxor, minneskortet Luxor: *Bruksanvisning Minneskort ABC*. Luxor, Motala, art. nr 66 79207-12 (svenska, engelska, tyska), 4 s.

Minneskortet 55 10762-01 för expansionsenheten: 4 KB DOS, PRINTER och IEC (2708), basadress 24 K.

Inskannad: <https://www.abc80.net/archive/luxor/ABC80/>

Luxors servicemanual Luxor: *ABC 80 Service Manual*. Luxor, Motala, art. nr 66 79589-01 (svenska, engelska, tyska), 54 s.

Tekniska data, datordelen med placeringsritning och reservdelslista, bildenheten, kassetminnet, testprogram och kopplingsschema. Underlag för skisserna av kretskortet, maskinen och baksidan.

Inskannad: <https://www.abc80.net/archive/luxor/ABC80/>

Microsoft 1980 Microsoft: *SoftCard. Volume I*. Microsoft Consumer Products, Bellevue 1980.

Z80-kortet för Apple II: minnet (s. 1-5), adressöversättningen (s. 2-5), klockan (s. 2-30), omkopplingen (s. 2-31).

Inskannad: https://archive.org/details/Microsoft_SoftCard_Volume_1

MYAB 1982 MYAB (Mikrokonsult AB): *Bruksanvisning för CP/M på ABC80; Installationsanvisning, CP/M på ABC80*. Mikrokonsult AB, 1982 (54 + 3 s.).

UNI-80: kortet, CP/M 2.2 med MYAB:s BIOS 3.3, RAM-skivan C:, flexskiveenheterna och ABC80-läget.

Inskannad: https://www.abc80.net/archive/luxor/ABC80/bruksanvisning_for_cpm_abc80.pdf

Inskannad: https://www.abc80.net/archive/luxor/ABC80/installationsanvisning_cpm_abc80.pdf

MYAB u.å. MYAB (Mikrokonsult AB): *Bruksanvisning [TKN80]; Monteringsanvisning för TKN80; Att skriva program som går med både 40 och 80 tecken*. Mikrokonsult AB, Göteborg, u.å. (8 + 6 + 2 s.).

TKN80: kort i tangentbordsenheten, bildminne 2 kilobyte på \$5800-\$5FFF (övre halvan även på \$7C00), byte med INP(3) och INP(4), PEEK(885), radtabellen på 884, avkodar-PROM:en som ersätter ABC80/13.

Inskannad: <https://www.abc80.net/archive/luxor/ABC80/TKN80-bruksanvisning.pdf>

Inskannad: https://www.abc80.net/archive/luxor/ABC80/monteringsanvisning_tkn80_abc80.pdf

Inskannad: https://www.abc80.net/archive/luxor/ABC80/skriva_program_med_40_80_tkn.pdf

Nyfors 1982 Nyfors Data Science: *Beskrivning av kommandona i ** SUPERBASIC ** 1982.07.28; Beskrivning av texteditorn "ED81"; Beskrivning av jobströmshanteraren "EXEC"*. Nyfors Data Science, 1982 (omsatt av Thomas Michanek 2021, 7 s.).

Alla kommandon med parametrar, skärmeditorns tangenter, EDIT med filkommandona, EXEC med parametrar och tecknen & / ? och exempel.

Inskannad: <https://www.abc80.net/archive/luxor/ABC80/Superbasic-manual.pdf>

OWOCO 1983 OWOCO: *Bruksanvisning till SUPER SMARTAID (ver 1 och 2)*. OWOCO Olle Wolkert AB, Stockholm Huddinge 1983-08-24 (31 s.).

Manual för Super Smartaid: skärmeditorn och dess CTRL-tangenter, kommandona (AUTO, DEL, REN ... FROM, LIST, VAR, CHANGE, FIND, ED, PEEK, OLD, START, RESUME, LIB/DIR, SYS, KEY, JOB, TIME), stegning med CTRL-C/S/T/D och skrivarrutinen.

Inskannad: <https://www.abc80.net/archive/luxor/ABC80/>

OWOCO 1985 OWOCO: *smartaid MAGNUM. "Korkar upp" nya möjligheter för din ABC80!* Annons i ABC-bladet (ABC-klubben) 1985 nr 3, s. 2.

Magnums DEBUG-funktioner (register, stack, minnesdump, stegning, brytpunkter, miniassembler och disassembler, decimal notering, loggfil) och Super Smartaids. Pris 2 450 kr, ombyggnad av en Super Smartaid 1 495 kr; Super Smartaid 1 450 kr, Smartaid 800 1 980 kr. Levereras i en låda som ansluts utan ingrepp i datorn.

Inskannad: <https://www.abc80.net/archive/luxor/ABC-bladet/>

Printerrutin i PROM Luxor: *Printerrutin i PROM*. Luxor, anvisning, 1 s. (odaterad).

Inställningarna efter PR: för skrivarkretsen 9704 (skrivartyp P/U/V/C/R, paritet, NUL, radlängd, perforerings-skip, formfeed, sidlängd, baud); radräknaren i 32255.

Inskannad: https://www.abc80.net/archive/luxor/misc_hw/

Radio Shack 1978a Radio Shack: *LEVEL II BASIC Reference Manual*. Radio Shack (Tandy Corporation), Fort Worth 1978, första upplagan.

Handboken till TRS-80:s Level II-BASIC. Kap. 10 (s. 10/1-10/4): Expansion Interface med andra kassetband-spelare, skrivare, upp till fyra skivminnen och 32 KB minne; med lådan ansluten försöker datorn läsa in från skivminnet, BREAK vid start hoppar över det. Reserverade ord (s. A/15): många har ingen funktion i Level II utan är reserverade för Disk BASIC. Felet L3 "DISK BASIC only" (s. B/3).

Inskannad: https://archive.org/details/Level_II_BASIC_Reference_Manual_1st_Ed._1978_Radio_Shack

Radio Shack 1978b Radio Shack: *TRS-80 Expansion Interface. Operator's Manual (26-1140, 26-1141, 26-1142)*. Radio Shack (Tandy Corporation), 1978.

Lådan: kretskort, två nåtdelar, plats för ytterligare ett kort, realtidsklocka, socklar för upp till 32 KB i steg om 16 KB; flatkabel med 40-poliga kontakter till porten längst till höger på datorns baksida. Skrivare, skivminnen och en andra kassetbandspelare ansluts till lådan.

Inskannad: <https://www.gutenberg.org/ebooks/27469>

Radio Shack 1978c Radio Shack: *TRS-80 Micro Computer Technical Reference Handbook*. Radio Shack (Tandy Corporation), 1978, första upplagan.

Hårdvaran i TRS-80 Model I: minneskartan med Level I-ROM \$0000-\$0FFF (s. 12-13); Z80 med drygt 1,774 MHz från en kristall på 10,6445 MHz (s. 14); 16 rader om 64 tecken (s. 28).

Inskannad: https://archive.org/details/TRS-80_Micro_Computer_Technical_Reference_Handbook_1st_Edition_1978_Radio_Shack

Sala Teknikutbildning 1987 Sala Teknikutbildning: *Datortillbehör. Produktblad och prislista*. Sala Teknikutbildning AB, Sala, prislista 1987-09-01.

Grafikkortet CW-Grafik (med Craftware Electronics) och ritprogrammet Leonardo för ABC80. Produktbladet är sidorna 5-6 och prislistan sidorna 14-18 i samlingen.

Inskannad: https://www.abc80.net/archive/luxor/marknad/prislistor/Prislista_Salateknikutbildning_AB.pdf

Texas Instruments 1981 Texas Instruments: *Editor/Assembler*. Handbok till Editor/Assembler-modulen för TI-99/4A, Texas Instruments, 1981.

TMS9900-assemblern och de maskinnära gränssnitten, bl.a. mot TI BASIC. Talen lagras i bas 100 i 8 bytes: en exponentbyte (potens av 100, plus 64) och sju bytes mantissa med en siffra 0-99 i varje (s. 279).

Inskannad: <https://archive.org/details/ea-manual>

Zilog 2016 Zilog: *Z80 CPU User Manual (UM0080)*. Zilog Inc., rev. 11, 2016.

Tillverkarens beskrivning av instruktionerna: verkan, flaggor och tider. Underlag för bilagan om Z80-instruktionerna.

<https://www.zilog.com/docs/z80/um0080.pdf>

Program och webbarkiv

ABC-klubben, diskett nr 21 och 27 ABC-klubben: *ABC-kassett/diskett nr 21 och 27*. ABC-klubbens programbibliotek, Bromma, 1985-1988.

Medlemmarnas program i BASIC. Bland spelen: "The Monster Room" (nr 21, "Grafik/aktion med BRA ljud") och HI-Q (nr 27, Dataindustrier AB 1979), som båda skriver till ljudgeneratoren med OUT 6.

Inskannad: <https://www.abc80.net/archive/luxor/sw/ABC-klubben/>

abc80.net abc80.net: *ROM-arkivet för Luxors datorer*. Webbarkiv, katalogen [archive/luxor/Prom/fw/](https://archive.org/details/luxor/Prom/fw/) med underkatalogerna ABC80, ABC800 och ABC802 (förteckning i 00index.txt).

Avbildningar av ROM-kretsarna. ABC 800:s BASIC finns i två uppsättningar, med ändelserna -1 och -12 i filnamnen; del 6 finns för färgskärm (C-1, C-12) och monokrom skärm (M-12). ROM:arna i böckerna är kontrollerade mot arkivet.

Inskannad: <https://www.abc80.net/archive/luxor/Prom/fw/>

abc80sim H. Peter Anvin m.fl.: *abc80sim*. Emulator för ABC80.

Dess ROM-filer för ABC80-BASIC:en är identiska med version 11273 och 9913. Uppgifter därifrån, främst om portarna, är markerade "(abc80sim)".

<https://github.com/sasq64/abc80sim>

Bjørn-Hansen Søren Bjørn-Hansen: *Piccoline-syndromet: Da en sær dansk computer (næsten) blev enden på EDB i folkeskolen*. DR (Danmarks Radio), webbartikel.

Regnecentralens Piccolo och Piccoline i de danska skolorna.

Inskannad: <https://www.dr.dk/viden/webfeature/piccoline>

MAME MAMEdev m.fl.: *MAME*. Emulator för många maskiner, bl.a. ABC80, ABC800-serien och kretskorten på ABC-bussen.

Används bara som ledtråd, t.ex. för ROM-uppsättningarnas kontrollsummor och för Luxors kontrollerkort 55 10828 (src/devices/bus/abcbus/lux10828.cpp: kopieringsskyddet via ULA:n på kortet).

<https://github.com/mamedev/mame>

Microsoft 1978 Microsoft: *BASIC-M6502 (källkoden till Microsofts BASIC för 6502)*. Assemblerkälla, copyright 1976-1978; utgiven som öppen källkod av Microsoft 2025.

Versionerna för bl.a. Commodore och Apple. Rutinen CRUNCH byter bara nyckelorden mot koder; GOTO söker upp raden i programmet.

<https://github.com/microsoft/BASIC-M6502>

NJSZT iTF Neumann János Számítógép-tudományi Társaság, Informatikatörténeti Fórum: *ABC 80 iskola-számítógép*. iTF Adattár (ungerska datahistoriska föreningens databas).

BRG:s licenstillverkning av ABC80 för ungerska skolor och bandspelarna som BRG levererade till Luxor.

Inskannad: https://itf.njszt.hu/termek_hardware/abc-80-iskola-szamitogep/

Norsk Teknisk Museum Norsk Teknisk Museum: *Datamaskin (Tiki 100)*, NTM 27833.1. DigitaltMuseum.

Tiki 100 och den norska upphandlingen av skoldatorer 1984.

Inskannad: <https://digitaltmuseum.no/011024239099/datamaskin>

Symbolförteckning

Förteckningen tar upp alla namn som satts på adresser i ROM:en: rutiner, hoppmål, tabeller och texter. Namnen är valda för den här genomgången; ROM:en själv innehåller inga. Varje rad ger namnet, adressen och sidan i listningen där adressen står. I den elektroniska utgåvan är namnet dessutom en länk dit.

Två slags namn saknas. Adresser som bara har fått ett namn efter sitt läge tas inte med, eftersom namnet redan säger var de står. Ett sådant namn är en bokstav följt av adressen: S för en rutin som anropas med CALL, L för ett hoppmål och D för data, till exempel L0709 för ett hoppmål på \$0709. Variablerna i RAM och adresserna utanför ROM:en står inte här utan i kapitlet Systemvariabler och externa adresser.

AADD	\$37A6	161	CASCLO	\$0428	13	CMPNOT	\$2826	115
ACOMP	\$37F6	161	CASGET	\$0509	16	CMPNUM	\$2689	110
ADDF	\$2AB7	123	CASHLF	\$0484	14	CMPNXT	\$1CB7	81
ADDI	\$2B5E	125	CASINT	\$0594	17	CMPON	\$1FBF	91
ADIV	\$383B	162	CASOPN	\$0491	15	CMPOP	\$205D	93
ALLOC	\$3374	149	CASPRE	\$03BF	13	CMPOUT	\$20FF	95
AMUL	\$38B8	163	CASPUT	\$040E	13	CMPPOK	\$2104	95
ARGWLK	\$33B0	149	CASRBL	\$052B	16	CMPPRT	\$2154	96
ASADDB	\$3ABA	168	CASTAB	\$03A4	12	CMPRD	\$1E6E	87
ASADDN	\$390D	164	CASW0	\$0476	14	CMPREF	\$1DC7	85
ASALG1	\$3A32	167	CASWBI	\$0477	14	CMPREM	\$226B	99
ASALGN	\$3A12	167	CASWBL	\$0439	14	CMPRES	\$2281	100
ASALST	\$3B72	170	CASWBY	\$046A	14	CMPSTR	\$25C6	108
ASBAD	\$39BF	165	CHAIN	\$0D3F	37	CMPTX	\$236C	103
ASBLST	\$3B6D	170	CLEAR	\$0A80	30	CMPSV	\$237F	103
ASBORN	\$3948	164	CLOSE	\$087F	25	CMPSVR	\$2392	103
ASCAR	\$2BBA	127	CLOSST	\$20BF	94	CMPTX	\$2372	103
ASCARG	\$2C6C	128	CLRDOT	\$22CF	101	CMPPAR	\$25EE	108
ASCARN	\$391A	164	CLRROW	\$0267	9	CMPPV	\$2397	103
ASCCMP	\$2C51	128	CLS	\$0276	9	COLD	\$00C6	5
ASCERR	\$2C4B	128	CLSALL	\$20CD	94	CONCAT	\$2F47	137
ASCMPS	\$3817	162	CMD	\$0AA1	30	CONST	\$2A36	121
ASCNT	\$3B68	170	CMDKW	\$0A20	29	CONSTI	\$2AE3	123
ASDIG	\$3B4D	170	CMDLIST	\$0AEF	31	CONSTS	\$2ED7	135
ASDIG2	\$395E	165	CMDLOAD	\$0BA8	32	COSSER	\$3FF4	182
ASDLP	\$3868	162	CMDLP	\$00DB	5	CRLF	\$0234	8
ASDOT	\$3B47	170	CMDMERG	\$0BA9	32	CURADR	\$0293	9
ASENTR	\$396F	165	CMDREN	\$0C8A	35	CURFIL	\$08FC	26
ASFIN	\$3B00	169	CMDRUN	\$0D4C	37	DATA	\$2273	99
ASGNF	\$2989	119	CMDSAVE	\$0DC6	38	DEF	\$1AAA	75
ASGNI	\$29B4	120	CMDTAB	\$0A5A	29	DETEXP	\$1258	51
ASGNS	\$29C3	120	CMPADD	\$27FB	114	DETOK	\$1195	49
ASLEN	\$38FC	164	CMPARG	\$264E	109	DETOP	\$1398	55
ASLEN1	\$3902	164	CMPB9	\$1CAF	81	DETTAB	\$12B6	52
ASMAADD	\$37B4	161	CMPB	\$1BB8	78	DETVAR	\$1422	56
ASMLP	\$38CC	163	CMPCLS	\$2071	93	DEVDFL	\$0193	7
ASMSUB	\$37C9	161	CMPDEF	\$1A62	74	DEVIEC	\$01A1	7
ASNEG	\$3AD6	169	CMPDIM	\$1BAA	78	DEVLST	\$018C	7
ASNZ	\$3B53	170	CMPDMI	\$1BBD	78	DEVPR	\$019A	7
ASPARS	\$39C5	166	CMPDOT	\$22B6	101	DIM	\$1BFA	79
ASPUT	\$3926	164	CMPERR	\$1BF7	79	DIM1	\$32E6	147
ASPUTB	\$3954	164	CMPEXB	\$2362	103	DIM2	\$3311	148
ASREST	\$3ACD	168	CMPEXP	\$2701	112	DIMELM	\$1BFE	79
ASRET	\$3B3E	170	CMPFLV	\$1BA5	78	DIMSTR	\$3338	148
ASSETB	\$38AB	163	CMPFN	\$1AB8	75	DIVF	\$2AC9	123
ASSUBN	\$393B	164	CMPFNC	\$261C	109	DIVI	\$2B73	125
ASUB	\$37AE	161	CMPFOR	\$1C83	81	DOCMD	\$0133	6
ASWIDE	\$3A67	167	CMPGET	\$23A8	104	DOTADR	\$22EE	101
ASWORK	\$3A9D	168	CMPIF	\$1DFB	86	DOTARG	\$22E3	101
ASZERA	\$37E6	161	CMPINL	\$1F6B	90	DVCALL	\$08A5	25
ATNPOS	\$3B8A	171	CMPINP	\$1E58	87	DVCLOS	\$0890	25
ATNSER	\$3C02	172	CMPINX	\$2370	103	DVGET	\$0899	25
BRKSTP	\$2323	102	CMPIVR	\$239F	104	DVINP	\$0893	25
CALLBC	\$2C47	128	CMPLET	\$1B89	77	DVOPEN	\$088A	25
CASBRS	\$03F4	13	CMPNOP	\$226A	99	DVPREP	\$088D	25

DVPRT	\$0896	25	FADDC	\$3402	151	FNKW	\$2431	105
DVPUT	\$089C	25	FADOT	\$17BB	67	FNKWTX	\$13F1	56
EDTAB	\$0348	11	FAEDGT	\$17EC	68	FNORM	\$3494	152
ELMADR	\$2CFA	130	FAEDIG	\$16E3	65	FNPART	\$33AB	149
ELMSIZ	\$1C6C	80	FAEFMT	\$16D1	65	FNPART	\$14DA	59
ENDDIM	\$1C7E	80	FAEND	\$17CE	67	FNRES	\$1B7B	77
ENDF	\$285F	116	FAEOUT	\$17E3	68	FNTEXT	\$13E9	56
ENDPRG	\$00C9	5	FAEPOS	\$1805	68	FNTYPE	\$24F2	106
ENDS	\$2877	116	FAEPRN	\$180D	68	FONE	\$3FBB	182
ENDVF	\$2884	116	FAEXP	\$16C1	65	FOR	\$1CC2	82
ENDVS	\$2887	116	FAEXPN	\$17F1	68	FORTSF	\$1D88	84
EQF	\$2A4C	122	FAFIX	\$16E7	65	FORTSI	\$1D59	84
EQI	\$2B2A	125	FAFIX1	\$16F5	65	FOVFL	\$3758	159
EQS	\$2FE4	139	FAFMT	\$16D5	65	FPI	\$3FA6	182
ERR65	\$2C4F	128	FAFORM	\$17A6	67	FPIH	\$3FA0	182
ERR66	\$2C49	128	FAFRAC	\$17C9	67	FPIQ	\$3FB4	182
ERRIDX	\$2D4F	131	FAINT	\$17D4	67	FPOLY	\$3F61	181
ERRMSG	\$069B	20	FALEAD	\$17BF	67	FPOW	\$2E07	133
ERROR	\$062E	19	FALIGN	\$36BB	157	FSIN	\$3D77	176
ERRSTP	\$0669	19	FAOUT	\$178A	67	FSINC	\$3D78	176
ERRTRP	\$0659	19	FAOVF1	\$1778	67	FSQR	\$3DF5	177
ESCARG	\$0227	8	FAOVF2	\$1783	67	FSTOR1	\$3437	151
EXABS	\$2D7F	131	FAOVFL	\$1766	66	FSTORE	\$3434	151
EXADD	\$2B9F	126	FAPOS	\$16C9	65	FSUB	\$3460	152
EXASC	\$2EEC	136	FARITH	\$2ACD	123	FSUBM	\$3473	152
EXATN	\$2DAF	132	FARND1	\$1741	66	FTAN	\$3EAD	179
EXCALL	\$31CC	144	FAROU1	\$1721	66	FTIERR	\$3EF1	180
EXCHR	\$2FC1	138	FAROUN	\$1719	66	FTILP	\$3EDE	180
EXCOMP	\$2BB3	126	FASGN1	\$17A4	67	FTISGN	\$3EF8	180
EXCOS	\$2DA5	132	FASIGN	\$1791	67	FTOI	\$2DE2	132
EXCUR	\$313B	142	FASMAL	\$16FD	65	FTOINT	\$3ECA	179
EXDIV	\$2BAE	126	FASTR1	\$1748	66	FZERO	\$375B	159
EXDOT	\$22D6	101	FASTRP	\$1747	66	GEF	\$2A75	122
EXEND	\$285B	116	FATINY	\$1703	65	GEI	\$2B4A	125
EXERR	\$204F	93	FATN	\$3B77	171	GES	\$300D	139
EXESKP	\$0DB8	38	FAZER1	\$175C	66	GET	\$1DB1	85
EXEST2	\$0DB3	38	FAZERO	\$1751	66	GETDIG	\$1837	69
EXESTM	\$0DA3	38	FAZERS	\$16C8	65	GETEND	\$1833	68
EXEXP	\$2842	115	FCLR5	\$3C39	173	GETF	\$29A1	120
EXEXP1	\$2DB4	132	FCMP	\$2A93	122	GETFLT	\$1513	60
EXEXPH	\$2841	115	FCMPST	\$2A7A	122	GETI	\$29BD	120
EXFIX	\$2D96	132	FCOPY	\$36FC	158	GETKEY	\$02EA	10
EXFUNC	\$2853	116	FCOS	\$3D70	175	GETLN	\$181E	68
EXIEC	\$016B	6	FDIV	\$35EE	155	GETLN1	\$181D	68
EXINP	\$31C1	144	FDLOOP	\$3622	156	GETLP	\$182A	68
EXINST	\$3075	140	FDRND	\$3689	157	GETOVF	\$183F	69
EXINT	\$2D9B	132	FENTER	\$3766	159	GETPOS	\$08EF	26
EXLEFT	\$2F07	136	FEXP	\$3C03	172	GETS	\$2A22	121
EXLEN	\$2EE3	136	FFIX	\$2DD5	132	GETSFL	\$14FB	59
EXLG10	\$2DBE	132	FFUNC	\$2DC6	132	GETSIN	\$1841	69
EXLOG	\$2DB9	132	FINDLN	\$0F39	42	GOSUB	\$1DE9	85
EXMID	\$2EFD	136	FINT	\$3C7F	173	GOTO	\$1DDB	85
EXMUL	\$2BA9	126	FINT1	\$3C8B	173	GTF	\$2A5E	122
EXNUM	\$3152	143	FINT2	\$3C8E	173	GTI	\$2B5A	125
EXPEEK	\$2B7E	126	FINTFL	\$3CA7	174	GTS	\$2FF6	139
EXPI	\$2D6E	131	FINTLP	\$3C93	173	IDIV	\$3788	160
EXPSE	\$3C7E	173	FLDX	\$3BC5	172	IF	\$1E4B	87
EXPTAB1	\$288B	117	FLGE	\$3D31	175	IM2VEC	\$0034	3
EXPTAB2	\$291F	118	FLN10	\$3D36	175	IMUL	\$3774	160
EXRGHT	\$2F1C	136	FLOG	\$3CC4	174	IMULDV	\$2B6B	125
EXRND	\$31E0	145	FLOG10	\$3CB7	174	INIT	\$0068	4
EXSGN	\$2D86	131	FLTAS6	\$1677	63	INLIN2	\$02AD	10
EXSIN	\$2DA0	132	FLTASC	\$1679	64	INLINE	\$02AA	10
EXSPC	\$30F7	142	FLTCH	\$15C4	62	INPLIN	\$1F82	90
EXSQR	\$2DC3	132	FLTEXP	\$1642	63	INPT	\$1E78	87
EXSTRG	\$30FB	142	FLTSV	\$1670	63	INPUT	\$08DD	26
EXSUB	\$2BA4	126	FMLN2	\$3D3B	175	INSCNV	\$27CF	114
EXSWAP	\$2B79	125	FMLLOOP	\$356C	154	INSLIN	\$0750	22
EXTAB	\$3126	142	FMRND	\$359D	155	INTDHI	\$3F5C	181
EXTAN	\$2DAA	132	FMUL	\$34DF	153	INTDIG	\$3F3F	181
EXTJP1	\$0156	6	FMULE	\$34FA	153	INTTOF	\$3F18	180
EXTJP2	\$017F	6	FN	\$1B2A	76	ITFPOS	\$3F36	181
EXVAL	\$3172	143	FNAME	\$1478	58	ITOF	\$2DF0	133
F2DPI	\$3FA5	182	FNDFIL	\$20E1	95	IXOFF	\$375F	159
FACARY	\$1734	66	FNEG	\$3BC0	171	JOINA	\$140C	56
FADD	\$33F7	151	FNEGX	\$3BBD	171	JOINC	\$140A	56

JPHL	\$2852	115	ONERRG	\$2041	93	RST30	\$0030	3
JPIX	\$2A91	122	OPAND	\$2B14	124	RST38	\$0038	3
KEYINT	\$031E	11	OPEN	\$07E2	23	RSTDAT	\$2293	100
KWADD	\$2421	105	OPENST	\$2074	94	RUBOUT	\$035B	11
KWAND	\$2407	104	OPEQV	\$2AF5	124	RUNLIN	\$0D71	37
KWAS	\$1998	72	OPIMP	\$2B00	124	RUNSTM	\$0D92	37
KWASF	\$1990	72	OPNOT	\$2B1D	124	SCRPOS	\$0338	11
KWCMP	\$2411	105	OPNSTR	\$07C2	23	SETDOT	\$22C7	101
KWDEFN	\$196D	72	OPOR	\$2B0B	124	SETEXT	\$08BD	25
KWELSE	\$1980	72	OPTEXT	\$13D6	55	SFALSE	\$2FE9	139
KWEQV	\$23F5	104	OPTYPE	\$27A0	113	SINCOS	\$3DDA	176
KWIMP	\$23FA	104	OPXOR	\$2B21	124	SINER	\$3FD4	182
KWL1ST	\$1973	72	OUTST	\$212A	96	SINSGN	\$3DE0	176
KWMUL	\$2426	105	POKE	\$2144	96	SKIPS0	\$25BF	108
KWNOT	\$240C	104	POLYLP	\$3F7D	182	SKIPSP	\$25BE	108
KWOR	\$23FF	104	POWF	\$2E00	133	SKPADD	\$10F9	47
KWPOW	\$242B	105	POWFI	\$2E5E	134	SKPARG	\$1B85	77
KWSRCH	\$09F5	29	POWI	\$2E18	133	SKPELM	\$0EFB	41
KWSTEP	\$1986	72	PRCRLF	\$21F6	98	SKPEXP	\$10C3	47
KWTHEN	\$197A	72	PRDIG	\$187D	69	SKPFN	\$10EA	47
KWTO	\$198C	72	PRDIG1	\$1889	69	SKPLEN	\$10F8	47
LEF	\$2A67	122	PREC1	\$23B5	104	SKPRET	\$10E8	47
LEI	\$2B56	125	PREC10	\$23F3	104	SKPSTR	\$10F4	47
LES	\$2FFF	139	PREC2	\$23BC	104	SKPTOK	\$10DD	47
LET	\$1BFD	79	PREC3	\$23C3	104	SMUL	\$2B83	126
LINF0	\$0238	9	PREC4	\$23CA	104	SMULDV	\$2B87	126
LNKADR	\$10A4	46	PREC5	\$23D1	104	SPACE	\$11EA	50
LNKDEF	\$0F59	43	PREC6	\$23D7	104	SPACES	\$224A	99
LNKELM	\$0EAA	41	PREC7	\$23DE	104	SQRADD	\$3E71	178
LNKEXP	\$1032	45	PREC8	\$23E5	104	SQRBOR	\$3E68	178
LNKFN	\$111D	48	PREC9	\$23EC	104	SQRBYT	\$3E8D	178
LNKFOR	\$0F7C	43	PRECLV	\$2705	112	SQRCLR	\$3E2A	177
LNKLST	\$106B	46	PRELNK	\$0E6E	40	SQRCY	\$3E53	178
LNKNEW	\$108E	46	PREPST	\$2077	94	SQRDIG	\$3E5B	178
LNKNXT	\$0FDB	44	PRIMARY	\$257F	107	SQREND	\$3E9E	179
LNKPAR	\$1050	45	PRINT	\$08D4	26	SQRLP	\$3E37	177
LNKREF	\$0D01	36	PRINTS	\$219F	97	SQRNXT	\$3E86	178
LNKSKP	\$102E	45	PRNUM	\$1862	69	SQRRLD	\$3E95	179
LNKSRC	\$1074	46	PRSGN	\$1860	69	SQRSHL	\$3E90	179
LNKVAR	\$103C	45	PRSIN	\$1855	69	SQRSUB	\$3E40	178
LNKWR	\$10B0	46	PRTCOM	\$221C	99	STEXE1	\$0E14	39
LOGDBL	\$3CCE	174	PRTEXP	\$21C0	98	STEXE2	\$0E42	39
LOGEXP	\$3D0D	175	PRTFIT	\$21F0	98	STKCHK	\$3389	149
LOGK	\$3D03	175	PRTSEP	\$223A	99	STKW1	\$1911	71
LOGRED	\$3CE3	174	PRTXT	\$01A8	7	STKW2	\$199C	72
LOGSER	\$3D6F	175	PUSH0	\$1C7F	80	STMEND	\$218D	97
LTF	\$2A70	122	PUTBC	\$239B	103	STOP	\$2320	102
LTI	\$2B3E	125	PUTTOK	\$2368	103	STRCMP	\$3012	139
LTS	\$3008	139	RANDOM	\$2258	99	STRCPN	\$3068	140
MADD3	\$3447	151	RCVB1T	\$061C	18	STRFIL	\$30FE	142
MEMFUL	\$3398	149	RCVBLK	\$05D0	18	STRPSH	\$2FBB	138
MEMRES	\$335A	148	RCVBYT	\$061A	18	STRSKP	\$2F25	136
MSHL	\$3736	159	RCVSTA	\$0556	17	STRUE	\$2FF2	139
MSHR	\$3714	158	RDBC	\$1017	45	STTAB1	\$1A08	73
MSUB3	\$34CB	153	RDBC1	\$1016	45	STTAB2	\$1A36	73
MUL10A	\$3F00	180	RDDE	\$101D	45	SUBF	\$2ABD	123
MUL10X	\$3F16	180	RDDE1	\$101C	45	SUBI	\$2B62	125
MULF	\$2AC3	123	RDVAL	\$1EFE	89	TOFLT	\$256E	107
MULI	\$2B67	125	READ	\$1EC5	88	TOINT	\$2561	107
NEF	\$2A55	122	READY	\$00CC	5	TOKKW	\$18D4	71
NEGF	\$2A43	121	REFADR	\$0D33	36	TOKLEN	\$1105	47
NEGHL	\$2AEE	124	REM	\$2276	99	TOKLIN	\$1890	70
NEGI	\$2AE9	123	RESET	\$0000	3	TOKSTM	\$18C7	70
NEI	\$2B34	125	RESTOR	\$2286	100	TRACE	\$2325	102
NES	\$2FED	139	RETURN	\$22B0	100	TRCPRT	\$232F	102
NEW	\$0A70	30	RNDADD	\$32E5	147	TSTBRK	\$033E	11
NEWLIN	\$0742	21	RNDINI	\$225D	99	TXBLK	\$0995	27
NEXT	\$1D21	83	RNDMUL	\$32E0	147	TXTEND	\$0985	27
NMI	\$0066	4	ROOM	\$339B	149	TXTINP	\$09A0	28
NMICLK	\$0047	4	ROTTXT	\$1449	57	TXTPRT	\$0912	26
NOCODE	\$22B5	100	ROWTAB	\$0374	12	TYPCNV	\$2551	106
NODATA	\$10FB	47	RST08	\$0008	3	VAR0	\$2CCD	129
NODIR	\$0DBF	38	RST10	\$0010	3	VAR1	\$2CE7	130
NOTRAC	\$232A	102	RST18	\$0018	3	VAR1F	\$2CD9	130
NXTLEV	\$27E8	114	RST20	\$0020	3	VAR1I	\$2CDF	130
ON	\$2009	92	RST28	\$0028	3	VAR1S	\$2CE4	130

VAR2	\$2D39	130	VARNAM	\$26CB	111	WRDE1	\$1022	45
VAR2F	\$2D2B	130	VARSIZ	\$10B8	46	WRDIG	\$1815	68
VAR2I	\$2D31	130	WRBC	\$1029	45	WRZERO	\$1761	66
VAR2S	\$2D36	130	WRBC1	\$1028	45			
VARIDX	\$2D11	130	WRDE	\$1023	45			